



UNIVERSIDAD DE LAS CIENCIAS INFORMÁTICAS
FACULTAD DE TECNOLOGÍAS INTERACTIVAS

**SERVICIO BACKEND BASADO EN EL PARADIGMA DE LÍNEAS DE PRODUCTOS
DE SOFTWARE Y MODELOS DE CARACTERÍSTICAS PARA LA GESTIÓN
SISTEMÁTICA DE LA VARIABILIDAD CURRICULAR EN INSTITUCIONES DE
EDUCACIÓN SUPERIOR**

Trabajo de diploma para optar por el título de Ingeniero en Ciencias Informáticas

Autor: Jose Luis Echemendia López

Tutores: MSc. Yadira Ramírez Rodríguez

Ing. Liany Sobrino Miranda

La Habana, 2026

What is well set on your mind will be for sure well set on your lips
José Martí

To my wife, my endless love

Agradecimientos

I am greatly indebted to professor John...

Declaración de auditoría

Declaramos ser autores de la presente tesis y reconocemos a la Universidad de las Ciencias Informáticas los derechos patrimoniales sobre esta, con carácter exclusivo.

Para que así conste firmamos la presente a los ____ días del mes de _____ del año _____.

Jose Luis Echemendia López
Autor

MSc. Yadira Ramírez Rodríguez
Tutor

Ing. Liany Sobrino Miranda
Tutor

En el contexto de la rápida evolución tecnológica y las transformaciones sociales del siglo XXI, la Educación Superior enfrenta el desafío de gestionar currículos flexibles que respondan a demandas cambiantes. Sin embargo, esta tarea se ve constantemente superada por el vertiginoso ritmo del cambio tecnológico, que amenaza con dejar obsoleto cualquier modelo educativo, por más innovador que sea en su concepción, antes de que pueda ser plenamente implementado y consolidado. Además, el mantenimiento de la variabilidad curricular se realiza actualmente mediante prácticas artesanales basadas en documentos estáticos y procesos manuales, un enfoque que resulta insostenible por su lentitud, propensión a errores e incapacidad para garantizar la trazabilidad y coherencia de los planes de estudio.

Esta investigación aborda este problema aplicando los principios de la Ingeniería de Líneas de Productos de Software (SPL) y los Modelos de Características (*Feature Models* - FM) al dominio de la planificación curricular. El trabajo se centra en el diseño, implementación y validación de un *backend* que permita a los gestores académicos modelar currículos dando lugar a la variabilidad curricular mediante FMs, con el objetivo de generar "Líneas de Productos Educativos". El sistema cuenta con una API RESTful que sirve como interfaz para las operaciones de creación, análisis, validación y configuración de FMs, permitiendo representar características y relaciones de obligatoriedad, optatividad y prerrequisitos como restricciones formales.

El resultado principal es un servicio backend especializado que no solo soporta el modelado y la persistencia de FMs adaptados al ámbito educativo, sino que también proporciona la base computacional para automatizar la generación y validación de planes de estudio diversos y consistentes a partir de un tronco común. Este desarrollo sienta las bases técnicas para una nueva generación de herramientas de gestión académica, demostrando la viabilidad de transferir técnicas avanzadas de ingeniería de software al dominio educativo.

Palabras clave: Líneas de Productos Educativos, Líneas de Productos de Software, Modelos Característicos, variabilidad curricular.

Introducción	1
1 Fundamentación teórica	5
1.1 Conceptos asociados al tema	5
1.1.1 Ingeniería de Líneas de Productos de Software (SPLE)	5
1.1.2 Modelos de Características (<i>Feature Models</i>)	7
1.1.3 Lenguaje Universal de Variabilidad (UVL)	13
1.1.4 Configuraciones y configurador curricular	16
1.1.5 Gestión y flexibilidad curricular. Línea de Productos Educativos	17
1.2 Fortalezas de los Modelo de Características (FM) aplicado en la gestión curricular	19
1.2.1 Analogía formal curricular	20
1.2.2 Beneficios de las Líneas de Productos Educativos	26
1.3 Análisis del mercado	26
1.3.1 Panorama general del mercado	27
1.3.2 Análisis de soluciones existentes	27
1.3.3 Oportunidades de mercado identificadas	29
1.4 Herramientas y tecnologías	31
1.4.1 Lenguaje de programación	31
1.4.2 <i>Framework</i> de desarrollo	32
1.4.3 Base de datos y gestor de base de datos	34
1.4.4 Contenerización y orquestación. Docker y Kubernetes	35
1.4.5 Almacenamiento en memoria, caché y mensajería	37
1.4.6 Almacenamiento de archivos y objetos	38
1.4.7 Procesamiento asíncrono	39
1.4.8 Algoritmos, motor de reglas y validación	41
1.4.9 Gestor de dependencias	42
1.4.10 Herramienta para el control de versiones	43
1.4.11 Lenguaje de modelado	44
1.4.12 Herramienta CASE	44

1.4.13	Herramientas para diseñar y ejecutar pruebas	45
1.4.14	Entorno integrado de desarrollo	46
1.5	Metodología de desarrollo del software	46
1.5.1	Metodologías de desarrollo de software tradicionales	47
1.5.2	Metodologías de desarrollo de software ágiles	47
1.5.3	Elección de la metodología	48
1.5.4	Programación Extrema	50
1.6	Conclusiones del capítulo	52
2	Análisis y diseño de la propuesta de solución	54
2.1	Descripción de la propuesta de solución	54
2.1.1	Sistemas de roles estáticos	55
2.1.2	Componentes funcionales de la lógica de negocio del sistema	58
2.2	Levantamiento de requisitos	61
2.2.1	Requisitos funcionales	61
2.2.2	Requisitos no funcionales	75
2.3	Historias de usuario	77
2.3.1	Estimación de esfuerzo por historia de usuarios	79
2.3.2	Desarrollo del plan de iteraciones	80
2.3.3	Desarrollo del plan de entregas	80
2.4	Tarjetas Clase-Responsabilidad-Colaboración (CRC)	81
2.5	Arquitectura de software	81
2.5.1	Representación arquitectónica de la solución	82
2.6	Patrones de diseños	87
2.6.1	Patrones GRASP	87
2.6.2	Patrones Banda de los Cuatro (GoF)	88
2.7	Conclusiones del capítulo	89
3	Implementación, despliegue y pruebas	90
3.1	Fase de implementación	90
3.1.1	Tareas de ingeniería	90
3.2	Despliegue del sistema	91
3.2.1	Diagrama de despliegue	91
3.3	Fase de pruebas	95
3.3.1	Estrategia de pruebas	96
3.3.2	Pruebas unitarias	96
3.3.3	Pruebas de integración	100
3.3.4	Pruebas de seguridad	103

3.3.5	Pruebas de rendimiento	105
3.3.6	Pruebas de aceptación	111
3.4	Conclusiones del capítulo	114
Conclusiones		115
Recomendaciones		116
Glossary		117
Acrónimos		120
Apéndices		122
A	Historias de usuario	123
B	Tarjetas CRC	133
C	Tareas de ingenierías	140
D	Pruebas unitarias	166
E	Pruebas de integración	181
F	Lista de verificación de seguridad	196
G	Pruebas de aceptación	198

Índice de figuras

1.1	Símbolos gráficos para representar las relaciones estructurales.	10
1.2	Representación gráfica del FM del modelo Sandwich especificado en UVL. Fuente: elaboración propia.	16
1.3	Representación simplificada #1 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.	23
1.4	Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.	24
2.1	Componentes de la lógica del negocio del sistema. Fuente: elaboración propia.	58
2.2	Arquitectura en capas de la propuesta de solución. Fuente: elaboración propia.	83
2.3	Arquitectura de Celery. Fuente: elaboración propia.	85
3.1	Diagrama de despliegue. Fuente: Elaboración propia.	92
3.2	Resultados de las Pruebas Unitarias. Fuente: Elaboración propia.	100
3.3	Resultados de las Pruebas de Integración. Fuente: Elaboración propia.	103
3.4	Resultados de las Pruebas de Seguridad. Fuente: Elaboración propia.	105
3.5	Resultados de la primera iteración de pruebas de rendimiento. Fuente: Elaboración propia.	108
3.6	Resultados de la segunda iteración de pruebas de rendimiento. Fuente: Elaboración propia.	110
3.7	Resultados de las Pruebas Aceptación. Fuente: Elaboración propia.	113

Índice de cuadros

1.1	Notación semántica de las relaciones entre características de los FMs.	8
1.2	Analogía de conceptos de Línea de Productos de Software (SPL) en el Dominio Educativo. Fuente: Elaboración propia.	20
1.3	Comparación entre <i>frameworks</i> de Python	33
1.4	Tabla Comparativa: FastAPI vs FastAPI + Celery. Fuente: Elaboración propia.	40
1.5	Comparación, Metodología Ágil VS Metodología Tradicional.	49
1.6	Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto.	49
1.7	Comparación de Metodología Ágiles. Fuente: Elaboración propia	50
2.1	Funciones y misión de los roles del sistema. Fuente: elaboración propia.	56
2.2	Jerarquía de roles estáticos del sistema. Fuente: elaboración propia.	56
2.3	Requisitos funcionales del sistema. Fuente: Elaboración propia.	61
2.4	Historia de usuario # 1	78
2.5	Estimación de esfuerzo por historia de usuario	79
2.6	Plan de duración de las iteraciones	80
2.7	Plan de entregas.	81
2.8	Tarjeta CRC # 1	81
3.1	Tarea de ingeniería # 1	91
3.2	Caso de Prueba Unitaria No1	96
3.3	Matriz de tipo de error contra componentes. Fuente: Elaboración propia	98
3.4	Caso de Prueba de Integración No1	101
3.5	Resultados de pruebas de seguridad - Primera iteración. Fuente: Elaboración propia.	104
3.6	Prueba de aceptación # 1	111
A.1	Historia de usuario # 2	123
A.2	Historia de usuario # 3	124
A.3	Historia de usuario # 4	124
A.4	Historia de usuario # 5	125
A.5	Historia de usuario # 6	126
A.6	Historia de usuario # 7	126

A.7	Historia de usuario # 8	127
A.8	Historia de usuario # 9	128
A.9	Historia de usuario # 10	129
A.10	Historia de usuario # 11	129
A.11	Historia de usuario # 12	130
A.12	Historia de usuario # 13	130
A.13	Historia de usuario # 14	131
A.14	Historia de usuario # 15	132
B.1	Tarjeta CRC # 2	133
B.2	Tarjeta CRC # 3	133
B.3	Tarjeta CRC # 4	133
B.4	Tarjeta CRC # 5	134
B.5	Tarjeta CRC # 6	134
B.6	Tarjeta CRC # 7	134
B.7	Tarjeta CRC # 8	135
B.8	Tarjeta CRC # 9	135
B.9	Tarjeta CRC # 10	135
B.10	Tarjeta CRC # 11	136
B.11	Tarjeta CRC # 12	136
B.12	Tarjeta CRC # 13	136
B.13	Tarjeta CRC # 14	136
B.14	Tarjeta CRC # 15	137
B.15	Tarjeta CRC # 16	137
B.16	Tarjeta CRC # 17	137
B.17	Tarjeta CRC # 18	138
B.18	Tarjeta CRC # 19	138
B.19	Tarjeta CRC # 20	138
B.20	Tarjeta CRC # 21	139
B.21	Tarjeta CRC # 22	139
C.1	Tarea de ingeniería # 2	140
C.2	Tarea de ingeniería # 3	140
C.3	Tarea de ingeniería # 4	141
C.4	Tarea de ingeniería # 5	141
C.5	Tarea de ingeniería # 6	141
C.6	Tarea de ingeniería # 7	142
C.7	Tarea de ingeniería # 8	142

C.8 Tarea de ingeniería # 9	143
C.9 Tarea de ingeniería # 10	143
C.10 Tarea de ingeniería # 11	143
C.11 Tarea de ingeniería # 12	144
C.12 Tarea de ingeniería # 13	144
C.13 Tarea de ingeniería # 14	144
C.14 Tarea de ingeniería # 15	145
C.15 Tarea de ingeniería # 16	145
C.16 Tarea de ingeniería # 17	145
C.17 Tarea de ingeniería # 18	146
C.18 Tarea de ingeniería # 19	146
C.19 Tarea de ingeniería # 20	146
C.20 Tarea de ingeniería # 21	147
C.21 Tarea de ingeniería # 22	147
C.22 Tarea de ingeniería # 23	148
C.23 Tarea de ingeniería # 24	148
C.24 Tarea de ingeniería # 25	148
C.25 Tarea de ingeniería # 26	149
C.26 Tarea de ingeniería # 27	149
C.27 Tarea de ingeniería # 28	149
C.28 Tarea de ingeniería # 29	150
C.29 Tarea de ingeniería # 30	150
C.30 Tarea de ingeniería # 31	150
C.31 Tarea de ingeniería # 32	151
C.32 Tarea de ingeniería # 33	151
C.33 Tarea de ingeniería # 34	152
C.34 Tarea de ingeniería # 35	152
C.35 Tarea de ingeniería # 36	152
C.36 Tarea de ingeniería # 37	153
C.37 Tarea de ingeniería # 38	153
C.38 Tarea de ingeniería # 39	153
C.39 Tarea de ingeniería # 40	154
C.40 Tarea de ingeniería # 41	154
C.41 Tarea de ingeniería # 42	154
C.42 Tarea de ingeniería # 43	155
C.43 Tarea de ingeniería # 44	155
C.44 Tarea de ingeniería # 45	155

C.45 Tarea de ingeniería # 46	156
C.46 Tarea de ingeniería # 47	156
C.47 Tarea de ingeniería # 48	156
C.48 Tarea de ingeniería # 49	157
C.49 Tarea de ingeniería # 50	157
C.50 Tarea de ingeniería # 51	157
C.51 Tarea de ingeniería # 52	158
C.52 Tarea de ingeniería # 53	158
C.53 Tarea de ingeniería # 54	159
C.54 Tarea de ingeniería # 55	159
C.55 Tarea de ingeniería # 56	159
C.56 Tarea de ingeniería # 57	160
C.57 Tarea de ingeniería # 58	160
C.58 Tarea de ingeniería # 59	160
C.59 Tarea de ingeniería # 60	161
C.60 Tarea de ingeniería # 61	161
C.61 Tarea de ingeniería # 62	161
C.62 Tarea de ingeniería # 63	162
C.63 Tarea de ingeniería # 64	162
C.64 Tarea de ingeniería # 65	162
C.65 Tarea de ingeniería # 66	163
C.66 Tarea de ingeniería # 67	163
C.67 Tarea de ingeniería # 68	163
C.68 Tarea de ingeniería # 69	164
C.69 Tarea de ingeniería # 70	164
C.70 Tarea de ingeniería # 71	165
C.71 Tarea de ingeniería # 72	165
C.72 Tarea de ingeniería # 73	165
D.1 Caso de Prueba Unitaria No1	166
D.2 Caso de Prueba Unitaria No2	167
D.3 Caso de Prueba Unitaria No3	167
D.4 Caso de Prueba Unitaria No4	168
D.5 Caso de Prueba Unitaria No5	168
D.6 Caso de Prueba Unitaria No6	169
D.7 Caso de Prueba Unitaria No7	169
D.8 Caso de Prueba Unitaria No8	170
D.9 Caso de Prueba Unitaria No9	170

D.10 Caso de Prueba Unitaria No10	171
D.11 Caso de Prueba Unitaria No11	171
D.12 Caso de Prueba Unitaria No12	172
D.13 Caso de Prueba Unitaria No13	172
D.14 Caso de Prueba Unitaria No14	173
D.15 Caso de Prueba Unitaria No15	173
D.16 Caso de Prueba Unitaria No16	173
D.17 Caso de Prueba Unitaria No17	174
D.18 Caso de Prueba Unitaria No18	174
D.19 Caso de Prueba Unitaria No19	175
D.20 Caso de Prueba Unitaria No20	176
D.21 Caso de Prueba Unitaria No21	176
D.22 Caso de Prueba Unitaria No22	177
D.23 Caso de Prueba Unitaria No23	177
D.24 Caso de Prueba Unitaria No24	178
D.25 Caso de Prueba Unitaria No25	178
D.26 Caso de Prueba Unitaria No26	178
D.27 Caso de Prueba Unitaria No27	179
D.28 Caso de Prueba Unitaria No28	179
D.29 Caso de Prueba Unitaria No29	180
E.1 Caso de Prueba de Integración HU4-P1	181
E.2 Caso de Prueba de Integración HU4-P2	182
E.3 Caso de Prueba de Integración HU5-P1	183
E.4 Caso de Prueba de Integración HU5-P2	183
E.5 Caso de Prueba de Integración HU7-P1	184
E.6 Caso de Prueba de Integración HU7-P2	185
E.7 Caso de Prueba de Integración HU8-P1	186
E.8 Caso de Prueba de Integración HU8-P2	187
E.9 Caso de Prueba de Integración HU9-P1	187
E.10 Caso de Prueba de Integración HU9-P2	188
E.11 Caso de Prueba de Integración HU10-P1	189
E.12 Caso de Prueba de Integración HU10-P2	190
E.13 Caso de Prueba de Integración HU11-P1	191
E.14 Caso de Prueba de Integración HU11-P2	191
E.15 Caso de Prueba de Integración HU6-P1	192
E.16 Caso de Prueba de Integración HU6-P2	193
E.17 Caso de Prueba de Integración HU12-P1	194

E.18 Caso de Prueba de Integración HU13-P1	195
F.1 Lista de Verificación de Seguridad	196
G.1 Prueba de aceptación # 2	198
G.2 Prueba de aceptación # 3	199
G.3 Prueba de aceptación # 4	199
G.4 Prueba de aceptación # 5	200
G.5 Prueba de aceptación # 6	201
G.6 Prueba de aceptación # 7	202
G.7 Prueba de aceptación # 8	203
G.8 Prueba de aceptación # 9	203
G.9 Prueba de aceptación # 10	204
G.10 Prueba de aceptación # 11	205
G.11 Prueba de aceptación # 12	206
G.12 Prueba de aceptación # 13	207
G.13 Prueba de aceptación # 14	207
G.14 Prueba de aceptación # 15	208
G.15 Prueba de aceptación # 16	209
G.16 Prueba de aceptación # 17	210
G.17 Prueba de aceptación # 18	211
G.18 Prueba de aceptación # 19	212
G.19 Prueba de aceptación # 20	213
G.20 Prueba de aceptación # 21	214
G.21 Prueba de aceptación # 22	214
G.22 Prueba de aceptación # 23	215
G.23 Prueba de aceptación # 24	216
G.24 Prueba de aceptación # 25	217
G.25 Prueba de aceptación # 26	218
G.26 Prueba de aceptación # 27	218
G.27 Prueba de aceptación # 28	219
G.28 Prueba de aceptación # 29	220
G.29 Prueba de aceptación # 30	221
G.30 Prueba de aceptación # 31	222
G.31 Prueba de aceptación # 32	223
G.32 Prueba de aceptación # 33	223
G.33 Prueba de aceptación # 34	224
G.34 Prueba de aceptación # 35	225

G.35 Prueba de aceptación # 36	226
G.36 Prueba de aceptación # 37	227
G.37 Prueba de aceptación # 38	228
G.38 Prueba de aceptación # 39	229
G.39 Prueba de aceptación # 40	230
G.40 Prueba de aceptación # 41	230
G.41 Prueba de aceptación # 42	231
G.42 Prueba de aceptación # 43	232
G.43 Prueba de aceptación # 44	233
G.44 Prueba de aceptación # 45	234
G.45 Prueba de aceptación # 46	235
G.46 Prueba de aceptación # 47	236
G.47 Prueba de aceptación # 48	236
G.48 Prueba de aceptación # 49	237
G.49 Prueba de aceptación # 50	238
G.50 Prueba de aceptación # 51	239
G.51 Prueba de aceptación # 52	240
G.52 Prueba de aceptación # 53	241
G.53 Prueba de aceptación # 54	241
G.54 Prueba de aceptación # 55	242
G.55 Prueba de aceptación # 56	243
G.56 Prueba de aceptación # 57	244
G.57 Prueba de aceptación # 58	245
G.58 Prueba de aceptación # 59	245
G.59 Prueba de aceptación # 60	246
G.60 Prueba de aceptación # 61	247
G.61 Prueba de aceptación # 62	248
G.62 Prueba de aceptación # 63	249
G.63 Prueba de aceptación # 64	249

Lista de algoritmos

Lista de códigos fuentes

1.1	Ejemplo de modelo de variabilidad en Lenguaje de Variabilidad Universal (UVL). Fuente universalvariabilitylanguage	15
1.2	Ejemplo de modelo de variabilidad en UVL para la asignatura Estructura de Datos I.	20

El panorama de la Educación Superior en el siglo XXI está inmerso en una dinámica de cambio sin precedentes, impulsada por rápidos avances tecnológicos y profundas transformaciones sociales. La aceleración tecnológica y las fluctuantes demandas del mercado laboral exigen que las instituciones académicas migren de currículos rígidos hacia planes de estudio dinámicos y flexibles (**unesco2022**). Esta necesidad de flexibilidad introduce un desafío crítico que se sitúa en el núcleo de la gestión universitaria moderna: la gestión de la variabilidad curricular.

Para abordar este desafío, es necesario comprender sus dos dimensiones fundamentales. Por un lado, la innovación educativa, la cual se refiere a la introducción de nuevas ideas, enfoques, metodologías y tecnologías en el ámbito educativo; busca mejorar la calidad de la enseñanza y el aprendizaje, fomentando la creatividad, el pensamiento crítico y la resolución de problemas en los estudiantes (**gomez2023**). Por otro lado, la gestión curricular, que implica el diseño, desarrollo e implementación de planes de estudio y programas educativos. Esta se ocupa de organizar y estructurar los contenidos, las habilidades y las competencias que se enseñan, así como la evaluación y mejora continua de los programas para asegurar su relevancia y efectividad (**gomez2023**). Ambas son aspectos fundamentales para el desarrollo de sistemas educativos actuales.

Para que la innovación educativa y la gestión curricular puedan responder efectivamente a las demandas de un entorno cambiante, la literatura especializada señala la necesidad de adoptar un enfoque de gestión por procesos y mejora continua en las instituciones de Educación Superior (**delaguila2014**). Este enfoque, inspirado en modelos de gestión de calidad como la norma ISO 9001, implica superar la visión fragmentada de la administración académica para concebir el currículo como un sistema integrado de procesos interrelacionados que deben ser evaluados y optimizados permanentemente (**soto2022**). La gestión por procesos permite identificar con precisión las actividades clave —desde el diseño de planes de estudio hasta la evaluación del aprendizaje—, establecer responsables, y documentar procedimientos para garantizar la trazabilidad y la coherencia institucional (**veiga2024**). Investigaciones aplicadas en universidades latinoamericanas demuestran que la implementación de este enfoque contribuye a reducir los “cuellos de botella” administrativos, optimizar el uso de recursos y alinear las operaciones cotidianas con los objetivos estratégicos de la institución (**gutama2025**). Bajo esta óptica, la gestión curricular deja de ser una tarea puramente administrativa para convertirse en un ejercicio estratégico de diseño y rediseño continuo, fundamentado en ciclos de planificación, ejecución, verificación y actuación.

Este enfoque estratégico de gestión por procesos cobra especial relevancia al comprender que la innova-

ción educativa y la gestión curricular no son dimensiones independientes, sino que se encuentran estrechamente interrelacionadas. La introducción de enfoques innovadores en la enseñanza y el aprendizaje requiere una gestión curricular flexible y adaptable, que permita revisar y actualizar constantemente los planes de estudio para integrar nuevas metodologías, tecnologías y recursos educativos. Un ejemplo claro es la integración de las [Tecnologías de la Información y Comunicación \(TIC\)](#) en el aula, cuyo máximo beneficio se logra cuando la gestión curricular planifica su uso estratégico ([gomez2023](#)). Esta interdependencia se ha vuelto crítica con la proliferación de nuevas modalidades educativas (presenciales, híbridas, en línea, [Curso en Línea Masivo y Abierto \(MOOC\)](#), *micro-learning*) y enfoques pedagógicos (aprendizaje basado en proyectos, *flipped classroom*, gamificación), lo que ha creado un panorama de extrema variabilidad instruccional ([bates2019teaching](#)).

A pesar de esta creciente complejidad, la definición y el mantenimiento de la variabilidad curricular en las instituciones de Educación Superior se gestionan actualmente mediante prácticas obsoletas. La dependencia de documentos estáticos, hojas de cálculo y procesos manuales constituye un enfoque fragmentado que resulta costoso, lento y altamente propenso a errores. Esta gestión artesanal dificulta la actualización oportuna de la información, genera inconsistencias entre versiones y complica la trazabilidad de los cambios. En la práctica, cada nuevo curso o recurso educativo se desarrolla prácticamente desde cero, desaprovechando la oportunidad de reutilizar componentes comunes y construir conocimiento de manera incremental sobre bases previamente validadas.

Las consecuencias de este paradigma obsoleto son profundas y de gran alcance. La falta de un enfoque sistemático provoca la divulgación de información desactualizada, compromete la coherencia institucional y puede llevar al diseño de rutas formativas incoherentes que afectan la progresión del estudiante. Esto incrementa la carga de trabajo administrativo y docente, limita la visibilidad global del currículo y tiene un impacto directo en la satisfacción del estudiante y en los procesos de acreditación. Irónicamente, la misma tecnología que ha posibilitado la explosión de modalidades y enfoques pedagógicos carece, en la actualidad, de los mecanismos sistemáticos para gestionar la complejidad que ella misma ha generado. Existe una batalla contrarreloj, donde la velocidad de la innovación tecnológica supera la capacidad de reacción de los sistemas educativos tradicionales, volviendo potencialmente obsoleta cualquier renovación curricular incluso antes de su implementación ([sanchez2020metodologia](#)). Esta situación deja de manifiesto que el paradigma artesanal de diseño curricular ha llegado a su límite.

De este análisis, se desprende la siguiente interrogante que define el **problema de investigación**: ¿Cómo gestionar sistemáticamente la variabilidad curricular en instituciones de Educación Superior, de manera que se superen las limitaciones de los enfoques basados en documentación manual y se garantice la coherencia, trazabilidad y sostenibilidad de los planes de estudio? Tomando como **objeto de estudio**: la gestión de la variabilidad curricular en instituciones de Educación Superior. Enmarcado en el **campo de acción**: La aplicación del paradigma de Líneas de Productos de Software para la modelación y gestión de la variabilidad curricular. Con el fin de solucionar la problemática planteada, se define como **objetivo general**: desarrollar un servicio *backend* basado en el paradigma de [SPL](#) y [FM](#) para la gestión sistemática de la variabilidad

curricular en instituciones de Educación Superior.

Para dar cumplimiento al objetivo planteado se proponen las siguientes **tareas investigativas**:

1. **Estudio** del estado del arte en la [Ingeniería de Líneas de Productos de Software \(SPLE\)](#), [FM](#) y su aplicación al dominio educativo.
2. **Análisis** de diferentes soluciones homólogas para la identificación de características generales y tendencias en este tipo de aplicaciones.
3. **Selección** de las herramientas, tecnologías y metodología a emplear, para garantizar una excelente y eficiente experiencia de desarrollo y alineado con los objetivos del proyecto.
4. **Identificación** de los requisitos funcionales y no funcionales necesarios para un sistema capaz de gestionar la variabilidad de Líneas de Productos Educativos.
5. **Diseño** de la arquitectura del servicio *backend*, definiendo los componentes principales y el modelo de datos para representar [FM](#) en el contexto educativo.
6. **Implementación** del servicio *backend* para materializar el sistema que dará respuesta a la problemática planteada, asegurando la creación, análisis, validación y configuración de [FM](#).
7. **Validación** del servicio desarrollado mediante casos de uso representativos en el ámbito educativo, evaluando que cumpla con el funcionamiento esperado.

Para darle solución a los objetivos trazados en las tareas de investigación se emplearon los siguientes **métodos científicos**:

- **Métodos teóricos:**

- **Modelación:** se utilizó para la elaboración de los diagramas del lenguaje de modelado y en la representación de las características de la solución.
- **Histórico-Lógico:** Se empleó para comprender la evolución de las técnicas de gestión de variabilidad en [SPL](#), desde sus orígenes en ingeniería de software hasta su aplicación potencial en dominios educativos, analizando su desarrollo histórico y lógica interna.
- **Analítico-sintético:** Utilizado en el examen crítico de la bibliografía especializada sobre [SPL](#), [FM](#) y sistemas educativos, permitiendo sintetizar un marco conceptual adaptado al dominio educativo.

- **Métodos empíricos:**

- **Observación:** Mediante este método se analizaron herramientas existentes de gestión [SPL](#) (como *pure::variants*, *FeatureIDE*) para identificar limitaciones técnicas y requisitos necesarios para el contexto educativo específico.
- **Entrevista:** Se entrevista especialistas responsables de diseños curriculares que compran aspectos técnicos para identificar los desafíos existentes en la flexibilidad y variabilidad de los currículos.
- **Experimentación:** Mediante la implementación de prototipos funcionales para validar diferentes enfoques arquitectónicos y algoritmos de validación.

Estructura de la investigación:

Capítulo 1: Fundamentación teórica, capítulo encargado de definir los elementos teóricos necesarios para el desarrollo de la investigación y los principales conceptos que se emplearán durante todo el trabajo. Se realiza un análisis de las soluciones similares y se selecciona la metodología de desarrollo de software y las herramientas y tecnologías a utilizar.

Capítulo 2: Características y diseño del sistema, capítulo poseedor de los diferentes requisitos funcionales y no funcionales de la aplicación a desarrollar, así como de la modelación de la solución mediante los artefactos planteados por la metodología de desarrollo a utilizar.

Capítulo 3: Implementación y pruebas, capítulo que contiene los estándares de codificación utilizados en la implementación, así como las diferentes pruebas a las que será sometida la aplicación una vez terminada para verificar su correcto funcionamiento.

Fundamentación teórica

La base teórica funciona como el almacén conceptual sobre el cual se erige todo trabajo investigativo. Este capítulo expone los conceptos asociados al tema, los fundamentos teóricos existentes y el estado del arte que respaldan el estudio, proporcionando un sistema de referencia que orienta la comprensión e interpretación del fenómeno abordado. En tal sentido, se establece el marco conceptual necesario para contextualizar y dar sentido a la propuesta de solución que se desarrolla en los capítulos subsiguientes.

Además, se efectúa una comparación entre soluciones homólogas existentes para determinar características predominantes y conocer las tendencias en este tipo de sistemas. También se realiza un análisis de las diferentes herramientas y tecnologías para dar a conocer el conjunto de utensilios que serán protagonistas en el desarrollo de la propuesta de solución, así como la metodología de desarrollo de software que guiará todo el proceso práctico.

1.1. Conceptos asociados al tema

En este epígrafe se desglosan los conceptos esenciales que convergen en el núcleo del estudio. La clarificación de estos términos no solo delimita el campo de acción, sino que también proporciona el lenguaje común y la base teórica necesaria para el diseño de una solución coherente y efectiva. Esta precisión conceptual resulta fundamental, pues sienta las bases sobre las cuales se articularán posteriormente las decisiones de diseño e implementación de la propuesta.

1.1.1. Ingeniería de Líneas de Productos de Software (SPLE)

La **SPLE** es una estrategia de ingeniería de software que busca producir familias de sistemas de software similares a partir de un conjunto común de activos centrales mediante un proceso gestionado de desarrollo y gestión de la variabilidad. El objetivo principal es lograr la reutilización a gran escala, lo que se traduce en una reducción de costos, menor tiempo de comercialización y mayor calidad de los productos (**sgbuzz_lps**). El concepto de **SPL** surgió a finales de los años 90 como respuesta directa a la crisis de

productividad que enfrentaba la industria del software a pesar de las técnicas de reutilización orientadas a objetos. La **SPL** formaliza una práctica que ya existía intuitivamente en muchas empresas: construir un software base (plataforma) para generar rápidamente múltiples productos similares con variaciones controladas (**academialab_linea_productos**)

Para comprender cabalmente el paradigma de **SPL**, es necesario examinar los conceptos fundamentales que constituyen su base teórica y que permiten articular la gestión de la variabilidad como eje central de la estrategia. Estos conceptos son:

1. **Línea de Productos**: Es la familia completa de sistemas de software que pueden ser generados. Representa el espacio de posibilidades definido por el conjunto de activos.
2. **Activos Centrales (Core Assets)**: Son los componentes fundamentales (código, arquitectura, modelos de diseño, documentación, planes de prueba) diseñados específicamente para ser compartidos y reutilizados en toda la línea. La inversión inicial en estos activos es clave para el éxito de la estrategia.
3. **Variabilidad**: Es la propiedad esencial de una **SPL**. Define la capacidad de la línea para ser adaptada, personalizada o configurada para generar productos específicos que satisfagan las necesidades particulares de un cliente o escenario. La variabilidad es, en esencia, el conjunto de diferencias controladas entre los productos posibles.
4. **Puntos de Variabilidad (Variability Points)**: Son las ubicaciones explícitas dentro de los Activos Centrales donde se deben tomar decisiones para configurar un producto. Estos puntos actúan como "interruptores" o "opciones" que guían el proceso de personalización.

La motivación detrás de la **SPL** es fundamentalmente económica y técnica, pues permite el ahorro de costos, ya que al compartir la gran mayoría de los activos (entre el 70 % y 90 % del sistema), se evita la reimplementación del mismo código y diseño para cada nuevo producto, además, provee de una reducción del tiempo de comercialización (*Time-to-Market*) pues la creación de un nuevo producto se reduce de un ciclo completo de desarrollo a un simple proceso de configuración y ensamblaje de activos preexistentes y permite una mejora de la calidad, pues como los activos centrales, al ser reutilizados y probados en múltiples productos, alcanzan un nivel de madurez y robustez superior al que podría lograrse en un desarrollo de producto único (**ingenieria_software_lps**).

La **SPL** opera a través de dos procesos interdependientes que deben gestionarse de manera continua, siendo el primero de estos la Ingeniería del Dominio (*Domain Engineering*), proceso hacia arriba que se enfoca en la creación y el mantenimiento de los Activos Centrales, cuya tarea principal es el modelado de la variabilidad para comprender y documentar todas las posibles configuraciones que la línea puede soportar y que constituye la base teórica y formal de la línea, y un segundo, la Ingeniería de la Aplicación (*Application Engineering*), el cual es el proceso hacia abajo que utiliza los Activos Centrales para configurar y generar productos específicos, que implica tomar las decisiones de variabilidad definidas en el modelo (seleccionar las opciones) para producir una instancia concreta de la línea de productos. La herramienta principal y estándar de facto para formalizar y gestionar la variabilidad durante la Ingeniería del Dominio es el **FM**.

Este modelo establece el puente formal entre las opciones de la línea de productos y la capacidad de generar una instancia válida, lo cual es el principio que se busca trasladar al diseño curricular.

La búsqueda de la variabilidad curricular ha llevado a la exploración de paradigmas de ingeniería de software. El concepto de **SPL** ofrece un enfoque estructurado y probado para gestionar familias de productos a partir de un conjunto común de activos (**apel2013feature**). Cuando este paradigma se aplica al dominio educativo, surge el concepto de "Líneas de Productos Educativos", donde un tronco curricular común puede derivar en múltiples planes de estudio especializados mediante la gestión explícita de su variabilidad.

Evolución y mantenimiento de líneas de productos

Las **SPL** no son entidades estáticas; evolucionan con el tiempo para adaptarse a nuevos requisitos, tecnologías y mercados. La evolución de **SPL** se refiere al proceso de modificar los activos centrales y los modelos de variabilidad para incorporar cambios, corregir defectos o mejorar la línea (**apel2013feature**).

Los principales desafíos en la evolución de **SPL** incluyen:

- Trazabilidad de cambios: Mantener registro de qué cambios se realizaron, por qué y cuándo, así como su impacto en los productos derivados.
- Impacto en productos existentes: Determinar cómo las modificaciones en la línea afectan a los productos ya desplegados y si es necesario migrarlos.
- Consistencia del modelo: Garantizar que después de los cambios, el **FM** siga siendo consistente y represente fielmente la variabilidad deseada.
- Gestión de versiones: Coordinar diferentes versiones de la línea que puedan coexistir para dar soporte a productos en distintos estados de madurez.

En el dominio educativo, la evolución curricular es una realidad constante: nuevos planes de estudio, actualización de contenidos, cambios en normativas, etc. Un sistema de gestión de variabilidad curricular debe contemplar mecanismos para:

- Registrar el histórico de modificaciones curriculares.
- Evaluar el impacto de los cambios en los planes de estudio existentes.
- Gestionar múltiples versiones de un mismo plan (por ejemplo, planes de estudio con diferentes años de implementación).
- Asegurar la trazabilidad entre las versiones del modelo y los productos generados.

1.1.2. Modelos de Características (*Feature Models*)

Es necesario contar con un modelo que represente de manera explícita las variaciones y opciones disponibles en la línea de productos. Este modelo debe ser capaz de capturar la complejidad de las relaciones entre las características y sus variaciones, así como las restricciones que pueden existir entre ellas

(pohl2005software). La representación gráfica de estas relaciones es fundamental para facilitar la comprensión y el análisis de la variabilidad en la línea de productos.

El Modelo de Características (*Feature Model* - **FM**) es la técnica de modelado central en **SPLE**. Su formalismo es indispensable para pasar de la noción abstracta de variabilidad a una representación estructurada y analizable (**ptc_feature_models**). Los **FMs** fueron formalmente introducidos por **kang1990foda** como parte de la metodología **Análisis de Dominio Orientado a Características (FODA)**, un enfoque pionero para la ingeniería de dominios. Desde su concepción, un **FM** ha sido reconocido como el artefacto primario para la captura de la variabilidad funcional de un sistema. En esencia, un **FM** es un modelo de dominio que no solo documenta las posibles funcionalidades, sino que también define las reglas de composición entre ellas (**gfg_feature_engineering**).

Formalmente, un **FM** se define como un árbol de decisiones jerárquico donde cada nodo es una característica (*feature*). Una característica se entiende como una unidad de funcionalidad, un requisito, o un aspecto distintivo de un sistema, tal como es percibido por el usuario final o el analista del dominio.

La estructura del **FM** es un árbol de composición, donde:

- La característica raíz representa el sistema o la línea de producto completa (ej., la carrera de Ingeniería Informática).
- Las características descendientes detallan la composición de sus características padre hasta alcanzar las características hoja, que son las unidades más pequeñas y configurables.

Relaciones estructurales

La potencia del **FM** reside en su notación gráfica concisa, la cual utiliza la posición y símbolos específicos para codificar las restricciones de configuración. Las relaciones entre una característica padre (letra P) y sus hijos (letra C) definen las decisiones de inclusión o exclusión; véase la tabla 1.1 (**pohl2005software**; **kang1990foda**; **benavides2010PrinciplesFM**; **benavides2010automated**; **Wang2007FormalSemanticsFM**).

Cuadro 1.1. Notación semántica de las relaciones entre características de los **FMs**.

Relación	Semántica Formal	Implicación en la Configuración
Obligatoria	$P \rightarrow C$	Si el padre P es seleccionado, el hijo obligatorio C debe incluirse. No hay variabilidad en este punto
Opcional	$C \rightarrow P$	El hijo C puede ser incluido o excluido, si se selecciona el padre P. Introduce variabilidad

Continúa en la siguiente página

Cuadro 1.1 – Continuación de la página anterior..

Relación	Semántica Formal	Implicación en la Configuración
Agrupación AND	$(P \rightarrow \bigwedge_{i=1}^n C_i) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Si se selecciona la característica padre P, todos los hijos C_i deben incluirse; cada hijo seleccionado implica la presencia del padre. No introduce variabilidad dentro del grupo
Agrupación OR	$(P \rightarrow \bigvee_{i=1}^n C_i) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Al seleccionar el padre P, debe elegirse al menos un hijo C_i . Cada hijo seleccionado requiere que el padre esté presente. Sí introduce variabilidad
Agrupación alternativa	$(P \rightarrow \bigvee_{i=1}^n C_i) \wedge \bigwedge_{i \neq j} \neg(C_i \wedge C_j) \wedge \bigwedge_{i=1}^n (C_i \rightarrow P)$	Cuando el padre P se selecciona, exactamente un hijo C_i puede activarse y la elección de cualquier hijo obliga a seleccionar al padre. Sí introduce variabilidad

Las relaciones descritas en la tabla 1.1 constituyen la base semántica sobre la cual se construye cualquier FM. Sin embargo, para que esta semántica sea útil en la práctica del modelado, es necesario contar con una notación gráfica estandarizada que permita visualizar de forma intuitiva la estructura jerárquica, las restricciones entre características e identificar los puntos de variabilidad. Esta representación visual proporciona una vista de alto nivel que complementa la especificación formal, pues es la encargada de dotar a los FMs de su poder comunicativo y analítico. La figura 1.1 ilustra la simbología convencionalmente aceptada en la literatura para representar las relaciones estructurales de los FMs, donde cada símbolo codifica visualmente una de las relaciones semánticas previamente explicadas.

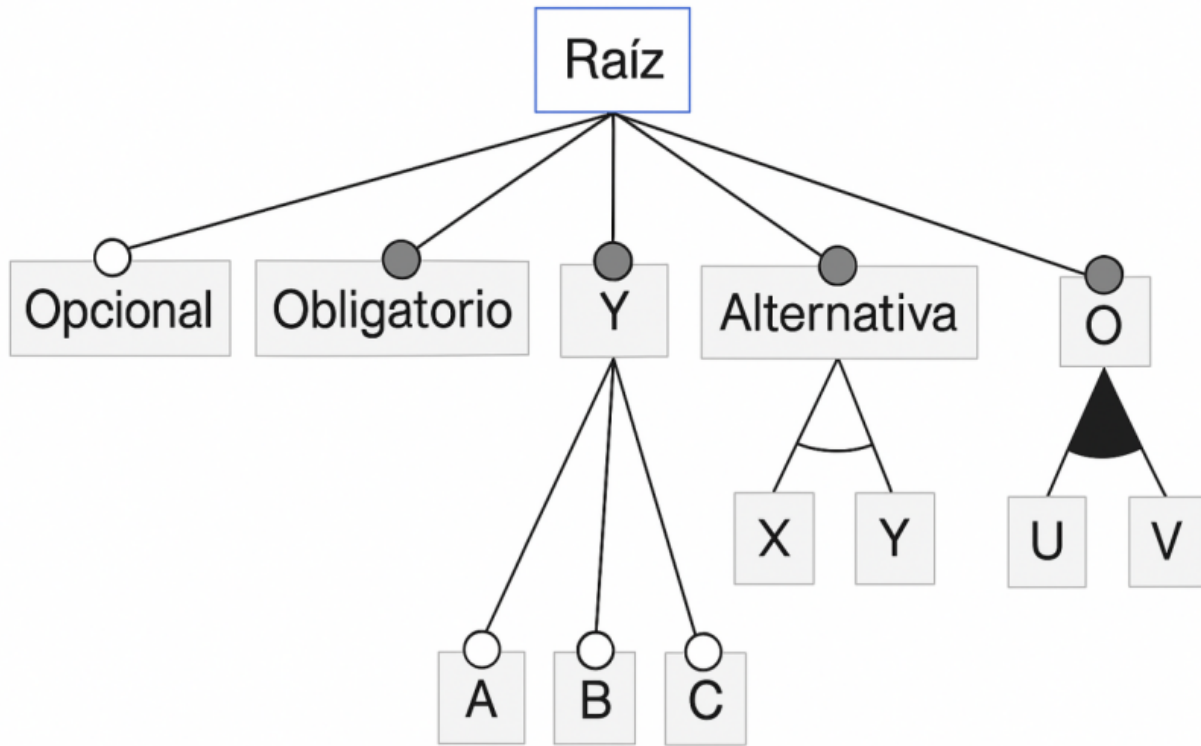


Figura 1.1. Símbolos gráficos para representar las relaciones estructurales.

Restricciones (*Cross-Tree*)

Las relaciones jerárquicas son insuficientes para capturar todas las reglas de negocio complejas. Por ello, los **FM**s permiten definir las restricciones transversales (*cross-tree*); que son reglas lógicas que definen dependencias entre características que no están directamente conectadas en la jerarquía padre-hijo del árbol de características. Su función es garantizar la validez de una configuración al capturar las relaciones complejas del mundo real que la estructura jerárquica por sí sola no puede expresar (**sundermann2023sat**; **benavides2005Taxonomy**).

Las dos formas más comunes y fundamentales son:

- **Requiere:** Esta restricción establece una dependencia de presencia. Si una característica A requiere una característica B, entonces siempre que A sea seleccionada en una configuración, B también debe estarlo. Por ejemplo, en un **FM** para un automóvil, la característica "Navegación" puede requerir la característica "Pantalla Táctil". Esta relación se modela en lógica proposicional como el condicional $A \rightarrow B$ (**sundermann2023sat**; **benavides2005Taxonomy**).
- **Excluye:** Esta restricción establece un conflicto o incompatibilidad. Si una característica C excluye una característica D, entonces ambas no pueden formar parte de la misma configuración. Por ejem-

plo, el motor "Diésel" puede excluir la transmisión eléctrica.^{en} un vehículo híbrido. Su representación lógica es la negación de la conjunción: $\neg(C \wedge D)$ (sundermann2023sat; benavides2005Taxonomy).

Semántica formal y solucionadores SAT (*SAT Solvers*)

La semántica formal de un FM proporciona un significado matemático preciso a todos sus elementos (jerarquía, relaciones y restricciones), lo que permite un análisis automático y libre de ambigüedades.

- Traducción a Lógica Proposicional: La base de la semántica formal es la traducción de todo el FM a una fórmula en lógica proposicional. Cada característica se representa como una variable proposicional (que puede ser Verdadera o Falsa, indicando si está seleccionada o no). Las relaciones padre-hijo y las restricciones *cross-tree* se convierten en una serie de fórmulas lógicas que, en conjunto, definen el conjunto de todas las configuraciones válidas (sundermann2023sat; thum2014featureide).
- Satisfacibilidad (*Satisfacibilidad Booleana (SAT)*) y *Conteo de Modelos (SHARPSAT)*: Una vez el FM está representado como una fórmula lógica F, se pueden realizar diversos análisis:
 - Análisis de SAT: Consiste en verificar si existe al menos una asignación de valores (Verdadero/Falso) a las variables que haga que la fórmula F sea verdadera. Esto responde a la pregunta fundamental de si el FM tiene al menos una configuración válida (sundermann2023sat; benavides2010automated; mendonca2009SAT).
 - Análisis de SHARPSAT: Consiste en el conteo de modelos (*Model Counting*) que satisfacen F. Esto es crucial para determinar el número total de configuraciones válidas que define un FM, una métrica esencial para comprender su variabilidad y complejidad (sundermann2023sat; benavides2010automated; mendonca2009SAT).
- Solucionadores SAT (*SAT solvers*): Un solucionador SAT (*SAT solver*) es una herramienta algorítmica altamente optimizada diseñada para resolver problemas de satisfacibilidad. Dada una fórmula F, el solucionador (*solver*) determina si es satisfacible (SAT) o insatisfacible (UNSAT). Para el análisis de SHARPSAT, se emplean solucionadores especializados de conteo de modelos (*#SAT solvers*) como sharpSAT y Ganak, que han demostrado un rendimiento excelente al calcular el número de configuraciones en modelos industriales (sundermann2023sat; soos2020Ganak).

Operaciones sobre FM

Para que un FM sea útil en un entorno de desarrollo, debe soportar un conjunto de operaciones que permitan manipularlo y consultarlo. Estas operaciones están agrupadas por grupos y son:

- **Operaciones de creación y edición:**
 - Creación del modelo: Inicialización de un FM con su característica raíz.
 - Inserción/eliminación de características: Añadir o remover nodos en la jerarquía.
 - Modificación de relaciones: Cambiar el tipo de relación entre características padre-hijo.

- Gestión de restricciones: Añadir, eliminar o modificar restricciones *cross-tree*.
- **Operaciones de consulta y análisis:**
 - Validación del modelo: Verificar la consistencia global del FM.
 - Cálculo del número de configuraciones: Determinar la cardinalidad del espacio de productos.
 - Extracción de configuraciones: Obtener configuraciones válidas aleatorias o siguiendo criterios específicos.
 - Verificación de restricciones: Comprobar si una configuración candidata cumple todas las reglas.
- **Operaciones de transformación:**
 - Exportación/importación: Serializar el modelo a formatos estándar (UVL, XML, JSON).
 - Refactorización: Reorganizar la estructura del modelo sin alterar su semántica.
 - Especialización: Generar un submáximo modelo a partir de decisiones parciales.

FM con atributos

Los FMs básicos, también conocidos como FMs booleanos, capturan únicamente la presencia o ausencia de características. Sin embargo, muchos dominios requieren modelar información adicional asociada a cada característica, como costos, pesos, tiempos, o en el caso educativo, créditos, horas lectivas o puntuaciones (**benavides2005Taxonomy**).

Los FMs con atributos extienden el formalismo básico permitiendo asociar a cada característica un conjunto de atributos con valores. Estos atributos pueden ser:

- Numéricos: Enteros o reales (ej., 4 créditos, 2.5 horas).
- Booleanos: Valores verdadero/falso.
- Enumerados: Conjuntos finitos de valores (ej., semestre: 1,2,3,4,5).
- Cadenas: Texto descriptivo.

La verdadera utilidad de los atributos emerge cuando se combinan con restricciones que los involucran. Por ejemplo, en un modelo curricular se pueden expresar reglas como:

- La suma de créditos de las asignaturas optativas no debe exceder 30.
- El costo de los materiales de laboratorio debe ser inferior a \$200.
- La carga horaria semanal debe estar entre 20 y 25 horas.

El análisis de FM con atributos requiere técnicas más sofisticadas, como la programación con restricciones (*constraint programming*) o la verificación de modelos (*model checking*), especialmente cuando se buscan configuraciones que optimicen funciones objetivo (**benavides2010automated**). Lenguajes como UVL incorporan soporte nativo para atributos y restricciones sobre ellos, como se ilustra en el ejemplo del Sándwich 1.1.

Análisis Automatizado de Modelos de Características (AAFMM)

La verdadera potencia de los **FM**s reside en la posibilidad de realizar análisis automatizados sobre ellos. El análisis de **FM**s (**Análisis Automatizado de Feature Models (AAFMM)**) es un área de investigación que estudia cómo extraer información útil de un **FM** mediante algoritmos y herramientas automáticas (**benavides2010automated**). Estos análisis permiten responder preguntas fundamentales sobre la línea de productos antes de invertir en su implementación.

benavides2010automated clasifican los análisis en varias categorías:

- **Análisis de propiedades básicas:** Verifican características elementales del modelo:
 - Consistencia (*consistency*): Determina si existe al menos una configuración válida (modelo no vacío).
 - Modelo muerto (*void model*): Detecta si el **FM** carece por completo de configuraciones válidas debido a restricciones contradictorias.
 - Características muertas (*dead features*): Identifica características que no pueden ser seleccionadas en ninguna configuración válida.
 - Características falsas opcionales (*false optional features*): Detecta características declaradas como opcionales pero que, debido a restricciones, resultan obligatorias en todas las configuraciones.
- **Análisis de configuración:** Asisten en el proceso de seleccionar características:
 - Validación de configuraciones: Verifica si una selección parcial o completa de características constituye una configuración válida.
 - Propagación de decisiones: Calcula automáticamente las consecuencias de seleccionar o excluir una característica.
 - Explicación de inconsistencias: Identifica las restricciones responsables de que una configuración sea inválida.
- **Análisis de optimización:** Buscan configuraciones que maximicen o minimicen ciertos criterios:
 - Selección óptima de características: Encuentra configuraciones que optimicen funciones objetivo (costo, rendimiento, calidad).
 - Análisis multi-objetivo: Considera múltiples criterios simultáneamente para encontrar el conjunto de configuraciones Pareto-óptimas.

1.1.3. Lenguaje Universal de Variabilidad (UVL)

La diversidad de notaciones y lenguajes para el modelado de la variabilidad ha sido históricamente un obstáculo para la colaboración científica y la transferencia tecnológica en el área de **SPL**. Ante esta situación, y como resultado de una iniciativa comunitaria denominada MODEVAR, surgió el Lenguaje Universal

de Variabilidad (*Universal Variability Language* - **UVL**), un esfuerzo colaborativo que ha involucrado a investigadoras e investigadores de universidades de España, Alemania, Austria y otros países durante más de seis años (**benavides2025uvl**; **sundermann2021yet**).

El **UVL** se define como un lenguaje abierto e independiente de plataforma, diseñado específicamente para especificar modelos de variabilidad de forma textual. Su principal objetivo es proporcionar una representación común que pueda ser utilizada tanto en el ámbito académico como en la industria, facilitando así el intercambio de modelos, la comparación de resultados y el desarrollo de herramientas interoperables (**universalvariabilitylanguage**). La creación de **UVL** responde a la necesidad de superar la fragmentación provocada por la existencia de múltiples lenguajes propietarios o académicos con capacidades y expresividad heterogéneas.

Aunque **UVL** nace en el contexto del software, su naturaleza genérica lo hace aplicable a cualquier dominio que requiera modelar variabilidad. El laboratorio *Diverso Lab* de la Universidad de Sevilla, liderado por el catedrático David Benavides, ha aplicado **UVL** en dominios tan diversos como las aplicaciones móviles, la automatización del marketing, la personalización de sensores para sistemas de riego, y recientemente, en la exploración de su aplicación al ámbito educativo para el modelado de currículos flexibles (**investigacionus2025**). Esta versatilidad convierte a **UVL** en una opción particularmente adecuada para la presente investigación, donde se utilizará como notación base para representar la variabilidad curricular.

Diseño del lenguaje y niveles de expresividad

UVL especifica modelos de variabilidad con una estructura similar a un árbol para representar la estructura jerárquica de los modelos de variabilidad. El núcleo del lenguaje viene con varios conceptos para especificar restricciones, y son exactamente los mismos que se explican en la tabla 1.1 junto a las limitaciones de árboles cruzados (restricciones *cross-tree*).

El diseño de **UVL** se estructura en torno, además de su núcleo, en un sistema de niveles lingüísticos (*language levels*) que permiten incrementar su expresividad de forma controlada. Su sintaxis básica representa la estructura jerárquica de un **FM** mediante una notación de árbol indentado, similar a la utilizada en lenguajes de programación como Python (**sundermann2023uvlparser**).

El lenguaje contempla los siguientes niveles principales (**universalvariabilitylanguage**):

- **Nivel Booleano (*Boolean Level*)**: Constituye la base del lenguaje e incluye:
 - *Núcleo Booleano (Boolean Core)*: Soporta las relaciones fundamentales (obligatoria, opcional, OR, alternativa) y restricciones transversales proposicionales.
 - *Cardinalidad de Grupo (Group Cardinality)*: Extiende las relaciones padre-hijo permitiendo especificar que se seleccionen entre n y m características hijas.
- **Nivel Aritmético (*Arithmetic Level*)**: Introduce la posibilidad de trabajar con atributos numéricos asociados a las características:

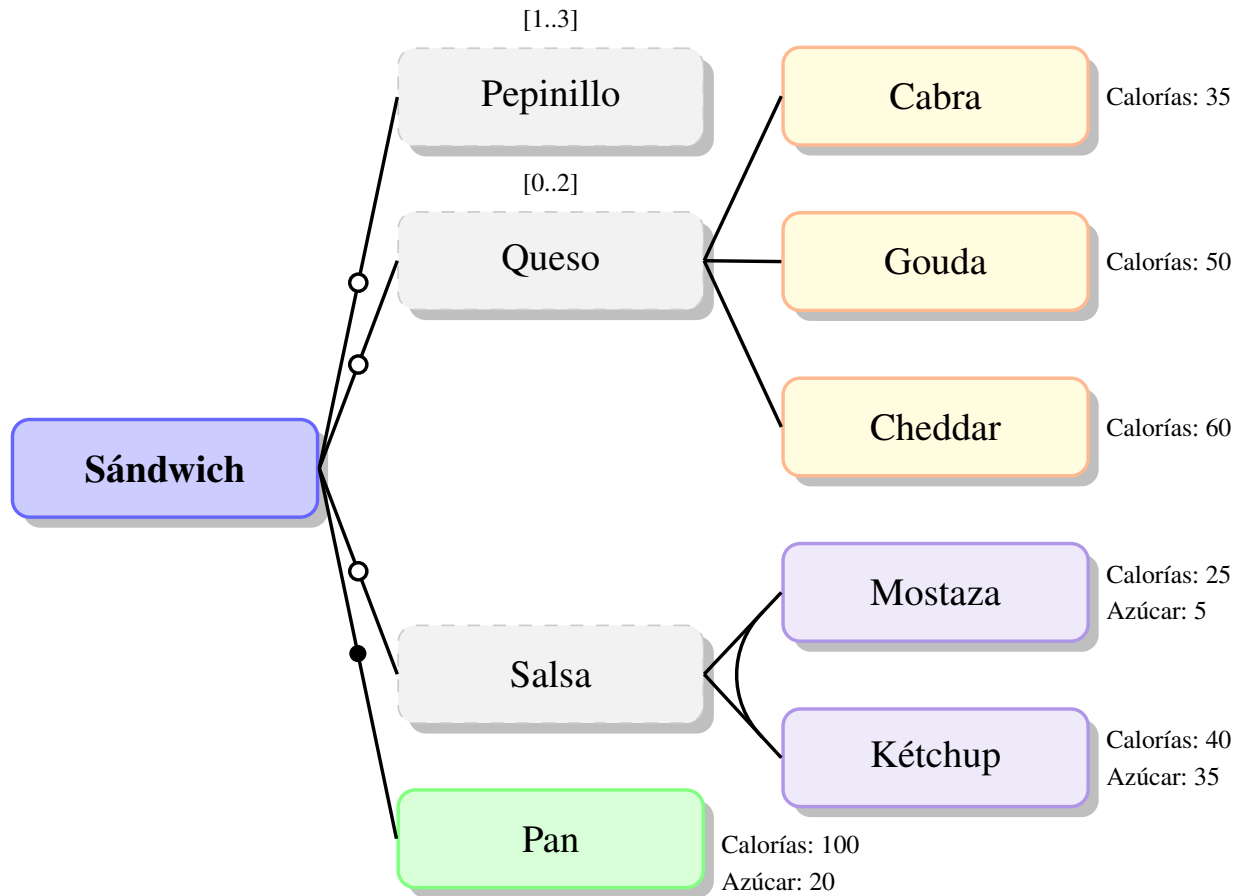
- *Núcleo Aritmético (Arithmetic Core)*: Permite definir restricciones sobre atributos mediante operaciones aritméticas estándar (+, -, *, /, =, !=, >, <).
- *Agregados de Atributos (Attribute aggregates)*: Simplifica la especificación de restricciones globales mediante funciones como `sum()` y `avg()`.
- *Cardinalidad de Característica (Feature Cardinality)*: Permite que una misma característica pueda ser seleccionada múltiples veces dentro de un rango $[n..m]$.
- **Nivel Tipado (Type Level)**: Amplía el lenguaje con tipos de datos más complejos:
 - *Núcleo Tipado (Type Core)*: Introduce características de tipo cadena (*String Features*).
 - *Restricciones Numéricas (Numeric-Constraints)*: Añade operaciones como `floor()` y `ceil()` dentro de las restricciones.
 - *Restricciones de Cadena (String-Constraints)*: Permite operaciones como `len()` y comparación entre cadenas.

Además, **UVL** incorpora un mecanismo de importación que permite referenciar submodelos, facilitando así la composición de modelos de variabilidad de gran tamaño a partir de módulos más pequeños y gestionables (**universalvariabilitylanguage**). El **FM** representado en las figuras 1.1 y 1.2 ejemplifica la sintaxis **UVL** y la correspondencia con su representación gráfica respectivamente.

Código fuente 1.1. Ejemplo de modelo de variabilidad en **UVL**. Fuente **universalvariabilitylanguage**.

```
include
  Boolean.group-cardinality
  Arithmetic.aggregate-function
  Arithmetic.feature-cardinality
  Type
features
  Sandwich
    mandatory
      Pan {Calorias 100, Azucar 20}
    optional
      Salsa
        or
          Ketchup {Calorias 40, Azucar 35}
          Mostaza {Calorias 25, Azucar 5}
      Queso
        [0..2] // Group cardinality
        Cheddar {Calorias 60}
        Gouda {Calorias 50}
        Cabra {Calorias 35}
      Pepinillo cardinality [1..3] // Feature cardinality
constraints
  Ketchup => Queso
  Pan.Azucar + Ketchup.Azucar + Mostaza.Azucar < 60 // Attribute constraints
```

```
sum(Calorias) < 160 // Attribute aggregate
```



ID	Restricciones Cruzadas (UVL / Lógica Proposicional)
R1	Kétchup $\Rightarrow$ Queso
R2	Pan.Azúcar + Kétchup.Azúcar + Mostaza.Azúcar $\leq$ 60
R3	sum(Calorías) < 60

Figura 1.2. Representación gráfica del FM del modelo Sandwich especificado en UVL. Fuente: elaboración propia.

1.1.4. Configuraciones y configurador curricular

En el contexto de las SPL, la configuración se refiere al proceso sistemático de seleccionar un conjunto específico de características de un FM para derivar un producto individual que satisfaga requisitos particulares (**apel2013feature**). Este proceso implica tomar decisiones de configuración que respeten todas las restricciones definidas en el modelo, asegurando que la selección final de características sea válida y consistente (**benavides2010automated**).

La configuración en **SPL** se caracteriza por ser un proceso guiado por restricciones donde las dependencias entre características (mandatorias, opcionales, alternativas) y las restricciones *cross-tree* (requiere, excluye) determinan el espacio de configuraciones posibles (**chen2020variability**). La complejidad de este proceso aumenta exponencialmente con el tamaño del modelo, haciendo necesario el uso de herramientas automatizadas para garantizar la corrección de las configuraciones generadas (**benavides2010automated**).

Un configurador curricular es una herramienta especializada que aplica los principios de configuración de **SPL** al dominio de la planificación educativa, permitiendo la generación semi-automatizada de planes de estudio personalizados a partir de un modelo base que captura la variabilidad curricular (**sanchez2020metodologia**). Este sistema opera como un motor de decisiones académicas que guía al usuario (gestor académico, coordinador de programa) a través del espacio de configuraciones válidas, garantizando que el plan de estudio resultante cumpla con todas las restricciones pedagógicas y administrativas (**chen2020variability**).

El valor fundamental de un configurador curricular reside en su capacidad para reducir la complejidad del diseño curricular mediante la aplicación de restricciones formales que previenen configuraciones inválidas (**rodriguez2019gestion_curricular**). Por ejemplo, el sistema puede asegurar automáticamente que:

- Se cumplan todos los prerrequisitos entre asignaturas y temáticas de las mismas.
- No existan asignaturas y/o temáticas que no esten relacionadas con otras o que sí lo estén, pero debido a las restricciones y reglas, es imposible seleccionar o incluir en cualquier configuración válida del producto.
- Los límites de créditos por semestre se respeten.
- La coherencia entre las competencias a desarrollar y las asignaturas seleccionadas se mantengan.
- Se eviten conflictos de horario y sobrecarga académica.
- No existan asignaturas o temáticas incompatibles en el mismo plan de estudio.
- Se garantice la inclusión de asignaturas obligatorias y la correcta selección de asignaturas optativas según las reglas definidas en el modelo curricular.
- Se cumplan las restricciones de diversidad temática y metodológica para asegurar un plan de estudio equilibrado y completo.

1.1.5. Gestión y flexibilidad curricular. Línea de Productos Educativos

La gestión curricular, como se expuso al inicio de este capítulo, constituye el conjunto de procesos mediante los cuales las instituciones educativas diseñan, implementan, evalúan y actualizan sus planes de estudio. Tradicionalmente, esta gestión se ha apoyado en modelos rígidos y documentos estáticos que dificultan la adaptación a contextos cambiantes (**gomez2023**). Sin embargo, las demandas del siglo XXI —caracterizadas por la aceleración tecnológica, la diversidad de perfiles estudiantiles y la necesidad de formación continua— exigen un enfoque radicalmente distinto: la flexibilización curricular.

La flexibilidad curricular puede entenderse como la capacidad de un plan de estudios para adaptarse a las necesidades, intereses y contextos de los estudiantes o bien por necesidades circunstanciales,

así como a las demandas del entorno social y productivo, sin perder coherencia ni calidad académica (**diaz2002flexibilidad**). Esta flexibilidad opera en múltiples dimensiones:

- Flexibilidad en el tiempo: Posibilidad de cursar asignaturas en diferentes momentos, ritmos y secuencias.
- Flexibilidad en el espacio: Modalidades presencial, híbrida o virtual.
- Flexibilidad en los contenidos: Selección de asignaturas optativas, énfasis, menciones o itinerarios formativos.
- Flexibilidad metodológica: Diversidad de enfoques pedagógicos (aprendizaje basado en proyectos, aula invertida, gamificación).
- Flexibilidad en la evaluación: Adaptación de los mecanismos de acreditación a las características del estudiante y del contexto.

Lograr esta flexibilidad de manera sistemática y sostenible requiere superar las prácticas artesanales basadas en documentos y hojas de cálculo. Es aquí donde los paradigmas de la ingeniería de software ofrecen un camino prometedor.

Hacia las Líneas de Productos Educativos

La aplicación del paradigma de **SPL** al dominio educativo da lugar al concepto de Líneas de Productos Educativos. Esta puede definirse como una familia de artefactos educativos (planes de estudio, cursos, materiales didácticos, actividades de aprendizaje) que comparten un núcleo común y que pueden ser configurados sistemáticamente para generar productos adaptados a contextos, perfiles o necesidades específicas (**okada2009methodology**; **chimalakonda2012accelerating**).

La idea fundamental consiste en trasladar los principios de la **SPL** —gestión de variabilidad, reutilización planificada, activos centrales, puntos de variación— al diseño y gestión curricular. Así, un tronco curricular común (asignaturas obligatorias, competencias básicas, estructura general) constituye los activos centrales de la línea, mientras que los itinerarios formativos, las asignaturas optativas, las menciones o las modalidades de impartición se modelan como puntos de variabilidad (**silva2017towards**).

Investigaciones pioneras en este campo han demostrado la viabilidad y los beneficios de este enfoque:

- **okada2009methodology** propusieron una metodología para mejorar la reutilización de materiales educativos mediante **SPL**. Su trabajo demostró que la separación de partes comunes y variables en los materiales docentes permite generar variantes adaptadas a diferentes escenarios de uso, resolviendo los problemas de consistencia que surgen cuando los educadores gestionan múltiples variantes de forma artesanal.
- **chimalakonda2012accelerating** exploraron la aplicación de **SPL** en tecnologías educativas para abordar la diversidad lingüística y contextual de la India. Su investigación evidenció que el desarrollo de soluciones educativas para 22 idiomas y múltiples contextos socioculturales puede beneficiarse enor-

mamente de un enfoque sistemático de reutilización, reduciendo el esfuerzo de desarrollo de años-persona a procesos de configuración guiados.

- **silva2017towards** diseñaron una arquitectura de línea de productos para construir aplicaciones de aprendizaje móvil (m-learning) orientadas a la enseñanza de programación. Su trabajo identificó las características comunes y variables en este dominio y demostró cómo el enfoque **SPL** puede facilitar la creación de herramientas educativas adaptadas a diferentes estilos de aprendizaje y contextos institucionales.
- **solispino2022software** aplicaron **SPL** como herramienta educativa para el aprendizaje de programación de robots industriales con Arduino. Su estudio con estudiantes de primer año universitario mostró que el enfoque permitió reutilizar entre el 38 % y el 43 % del código total necesario para programar los robots, mejorando significativamente el tiempo de desarrollo y facilitando el aprendizaje de conceptos de abstracción y modularidad.
- **galan2014building** refactorizaron una aplicación de e-learning (E-Learning Web Miner) como una línea de productos, demostrando que el paradigma **SPL** puede aliviar los costosos y propensos a errores procesos de adaptación manual que caracterizan a muchas plataformas educativas cuando deben ajustarse a diferentes contextos de enseñanza-aprendizaje.

1.2. Fortalezas de los **FM** aplicado en la gestión curricular

La verdadera potencia del **FM** radica en que, una vez formalizado, se convierte en un modelo lógico-matemático que define explícitamente el espacio de diseño o el conjunto de configuraciones válidas (2^n) que se pueden generar (donde n es el número de características). Esta formalidad permite realizar análisis automatizados esenciales mediante herramientas conocidas como solucionadores de satisfacibilidad (*SAT Solvers*) o restricciones, permitiendo la validación de consistencia, que se encarga de determinar si un **FM** es consistente, es decir, si existe al menos una configuración válida; la Identificación de características muertas, lo cual permite encontrar las características que nunca pueden ser seleccionadas debido a la severidad de las restricciones y posibilitan una configuración guiada, garantizándose que el producto final (ej. el plan de estudios) sea siempre válido.

Aunque existen otros modelos de variabilidad (como las tablas de decisión o los diagramas UML de variabilidad), el **FM** es el formalismo preferido por su simplicidad visual y su potencia analítica. A diferencia de modelos como las tablas, que pueden volverse inmanejables al aumentar el número de características, el **FM** provee una visión de alto nivel y, simultáneamente, una base lógica binaria robusta para el análisis automatizado de la consistencia.

En los próximos epígrafes se profundiza en esta línea argumental desde dos perspectivas complementarias. En primer lugar, se establece una analogía formal curricular que traduce los principales conceptos de la **SPL** al dominio educativo, demostrando mediante un ejemplo (figuras 1.3 y 1.4) concreto —como el modelado de la asignatura Estructura de Datos I— cómo los **FMs** pueden capturar la variabilidad intrínseca

de los planes de estudio. Posteriormente, se sintetizan los beneficios de las **SPL**, destacando las ventajas que este enfoque aporta a la gestión curricular en términos de reutilización, consistencia, trazabilidad y exploración del espacio de configuraciones.

1.2.1. Analogía formal curricular

La aplicación de los **FM**s a la gestión y diseño curricular constituye el principal aporte metodológico de esta investigación. Esta extensión se fundamenta en la premisa de que la variabilidad, concepto central de la **SPLE** es intrínseca y necesaria en los planes de estudio modernos de la enseñanza superior.

Para fundamentar la propuesta, es imprescindible establecer una analogía (tabla 1.2) formal y coherente que permita sustituir los constructos de **SPL** por sus equivalentes en el dominio educativo, elevando el plan de estudios de un simple documento descriptivo a un activo configurable y analizable.

Cuadro 1.2. Analogía de conceptos de **SPL** en el Dominio Educativo. Fuente: Elaboración propia.

Concepto de SPL	Concepto análogo en planificación curricular	Justificación
Línea de Productos	Carrera o área de estudio	El tronco común de conocimientos que define una familia de perfiles profesionales
Producto Específico	Plan de estudio configuracional	La ruta académica única y válida para un estudiante o perfil de egreso definido
Característica	Unidad curricular(asignatura, módulo, tema de estudio)	La unidad funcional mínima que puede ser seleccionada u omitida
Característica Raíz	El currículo base (ej. la carrera Ingeniería en Ciencias Informáticas o en caso que se esté modelando una asignatura, el nombre de la misma)	La entidad que engloba todos los módulos y asignaturas posibles de la carrera
Variabilidad	Opciones académicas	El conjunto de decisiones disponibles (optatividad, electividad, elección de menciones) que permiten personalizar la ruta del estudiante

Yendo un paso más adelante para evidenciar la aplicabilidad del **FM** al dominio curricular, las figuras 1.3 y 1.4 muestran un ejemplo concreto de modelado de la variabilidad curricular basado en la asignatura Estructura de Datos I. Se expone el código 1.2 **UVL** y su esquema jerárquico conceptual con el fin ilustrativo y suficiente para comprender las decisiones de variabilidad del **FM**.

Código fuente 1.2. Ejemplo de modelo de variabilidad en **UVL** para la asignatura Estructura de Datos I.

```
namespace EstructuraDatos
include
```

```
Boolean.group-cardinality
Arithmetic.aggregate-function
Arithmetic.feature-cardinality
Type
features
  Estructura de Datos I
    mandatory
      Fundamentos y Análisis {Horas 6}

      Estructuras Lineales
        cardinality [1..*]
          mandatory
            Arreglos
            Listas Enlazadas
          optional
            Listas Circulares

      Pilas {Horas 6}
        mandatory
          Implementacion
            alternative
              Con Arreglos {Horas 4, Dif 1}
              Con Listas {Horas 6, Dif 2}

      Colas {Horas 8}
        mandatory
          FIFO {Dif 2}
          Prioridad {Dif 3}
        optional
          Deque {Dif 2}

      Árboles Binarios {Horas 12, Dif 4}

      Algoritmos Ordenamiento {Horas 10}

      Lenguaje
        alternative
          C/C++ {Carga 9}
          Java {Carga 6}
          Python {Carga 4}

      Técnicas de Programación
        mandatory
          Recursión Avanzada {Carga 8}
          Generica {Carga 10}
          Manejo de Memoria {Carga 8}
```



```
Herramientas
    or
        IDE {Carga 5}
        Debugger {Carga 2}
        Git {Carga 4}

Análisis de Complejidad
    mandatory
        Notación Big-O
        Análisis Temporal
        Análisis Espacial
constraints
    // R1: Prerrequisito pedagógico
    Árboles Binarios => Recursión Avanzada

    // R2: Dependencia técnica de bajo nivel
    C/C++ <=> Manejo de Memoria

    // R3: Restricción por abstracción del lenguaje
    Python => !Manejo de Memoria

    // R4: Restricción de recursos (Suma de atributos)
    sum(Horas) <= 64

    // R5: Restricción de viabilidad (Cálculo entre atributos)
    (avg(Dif) * Lenguaje.Carga) < 50
```

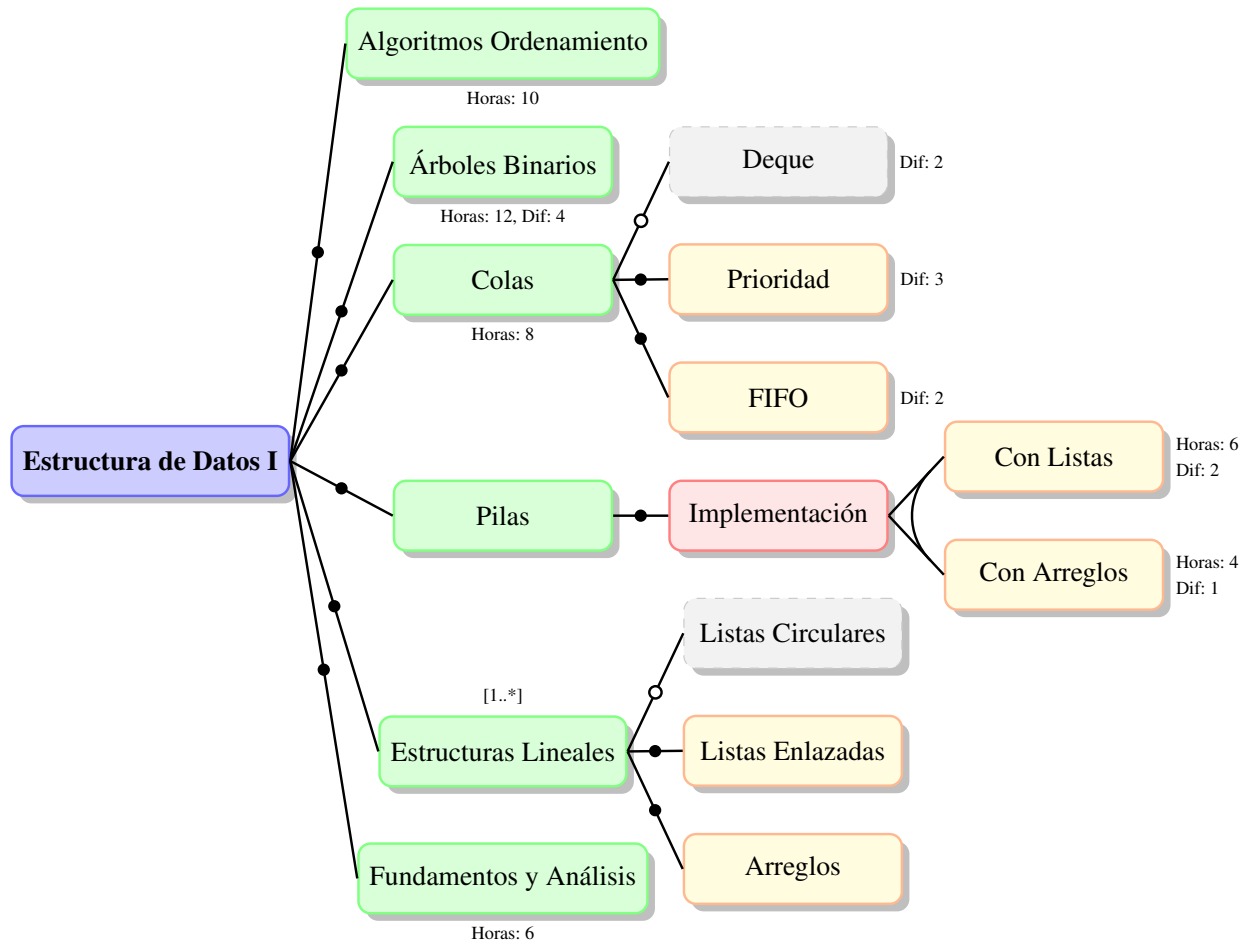
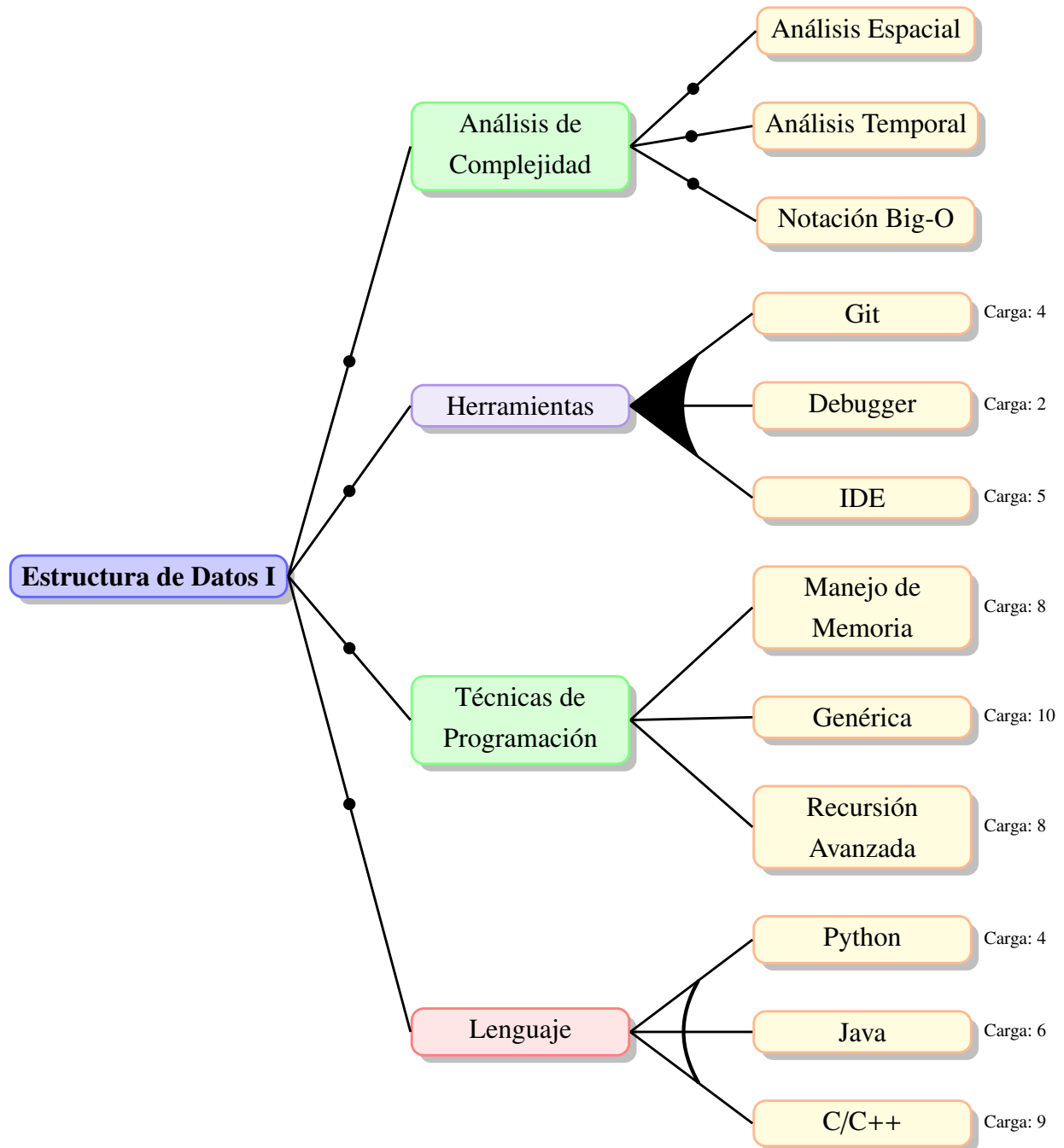


Figura 1.3. Representación simplificada #1 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.



ID	Restricciones Cruzadas (UVL / Lógica Proposicional)
R1	Árboles Binarios $\Rightarrow$ Recursión Avanzada
R2	C/C++ $\Leftrightarrow$ Manejo de Memoria
R3	Python $\Rightarrow \neg$ Manejo de Memoria
R4	$\text{sum}(\text{Horas}) \leq 64$ (Límite semestral)
R5	$\text{avg}(\text{Dificultad}) * \text{Lenguaje.Carga} < 50$ (Umbral de viabilidad)

Figura 1.4. Representación simplificada #2 del FM de la asignatura Estructura de Datos I. Fuente: elaboración propia.

Este caso permite demostrar cómo una asignatura, lejos de ser un listado estático de contenidos, posee una estructura jerárquica, elementos obligatorios y opcionales, variabilidad controlada, dependencias de prerrequisitos y restricciones internas; todas ellas propiedades capturables dentro de un FM formal. El modelo mostrado (figuras 1.3 y 1.4) constituye una representación declarativa de la asignatura, estructurada como un árbol donde la característica raíz corresponde al curso completo y las características hijas representan módulos, temas o decisiones didácticas. Esta representación no solo captura los contenidos, sino que los organiza con semántica formal, permitiendo análisis automatizable. Los ejemplos ilustra distintos tipos de variabilidad presentes en currículos reales: contenidos obligatorios (módulos troncales), opcionales (tópicos avanzados), alternativas mutuamente excluyentes (implementación de pilas), configuraciones tipo OR (técnicas complementarias) y variantes dependientes (manejo de memoria solo si la implementación se realiza en C/C++).

La estructura del FM muestra claramente cómo una asignatura puede contener subniveles jerárquicos complejos, donde cada nivel constituye un subconjunto configurable. Esto permite modelar asignaturas que se imparten según distintas líneas, enfoques o lenguajes de implementación (C/C++, Java, Python), conservando la validez curricular y evitando inconsistencias.

La variabilidad no solo define qué temas se abordan, sino cómo se implementan, qué carga cognitiva poseen y qué prerrequisitos deben cumplirse. La presencia de relaciones cruzadas de árboles, restricciones lógicas e implicaciones capturan dependencias reales, tales como: para comprender árboles es necesario estudiar recursión (relación R1), o bien optar por C/C++ implica la posibilidad de manejo manual de memoria (relación R2). Esta lógica curricular, que en los planes tradicionales queda implícita o depende de interpretación docente, aquí se vuelve explícita, formal y verificable. Además, la inclusión de atributos numéricos (Horas, Dificultad, Carga) permite que el FM funcione como un sistema de soporte a la decisión. Restricciones como R4 y R5 demuestran cómo el formalismo puede garantizar que una configuración curricular no solo sea lógicamente válida, sino también factible en términos de carga cognitiva para el estudiante y disponibilidad horaria del programa.

De este modo, la asignatura deja de ser un documento textual estático para convertirse en un componente configurable. El FM presenta la asignatura como un micro-espacio de variabilidad curricular, análogo a un “producto configurable” dentro de una línea de productos académicos. Diferentes configuraciones del FM representan distintas adaptaciones válidas de la asignatura según perfiles de estudiante, competencias, niveles de profundidad o enfoques pedagógicos. Este modelo puede procesarse mediante un motor lógico para verificar su consistencia, detectar características inaccesibles y generar configuraciones válidas automáticamente.

Por tanto, los FMs no solo son adecuados para representar planes de estudios completos, sino también para modelar unidades académicas individuales con variabilidad interna. Esto abre la puerta a una flexibilidad curricular controlada, verificable y sustentada en formalismos computacionales, alineándose con los principios de SPLE aplicados a la planificación educativa.

1.2.2. Beneficios de las Líneas de Productos Educativos

En una Línea de Productos Educativos, la variabilidad curricular se convierte en el objeto central de modelado y gestión. Retomando los conceptos de [FM](#) y [UVL](#) desarrollados en secciones anteriores, es posible representar explícitamente:

- Características obligatorias: Asignaturas troncales, competencias básicas, prácticas profesionales obligatorias.
- Características opcionales: Asignaturas optativas, seminarios complementarios, actividades extracurriculares.
- Relaciones de alternancia: Menciones o itinerarios excluyentes entre sí (ej., mención en Ingeniería de Software vs. mención en Inteligencia Artificial).
- Agrupaciones OR: Conjuntos de asignaturas entre las cuales debe elegirse un mínimo (ej., elegir al menos 3 de 5 asignaturas de un bloque).
- Restricciones (*cross-tree*): Prerrequisitos entre asignaturas, incompatibilidades, correlatividades, límites de créditos.
- Atributos: Créditos, horas lectivas, semestre recomendado, área de conocimiento, modalidad de impartición.

La adopción del enfoque [SPL](#) en la gestión curricular reporta beneficios análogos a los observados en el desarrollo de software ([apel2013feature](#)). De los cuales se tiene:

- Reutilización sistemática: Los diseños curriculares, una vez validados, pueden ser reutilizados para generar nuevos programas académicos sin partir desde cero.
- Consistencia y trazabilidad: La representación formal de la variabilidad garantiza que todos los planes de estudio derivados respeten las mismas reglas y restricciones, facilitando la trazabilidad de los cambios.
- Reducción de errores: Al automatizar la validación de configuraciones, se eliminan los errores humanos derivados de la gestión manual (inconsistencias, omisiones, conflictos).
- Agilidad en la actualización: Las modificaciones al modelo central se propagan automáticamente a todos los productos derivados, manteniendo la coherencia institucional.
- Exploración del espacio de configuraciones: Herramientas de análisis automatizado pueden responder preguntas como “¿cuántos planes de estudio válidos pueden generarse?” o “¿qué asignaturas quedan sin posibilidad de ser cursadas?”.

1.3. Análisis del mercado

El presente epígrafe desarrolla un análisis sistemático del mercado relacionado con los sistemas de gestión curricular y herramientas de modelado de variabilidad. Se examinan tanto las soluciones tradicionales

empleadas por instituciones académicas como aquellas provenientes del ámbito de la [SPLE](#), identificando sus fortalezas, limitaciones y brechas funcionales. A partir de esta revisión, se fundamenta la oportunidad tecnológica que sustenta la propuesta investigativa, destacando el vacío existente entre las plataformas de administración académica y los mecanismos de planificación curricular basados en variabilidad, lo que justifica la necesidad y pertinencia de la solución desarrollada.

1.3.1. Panorama general del mercado

El mercado de sistemas de gestión curricular presenta una segmentación clara entre soluciones genéricas de administración académica y herramientas especializadas en modelado de variabilidad. Mientras plataformas como Moodle, Blackboard y Canvas dominan el segmento de los [Sistema de Gestión del Aprendizaje \(LMS\)](#) ([moodle2025overview](#); [blackboard2025overview](#); [canvas2025overview](#)), existen vacíos significativos en soluciones específicas para la planificación curricular basada en Líneas de Productos Educativos ([sanchez2020metodologia](#)).

1.3.2. Análisis de soluciones existentes

El análisis de soluciones existentes se organiza en dos grupos claramente diferenciados, cada uno representativo de perspectivas y alcances funcionales opuestos dentro del dominio investigado. Por un lado, se examinan los sistemas institucionales de gestión curricular tradicional, ampliamente implementados en universidades, cuyo propósito principal es administrar información académica pero que presentan limitaciones para modelar y validar variabilidad curricular. Por otro lado, se estudian herramientas avanzadas de [SPLE](#), diseñadas para gestionar características y restricciones complejas, pero que requieren conocimientos técnicos elevados y carecen de adaptación al contexto educativo. Esta comparación permite identificar brechas tecnológicas y conceptuales que fundamentan la necesidad de una solución especializada que integre fortalezas de ambos mundos mientras atiende las demandas propias de la planificación curricular.

Sistemas de gestión curricular tradicionales

- SIGA y Banner: Los sistemas como Banner son fundamentalmente sistemas de información transaccionales diseñados para la gestión operativa de una institución. Su arquitectura está optimizada para manejar grandes volúmenes de datos de estudiantes, matrículas, registros de cursos y pagos. La evidencia de su funcionamiento, como se observa en la documentación de "Banner Student", muestra una estructura basada en bloques, ventanas y secciones para la gestión de admisiones, currículos y campos de estudio ([ellucian2015b_banner](#)). Esta lógica está orientada a capturar y recuperar información de forma consistente, no a modelar relaciones complejas y variables entre componentes curriculares. Carecen de un motor de reglas nativo que permite definir y verificar restricciones complejas como las que se modelan con "requiere" o "excluye" en un [FM](#). Su limitación principal es que operan sobre una estructura de datos fija y predefinida, lo que los hace rígidos para experimentar con diferentes

configuraciones curriculares o para gestionar una línea de productos educativos donde las relaciones entre asignaturas y competencias son dinámicas y variables.

- Moodle: como [LMS](#), es una plataforma excelente para la ejecución y gestión de un plan de estudios ya definido. Su fortaleza reside en organizar cursos, desglosarlos en unidades y lecciones, gestionar la entrega de contenidos, las actividades de evaluación y la interacción entre estudiantes y profesores ([moodle2025overview](#)). Sin embargo, su funcionalidad no se extiende al diseño instruccional ni a la planificación curricular basada en características. Moodle asume que el diseño del curso (los objetivos, la secuencia de contenidos, las actividades de evaluación) ya ha sido realizado externamente. No proporciona herramientas para que los gestores académicos modelen visualmente un plan de estudios, definan características opcionales u obligatorias, o establezcan restricciones entre ellas. En el contexto de la investigación, Moodle sería un consumidor potencial de la configuración curricular final generada por la propuesta de solución, pudiéndose integrar como plataforma de entrega y gestión de los cursos derivados, pero no como herramienta para diseñar o validar la estructura curricular en sí.

Herramientas de modelado de variabilidad

- Pure::variants: es una solución comercial robusta para [SPLE](#). Proporciona un entorno completo para definir características, crear modelos, gestionar restricciones y, crucialmente, derivar productos concretos a partir de configuraciones válidas. Su potencia es innegable en el dominio técnico para el que fue diseñado ([purevariants2024overview](#)). El problema central para el contexto educativo es su alta barrera de entrada para usuarios no técnicos. Los gestores académicos y diseñadores curriculares, que son expertos en pedagogía pero no necesariamente en [SPLE](#), se verían enfrentados a una terminología y flujos de trabajo propios de la ingeniería de software. Adaptar pure::variants requeriría una personalización significativa de la interfaz y una capacitación intensiva, lo que dificulta su adopción ágil en un entorno académico, validando la necesidad de una solución más especializada y accesible.
- FeatureIDE: es un entorno de código abierto construido como un complemento (*plugin*) para Eclipse, lo que lo hace muy popular en entornos de investigación y enseñanza de la ingeniería de software ([thum2014featureide](#)). Al igual que pure::variants, es poderoso para el análisis y la configuración de [FMs](#). Sin embargo, su dependencia de Eclipse y su orientación hacia desarrolladores lo convierten en una herramienta que requiere especialización técnica. Para un coordinador de carrera o un vicerrector académico, interactuar con FeatureIDE supondría una curva de aprendizaje muy pronunciada. Su arquitectura no está diseñada para integrarse de manera natural con los sistemas de información universitarios existentes (como los propios SIGA o Banner), lo que limita su utilidad práctica para la gestión curricular del día a día. Su uso en educación se limita predominantemente a la enseñanza de conceptos de [SPL](#), no a la gestión operativa de currículos.
- Flamapy / FlamapyIDE: es un ecosistema de código abierto para el análisis automatizado de [FMs](#), desarrollado activamente por el grupo de investigación Diverso Lab de la Universidad de Sevilla, liderado por el catedrático David Benavides ([galindo2023flama](#)). A diferencia de pure::variants y FeatureI-

DE, Flamapy se concibe desde su origen como un “*framework* Python” modular y extensible, diseñado para facilitar la integración de capacidades de análisis de variabilidad en otras aplicaciones, lo que lo emparenta directamente con el objetivo de la presente investigación. El *framework* soporta múltiples formatos de entrada, incluyendo UVL, FeatureIDE (.fide) y otros ([flamapyfeatureidereader2025](#)), y proporciona transformaciones a representaciones intermedias como [Diagrama de Decisión Binaria \(BDD\)](#) para análisis eficientes ([flamapyfm2bdd2025](#)).

Flamapy ofrece tres interfaces principales que lo convierten en una propuesta particularmente relevante ([flamapyrest2024](#)):

- FlamapyIDE: un entorno de edición y análisis basado en web, accesible en <https://ide.flamapy.org/editor>, que permite cargar modelos UVL o FeatureIDE, visualizar su estructura, y ejecutar operaciones de análisis (validación de consistencia, detección de características muertas, conteo de configuraciones, etc.) sin necesidad de instalar software local. La herramienta permite crear configuraciones manualmente o mediante un enfoque interactivo, en el cual le permite al usuario ir seleccionando características del árbol, primeramente, desde el nivel uno hasta la hoja; estas configuraciones son validadas y pueden descargarse. Esta interfaz gráfica reduce significativamente la barrera de entrada para usuarios no técnicos.
- [Interfaz de Programación de Aplicaciones \(API\)](#) de [Transferencia de Estado Representacional \(REST\)](#): Flamapy proporciona una API RESTful que expone sus capacidades de análisis como servicios web, permitiendo que otras aplicaciones (como el *backend* propuesto en esta investigación) integren funcionalidades avanzadas de validación y configuración de modelos de características mediante llamadas de [Protocolo de Transferencia de Hipertexto \(HTTP\)](#) ([flamapyrest2024](#)). La API puede desplegarse localmente mediante Docker o accederse a versiones alojadas, facilitando su adopción en arquitecturas de microservicios.
- Línea de comandos y biblioteca Python: para integraciones más profundas, Flamapy ofrece una interfaz de línea de comandos y una biblioteca Python que permite incorporar directamente sus capacidades en aplicaciones desarrolladas en este lenguaje ([flamapydocs2025](#)).

1.3.3. Oportunidades de mercado identificadas

Del análisis realizado se evidencia que el mercado asociado a la temática padece de un vacío crítico en herramientas capaces de integrar la gestión de variabilidad, propia de las [SPL](#), con la planificación curricular educativa, creando una oportunidad para sistemas especializados ([sanchez2020metodologia](#)). Las herramientas existentes de [SPL](#) están diseñadas para ingenieros de software, no para profesionales de la educación ni mucho menos para la gestión curricular, limitando su adopción en instituciones educativas y obligando a comprender los conceptos fundamentales de [SPLE](#) ([chen2020variability](#)). En el caso de Flamapy, destacar que no permite gestionar y operar el FM desde la vista gráfica (visualización del árbol), sino mediante la escritura de UVL. En este escenario, la necesidad de proponer una solución propia no surge de la ausencia total de tecnologías, sino de la falta de una alternativa que articule las fortalezas identificadas en

ambos tipos de herramientas y las adapte al dominio educativo.

De los sistemas analizados se reconoce como indispensable la capacidad de representar explícitamente características con relaciones jerárquicas y restricciones, la validación automática de configuraciones, la posibilidad de derivar múltiples variantes a partir de un modelo común y la interoperabilidad con ecosistemas académicos existentes. Flamapy, en particular, se destaca su funcionamiento en capturar y modelar los FMs a partir de escribir código en UVL, su amplio abanico de exportación a diferentes formatos de archivos y la capacidad de diagnosticar los FMs (funcionalidad no ausente en los otros 2 sistemas analizados) en búsquedas de estadísticas, como cantidad de configuraciones, características muertas, etc. La relevancia de Flamapy para la presente investigación es múltiple:

- Su naturaleza como *framework* Python facilita la integración con la propuesta solución, permitiendo reutilizar sus capacidades de análisis en lugar de implementarlas desde cero.
- La existencia de una API REST validada y documentada proporciona un referente arquitectónico para el diseño del servicio.
- Su enfoque en accesibilidad (interfaz web, API) lo alinea con la necesidad de acercar las técnicas de SPL a gestores académicos no especializados.
- El equipo de desarrollo (Diverso Lab) es el mismo que lidera el estándar UVL, garantizando coherencia con las notaciones adoptadas en esta investigación.

La solución propuesta se posiciona estratégicamente en el mercado educativo como un “Motor de Variabilidad Curricular” especializado, que capitaliza las tendencias actuales de crecimiento en educación híbrida y multimodal, así como las demandas crecientes de personalización educativa y eficiencia en la gestión académica post-pandemia. Su propuesta de valor se sustenta en fortalezas técnicas diferenciadoras, ofreciendo un servidor de aplicaciones (*backend*) que permita diseñar y gestionar modelos de características aplicados a planes de estudio, y derivar itinerarios formativos dinámicos y personalizables. Su valor diferencial radica en su especialización en gestión de variabilidad curricular, incorporar motores de reglas pedagógicas que comprenden prerequisites, competencias y secuencias, una arquitectura de API para integración con sistemas existentes y un modelo de datos optimizado para planificación académica. Estas capacidades se traducen en ventajas competitivas tangibles como la automatización de validaciones que reduce errores en diseño curricular, la generación de múltiples variantes de planes de estudio desde un tronco común y su adaptabilidad a contextos institucionales diversos, posicionándose como complemento esencial -no competencia- de los sistemas LMS existentes, al enfocarse específicamente en el diseño y planificación académica más que en la ejecución curricular. En consecuencia, la propuesta investigativa no se limita a replicar herramientas existentes, sino que se posiciona como un puente necesario entre el rigor formal de las SPL y la flexibilidad curricular demandada por la educación contemporánea, constituyendo así una innovación con pertinencia tecnológica y académico-formativa.

1.4. Herramientas y tecnologías

La implementación de la solución propuesta requiere la selección cuidadosa de tecnologías, herramientas y *frameworks* que permitan materializar de manera eficiente los modelos curriculares, soportar reglas de variabilidad y garantizar una experiencia de desarrollo fluida. En este epígrafe se describen los componentes tecnológicos que conformarán la plataforma, justificando su elección. Asimismo, se analizan los entornos de desarrollo, lenguajes de programación, sistemas de persistencia y bibliotecas especializadas que facilitan la representación formal de restricciones curriculares y la validación automática de configuraciones. Esta caracterización permite comprender la base tecnológica que sustenta la solución y cómo cada herramienta contribuye al logro de sus objetivos funcionales y no funcionales.

1.4.1. Lenguaje de programación

Los lenguajes de programación son herramientas formales que permiten expresar instrucciones de manera estructurada para que un sistema computacional ejecute tareas determinadas (**sebesta2016**). A través de ellos se implementan algoritmos, modelos de datos, reglas de negocio y mecanismos de interacción con el usuario u otros sistemas. La elección de un lenguaje influye directamente en la escalabilidad, el rendimiento, la mantenibilidad y la capacidad de expresar soluciones complejas de manera eficiente. En el contexto del sistema propuesto, el lenguaje seleccionado debe ofrecer flexibilidad, capacidad de representación estructural y soporte robusto para lógica y concurrencia (**sommerville2016se**).

Entre los lenguajes de programación más utilizados para el desarrollo de aplicaciones del lado del servidor se encuentran Java, Python, C#, JavaScript\TypeScript (Node.js), Go, PHP, entre otros. Python será el lenguaje principal utilizado para el desarrollo del *backend* de la solución propuesta. Su elección se fundamenta en su madurez tecnológica, su amplio ecosistema de bibliotecas y la claridad sintáctica que facilita la implementación de modelos complejos manteniendo niveles altos de legibilidad y mantenibilidad del código (**van1995python**; **lutz2013learning**). Python es ampliamente utilizado en el ámbito académico y profesional, lo que garantiza disponibilidad de documentación, ejemplos de buenas prácticas y una sólida comunidad de soporte (**ray2014large**).

Una de las principales fortalezas de Python radica en su capacidad de integración con múltiples paradigmas de desarrollo, permitiendo tanto programación estructurada como orientada a objetos y funcional (**lutz2013learning**). Esto resulta particularmente útil para el diseño modular del sistema, la definición formal de **FM** y la implementación de validaciones de reglas en motores lógicos.

La representación estructurada de **FMs** requiere un lenguaje capaz de expresar jerarquías, relaciones lógicas, restricciones transversales y cardinalidades con claridad y sin ambigüedad. Python ofrece capacidades idóneas para este propósito debido a su flexibilidad en estructuras de datos y su riqueza semántica para definir invariantes y reglas de negocio. La cantidad de líneas de código que se requiere para operar los **FMs** en Python es drásticamente menor que en comparación con otros lenguajes como pueden ser Java y C#, lo que mejora la experiencia de desarrollo.

En primer lugar, Python permite modelar **FM**s de forma natural mediante diccionarios, listas y clases, manteniendo la semántica jerárquica del dominio sin necesidad de sintaxis complejas ni compilación intermedia. Esto facilita representar nodos, ramas, grupos OR/XOR, atributos y relaciones *cross-tree* como estructuras anidadas. Gracias a esta expresividad, resulta posible describir distintos niveles del modelo —desde características obligatorias y opcionales hasta restricciones de cardinalidad— de manera directa y legible, lo que favorece tanto su mantenimiento como su inspección. Adicionalmente, esta flexibilidad estructural de Python facilita la implementación de mecanismos de serialización y deserialización que permiten la interoperabilidad con formatos estándar de la comunidad de **SPL**, como **UVL** ([sundermann2021uvl](#)). **UVL** se ha propuesto como un formato independiente del lenguaje para representar modelos de variabilidad, y Python, gracias a su soporte para procesamiento de texto, análisis sintáctico y generación de estructuras de datos, permite construir traductores bidireccionales que convierten modelos **UVL** en representaciones internas del sistema y viceversa, asegurando la portabilidad de los modelos entre distintas herramientas y entornos de desarrollo.

Además, Python ofrece un ecosistema amplio de herramientas para la validación programática de configuraciones. El sistema puede implementar algoritmos de selección y verificación que comprueben automáticamente consistencia lógica, satisfacibilidad de prerrequisitos, exclusiones y cumplimiento de restricciones. La sintaxis de Python resulta especialmente adecuada para traducir expresiones lógicas como requerimientos, exclusiones o implicaciones, ya sea mediante evaluaciones directas o a través de módulos que permiten construir árboles lógicos, satisfacibilidad booleana o incluso resolución por *backtracking*.

Adicionalmente, la flexibilidad del lenguaje, junto con el respaldo de su amplio ecosistema de bibliotecas, facilita la construcción de algoritmos que no solo derivan configuraciones de forma eficiente, sino que también las verifican mediante evaluaciones lógicas, análisis de dependencias y resolución de conflictos. Este mismo soporte permite implementar mecanismos de versionado y *snapshots* del modelo, conservando estados históricos y garantizando que la evolución del **FM** no invalide configuraciones previas, preservando así la consistencia y trazabilidad del sistema en el tiempo.

En conjunto, estas características hacen de Python un lenguaje altamente apropiado tanto para la representación formal de **FM**s como para la generación y validación automatizada de configuraciones dentro del sistema propuesto ([ray2014large](#)). Por su equilibrio entre simplicidad expresiva y potencia funcional, Python constituye una base tecnológica idónea para el *backend* del sistema, permitiendo acelerar el desarrollo sin sacrificar calidad, extensibilidad ni alineación con los requisitos funcionales de la solución.

1.4.2. *Framework* de desarrollo

Los *frameworks* de desarrollo son estructuras software que proporcionan un conjunto de herramientas, patrones, módulos y convenciones que simplifican la construcción de aplicaciones ([sommerville2016se](#)). Permiten acelerar la implementación de funcionalidades complejas, garantizando buenas prácticas de diseño y facilitando la mantenibilidad del software ([pressman2014se](#)). Un *framework* adecuado reduce la repetición de código, mejora la seguridad, permite la modularización y posibilita el desarrollo escalable. En

este proyecto, el *framework* seleccionado debe soportar concurrencia, validación, comunicación con bases de datos y un desarrollo claro basado en APIs ([gamma1995design](#)).

Algunos de los *frameworks backend* con los que cuenta Python son Django\Django REST Framework (DRF), FastAPI, AIOHTTP, Flask, Tornado, entre otros no tanto conocidos o aclamados por la comunidad. Se evaluaron (tabla 1.3) estas opciones considerando criterios como rendimiento, facilidad de desarrollo, soporte para tipado, validación automática y capacidades asíncronas.

Cuadro 1.3. Comparación entre *frameworks* de Python. Fuente: elaboración propia.

Framework	Tipo	Rendimiento	Curva de aprendizaje	Ideal para
Django	Full-stack	Medio	Media	Sistemas completos
DRF	Full-stack API	Medio	Media	APIs sobre Django
FastAPI	Asíncrono	Alto	Baja	APIs modernas
Flask	Micro	Medio	Muy baja	Prototipos, servicios
Tornado	Asíncrono	Alto	Alta	Tiempo real
AIOHTTP	Asíncrono	Alto	Media	Clientes/servidores asíncronos

De la tabla 1.3 se puede concluir que, si bien DRF ofrece una solución robusta y madura para construir APIs sobre el ecosistema Django, su naturaleza síncrona y su curva de configuración pueden representar limitaciones para escenarios que requieren alta concurrencia y validación estricta de FMs. Por su parte, Tornado y AIOHTTP proporcionan excelente rendimiento asíncrono, pero demandan un desarrollo más manual y complejo. Flask, aunque sencillo y flexible, carece de tipado fuerte y validación automática, aspectos cruciales para garantizar la consistencia de los FMs y las configuraciones derivadas. En contraste, FastAPI fue seleccionado como el *framework* principal para el desarrollo del *backend* debido a su combinación equilibrada de alto rendimiento asíncrono, sintaxis intuitiva, simplicidad de desarrollo y capacidades avanzadas de validación mediante Pydantic ([ramirez2021fastapi](#)).

Un factor determinante en la selección de FastAPI es su rendimiento comparable a implementaciones escritas en Go y Node.js, gracias a su construcción sobre Starlette (para la capa asíncrona) y Pydantic (para validación y serialización) ([pesce2021benchmark](#)). Esta combinación permite manejar múltiples peticiones concurrentes, realizar verificaciones de reglas e interacciones dinámicas con el motor de configuraciones sin degradación perceptible del servicio, incluso bajo cargas elevadas.

Un argumento especialmente relevante para seleccionar FastAPI es la naturaleza intensiva en validaciones y operaciones Entrada/Salida (I/O) que caracteriza al sistema propuesto. La plataforma no solo debe recibir configuraciones y verificar reglas, sino también consultar persistencia, aplicar evaluaciones lógicas, generar respuestas alternativas y registrar versiones de modelo, todo ello de manera concurrente y potencialmente bajo múltiples usuarios. Debido a su arquitectura asíncrona nativa basada en `async/await`, FastAPI permite manejar este patrón de carga sin bloqueo, maximizando la eficiencia en tareas I/O y reduciendo latencias ([eugene2023fastapi](#); [pesce2021benchmark](#)). Esto se traduce en una capacidad real de escalar, sin

incurrir en cuellos de botella característicos de *frameworks* síncronos, lo que resulta esencial en un sistema donde las consultas y validaciones son frecuentes, complejas y potencialmente concurrentes.

FastAPI aprovecha las características más recientes de Python (*type hints*) para ofrecer validación automática de datos, documentación interactiva generada dinámicamente (OpenAPI/Swagger) y soporte nativo para concurrencia, lo que resulta particularmente ventajoso para implementar operaciones de validación y derivación de configuraciones en FM ([fastapi2023docs](#)). Esta capacidad de tipado fuerte permite mapear directamente las estructuras de los FMs —como características, restricciones y relaciones— a modelos Pydantic, asegurando que los datos procesados cumplan consistentemente con las reglas del dominio. Además, su enfoque modular y su integración fluida con bases de datos relacionales y no relacionales mediante bibliotecas como SQLAlchemy y Motor lo convierten en una opción versátil y preparada para escalar junto con los requisitos del sistema ([lubanovic2019python](#)). Por estas razones, FastAPI constituye la base tecnológica idónea para implementar la propuesta de solución, alineado con los objetivos funcionales y no funcionales del sistema.

1.4.3. Base de datos y gestor de base de datos

Las bases de datos son sistemas diseñados para almacenar, organizar y proteger información de manera eficiente y estructurada, garantizando su disponibilidad a lo largo del tiempo ([sciencedirect_dbms](#)). La persistencia de los modelos curriculares, configuraciones generadas, versiones históricas y metadatos asociados requiere de un sistema robusto, seguro y con garantías de consistencia transaccional.

Entre los sistemas de bases de datos más utilizados se encuentran MySQL, PostgreSQL, MongoDB y SQL Server. Se selecciona PostgreSQL como gestor de base de datos relacional, debido a su madurez, fiabilidad y su capacidad de manejar estructuras complejas con fuertes requisitos de integridad referencial ([postgres2024](#); [momjian2001postgresql](#)). PostgreSQL destaca por su cumplimiento estricto del estándar Lenguaje de Consulta Estructurado (SQL), su modelo Atomicidad, Consistencia, Aislamiento, Durabilidad (ACID), y su eficiente gestión de transacciones, elementos imprescindibles para representar relaciones jerárquicas entre características, restricciones *cross-tree*, y *snapshots* versionados del modelo sin comprometer la coherencia global del sistema ([momjian2001postgresql](#)).

Además, PostgreSQL ofrece capacidades avanzadas como índices compuestos, consultas optimizadas, vistas materializadas, *triggers* y procedimientos almacenados, que resultan útiles para procesar datos curriculares con alto grado de estructuración ([postgres2024](#)). El motor también soporta tipos de datos Notación de Objetos JavaScript (JSON) y JSON Binario (JSONB), permitiendo almacenar representaciones semiestructuradas de configuraciones y estados transitorios, por lo que proporciona flexibilidad para almacenar tanto esquemas estáticos como *snapshots* dinámicos relacionados con la evolución del FM ([postgres2024](#)).

Adicionalmente, PostgreSQL se integra de forma natural con SQLAlchemy como Mapeo Objeto - Relacional (ORM) dentro del *backend* desarrollado en FastAPI, facilitando el diseño declarativo de entidades, la gestión del ciclo de persistencia, la validación de relaciones y la migración segura del esquema. Esta integración favorece una arquitectura limpia y escalable, desacoplando las capas de datos, servicios y lógica

de negocio.

Para la administración visual del motor se empleará pgAdmin, interfaz ampliamente utilizada para la gestión de bases de datos PostgreSQL. Mediante esta herramienta es posible inspeccionar tablas, ejecutar consultas SQL, visualizar índices y restricciones, supervisar cargas de trabajo y realizar respaldos de forma controlada. En el contexto de la solución propuesta, pgAdmin permite auditar el estado de los modelos almacenados, verificar reglas impuestas, revisar configuraciones generadas y garantizar la trazabilidad del sistema.

En conjunto, la elección de PostgreSQL como sistema de persistencia y pgAdmin como herramienta administrativa proporciona una base confiable y alineada con las necesidades de integridad, consistencia y trazabilidad del sistema a desarrollar.

1.4.4. Contenerización y orquestación. Docker y Kubernetes

La contenerización es una tecnología que permite empaquetar aplicaciones junto con todas sus dependencias en unidades aisladas, reproducibles y portables llamadas contenedores. Esto garantiza que una aplicación se ejecute de la misma manera en cualquier entorno, eliminando problemas de incompatibilidad. Para ello se emplean herramientas especializadas, donde Docker se ha consolidado como estándar industrial. Por su parte, la orquestación permite administrar múltiples contenedores, asegurando coordinación, escalabilidad y resiliencia; funciones que normalmente se alcanzan mediante tecnologías como Docker Compose o Kubernetes. Este enfoque es crucial para despliegues confiables, mantenibles y escalables.

Para garantizar la portabilidad, reproducibilidad y facilidad de despliegue del sistema propuesto, se adoptará Docker como tecnología base de contenedorización (**boettiger2015introduction; docker2024docs**). Docker permite encapsular los servicios de la aplicación —incluyendo el *backend* FastAPI, la base de datos PostgreSQL, la herramienta administrativa pgAdmin y otros servicios de utilidad o futuros módulos asociados— dentro de entornos aislados y autocontenidos. Esto elimina problemas de dependencias, divergencias de entorno y configuraciones inconsistentes entre despliegues locales y productivos (**boettiger2015introduction**). La capacidad de reproducir una imagen de forma determinística contribuye directamente a la robustez y mantenibilidad de la solución.

La contenedorización resulta especialmente útil para aplicaciones multicapa como la propuesta, donde distintos servicios deben colaborar manteniendo canales de comunicación estables. Docker permite garantizar uniformidad tanto en las dependencias del *backend*, como versiones de bibliotecas, configuración del servidor de aplicación y parámetros del motor de base de datos, disminuyendo errores derivados de diferencias de entorno y facilitando diagnósticos (**docker2024docs**).

Sobre Docker se construye Docker Compose, herramienta de orquestación ligera que permite describir, mediante un único archivo declarativo, toda la topología de servicios del sistema (**docker2024docs**). Esto incluye la configuración de redes internas, gestión de persistencia en volúmenes, variables de entorno, reglas de dependencias entre contenedores, puntos de exposición de puertos, así como políticas de reinicio. En el contexto del sistema de modelado curricular, Docker Compose facilita levantar el entorno completo de de-

sarrollo o pruebas con un solo comando, garantizando reproducibilidad y reduciendo tiempos de despliegue ([boettiger2015introduction](#)).

Cabe precisar que Docker Compose se empleará exclusivamente durante la fase de desarrollo y para simular un entorno de producción local, permitiendo validar el comportamiento integral del sistema antes de su despliegue final. Sin embargo, para el despliegue en un entorno productivo real —donde se requiere alta disponibilidad, escalabilidad automática y tolerancia a fallos— se utilizará Kubernetes, la plataforma de orquestación de contenedores más extendida en la industria ([kubernetes2025docs](#)).

Kubernetes es un sistema de código abierto para la automatización del despliegue, escalado y gestión de aplicaciones contenedorizadas. Originalmente desarrollado por Google, actualmente es mantenido por la Cloud Native Computing Foundation (CNCF) ([kubernetes2025overview](#)). A diferencia de Docker Compose, que opera sobre un único nodo, Kubernetes gestiona clústeres completos de máquinas, distribuyendo los contenedores entre múltiples servidores y garantizando que la aplicación se mantenga operativa incluso si alguno de ellos falla.

La adopción de Kubernetes para el despliegue productivo del servicio backend aporta múltiples ventajas:

- **Escalado automático:** Kubernetes permite aumentar o reducir el número de réplicas de los contenedores en función de la demanda (uso de CPU, memoria o métricas personalizadas). Esto resulta especialmente valioso para el sistema de modelado curricular, donde la carga de trabajo puede incrementarse significativamente durante períodos de planificación académica intensiva.
- **Alta disponibilidad y tolerancia a fallos:** El orquestador monitorea continuamente el estado de los contenedores y reinicia automáticamente aquellos que fallen. Asimismo, puede redistribuir las réplicas entre nodos sanos en caso de caída de un servidor, minimizando el tiempo de inactividad del servicio.
- **Balanceo de carga interno:** Kubernetes expone los contenedores como servicios con una dirección [Protocolo de Internet \(IP\)](#) virtual y distribuye el tráfico entrante entre todas las réplicas disponibles, evitando la sobrecarga de una instancia específica.
- **Actualizaciones continuas sin tiempo de inactividad:** Mediante estrategias como *rolling updates*, Kubernetes puede actualizar la versión de los contenedores sin interrumpir el servicio, reemplazando gradualmente las réplicas antiguas por las nuevas mientras el sistema sigue respondiendo peticiones.
- **Gestión declarativa de la configuración:** Al igual que Docker Compose, Kubernetes utiliza archivos de configuración declarativos (YAML) que describen el estado deseado del sistema. El orquestador se encarga de reconciliar el estado actual con el deseado, simplificando la administración y auditoría de los despliegues.
- **Portabilidad entre infraestructuras:** Kubernetes está soportado por los principales proveedores de nube (AWS, Azure, Google Cloud) y también puede desplegarse sobre [Servidores Privados Virtuales \(VPS\)](#) convencionales mediante distribuciones ligeras como K3s o MicroK8s. Esto permite ejecutar el sistema tanto en nubes públicas como en infraestructura propia o alquilada sin modificar la configuración de orquestación.

Un beneficio adicional de esta estrategia es su contribución a las prácticas de [Integración Continua y](#)

Despliegue Continuo (CI/CD). Gracias a la naturaleza declarativa y reproducible de los contenedores, el sistema puede desplegarse automáticamente mediante *pipelines* controlados sin intervención manual, asegurando que las versiones del modelo curricular, las configuraciones generadas y los *snapshots* almacenados sean accesibles en entornos iguales o equivalentes entre sí (**docker2024docs**).

1.4.5. Almacenamiento en memoria, caché y mensajería

El almacenamiento en memoria es una técnica que permite guardar temporalmente datos en la **Memoria de Acceso Aleatorio (RAM)** para acelerar el acceso a información que se consulta de forma recurrente, evitando operaciones de disco o consultas de base de datos innecesarias. El *caching* se basa precisamente en optimizar velocidad, reduciendo latencias y mejorando el rendimiento global del sistema. Existen herramientas especializadas que permiten gestionar este almacenamiento temporal, soportar estructuras clave-valor y facilitar comunicación entre servicios. Redis destaca entre ellas por su eficiencia, simplicidad y capacidades adicionales como mensajería, *pub/sub* y persistencia configurable, lo que la convierte en una pieza clave para arquitecturas modernas.

Redis será incorporado como componente clave para el almacenamiento temporal y procesamiento eficiente de información crítica durante las fases de generación y validación de configuraciones curriculares (**carlson2013redis**). Su naturaleza de base de datos en memoria permite alcanzar tiempos de respuesta extremadamente bajos, lo cual resulta particularmente valioso en un sistema donde las operaciones de verificación de restricciones, derivación de configuraciones válidas y resolución de incompatibilidades pueden ejecutarse de forma intensiva y repetitiva.

A diferencia de un motor relacional tradicional, Redis está optimizado para operaciones clave-valor y estructuras avanzadas como listas, conjuntos, mapas *hash* y *sorted sets* (**carlson2013redis**). Estas estructuras permiten modelar transitoriamente elementos como:

- Configuraciones candidatas generadas por el usuario.
- Listas de incompatibilidades detectadas.
- Descriptores internos del **FM**.
- Estados de sesión durante la exploración de alternativas.
- Cachés de resultados de validaciones repetitivas.

Este almacenamiento en memoria reduce significativamente la latencia, lo que contribuye a experiencias interactivas y a la capacidad del sistema de responder en tiempo real a decisiones del usuario (por ejemplo, activar o desactivar características, consultar prerequisites, etc.) (**carlson2013redis**).

Otro aspecto clave es que Redis permite persistencia opcional mediante *snapshots*, lo que lo convierte en un aliado directo del soporte de versionado de modelos y configuraciones (**carlson2013redis**). Es decir, puede almacenar estados intermedios del proceso de modelado curricular sin interferir con la base de datos principal, permitiendo restaurar fácilmente versiones anteriores o explorar ramas alternativas del mismo modelo.

Además, Redis implementa un sistema de Time-To-Live (**Tiempo de Vida (TTL)**), lo que permite gestionar la caducidad automática de configuraciones temporales, sesiones y cálculos ya procesados. Esta funcionalidad contribuye a la integridad global del sistema y evita acumulación de datos residual (**carlson2013redis**).

Finalmente, una característica determinante para su elección es su excelente integración con FastAPI, permitiendo aprovechar plenamente la naturaleza asíncrona del *backend* (**ramirez2019fastapi**). Gracias a bibliotecas como **aioredis**, las operaciones de lectura/escritura se realizan sin bloquear el `event loop`, lo que maximiza la concurrencia cuando múltiples usuarios exploran configuraciones simultáneamente.

1.4.6. Almacenamiento de archivos y objetos

El almacenamiento persistente de archivos, versiones históricas de modelos y resultados derivados de las evaluaciones requiere una tecnología que ofrezca acceso distribuido, tolerancia a fallos, escalabilidad y compatibilidad con estándares industriales. Contar con una herramienta que permita aislar objetos de la base de datos es sumamente importante para evitar sobrecargar el sistema de gestión de datos relacional con información que no requiere consultas estructuradas, sino almacenamiento eficiente y recuperación rápida, por lo que PostgreSQL se encargaría de almacenar la dirección **Localizador Uniforme de Recursos (URL)** del objeto y sus metadatos; y en otro sistema se almacenaría el objeto como tal. Para satisfacer esta necesidad se incorporará MinIO, un sistema de almacenamiento de objetos compatible con la **API** de Amazon **Simple Storage Service (S3)**, ampliamente adoptado en entornos de producción y servicios en la nube (**minio2024docs**).

MinIO ofrece una interfaz uniforme para almacenar objetos binarios, documentos, *snapshots* de modelos y resultados asociados al proceso de validación. Su compatibilidad con **S3** permite integrar la plataforma con herramientas estándar sin requerir dependencias propietarias (**adler2023evaluating**). Esta característica asegura interoperabilidad con sistemas externos, posibilitando que módulos de análisis, auditoría o visualización operen sobre un *backend* de almacenamiento robusto y escalable.

En el contexto de la solución propuesta, MinIO será empleado para almacenar:

- *Snapshots* persistentes de versiones del **FM**.
- Configuraciones válidas generadas por los motores **SAT/Satisfacibilidad Módulo Teorías (SMT)**.
- Artefactos derivados del análisis estructural.
- Archivos de auditoría e informes.
- Registros de pruebas y diagnósticos.

A diferencia del almacenamiento en memoria provisto por Redis, MinIO garantiza persistencia a largo plazo, redundancia y recuperación confiable de información. Su arquitectura distribuida permite configurar réplicas, dispersión de datos y mecanismos de resiliencia sin sacrificar rendimiento (**adler2023evaluating**). Además, sus políticas de control de acceso simplifican la gestión de permisos, evitando inconsistencias o pérdida de trazabilidad a lo largo del ciclo de vida del currículo.

Desde el punto de vista operativo, MinIO permite integrarse con Docker y Docker Compose de manera directa, facilitando el despliegue, escalado y administración sin requerir configuración adicional. Esto

habilita una solución completa de almacenamiento distribuido que puede operar tanto en entornos locales (*on-premise*) como en infraestructuras institucionales o nube privada.

1.4.7. Procesamiento asíncrono

El procesamiento asíncrono permite ejecutar tareas en segundo plano sin bloquear la operación principal del sistema, lo cual es esencial cuando existen tareas costosas, como validaciones complejas, consultas externas, cálculos intensivos o envío de notificaciones. En el ecosistema de Python existen herramientas especializadas para administrar colas de trabajo, distribuir tareas y realizar ejecución diferida. Entre ellas, Celery se ha posicionado como la solución estándar para orquestar tareas asíncronas, ofreciendo escalabilidad, distribución y tolerancia a fallos. Su integración con brokers como Redis permite construir sistemas altamente eficientes, modulares y no bloqueantes.

En la arquitectura propuesta, Celery desempeñará un rol esencial como orquestador de tareas asíncronas, permitiendo que operaciones complejas, lentas o intensivas en cómputo se ejecuten fuera del ciclo de solicitud-respuesta del [API](#), mejorando sustancialmente la experiencia del usuario y la disponibilidad del sistema ([celery2025docs](#)).

Dado que el proceso de generación y validación de configuraciones curriculares incluye fases donde se deben:

- Recorrer árboles completos del [FM](#).
- Validar múltiples restricciones cruzadas.
- Propagar efectos lógicos por dependencias.
- Explorar configuraciones alternativas válidas.
- Verificar versiones y retrocompatibilidad.

Resulta natural que estas tareas no se ejecuten directamente en una petición [HTTP](#), ya que podrían bloquear el servidor o provocar tiempos de respuesta superiores a lo aceptable. Celery permite delegar estos procesos a “*workers*” paralelos que pueden ejecutarse en segundo plano, liberando a FastAPI para seguir atendiendo solicitudes sin congestión.

La integración de Celery con Redis (como *message broker*) permite que las tareas se publiquen, distribuyan, encolen, ejecuten y documenten de forma confiable. Asimismo, Redis funge como *backend* de resultados, permitiendo al sistema:

- Consultar el estado de una ejecución.
- Recuperar resultados parciales o finales.
- Cancelar o reprogramar tareas.
- Mostrar progresos al usuario.

Otra ventaja crítica es que Celery soporta escalamiento horizontal, lo cual es relevante para el sistema, en el cual múltiples usuarios pueden realizar evaluaciones simultáneamente sobre modelos distintos o versiones

diferentes. Con más carga, simplemente se añaden más *workers*, sin necesidad de cambios estructurales (**celery2025docs**). Desde el punto de vista de robustez, Celery ofrece herramientas para:

- Reintentos automáticos ante fallos.
- Límite de concurrencia.
- Priorización de colas.
- Periodicidad (con Celery Beat) para ejecutar tareas programadas.

Esto permite automatizar procesos como:

- Generación periódica de *snapshots* de [FM](#).
- Validaciones nocturnas de consistencia.
- Limpieza de configuraciones expiradas.
- Copias de seguridad de estados intermedios.

Celery es altamente compatible con FastAPI y Python, manteniendo coherencia tecnológica con la decisión de usar un *stack* asíncrono y un modelo de validación compleja (**celery2025docs**; **ramirez2019fastapi**). Su adopción garantiza que la capa lógica pesada no degrade la utilidad interactiva del sistema, sino que se ejecute de forma paralela, programada y resistente. La tabla 1.4 respalda la justificación de por qué adicionar celery al sistema.

Cuadro 1.4. Tabla Comparativa: FastAPI vs FastAPI + Celery. Fuente: Elaboración propia.

Criterio	Solo FastAPI	FastAPI + Celery
Manejo de operaciones pesadas	Bloquea el servidor si la tarea tarda mucho	Delegación a <i>workers</i> externos sin bloqueo
Experiencia del usuario	Tiempos de respuesta largos para validaciones	Respuestas inmediatas + seguimiento del progreso
Escalabilidad	Limitada al número de <i>workers</i> del servidor	Escalamiento horizontal agregando <i>workers</i> Celery
Naturaleza de carga	Orientada a I/O y sincronización	Orientada a cómputo intensivo y tareas diferidas
Validación de modelos complejos	Puede agotar recursos del servidor	Optimización mediante <i>workers</i> paralelos
Generación de <i>snapshots</i> o versiones	Bajo rendimiento si se ejecuta en el <i>request</i>	Tareas programadas y automáticas (Celery Beat)
Robustez ante fallos	Fallos interrumpen solicitudes activas	Reintentos automáticos e independientes
Concurrencia	Limitada	Altamente concurrente
Distribución de carga	Manual, difícil de gestionar	Balance natural mediante colas y <i>brokers</i>

Continúa en la siguiente página

Cuadro 1.4 – Continuación de la página anterior.

Criterio	Solo FastAPI	FastAPI + Celery
Estado de procesos largos	No persistente, no rastreable	Persistente, rastreable, recuperable
Integración con Redis	Opcional	Ideal, Redis como <i>broker</i> y <i>backend</i>
Mantenimiento	Monolítico	Arquitectura desacoplada y modular

Esta comparación evidencia que mientras FastAPI permite construir servicios web altamente eficientes para operaciones sin bloqueo y validaciones ligeras, la incorporación de Celery habilita un procesamiento distribuido ideal para la naturaleza intensiva de la manipulación y validación de **FM**s, generando configuraciones, *snapshots* y cálculos alternativos sin afectar la disponibilidad del sistema.

1.4.8. Algoritmos, motor de reglas y validación

Para garantizar la consistencia, completitud y validez de las configuraciones generadas a partir del **FM**, la plataforma integrará un conjunto de algoritmos especializados, combinando enfoques deterministas y heurísticos según la naturaleza del problema a resolver.

En primer lugar, se utilizará SymPy como motor base para la representación simbólica y evaluación lógica de restricciones asociadas al **FM** (**meurer2017sympy**). SymPy permite manipular expresiones booleanas, aplicar simplificaciones, detectar contradicciones y verificar satisfacibilidad, lo que resulta esencial para la evaluación de relaciones *cross-tree* como *requires*, *excludes*, *implies* y restricciones de cardinalidad. Su uso asegura una verificación formal, basada en lógica proposicional y álgebra simbólica, manteniendo precisión y trazabilidad en los análisis (**meurer2017sympy**).

De manera complementaria, la plataforma empleará NetworkX para modelar y procesar las dependencias entre características como grafos dirigidos, permitiendo ejecutar algoritmos de recorrido, detección de ciclos, análisis de conectividad, propagación de condiciones y determinación de componentes fuertemente conexas (**hagberg2008networkx**). Este enfoque orientado a grafos habilita la inspección estructural del **FM**, la detección temprana de inconsistencias y la aplicación eficiente de validaciones transversales.

En escenarios donde la validación determinista no sea suficiente para explorar el espacio de configuraciones posibles —especialmente cuando existan alternativas múltiples y restricciones complejas— se utilizarán técnicas heurísticas como algoritmos genéticos. Estos permiten buscar soluciones aproximadas optimizando criterios como completitud, compatibilidad y minimización de violaciones. Su empleo resulta valioso para sugerir configuraciones válidas, proponer versiones corregidas y explorar variantes del **FM** bajo metas específicas.

Si bien el sistema incorpora herramientas como SymPy para análisis simbólico, NetworkX para la evaluación estructural del modelo y algoritmos genéticos para la exploración heurística del espacio de soluciones, estas tecnologías no sustituyen un motor formal de satisfacibilidad. Por su naturaleza, SymPy resulta adecuado para verificar restricciones locales o detectar inconsistencias obvias, pero no está diseñado para

resolver problemas de variabilidad a gran escala. De igual forma, los algoritmos evolutivos permiten sugerir configuraciones o completar decisiones, aunque sus resultados son aproximados. Por ello, la plataforma complementa este enfoque híbrido con motores lógicos especializados basados en SAT/SMT para garantizar corrección formal y escalabilidad en los análisis.

Para validar y generar configuraciones curriculares consistentes, se adoptó por un motor lógico basado en técnicas SAT y resolución de restricciones (CSP). Estas técnicas son exactas, determinísticas y ampliamente empleadas en entornos SPL industriales. Librerías como PySAT (python-sat) y Z3 Solver permiten representar cardinalidades, dependencias, exclusiones y restricciones complejas entre características, garantizando un análisis consistente independientemente del tamaño del modelo.

Para la validación formal del FM se hará uso de PySAT, un *toolkit* industrial que integra múltiples solucionadores SAT de alto rendimiento (MiniSAT, Glucose, Lingeling, Maple, entre otros). PySAT permite formular restricciones curriculares como cláusulas booleanas y delegar su resolución a técnicas probadas a nivel de investigación y producción (**pysat2018**). Su uso mejora la escalabilidad del sistema, ya que permite manejar modelos con miles de dependencias de forma eficiente, además de habilitar análisis avanzados como detección de dependencias redundantes, pruebas de satisfacibilidad incremental y extracción de explicaciones (UNSAT cores), funcionalidad útil para asistir al usuario en la corrección de configuraciones inválidas.

Los solucionadores SAT-based han demostrado ser una técnica altamente efectiva en SPL para la verificación automática de configuraciones (**benavides2010automated**). Su aplicación consiste en reducir el FM a una fórmula booleana y comprobar su satisfacibilidad mediante algoritmos especializados. Este enfoque es determinista, completo y garantiza la detección de inconsistencias globales, no sólo locales. A diferencia de técnicas heurísticas, los solucionadores SAT permiten decisiones exactas, prueba de imposibilidad, generación de configuraciones alternativas, evaluación de cardinalidades y análisis de impacto ante cambios, aportando rigor matemático y soporte computacional sólido para la variabilidad curricular.

Mientras los solucionadores SAT determinan si una configuración es válida o no, el uso de técnicas Max-SAT permite optimizar soluciones, seleccionar variantes preferidas y resolver conflictos minimizando el número de violaciones. El solucionador Z3, desarrollado por Microsoft Research, soporta Max-SAT, SMT y razonamiento híbrido, lo que lo convierte en una herramienta idónea para el análisis curricular (**z3_2008**). Esto permite no sólo verificar configuraciones, sino sugerir alternativas cercanas a las decisiones originales del usuario, resolver conflictos automáticamente y explorar escenarios de “mejor esfuerzo” cuando no existe una solución exacta. En términos prácticos, Max-SAT habilita un sistema inteligente que auxilia al gestor curricular en decisiones complejas, proporcionando soluciones justificadas y transparentes.

1.4.9. Gestor de dependencias

Para la gestión eficiente de dependencias, aislamiento de entornos y versionado reproducible de bibliotecas, la plataforma adoptará Poetry y UV como gestores complementarios (**poetry2024**). La combinación de Poetry y UV permite una gestión reproducible y eficiente de dependencias, alineada con las necesidades

de mantenibilidad, escalabilidad y automatización del sistema (**poetry2024**; **uv2024**).

Poetry permite mantener un control declarativo de las dependencias del proyecto mediante archivos `pyproject.toml`, garantizando la trazabilidad del *stack* tecnológico y la compatibilidad de versiones. Su mecanismo de resolución de dependencias asegura consistencia en los entornos de desarrollo y producción, reduciendo incidencias asociadas a incompatibilidades entre bibliotecas (**poetry2024**). Además, Poetry facilita la generación de entornos virtuales aislados, el bloqueo de versiones exactas (`poetry.lock`), la publicación del proyecto como paquete y la automatización de *scripts* relacionados con el ciclo de vida del sistema.

En conjunto con Poetry, se utilizará UV como herramienta moderna para instalación ultrarrápida de dependencias y creación de entornos (**uv2024**). UV destaca por su rendimiento de orden de magnitud superior a herramientas tradicionales, lo cual mejora tiempos de despliegue, entornos **CI/CD** y operaciones de reconstrucción.

1.4.10. Herramienta para el control de versiones

Los sistemas de control de versiones más populares son Git, Subversion y Mercurial. Se eligió Git debido a que es un sistema de control de versiones distribuido utilizado en el desarrollo de software que rastrea las modificaciones realizadas en el código fuente. Git, a diferencia de los sistemas de control de versiones centralizados, permite que varios desarrolladores colaboren de forma independiente en un solo proyecto al permitirles mantener repositorios locales separados que se sincronizan con un repositorio remoto. Los repositorios, que contienen los archivos del proyecto y el historial de cambios, son esenciales para el funcionamiento de Git (**ponuthorai2022git**). Una confirmación en un repositorio es una instantánea del proyecto en ese momento específico; cada confirmación se distingue por un *hash* distinto y va acompañada de un mensaje de explicación. Las ramas permiten a los desarrolladores trabajar de forma independiente en varias funciones o correcciones (**loeliger2012version**).

Mientras que otras ramas se utilizan para el desarrollo, el código estable generalmente se encuentra en la rama principal, también conocida como *main*. La reorganización y la combinación son dos formas en que los cambios de las ramas se pueden combinar en la rama principal, cada una con una función distinta para preservar el historial del proyecto. Se crea una copia local de un repositorio clonándolo, y los cambios locales se sincronizan con un repositorio remoto mediante la inserción y la extracción (**ghodke2024git**). El área de preparación sirve como zona de influencia donde se realizan modificaciones antes de la confirmación. Los submódulos, `cherry-pick` y `Git hooks` son ejemplos de funciones avanzadas que proporcionan más personalización y control. En resumen, Git permite llevar un historial exhaustivo de cambios, ramificación para desarrollo de nuevas características, fusiones controladas, auditoría de modificaciones y recuperación de estados previos del sistema. Los conocidos servicios de alojamiento de Git, como GitHub, GitLab y Bitbucket, ofrecen plataformas para la revisión de código, la gestión de repositorios y la integración de herramientas, que promueven el desarrollo colaborativo. El uso de Git mejora los procesos de desarrollo al facilitar el trabajo en equipo y la gestión de código efectivos (**ponuthorai2022git**).

GitHub es una plataforma de alojamiento de código basada en la web que utiliza Git como tecnología subyacente para el control de versiones. Actúa como un servicio de repositorio remoto centralizado, ampliando las capacidades de Git con herramientas de colaboración, gestión de proyectos y características sociales. Como una de las plataformas más populares para el desarrollo de software colaborativo, GitHub permite a los equipos alojar repositorios públicos y privados, facilitando la revisión de código mediante *pull requests*, el seguimiento de problemas con *issues* y la automatización de flujos de trabajo mediante GitHub Actions. Su interfaz gráfica y sus herramientas de gestión hacen que las operaciones de Git, como la fusión de ramas y la resolución de conflictos, sean más accesibles para los desarrolladores, al tiempo que proporcionan integraciones robustas con herramientas de [CI/CD](#) y servicios de terceros. GitHub también fomenta la comunidad *open-source* mediante funciones como la bifurcación (*forking*) de repositorios y las contribuciones externas, consolidándose como un entorno integral para el ciclo de vida del desarrollo de software ([github2024](#); [ghodke2024git](#)).

1.4.11. Lenguaje de modelado

El lenguaje de modelado se refiere a un lenguaje formal utilizado para crear modelos que representan sistemas, procesos o estructuras de datos. Estos lenguajes proporcionan un conjunto de reglas sintácticas y semánticas que facilitan la comprensión de fenómenos complejos de manera estructurada ([rumbaugh2004uml](#)).

El lenguaje de modelado a utilizar es el [Lenguaje de Modelado Unificado \(UML\)](#), este lenguaje ofrece un estándar para describir un plano del sistema, incluyendo aspectos conceptuales tales como procesos de negocios, funciones del sistema, expresiones de lenguajes de programación, esquemas de bases de datos y componentes reutilizables. Este lenguaje de modelo es el seleccionado para especificar o describir métodos o procesos, definir el sistema y sus artefactos para la documentación y construcción de la aplicación web a desarrollar ([omg2017](#)).

1.4.12. Herramienta CASE

Una herramienta de [Ingeniería de Software Asistida por Computadora \(CASE\)](#) es un programa informático destinado a aumentar la productividad durante todo el proceso de desarrollo de software reduciendo el coste de este en términos de tiempo y de dinero ([pressman2014se](#)). Según Sommerville, estas herramientas nos pueden ayudar en todos los aspectos del ciclo de vida de desarrollo del software en tareas como el diseño de proyectos, cálculo de costes, implementación de parte del código automáticamente con el diseño dado, compilación automática, documentación o detección de errores, automatizar actividades de análisis, mejorando así la productividad y calidad del software. Para el diseño de la solución se empleará la herramienta [CASE Visual Paradigm](#) en su versión 16.2 ([sommerville2016se](#)).

La herramienta Visual Paradigm es reconocida mundialmente por sus soluciones de software para la transformación de negocio. Propicia un conjunto de ayudas para el desarrollo de programas informáticos, desde la planificación, pasando por el análisis y el diseño, hasta la generación de código. Soporta los principales estándares de la industria tales como el [UML](#), la notación [Notación de Modelado de Procesos de](#)

Negocio (BPMN) y un sinfín de diagramas. Por lo que Visual Paradigm es una herramienta CASE integral y flexible para el modelo y gestión en el desarrollo de software, procesos de negocio y arquitectura empresarial (visualparadigm2024).

1.4.13. Herramientas para diseñar y ejecutar pruebas

Para garantizar la calidad y correcto funcionamiento del sistema propuesto, se emplearán herramientas especializadas tanto para pruebas de API como para pruebas unitarias del código. Existen múltiples alternativas en el mercado —como JUnit para Java, Mocha para JavaScript, Pytest para Python, o Postman y SoapUI para pruebas de API—, por lo que la selección de las herramientas para este proyecto se basó en su compatibilidad con el ecosistema tecnológico adoptado, su madurez y el soporte documental disponible.

Postman será utilizado como plataforma de desarrollo y pruebas de APIs (postman2025docs). Esta herramienta permite diseñar, documentar y probar *endpoints* RESTful de forma interactiva, facilitando la verificación de contratos de datos, validación de respuestas, gestión de variables de entorno y automatización de colecciones de pruebas. Postman resulta esencial para validar el comportamiento del *backend* FastAPI, pues el servidor desarrollado en la tecnología seleccionada permite con facilidad descargar o importar la documentación en la herramienta, lo que brinda mayor agilidad a la hora de escribir las pruebas, asegurando que las operaciones de generación y validación de configuraciones curriculares respondan correctamente bajo distintos escenarios en etapas tempranas (postman2025docs; fastapi2023docs). Postman será empleado para la ejecución de pruebas de aceptación manuales y semiautomáticas.

Complementariamente, se utilizará Pytest como *framework* para la ejecución de pruebas unitarias en Python (pytest2025docs). Pytest destaca por su sintaxis clara, capacidad de parametrización, gestión de *fixtures* y generación de reportes detallados, lo que facilita la escritura y mantenimiento de pruebas automatizadas. Su integración con el ecosistema Python permite validar lógica de negocio, algoritmos de validación, operaciones sobre modelos de características y comportamiento de componentes aislados del sistema (pytest2025docs).

Para las pruebas de integración se determinó emplear TestClient proporcionado por FastAPI con Pytest para simular peticiones HTTP reales a los *endpoints* del sistema sin necesidad de levantar un servidor en cada ejecución. TestClient permite enviar peticiones directamente a la aplicación FastAPI, verificando el enrutamiento, la validación de datos de entrada, la ejecución de la lógica de negocio y el formato de las respuestas (fastapi2025testing). Esta aproximación garantiza que las pruebas de integración sean rápidas, repetibles y fácilmente automatizables dentro del mismo ecosistema que las pruebas unitarias.

Se escogió Apache JMeter como herramienta para las pruebas de carga y estrés. JMeter permite diseñar planes de prueba que simulan múltiples usuarios concurrentes ejecutando operaciones sobre la API REST del sistema, registrando métricas como tiempo de respuesta (promedio, percentiles 90 y 95), rendimiento (peticiones por segundo), tasa de error y consumo de recursos del servidor (jmeter2025docs). Los planes de prueba se configuraron para simular escenarios de carga creciente (de 10 a 500 usuarios concurrentes) y para evaluar el comportamiento del sistema bajo condiciones de estrés sostenido.

Esta combinación de herramientas y técnicas proporciona una cobertura integral de los diferentes niveles de prueba por los que será sometido el sistema, asegurando que los defectos sean detectados en la fase más temprana posible y que el producto final cumpla con los estándares de calidad esperados antes de su despliegue en producción.

1.4.14. Entorno integrado de desarrollo

Los **Entorno de Desarrollo Integrado (IDE)** son las herramientas que todo desarrollador debe tener a mano. Permiten editar y gestionar código fuente en diversos lenguajes de programación y ofrecen múltiples herramientas para facilitar el trabajo y aumentar la productividad, pues permiten depurar, resaltar sintaxis, autocompletado, integración con control de versiones y la refactorización. Por lo general cada **IDE** está asociado a un único lenguaje de programación o a un conjunto pequeño de lenguajes, cuyo factor provoca que la gran mayoría de desarrolladores opten por otras alternativas, ya que en proyectos donde el uso de múltiples lenguajes es evidente, puede ser un obstáculo (**ibm_IDE**).

Se escogió como entorno de desarrollo Visual Studio Code (**vscode2025**), cuya herramienta es no convencional pero muy usada por muchos programadores, siendo señalado como el editor de código más popular entre proyectos de código abierto (*open source*) (**ramel2021VSCodePopularity**). El creador del editor de código multiplataforma, VSCode, es Microsoft. Este software es gratuito y de código abierto. Incluye soporte para la depuración, control integrado de Git, resaltado de sintaxis, finalización inteligente de código, fragmentos y refactorización de código. Además, su nivel de personalización permite adaptar el entorno a las preferencias de cada programador mediante temas, atajos y configuraciones avanzadas (**vscode_docs2025**).

VSCode presenta extensiones importantísimas que permite adaptar el entorno de desarrollo para una enorme infinidad de tareas, tecnologías y lenguajes. Este conjunto, prácticamente ilimitado, es totalmente accesible dentro del propio editor. Tiene soporte integrado para aplicaciones web; por lo tanto, se puede llevar a cabo en este entorno de trabajo la codificación de la aplicación a desarrollar con las tecnologías FastAPI y Python. VSCode es considerado por brindar eficiencia y agilidad en la programación, pues funciona muy bien en incluso equipos con recursos limitados; además, los desarrolladores lo aprecian porque su interfaz de usuario es muy intuitiva y permite comenzar, prácticamente, a trabajar sin necesidad de alguna experiencia (**vscode2025**; **vscode_docs2025**).

1.5. Metodología de desarrollo del software

Las metodologías de desarrollo de software son enfoques estructurados que guían las distintas fases del ciclo de vida de un proyecto, desde la planificación y análisis hasta la implementación y mantenimiento. Estas metodologías establecen principios, prácticas y procedimientos para gestionar el desarrollo de manera organizada y eficiente, promoviendo la colaboración en equipo y la calidad del producto final (**pacienza2015metodologias**).

A lo largo de esta sección se realiza un análisis de las metodologías de software disponibles y, finalmente,

se detallan los principios, fases y prácticas de la metodología seleccionada para guiar el diseño e implementación del sistema propuesto. La elección de la metodología es fundamental para asegurar un proceso de desarrollo ágil, eficiente y adaptable a posibles cambios en los requisitos del proyecto. Estas metodologías facilitan además la detección temprana de errores y el ajuste progresivo de funcionalidades, garantizando la calidad final del sistema.

En la actualidad se pueden diferenciar dos grandes grupos de metodologías de desarrollo de software: las ágiles y las tradicionales. A continuación, se explican las características de cada una de ellas.

1.5.1. Metodologías de desarrollo de software tradicionales

Las metodologías de desarrollo de software tradicionales se caracterizan por definir total y rígidamente los requisitos al inicio de los proyectos de ingeniería de software. Los ciclos de desarrollo son poco flexibles y no permiten realizar cambios, al contrario que las metodologías ágiles; lo que ha propiciado el incremento en el uso de las segundas.

La organización del trabajo de las metodologías tradicionales es lineal, es decir, las etapas se suceden una tras otra y no se puede empezar la siguiente sin terminar la anterior. Tampoco se puede volver hacia atrás una vez se ha cambiado de etapa. Estas metodologías, no se adaptan nada bien a los cambios, y el mundo actual cambia constantemente (**awad2005agile_v_traditional**). Las principales metodologías tradicionales o clásicas son:

- **Modelo en Cascada:** también conocido como el ciclo de vida clásico del software, es una de las metodologías de desarrollo de software más antiguas y ampliamente utilizadas. Este enfoque se basa en la idea de que el desarrollo de software debe seguir una secuencia lineal y predefinida de etapas de riguroso orden, cada una de las cuales debe completarse antes de pasar a la siguiente. Los requisitos y especificaciones iniciales no están predispuestos para cambiarse.
- **Incremental:** en esta metodología de desarrollo de software se va construyendo el producto final de manera progresiva. En cada etapa incremental se agrega una nueva funcionalidad, lo que permite ver resultados de una forma más rápida en comparación con el modelo en cascada. El software se puede empezar a utilizar incluso antes de que se complete totalmente y, en general, es mucho más flexible que las demás metodologías.
- **Espiral:** es una combinación de los dos modelos anteriores, que añade el concepto de análisis de riesgo. Se divide en cuatro etapas: planificación, análisis de riesgo, desarrollo de prototipo y evaluación del cliente. El nombre de esta metodología da nombre a su funcionamiento, ya que se van procesando las etapas en forma de espiral. Cuanto más cerca del centro se está, más avanzado está el proyecto.

1.5.2. Metodologías de desarrollo de software ágiles

Hoy en día las metodologías ágiles de desarrollo de software son las más utilizadas debido a su alta flexibilidad y agilidad. Los equipos de trabajo que las utilizan son mucho más productivos y eficientes, ya

que saben lo que tienen que hacer en cada momento. Además, la metodología permite adaptar el software a las necesidades que van surgiendo por el camino, lo que facilita construir aplicaciones más funcionales.

Las metodologías ágiles se basan en la metodología incremental, en la que en cada ciclo de desarrollo se van agregando nuevas funcionalidades a la aplicación final. Sin embargo, los ciclos son mucho más cortos y rápidos, por lo que se van agregando pequeñas funcionalidades en lugar de grandes cambios.

Este tipo de metodologías permite construir equipos de trabajo autosuficientes e independientes que se reúnen cada poco tiempo para poner en común las novedades. Poco a poco, se va construyendo y puliendo el producto final, a la vez que el cliente puede ir aportando nuevos requerimientos o correcciones, ya que puede comprobar cómo avanza el proyecto en tiempo real (**paciencia2015metodologias**). Las principales metodologías ágiles son:

- Kanban: metodología de trabajo inventada por la empresa de automóviles Toyota. Consiste en dividir las tareas en porciones mínimas y organizarlas en un tablero de trabajo dividido en tareas pendientes, en curso y finalizadas. De esta forma, se crea un flujo de trabajo muy visual basado en tareas prioritarias e incrementando el valor del producto.
- Scrum: es también una metodología incremental que divide los requisitos y tareas de forma similar a Kanban. Se itera sobre bloques de tiempos cortos y fijos (entre dos y cuatro semanas) para conseguir un resultado completo en cada iteración. Las etapas son: planificación de la iteración (*planning sprint*), ejecución (*sprint*), reunión diaria (*daily meeting*) y demostración de resultados (*sprint review*). Cada iteración por estas etapas se denomina también *sprint*.
- **Extreme Programming (XP)**: es una metodología de desarrollo de software basada en las relaciones interpersonales, que se consideran la clave del éxito. Su principal objetivo es crear un buen ambiente de trabajo en equipo y que haya un *feedback* constante del cliente. El trabajo se basa en 12 conceptos: diseño sencillo, *testing*, refactorización y codificación con estándares, propiedad colectiva del código, programación en parejas, integración continua, entregas semanales e integridad con el cliente, *cliente in situ*, entregas frecuentes y planificación.

1.5.3. Elección de la metodología

En las metodologías tradicionales el principal problema es que nunca se logra planificar bien el esfuerzo requerido para seguir la metodología. Pero entonces, si se logran definir métricas que apoyen la estimación de las actividades de desarrollo, muchas prácticas de metodologías tradicionales podrían ser apropiadas. El no poder predecir siempre los resultados de cada proceso no significa que se esté frente a una disciplina de azar. Lo que significa es que se está frente a la necesidad de adaptación de los procesos de desarrollo que son llevados por parte de los equipos que desarrollan software (**awad2005agile_v_traditional**).

Tener metodologías diferentes para aplicar de acuerdo con el proyecto que se desarrolle resulta una idea interesante. Estas metodologías pueden involucrar prácticas tanto de metodologías ágiles como de metodologías tradicionales. De esta manera se podría tener una metodología por cada proyecto, la problemática sería definir cada una de las prácticas, y en el momento preciso definir parámetros para saber cuál usar. Es

importante tener en cuenta que el uso de un método ágil no vale para cualquier proyecto. Sin embargo, una de las principales ventajas de los métodos ágiles es su peso inicialmente ligero y por eso las personas que no estén acostumbradas a seguir procesos encuentran estas metodologías bastante agradables. La tabla 1.5 explica de forma resumida las diferencias que existen entre las metodologías ágiles y las tradicionales.

Cuadro 1.5. Comparación, Metodología Ágil VS Metodología Tradicional (awad2005agile_v_traditional).

Parámetros	Metodologías Ágiles	Metodologías Tradicionales
Acercamiento	Adaptivo	Predictivo
Medición del objetivo	Valor para el negocio	Conformación del plan
Tamaño del proyecto	Pequeño	Largo
Tipo de administración	Descentralizado	Autocrático
Perspectiva de cambios	Adaptable a cambio	Sostenible al cambio
Cultura del equipo	Liderazgo\Colaboración	Comandada\Controlada
Documentación	Baja	Pesada
Orientada	Personas	Procesos
Ciclos de desarrollo	Numerosos	Limitados
Dominio de desarrollo	Impredecibles	Predecibles
Planificación inicial	Mínima	Comprensiva
Retorno de la inversión	Temprano en el proyecto	Al final del proyecto
Tamaño del equipo	Pequeño	Grande

Para la selección de la metodología a utilizar se toma como referencia la tabla 1.5. Para ello se analizan las características del proyecto atendiendo a los diferentes aspectos y se selecciona la metodología con la que haya más coincidencias, dando como resultado de este análisis la tabla 1.6.

Cuadro 1.6. Comparación de la Metodología Ágil y la Metodología Tradicional con respecto al proyecto. Fuente: Elaboración propia.

Parámetros	Ágiles	Pesadas
Acercamiento	X	
Medición del objetivo	X	
Tamaño del proyecto	X	
Tipo de administración	X	
Perspectiva de cambios	X	
Cultura del equipo	X	
Documentación	X	
Orientada	X	
Ciclos de desarrollo	X	
Dominio de desarrollo	X	
Planificación inicial	X	

Parámetros	Ágiles	Pesadas
Retorno de la inversión		X
Tamaño del equipo	X	
Total:	12	1

Como resultado se obtiene que lo mejor es utilizar una metodología ágil. Ya que, según el análisis realizado, se acomoda mejor a las necesidades y características del proceso a llevar a cabo. Para decidir qué metodología ágil se utilizará, se realizó una comparación entre algunas de las metodologías ágiles más comúnmente utilizadas, la cual se refleja en la tabla 1.7.

Cuadro 1.7. Comparación de Metodología Ágiles. Fuente: Elaboración propia.

Parámetros	XP	Scrum
Duración	1 a 2 semanas	2 semanas a 1 mes
Cambios	Sensibles al cambio	No permite cambios
Prioridad	Estricta, priorizada por el cliente	Determinada por el equipo
Prácticas de ingeniería	Prescribe prácticas (pruebas, programación en parejas)	No prescribe prácticas
Tamaño de los proyectos	Pequeños y medianos	Pequeños, medianos y grandes
Tamaño de equipo	Menor que 10	Múltiples equipos menores que 10

Teniendo en cuenta la tabla 1.7 se decide utilizar la metodología **XP**. Esto se debe a que es muy eficiente durante el proceso de pruebas y planificación, su tasa de error es muy pequeña, facilita los cambios, origina una programación muy organizada, además de fomentar la comunicación entre los desarrolladores y los clientes. También se puede aplicar a cualquier lenguaje de programación, el cliente tiene control sobre las prioridades, durante todo el proyecto se pueden realizar diversas pruebas y, sobre todo, permite ahorrar mucho tiempo y dinero.

1.5.4. Programación Extrema

XP es una metodología ágil de desarrollo de software con bases en la comunicación constante y la retroalimentación. Uno de sus fines principales es el de construir un producto que vaya en línea con los requerimientos del cliente. En ese sentido es adaptable a los cambios, generando una rápida respuesta frente a cualquier inconveniente. Por otro lado, el equipo de trabajo tiene la ventaja de potenciar sus relaciones, ya que el proceso que de este se desprende es abierto, conjunto y de aprendizaje continuo (**pressman2014se**).

Las 4 etapas que sigue **XP** son:

- Planificación: toma como referencia la identificación de las **Historias de Usuario (HU)** con pequeñas versiones que se irán revisando en periodos cortos con el fin de obtener un software funcional.
- Diseño: trabaja el código orientado a objetivos y, sobre todo, usando los recursos necesarios para que funcione.

- Codificación: se refiere al proceso de programación organizada en parejas, estandarizada y que resulte en un código universal entendible.
- Pruebas: consiste en un testeo automático y continuo en el que el cliente tiene voz para validar y proponer.

Los principios de **XP** comprenden diez buenas prácticas que involucran al equipo de trabajo, los procesos y el cliente (**pressman2014se**); los cuales son:

- Planificación incremental: Se toman los requerimientos en **HU**, las cuales son negociadas progresivamente con el cliente.
- Entregas pequeñas: Se desarrolla primero la más mínima parte útil que le proporcione funcionalidad al sistema, y poco a poco se efectúan incrementos que añaden funcionalidad a la primera entrega, cada ciclo termina con una entrega del sistema.
- Diseño sencillo: Solo se efectúa el diseño necesario para cumplir con los requerimientos actuales, es decir, no se abordan requerimientos futuros.
- Desarrollo previamente aprobado: Una de las características relevantes y propias de **XP** es que primero se escriben las pruebas y luego se da la codificación, esto con la finalidad de asegurar la satisfacción del requerimiento.
- Limpieza del código o refactorización: Consiste en simplificar y optimizar el programa sin perder funcionalidad, es decir, alterar su estructura interna sin afectar su comportamiento externo.
- Programación en parejas: Es otra de las características de esta metodología, que propone que los desarrolladores trabajen en parejas en una terminal, verificando cada uno el trabajo del otro y ayudándose para buscar las mejores soluciones. Se entiende que de esta forma el trabajo será más eficiente y de mayor calidad.
- Propiedad colectiva: El conocimiento y la información deben ser de todos, por lo tanto no se desarrollan islas de conocimiento, todos los programadores poseen todo el código y cualquiera puede sugerir y realizar mejoras.
- Integración continua: Al terminar una tarea, esta se integra al sistema entero y se realizan pruebas de unidad a todo el sistema, esta práctica permite que la aplicación sea más funcional en cada iteración y garantiza su funcionamiento con los demás módulos del sistema.
- Ritmo sostenible: No es aceptable trabajar durante grandes cantidades de horas ya que se considera que puede reducir la calidad del código y la productividad del equipo a mediano plazo, se sugieren 40 horas semanales.
- Cliente presente: Se debe tener un representante (cliente o usuario final) a tiempo completo, ya que en **XP** este hace parte del equipo de desarrollo y es responsable de formular los requerimientos para el desarrollo del sistema.

Artefactos

Las metodologías utilizan artefactos para describir las actividades o procedimientos que se deben seguir durante su desarrollo. A continuación, se describen los artefactos de **XP**:

- **HU**: Representan una breve descripción del comportamiento del sistema. Emplea terminología del cliente sin lenguaje técnico. Se realiza una por cada característica principal del sistema. Se emplean para hacer estimaciones de tiempo y para el plan de lanzamientos. Reemplazan un gran documento de requisitos y presiden la creación de las pruebas de aceptación. Estas deben proporcionar sólo el detalle suficiente como para poder hacer razonable la estimación de cuánto tiempo requiere la implementación de la **HU**, difiere de los casos de uso porque son escritos por el cliente, no por los programadores, empleando terminología del cliente (**pressman2014se**).
- Tareas de Ingeniería: Una **HU** se descompone en varias tareas de ingeniería, que describen las actividades que se realizarán en cada una de ellas. Así mismo, las tareas de ingeniería se vinculan más al desarrollador, ya que permite tener un acercamiento con el código (**pressman2014se**).
- Tarjetas **Clase, Responsabilidad y Colaboración (CRC)**: Estas tarjetas se dividen en tres secciones que contienen la información del nombre de la clase, sus responsabilidades y sus colaboradores. Una clase es cualquier persona, cosa, evento, concepto, pantalla o reporte. Las responsabilidades de una clase son las cosas que conoce y las que realiza, sus atributos y métodos. Los colaboradores son las demás clases con las que trabaja en conjunto para llevar a cabo sus responsabilidades (**pressman2014se**).

En síntesis, actualmente **XP** es una de las metodologías de mayor aceptación en la industria del software. Su enfoque basado en los métodos ágiles, su énfasis en la gestión del recurso humano, el cual es uno de los puntos más críticos en todo proyecto, y sus principios de previsibilidad y adaptabilidad hacen de esta metodología una buena opción a seguir.

1.6. Conclusiones del capítulo

En el desarrollo del capítulo se obtuvo una mejor visión acerca del problema planteado y se dieron cumplimiento a las primeras tareas de investigación llegando a las siguientes conclusiones:

- El análisis de los conceptos asociados a la solución permitió un acercamiento a los temas relacionados con la **SPLE**, **FM** y el **UVL**, estableciendo una base teórica sólida que fundamenta la propuesta. La analogía formal desarrollada entre estos conceptos y el dominio educativo demostró la viabilidad de aplicar técnicas de variabilidad a la gestión curricular, abriendo nuevas posibilidades para la flexibilización controlada de planes de estudio.
- El estudio de los sistemas homólogos relacionados con el marco de la investigación, permitió identificar una brecha significativa en el mercado: mientras existen herramientas maduras para la gestión administrativa académica (**SIGA**, **Banner**, **Moodle**) y potentes *frameworks* de modelado de variabilidad (**pure::variants**, **FeatureIDE**, **Flamapy**), ninguna de ellas integra de manera natural la capacidad de

modelar, validar y configurar currículos educativos con un enfoque accesible para gestores académicos no especializados en ingeniería de software. Esta constatación justifica la necesidad y pertinencia de la solución propuesta.

- La selección tecnológica resultante, respaldada por un análisis comparativo riguroso, define las bases para el diseño e implementación satisfactoria de la aplicación a desarrollar, garantizando rendimiento, escalabilidad, mantenibilidad y alineación con los requisitos funcionales del sistema.
- El proyecto tiene las características de tener un tiempo de desarrollo corto y la necesidad de realizar entregas constantes en ciclos cortos, por lo que es necesario el uso de una metodología ágil. Luego de un análisis comparativo entre metodologías tradicionales y ágiles, se decide utilizar la metodología [XP](#), permitiendo esto cierta flexibilidad a lo largo del desarrollo de la aplicación, una interacción constante con el usuario, la incorporación de cambios durante el proceso, y la garantía de calidad mediante prácticas como programación en parejas, desarrollo dirigido por pruebas e integración continua.

Análisis y diseño de la propuesta de solución

El presente capítulo constituye el núcleo de la fase de análisis y diseño de la investigación, traduciendo los fundamentos teóricos establecidos en el capítulo anterior en una propuesta técnica concreta y viable. Partiendo del diagnóstico de la problemática y del estudio de sistemas homólogos, se procede a definir de manera formal y estructurada la solución de software que dará respuesta al objetivo general de la investigación.

El capítulo se inicia con la propuesta de solución, que describe a alto nivel el sistema a desarrollar. A continuación, se realiza un levantamiento detallado de requisitos, clasificados en funcionales y no funcionales, para precisar el comportamiento y las cualidades del sistema. Estas especificaciones se refinan a través de [HU](#), las cuales, junto con su estimación de esfuerzo, sirven de base para el desarrollo del plan de iteraciones que guiará la implementación. Posteriormente, el diseño orientado a objetos se plasma mediante tarjetas [CRC](#), sentando las bases para la definición de la arquitectura de software basada en el patrón arquitectónico por capas. Finalmente, se especifican los [Patrones de Software Generales para la Asignación de Responsabilidades \(GRASP\)](#) y los patrones de diseño [GoF](#), que se aplicarán para resolver problemas comunes en el desarrollo de software y otros ámbitos referentes al diseño.

2.1. Descripción de la propuesta de solución

En función de dar cumplimiento al objetivo planteado y ajustándose a las necesidades presentes, se propone desarrollar un servicio *backend* que, basado en los principios de [SPLE](#) y la técnica de modelado [FM](#), permita a los profesionales de la educación representar y gestionar la variabilidad curricular de forma sistemática utilizando [UML](#) como lenguaje de modelado u operando directamente sobre la representación gráfica. Esta solución permitirá representar formalmente planes de estudio, definir variabilidad entre módulos, establecer dependencias, restricciones y rutas formativas alternativas, así como derivar configuraciones válidas que representen itinerarios personalizados. En esencia, el sistema habilita una ingeniería curricular basada en la lógica [SPLE](#), donde los planes de estudio se conciben como familias de productos y cada configuración curricular corresponde a un producto derivado.

El servicio abordará todas las fases necesarias para este proceso: diseño del modelo curricular (equivalente al análisis de dominio), definición de restricciones y cardinalidades (gestión de variabilidad), validación automatizada de configuraciones (ensamblaje de productos) y versionamiento del modelo (gestión de activos). Para ello se implementará un motor central que permita representar jerarquías de características, aplicar reglas transversales, procesar requerimientos, exclusiones e implicaciones, y generar configuraciones consistentes, detectando violaciones y proponiendo alternativas.

El sistema incorporará funcionalidades que responden directamente a los retos del dominio educativo: evitar inconsistencias curriculares, detectar prerrequisitos no satisfechos, prevenir combinaciones inviables, explorar rutas posibles, permitir adaptaciones por niveles, perfiles o competencias, y almacenar *snapshots* versionados que faciliten la evolución del plan de estudios sin invalidar configuraciones previas. Este enfoque no se limita a documentar un currículo, sino que lo convierte en un modelo computable y verificable, habilitando su reutilización y variación controlada.

2.1.1. Sistemas de roles estáticos

Un sistema de roles estáticos constituye un mecanismo de control de acceso y organización funcional, en el cual los permisos, responsabilidades y alcances de cada usuario se definen de manera fija durante la fase de diseño del sistema y permanecen invariantes a lo largo del tiempo, salvo intervención explícita de un administrador (**ferraiolo2007role**; **sandhu1996role**). Este paradigma, ampliamente adoptado en sistemas empresariales y académicos, asigna a cada usuario uno o varios roles predefinidos, y cada rol posee un conjunto determinado de privilegios sobre los recursos y operaciones del sistema.

La principal característica de los sistemas de roles estáticos es su predictibilidad y estabilidad: las acciones que un usuario puede realizar están claramente delimitadas por el rol que se le ha asignado, sin que exista una lógica dinámica que modifique estos permisos en tiempo de ejecución en función del contexto, el estado del sistema o el historial de acciones del usuario (**hu2006assessment**). Esta rigidez, que podría considerarse una limitación en entornos altamente dinámicos, se convierte en una ventaja en dominios donde la separación de responsabilidades debe ser clara, auditable y fácil de comprender por los propios usuarios.

En el contexto de los sistemas de gestión, los roles estáticos más comunes suelen incluir:

- Administrador: Usuario con privilegios máximos, responsable de la configuración del sistema, la gestión de usuarios y la asignación de roles.
- Usuario estándar: Rol con permisos básicos para operar dentro del sistema según su función específica (lectura, escritura, modificación de ciertos recursos).
- Invitado o visitante: Rol con acceso limitado a información pública o de solo lectura, sin capacidad de modificación.

La adopción de un modelo de roles estáticos en una aplicación software implica, desde el punto de vista arquitectónico, la necesidad de implementar un subsistema de autenticación y autorización que valide cada solicitud del usuario contra la matriz de permisos definida para su rol. Este enfoque simplifica el desarrollo

al eliminar la complejidad asociada a los sistemas de roles dinámicos (donde los permisos pueden cambiar según reglas de negocio, tiempo, ubicación o estado del sistema), pero requiere una planificación cuidadosa de la jerarquía y granularidad de los roles para cubrir todos los casos de uso previstos (**colantonio2010role**).

En el servicio *backend* objeto de esta investigación, se han definido los roles estáticos mostrados en la tabla 2.1, cada uno con una misión y función principal.

Cuadro 2.1. Funciones y misión de los roles del sistema. Fuente: elaboración propia.

Rol	Función principal	Misión
Administrador	Gestión y control total de plataforma y datos.	Posee el control total de la plataforma y por tanto es el responsable de su salud, configuración y gestión de usuarios.
Diseñador de modelo	Propietario funcional del ciclo de diseño del modelo.	El arquitecto creativo encargado de idear, diseñar y liderar la construcción de los FM.
Editor de modelo	Colaborador con capacidad de edición acotada al ámbito invitado.	Aporta su conocimiento editando modelos a los que ha sido invitado.
Revisor	Rol de control de calidad y publicación, sin edición estructural.	Vela por la calidad del modelo, valida y aprueba los modelos antes de que puedan ser utilizados.
Configurador	Consumidor experto de modelos publicados para derivar configuraciones.	Constructor práctico que utiliza los FM aprobados para ensamblar soluciones finales.
Observador	Acceso de solo lectura a artefactos finales/publicados.	Consulta la información pública y final sin poder alterarla.

Esta estratificación de roles garantiza una clara separación de responsabilidades y sigue el principio de privilegio mínimo, según el cual cada usuario debe tener únicamente los permisos necesarios para cumplir con sus funciones (**saltzer1975protection**). En la tabla 2.2 se muestra los permisos estáticos de escritura y lectura que posee cada rol.

Cuadro 2.2. Jerarquía de roles estáticos del sistema. Fuente: elaboración propia.

Tabla	Administrador	Diseñador de modelo	Editor de modelo	Revisor	Configurador	Observador
User	R=Sí / W=Sí (global)	R=Sí / W=No	R=Sí / W=No	R=No / W=No	R=No / W=No	R=No / W=No
Domain	R=Sí / W=Sí (global)	R=Sí / W=No	R=Sí / W=No	R=Sí / W=No	R=Sí / W=No	R=Sí / W=No
Continúa en la siguiente página						

Cuadro 2.2 – Continuación de la página anterior.

Tabla / Entidad	Administrador	Diseñador de modelo	Editor de modelo	Revisor	Configurador	Observador
FeatureModel	R=Sí / W=Sí (global)	R=Sí / W=Sí (solo propios)	R=Sí / W=Sí (solo donde colabora; sin crear ni borrar)	R=Sí / W=No	R=Sí (publicados) / W=No	R=Sí (publicados) / W=No
FeatureModel Version	R=Sí / W=Sí (global)	R=Sí / W=Parcial	R=Sí / W=Parcial	R=Sí / W=Parcial	R=Sí / W=No	R=No / W=No
Feature	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
FeatureRelation	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
FeatureGroup	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
restricción	R=Sí / W=Sí (global)	R=Sí / W=Sí (en sus modelos)	R=Sí / W=Sí (en modelos permitidos)	R=Sí / W=No	R=Sí / W=No	R=No / W=No
FeatureModel Collaborator	R=Sí / W=Sí (global)	R=Sí / W=Sí (gestiona colaboradores de sus modelos)	R=Sí / W=No	R=No / W=No	R=No / W=No	R=No / W=No
Resource / Tag	R=Sí / W=Sí (global)	R=Sí / W=Sí	R=Sí / W=Sí	R=Sí / W=Sí	R=Sí / W=Sí (acotado según configuración propia)	R=No / W=No
Configuration	R=Sí / W=Sí (global)	R=Sí / W=No	R=Sí / W=No	R=No / W=No	R=Sí / W=Sí (propias)	R=Sí (públicas) / W=No

Convención utilizada en la tabla: R = lectura, W = escritura.

2.1.2. Componentes funcionales de la lógica de negocio del sistema

Dentro de la arquitectura general del sistema existe una separación funcional de componentes fundamentales que encapsulan en requisitos de alto nivel las funcionalidades principales del sistema. Cada uno respaldado por estrategias tecnológicas y buenas prácticas del desarrollo de software que optimizan su ejecución y conexión entre los mismos.

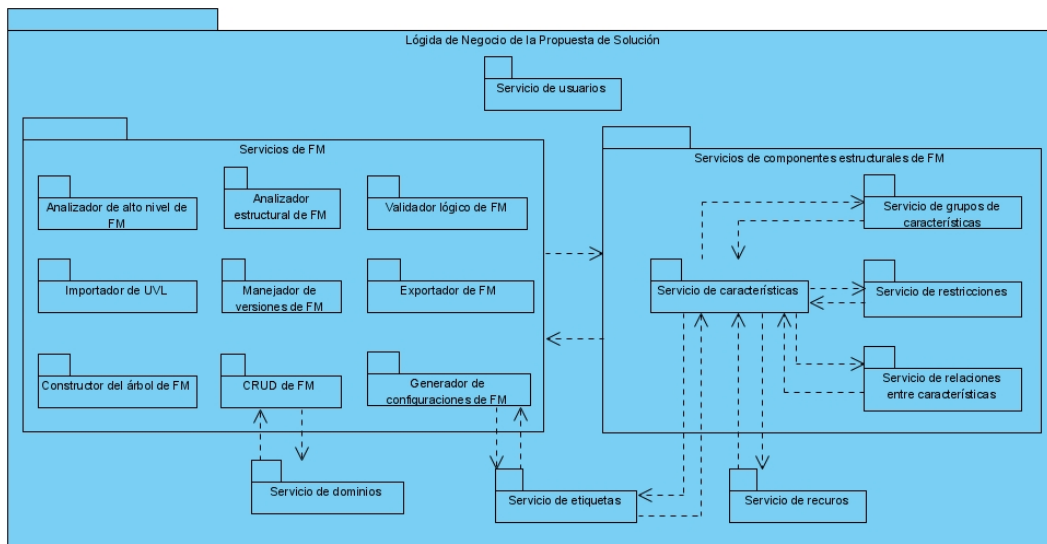


Figura 2.1. Componentes de la lógica del negocio del sistema. Fuente: elaboración propia.

La figura 2.1 ilustra los elementos que componen la lógica del negocio del sistema. En ella se ve claramente dividido el núcleo funcional en dos principales responsabilidades: los servicios de componentes estructurales de **FM** y los servicios que operan directamente sobre el **FM** en su totalidad. Ambos son dependientes del otro para su correcto funcionamiento.

Los servicios de componentes estructurales de **FM** proveen las funcionalidades básicas (**Creación, Lectura, Actualización y Eliminación (CRUD)** y otras) que permiten construir y modelar los **FM**. Por otro lado, los servicios de **FM**, están descompuestos estratégicamente como parte del principio de alta cohesión. A continuación, se describen los servicios de **FM**:

CRUD de FM

Tiene como objetivos exponer operaciones completas de gestión de **FM** y centralizar control de permisos, validaciones y respuesta de datos para los clientes. Tiene como funcionalidades:

- Proveer *endpoints* para administrar **FM**.
- Aplicar control de permisos por rol y propiedad del modelo.

Validador lógico de FM

Tiene como objetivos verificar la consistencia lógica global de un **FM** y asegurar satisfacibilidad de restricciones y relaciones. Entre sus funcionalidades se encuentran:

- Construir variables booleanas para cada característica.

- Codificar jerarquías, grupos y restricciones.
- Verificar satisfacibilidad completa del modelo y de configuraciones específicas.
- Soportar validación de selecciones parciales con caché.
- Permitir enumerar configuraciones válidas (cuando se utiliza en conjunto con el generador).
- Permitir validaciones en tres niveles:
 - Nivel 1 (SymPy): validación simbólica para modelos pequeños.
 - Nivel 2 (PySAT): solucionador [SAT](#) escalable para modelos medianos/grandes.
 - Nivel 3 (Z3): [SMT](#)/optimización para análisis avanzados.

Analizador estructural de [FM](#)

Se encarga de analizar propiedades topológicas del modelo (más allá de la lógica pura) y detectar problemas estructurales y métricas de complejidad. Cuenta con las funcionalidades de:

- Construir grafos de dependencias y jerarquía.
- Analizar alcanzabilidad, redundancias y relaciones implícitas.
- Calcular métricas de impacto y complejidad.
- Proveer los siguientes tipos de análisis:
 - Características muertas: Características inaccesibles desde la raíz.
 - Redundancias: relaciones o restricciones duplicadas.
 - Relaciones implícitas: dependencias no explícitas en la jerarquía.
 - Dependencias transitivas: efectos indirectos entre características.
 - Componentes fuertemente conexas: ciclos y conectividad.
 - Métricas: centralidad, complejidad, profundidad, impacto.

Analizador de alto nivel de [FM](#)

Tiene el objetivo principal de orquestar análisis de alto nivel combinando validación lógica, análisis estructural y métricas derivadas. Entre sus funcionalidades se encuentran:

- Construir *payloads* desde la versión.
- Ejecutar validación lógica y análisis estructural.
- Calcular métricas derivadas: núcleo / comunalidad, conjuntos atómicos y estimación de configuraciones.
- Integrar validaciones [UVL](#) y validación adicional con Flamapy.

Constructor del árbol de [FM](#)

Encargado únicamente de construir la representación completa del [FM](#) para consumo de los clientes, por lo que tiene como funcionalidades:

- Construir el árbol jerárquico anidado con metadatos.
- Generar lista de relaciones y restricciones con descripciones legibles.
- Calcular estadísticas agregadas (total de nodos, profundidad del árbol, grupos, etc.).
- Incluir recursos y etiquetas asociados a características.

Manejador de versiones de FM

Tiene como objetivos gestionar el ciclo de vida de versiones del FM y garantizar trazabilidad, inmutabilidad y reproducibilidad. Cuenta con las siguientes funcionalidades:

- Crear nuevas versiones (vacías o clonadas).
- Publicar, archivar y restaurar versiones.
- Genera *snapshots* completos con mapeos $UUID \leftrightarrow Integer$ y estadísticas.
- Controla estados (borrador, en revisión, publicado, archivado).

Exportador de FM

Su objetivo es exportar el FM a formatos estándar para herramientas externas. Tiene las funciones de:

- Mapea Identificador Único Universal (UUID)s a identificadores numéricos para formatos SAT/externos.
- Generar representaciones en múltiples estándares, tales como:
 - UVL.
 - Lenguaje de Marcado Extensible (XML).
 - Lenguaje Textual de Variabilidad (TVL).
 - Formateo DIMACS (DIMACS).
 - JSON.
 - Lenguaje DOT de Graphviz (DOT).
 - Mermaid.

Importador de UVL

Tiene como objetivos importar especificaciones UVL y materializarlas como estructura de FM; y validar UVL en fases: análisis sintáctico, estructura, restricciones simples y diferencia. Tiene como funcionalidades:

- Analizar sintácticamente UVL en una estructura intermedia (nodos, grupos, restricciones).
- Validar estructura: raíz única, ciclos, cardinalidad mínima de grupos.
- Convertir UVL a entidades del modelo (características, grupos, relaciones, restricciones).
- Permitir comparar un UVL contra una versión existente.

Generador de configuraciones de FM

Encargado de generar configuraciones válidas completas o completar decisiones parciales, y proponer soluciones diversas o bien optimizadas según estrategia. Cuenta con las funcionalidades de:

- Usar validación lógica para asegurar que las configuraciones cumplan todas las restricciones.
- Generar una configuración única o múltiples configuraciones diversas.
- Permitir diferentes estrategias para generar las configuraciones:
 - GREEDY: determinista, rápida y con prioridad en relaciones obligatorias; especializa en rapidez y consistencia sobre diversidad.
 - RANDOM: estocástica; prioriza diversidad y exploración, útil para muestras variadas.

- BEAM_SEARCH: explora varios caminos en paralelo con poda; balancea exploración/eficiencia y prioriza soluciones prometedoras.
- GENETIC ([Algoritmos Evolutivos Distribuidos en Python \(DEAP\)](#)): búsqueda evolutiva multi-objetivo; prioriza optimización de criterios definidos (p. ej., minimizar costos o maximizar cobertura).
- SAT_ENUM: enumeración por SAT; prioriza exhaustividad controlada, ideal para listar configuraciones válidas.
- PAIRWISE / UNIFORM / STRATIFIED: muestreo para cobertura; prioriza representatividad (cobertura de pares, distribución uniforme o estratos definidos).
- CP_SAT ([Herramientas de Investigación Operativa \(OR-Tools\)](#)): programación con restricciones; prioriza factibilidad bajo restricciones complejas y objetivos declarativos.
- BDD: muestreo con diagramas de decisión binaria; prioriza eficiencia en conteo/muestreo cuando el modelo es grande.
- [Algoritmo Genético de Ordenamiento No-Dominado II \(NSGA2\)](#): optimización multi-objetivo con Pareto; prioriza soluciones no dominadas y *trade-offs* entre objetivos.

2.2. Levantamiento de requisitos

El levantamiento de requisitos constituye una fase crítica dentro del ciclo de vida del desarrollo de software, al permitir identificar de forma estructurada las necesidades, restricciones y expectativas de los usuarios en relación con el sistema a construir. Esta etapa no solo define el alcance funcional del producto, sino que también condiciona su calidad, adaptabilidad y éxito posterior. ([pressman2014se](#))

2.2.1. Requisitos funcionales

Los [Requisitos Funcionales \(RF\)](#) son especificaciones detalladas que describen las funciones y comportamientos que un sistema, software o producto debe realizar para satisfacer las necesidades y expectativas de los usuarios. Su principal función es definir qué debe hacer el sistema para cumplir con los objetivos del negocio y las demandas del usuario, estableciendo claramente las tareas, procesos y respuestas que el sistema debe ejecutar ante diferentes situaciones. Estos requisitos son esenciales para asegurar que el sistema cumpla su propósito y funcione de acuerdo a los objetivos especificados del proyecto. (Maciaszek y Liong, 2005). La tabla 2.3 muestra los RF que debe cumplir la aplicación propuesta para dar solución a la problemática planteada.

Cuadro 2.3. Requisitos funcionales del sistema. Fuente: Elaboración propia.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#1	Autenticar usuario	Permite acceder al sistema introduciendo los siguientes datos: • correo electrónico • contraseña	Alta	Baja
Continúa en la siguiente página				

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#2	Cerrar sesión	Permite cerrar la sesión de un usuario anteriormente autenticado	Baja	Baja
RF#3	Registrar usuario	Permite registrar un usuario en el sistema con los siguientes datos: • correo electrónico • contraseña	Alta	Baja
RF#4	Solicitar recuperación de contraseña	Genera un token de recuperación y lo envía por correo para restablecer el acceso de forma controlada.	Baja	Media
RF#5	Restablecer contraseña	Permite definir una nueva contraseña válida de forma segura a partir de uno de los siguientes escenarios: • Escenario con token: token de recuperación y nueva contraseña. • Escenario autenticado: contraseña actual y nueva contraseña.	Baja	Media
RF#6	Listar usuarios	Proporciona el listado paginado de usuarios para perfiles con permisos autorizados.	Alta	Baja
RF#7	Filtrar usuarios	Permite filtrar usuarios por rol o mediante un término de búsqueda.	Media	Baja
RF#8	Actualizar perfil propio	Permite al usuario editar su información personal sin depender de administración externa. Campo de entrada: • correo electrónico (opcional)	Alta	Baja
Continúa en la siguiente página				

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#9	Asignar rol de usuario	Permite al administrador del sistema modificar el rol asignado a un usuario. Parámetros de entrada: • identificador de usuario (ruta) • rol (consulta)	Alta	Baja
RF#10	Desactivar usuario	El sistema debe permitir inhabilitar cuentas activas.	Media	Baja
RF#11	Activar usuario	Reactiva cuentas inactivas para restablecer su acceso al sistema.	Media	Baja
RF#12	Listar dominios	El sistema debe permitir listar los dominios activos en el sistema.	Alta	Baja
RF#13	Obtener dominio	Obtiene la información detallada de un dominio específico por su identificador.	Media	Baja
RF#14	Crear dominio	El sistema debe permitir crear nuevos dominios desde perfiles administrativos. Campos de entrada: • nombre • descripción (opcional)	Alta	Baja
RF#15	Actualizar dominio	Permite editar la información de un dominio ya existente a un administrador. Parámetros y campos de entrada: • identificador de dominio (ruta) • nombre (opcional) • descripción (opcional)	Alta	Baja
RF#16	Buscar dominios	Permite localizar dominios por nombre o por descripción.	Media	Baja
RF#17	Obtener dominio con FM	Devuelve un dominio junto a sus FM relacionados.	Media	Baja
RF#18	Activar dominio	Habilita un dominio previamente desactivado.	Media	Baja
Continúa en la siguiente página				

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#19	Desactivar dominio	Inhabilita un dominio para evitar su uso temporal o definitivo.	Media	Baja
RF#20	Listar FM	Permite listar FM con paginación y filtros por dominio para facilitar su navegación.	Alta	Media
RF#21	Crear FM	Crea un FM asociado a un dominio válido y consistente. Campos de entrada: • nombre • descripción (opcional) • identificador de dominio	Alta	Media
RF#22	Consultar FM	Muestra el detalle del FM incluyendo sus versiones.	Alta	Media
RF#23	Actualizar FM	El sistema debe permitir actualizar metadatos del FM manteniendo integridad de la información. Parámetros y campos de entrada: • identificador de modelo (ruta) • nombre (opcional) • descripción (opcional) • estado activo (opcional)	Alta	Media
RF#24	Activar FM	Activa un FM para habilitar su uso operativo.	Alta	Baja
RF#25	Desactivar FM	Desactiva un FM cuando se requiere restringir su uso.	Alta	Baja
RF#26	Listar versiones de modelo	Lista las versiones disponibles de un FM específico.	Alta	Baja
RF#27	Consultar versión específica de un modelo	Permite recuperar el detalle de una versión concreta de FM.	Alta	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#28	Crear versión de un modelo	Crea una nueva versión, con soporte de <i>copy-on-write</i> desde una versión origen. Parámetros y campos de entrada: • identificador de modelo (ruta) • identificador de versión origen (opcional) • descripción (opcional)	Alta	Media
RF#29	Publicar versión de un modelo	El sistema debe permitir publicar una versión y ejecuta validaciones necesarias antes de su disponibilidad oficial.	Alta	Alta
RF#30	Achivar versión de un modelo	Cambia el estado de una versión a archivada para su preservación histórica.	Media	Baja
RF#31	Restaurar versión de un modelo	EL sistema debe permitir recuperar una versión archivada y la devuelve al flujo de trabajo activo.	Media	Baja
RF#32	Obtener estructura completa de un FM	El sistema debe entregar el árbol completo para consulta integral de una versión concreta ya sea por identificar o por la versión publicada más reciente.	Alta	Alta
RF#33	Obtener Estadísticas de un FM	Obtiene métricas principales de la versión publicada más reciente o de una versión específica.	Alta	Media

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#34	Exportar un FM	El sistema debe permitir exportar a la versión publicada más reciente o a una versión concreta en los siguientes formatos: • UVL. • XML. • TVL. • DIMACS. • JSON. • DOT. • mermaid.	Alta	Media
RF#35	Obtener UVL efectivo de un FM	Permite devolver el UVL final, ya sea almacenado o generado desde la estructura vigente.	Alta	Media
RF#36	Guardar UVL	El sistema debe permitir guardar un UVL analizado y validado. Parámetros y campos de entrada: • identificador de modelo (ruta) • identificador de versión (ruta) • contenido UVL	Alta	Alta
RF#37	Sincronizar UVL desde estructura	Regenera y persiste UVL a partir del modelo visual para mantener coherencia.	Alta	Alta
RF#38	Aplicar UVL a estructura	El sistema debe permitir construir una nueva versión estructural tomando como entrada una definición UVL. Parámetros y campos de entrada: • identificador de modelo (ruta) • identificador de versión (ruta) • contenido UVL	Alta	Alta
Continúa en la siguiente página				

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#39	Validar lógica del modelo	El sistema debe permitir evaluar la satisfacibilidad y consistencia lógica global del modelo.	Alta	Alta
RF#40	Validar estructura del modelo	Verifica integridad estructural, incluyendo raíz, ciclos y nodos huérfanos de un FM.	Alta	Media
RF#41	Validar configuración seleccionada	Comprueba que una selección concreta de características sea válida y consistente.	Alta	Alta
RF#42	Análisis estructural de validación	El sistema debe permitir ejecutar análisis de características muertas, redundancias y componentes fuertemente conectados.	Alta	Alta
RF#43	Crear configuración persistente	Crea y almacena configuraciones asociadas a una versión tras validar consistencia. Campos de entrada: • nombre • descripción (opcional) • identificador de versión del modelo de características • lista de identificadores de características	Alta	Alta
RF#44	Obtener configuración por identificador	Recupera una configuración específica junto con sus características relacionadas.	Media	Baja
RF#45	Listar configuraciones	Lista configuraciones disponibles con soporte de paginación.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#46	Actualizar configuración	Modifica metadatos y características de una configuración existente. Parámetros y campos de entrada: • identificador de configuración (ruta) • nombre (opcional) • descripción (opcional) • lista de identificadores de características (opcional)	Media	Baja
RF#47	Activar configuración	Marca una configuración como activa para su uso.	Media	Baja
RF#48	Desactivar configuración	Permite al sistema eliminar lógicamente una configuración.	Media	Baja
RF#49	Validar configuración propuesta	El sistema debe permitir evaluar una selección de características sin necesidad de persistirla.	Alta	Alta
RF#50	Generar configuraciones válidas	Genera una o varias configuraciones válidas según estrategia definida.	Alta	Alta
RF#51	Optimizar configuraciones	Ejecuta algoritmos de optimización para obtener configuraciones de mejor calidad.	Alta	Alta
RF#52	Listar características	Lista características mediante filtros y representación de árbol por versión.	Alta	Media

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#53	Crear característica	El sistema debe permitir crear nuevas características aplicando validaciones y permisos sobre la versión objetivo del modelo. Campos de entrada: • nombre • tipo • identificador de versión del modelo de características • identificador de padre (opcional) • identificador de grupo (opcional) • identificador de recurso (opcional) • propiedades (opcional)	Alta	Alta
RF#54	Obtener característica	Recupera el detalle completo de una característica específica.	Media	Baja
RF#55	Obtener subárbol de característica	Devuelve una característica junto con toda su descendencia.	Media	Media
RF#56	Consultar relaciones de característica	Lista relaciones en las que participa una característica determinada.	Media	Baja
RF#57	Listar etiquetas de característica	Recupera las etiquetas asociadas a una característica.	Media	Baja
RF#58	Asociar etiqueta con característica	El sistema debe permitir vincular una etiqueta a una característica.	Media	Baja
RF#59	Desasociar etiqueta de una característica	Permite eliminar el vínculo entre una etiqueta y una característica.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#60	Actualizar característica	El sistema debe permitir actualizar campos puntuales de una característica. Parámetros y campos de entrada: • identificador de característica (ruta) • nombre (opcional) • tipo (opcional) • identificador de padre (opcional) • identificador de grupo (opcional)	Alta	Alta
RF#61	Mover característica	El sistema debe permitir reubicar una característica cambiando su nodo padre sin perder trazabilidad.	Alta	Alta
RF#62	Desactivar característica	El sistema debe permitir eliminar lógicamente una característica y generar una nueva versión consistente del modelo.	Alta	Alta
RF#63	Activar característica	El sistema debe permitir recuperar una característica anteriormente desactivada.	Alta	Alta
RF#64	Listar grupos de características	El sistema debe permitir listar grupos de características con filtros por versión, padre y tipo de agrupación.	Media	Baja
RF#65	Obtener grupo de características	Recupera la información de un grupo de características por su identificador.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#66	Crear grupo de características	Crea grupos de características con validaciones para un modelo. Campos de entrada: • tipo de grupo • identificador de característica padre • cardinalidad mínima (opcional, valor por defecto 1) • cardinalidad máxima (opcional)	Alta	Alta
RF#67	Actualizar grupo de características	Actualiza grupos de características conservando consistencia y versionado en el modelo. Parámetros y campos de entrada: • identificador de grupo (ruta) • tipo de grupo (opcional) • identificador de característica padre (opcional) • cardinalidad mínima (opcional) • cardinalidad máxima (opcional)	Alta	Alta
RF#68	Desactivar grupo de características	Elimina lógicamente grupos de características aplicando reglas de integridad y versionado.	Alta	Alta
RF#69	Activar grupo de características	Permitir recuperar grupos de características desactivados.	Alta	Alta
RF#70	Listar relaciones de características	Lista relaciones entre características con criterios de filtrado.	Media	Baja
RF#71	Obtener relación de características	Obtiene el detalle de una relación de características por identificador.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#72	Crear relación de características	El sistema debe permitir crear relaciones entre características con validaciones semánticas y <i>copy-on-write</i> . Campos de entrada: • tipo • identificador de característica origen • identificador de característica destino	Alta	Alta
RF#73	Actualizar relación de características	El sistema debe permitir actualizar relaciones de características existentes. Parámetros y campos de entrada: • identificador de relación (ruta) • tipo (opcional) • identificador de característica origen (opcional) • identificador de característica destino (opcional)	Alta	Alta
RF#74	Activar relaciones de características	Recupera una relación de características anteriormente desactivada.	Alta	Alta
RF#75	Desactivar relaciones de características	Elimina lógicamente una relación de características manteniendo la consistencia global del modelo.	Alta	Alta
RF#76	Listar de restricciones	Lista restricciones aplicables según filtros definidos.	Media	Baja
RF#77	Obtener restricción	Recupera el detalle de una restricción por identificador.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#78	Crear restricción	El sistema debe permitir crear restricciones complejas con validaciones y control de versiones. Campos de entrada: • identificador de versión del modelo de características • expresión lógica • descripción (opcional)	Alta	Alta
RF#79	Actualizar restricción	Actualiza restricciones existentes preservando coherencia lógica. Parámetros y campos de entrada: • identificador de restricción (ruta) • expresión lógica (opcional) • descripción (opcional)	Alta	Alta
RF#80	Desactivar restricción	Elimina restricciones aplicando <i>copy-on-write</i> para mantener trazabilidad.	Alta	Alta
RF#81	Activar restricción	Elimina restricciones aplicando <i>copy-on-write</i> para mantener trazabilidad.	Alta	Alta
RF#82	Listar etiquetas	Lista etiquetas activas disponibles para clasificación funcional.	Media	Baja
RF#83	Obtener etiqueta	Recupera una etiqueta específica mediante su identificador.	Media	Baja
RF#84	Crear etiqueta	Crea nuevas etiquetas asegurando unicidad de nombre. Campos de entrada: • nombre • descripción (opcional)	Media	Baja
RF#85	Listar recursos	Lista recursos registrados con soporte de paginación.	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#86	Consultar recurso	Obtiene el detalle de un recurso específico por su identificador.	Media	Baja
RF#87	Crear recurso	El sistema debe permitir registrar recursos y asigna propietario. Campos de entrada: • título • tipo • URL o contenido del recurso • descripción (opcional) • idioma (opcional) • duración en minutos (opcional) • versión (opcional) • estado (opcional) • fecha de publicación (opcional) • nombre del autor (opcional) • identificador de propietario (opcional) • licencia (opcional) • válido hasta (opcional) • etiquetas (opcional) • notas de accesibilidad (opcional)	Media	Baja

Continúa en la siguiente página

Cuadro 2.3 – Continuación de la página anterior.

No.	Nombre	Descripción	Prioridad	Complejidad
RF#88	Actualizar recurso	El sistema debe permitir actualizar parcial o totalmente la información de un recurso y mantener el último documento guardado para el mismo recurso. Parámetros y campos de entrada: • identificador de recurso (ruta) • título (opcional) • tipo (opcional) • URL o contenido del recurso (opcional) • descripción (opcional) • idioma (opcional) • duración en minutos (opcional) • versión (opcional) • estado (opcional) • fecha de publicación (opcional) • nombre del autor (opcional) • identificador de propietario (opcional, solo superusuario) • licencia (opcional) • válido hasta (opcional) • etiquetas (opcional) • notas de accesibilidad (opcional)	Media	Media

2.2.2. Requisitos no funcionales

Los **Requisitos no Funcionales (RnF)** especifican criterios que permiten juzgar la operación de un sistema, en lugar de sus comportamientos específicos. Estos requisitos, clasificados en categorías como rendimiento, seguridad, fiabilidad, usabilidad, etc; actúan como restricciones o atributos de calidad que el software debe cumplir. A diferencia de los funcionales, que definen qué hace el sistema, los no funcionales establecen cómo debe hacerlo, garantizando propiedades medibles como tiempos de respuesta, capacidad de carga o estándares de protección de datos. Su correcta definición es crítica, pues impactan directamente en la arquitectura del sistema y en la experiencia del usuario final. (serna-montoya2012). De forma alternativa definen las restricciones del sistema (sommerville2005).

A continuación se definen los requisitos no funcionales del sistema:

Seguridad

- **RnF#1:** El sistema debe implementar autenticación basada en tokens **JSON Web Token (JWT)** para garantizar la seguridad de las sesiones de usuario.
- **RnF#2:** El sistema debe implementar un ciclo de vida completo para los tokens **JWT** que incluya operaciones de emisión, renovación e invalidación, garantizando que los tokens tengan un tiempo de expiración limitado y puedan ser revocados explícitamente por el usuario.
- **RnF#3:** El sistema debe implementar un mecanismo de eliminación lógica mediante un cambio de estado (activo/inactivo) para conservar la integridad referencial de la **Base de datos (BD)**.
- **RnF#4:** Uso exclusivo de **UUID** como tipo de dato para llaves primarias en las tablas de la **BD**, no valores numéricos. Su generación se realizará automáticamente a nivel de aplicación o **BD**. Esto garantiza unicidad global y mejora la seguridad al no exponer patrones secuenciales.
- **RnF#5:** Solo usuarios autorizados deben poder crear, modificar y eliminar registros del sistema.
- **RnF#6:** El acceso al sistema a través de cualquier cliente debe ser mediante una comunicación segura **TLS¹/SSL²** (**Protocolo de Transferencia de Hipertexto Seguro (HTTPS)**) para proteger la confidencialidad e integridad de los datos transmitidos.
- **RnF#7:** Toda configuración variable (**BD**, credenciales, **URLs** externas) debe inyectarse vía variables de entorno, no **hardcodeada**.

Rendimiento

- **RnF#8:** El 90 % de las peticiones a *endpoints* críticos deben responderse en ≤ 200 ms, y el 95 % en ≤ 500 ms.
- **RnF#9:** El *backend* debe manejar al menos 1000 usuarios simultáneos sin degradación superior al 20 %.
- **RnF#11:** El sistema debe soportar al menos 100 peticiones por segundo en condiciones de alta demanda y 10 peticiones por segundos en condiciones normales de operación.
- **RnF#10:** La latencia p95 entre capas (repositorios $\rightarrow$ **BD**) no debe superar los 50 ms en la misma región.

Integración

- **RnF#12:** Todas las **APIs** públicas deben seguir **RESTful** con códigos **HTTP** estandarizados y formato **JSON** consistente.
- **RnF#13:** Todas las **APIs** públicas deben incluir la versión en la ruta (**/api/v1**) para cambios rompientes y mantener compatibilidad hacia atrás para parches/pequeñas mejoras.

Mantenibilidad y evolución

- **RnF#14:** Todo *endpoint* debe estar documentado con OpenAPI/Swagger; el código debe seguir una guía de estilo definida.
- **RnF#15:** El código debe estar modularizado para poder actualizarse o reemplazarse sin afectar al resto.
- **RnF#16:** El sistema debe soportar **CI/CD**.

¹Seguridad de la Capa de Transporte (TLS)

²Capa de Sockets Seguros (predecesor de TLS) (SSL)

Disponibilidad y confiabilidad

- **RnF#17**: Ante la caída de una instancia del *backend*, el tráfico debe redirigirse automáticamente a otra sin pérdida de peticiones activas.
- **RnF#18**: El *lifespan* de la aplicación debe garantizar arranque y apagado ordenados: inicializar dependencias, exponer *readiness/liveness*, permitir drenado de peticiones activas, limpiar recursos y permitir reanudación segura de trabajos en otra instancia.
- **RnF#19**: Todos los trabajos en curso (tareas asíncronas, procesos en memoria, sesiones transaccionales) deben poder persistir su estado o reencolarse para permitir reanudación en otra instancia sin pérdida.

Fiabilidad

- **RnF#20**: Los resultados de exportación de los **FM** deben ser consistentes en consultas repetidas.
- **RnF#21**: Las estructuras completas de un **FM** deben ser consistentes en consultas repetidas.

Monitoreo y observabilidad

- **RnF#22**: El sistema debe exponer una ruta pública (chequeo de vida) para comprobar de forma rápida que está en ejecución y respondiendo correctamente.
- **RnF#23**: El sistema debe mostrar el estado detallado de los servicios principales (PostgreSQL, Redis, Celery y MinIO) para saber la operatividad de los mismos, así como las métricas clave del *host* para facilitar el monitoreo.
- **RnF#24**: Todos los *logs* deben ser estructurados en formato **JSON**, con nivel (*debug/info/warn/error*) y *timestamp* ISO.
- **RnF#25**: Se debe alertar automáticamente ante cualquier error del sistema.

Escalabilidad

- **RnF#26**: El sistema debe permitir agregar o quitar instancias del *backend* sin detener el servicio ni perder sesiones activas (escalado horizontal).
- **RnF#27**: En horas pico, el sistema debe auto-escalar de 2 a 10 réplicas en menos de 3 minutos.
- **RnF#28**: La **BD** soportar particionamiento o réplicas de lectura sin cambios en la lógica de negocio.

Portabilidad

- **RnF#29**: El *backend* debe poder ejecutarse idénticamente en desarrollo, *staging* y producción usando Docker/-Kubernetes bajo **Infraestructura como Código (IaC)** reproducible.

2.3. Historias de usuario

Las **HU**, dentro del enfoque **XP**, constituyen una técnica ágil empleada para la recolección y especificación de requisitos funcionales desde la perspectiva del usuario final. Estas se expresan como descripciones breves, usualmente en una o dos frases, utilizando un lenguaje natural no técnico que refleja directamente las necesidades, expectativas o problemas del usuario con relación al sistema. Su propósito es capturar de forma rápida, comprensible y accionable qué funcionalidad requiere el usuario y por qué, sin profundizar en detalles de implementación. (**menzinsky2018historias**)

Las **HU** son escritas para ser implementadas en un tiempo de 1 a 3 semanas como máximo. Cuando una **HU** esté compuesta por diferentes funcionalidades, será dividida en varias **HU**. Algunas no tienen un alto grado de complejidad para implementarlas. Entonces, se determinará que su período de desarrollo sea de 1, 2 o 4 días en un equivalente de 0.2, 0.4 y 0.8 en semanas, respectivamente, para así calcular el tiempo estimado del proyecto. (**beck2000extreme**)

Las **HU** se agrupan y organizan en épicas, las cuales son etiquetas que representan conjuntos de **HU** relacionadas entre sí por un objetivo funcional o temático de mayor envergadura. Una épica no es más que una **HU** de gran tamaño que aún no ha sido desglosada en historias más pequeñas y manejables (**cohn2004user**). Las épicas sirven como mecanismo de organización jerárquica dentro del *backlog* de producto, permitiendo a los equipos ágiles planificar y priorizar a niveles estratégicos sin perder de vista la trazabilidad entre las historias individuales y los grandes bloques de funcionalidad. En el contexto de esta investigación, las épicas constituyen la capa superior de organización de los requisitos funcionales, agrupando las **HU** que contribuyen a un mismo propósito dentro del sistema.

Las épicas definidas para esta investigación son las siguientes:

- **Épica 1: Gestión de Identidad y Acceso:** Agrupa las historias relacionadas con la autenticación, gestión de usuarios y control de acceso al sistema.
- **Épica 2: Gestión de Dominios Curriculares:** Incluye las historias que abordan la administración de dominios como contenedores de características.
- **Épica 3: Ciclo de Vida del FM:** Contiene las historias que tratan sobre la creación, gestión, versionado, métricas, exportación y publicación de los **FM** dentro del sistema.
- **Épica 4: Modelado de Variabilidad Estructural:** Reúne las historias enfocadas en la construcción del árbol (características), grupos y relaciones.
- **Épica 5: Restricciones y Lógica Avanzada:** Agrupa las historias relacionadas con la gestión de restricciones transversales y sincronización con el lenguaje **UVL**.
- **Épica 6: Validación y Análisis de Modelos:** Análisis de consistencia, detección de errores estructurales y características muertas.
- **Épica 7: Configuración, Optimización y Recursos:** Contiene las historias relacionadas con la derivación de productos, validación de selecciones, algoritmos de optimización, etiquetas y recursos pedagógicos.

A continuación, se presenta un ejemplo de las **HU** confeccionadas, el resto se puede encontrar en el anexo **A**.

Cuadro 2.4. Historia de usuario # 1

Historia de usuario	
Número: 1	Nombre: Validación Lógica y Estructural del Modelo
Épica: Validación y Análisis Lógica Avanzada	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 40, 41, 43	
Programador responsable: Jose Luis Echemendia López	
Descripción: El revisador y el diseñador deben poder ejecutar análisis de consistencia, detectar características muertas y redundancias para asegurar que el plan de estudios sea lógicamente posible y no presente errores.	

Continúa en la próxima página

Cuadro 2.4. Continuación de la página anterior.

Criterios de aceptación: • Evaluar la satisfacibilidad global del modelo mediante un solucionador SAT. • Detectar ciclos y nodos huérfanos. • Identificar características muertas y redundancias.
Observaciones: El análisis debe advertir problemas estructurales y semánticos antes de publicar o derivar configuraciones.

2.3.1. Estimación de esfuerzo por historia de usuarios

A lo largo de los años se han llevado a cabo numerosos estudios relacionados con la estimación de esfuerzo en el desarrollo de software ágil, los cuales permiten comprender los avances que han tenido y la necesidad de su uso en las empresas (prieto2021estimacion). La metodología XP propone que la estimación de esfuerzo se realice de acuerdo a las HU. Para la solución informática a desarrollar se asignó una duración aproximada a cada una de las HU las cuales dieron como resultado una duración total aproximada de 13 semanas, como se muestra en la tabla 2.5.

Cuadro 2.5. Estimación de esfuerzo por historia de usuario

Iteración	Historias de usuario		Puntos estimados (semanas)
1	1	Administración de Usuarios y Permisos	0.3
	2	Autenticación y Autogestión de cuenta	0.4
2	3	Gestión de Dominios de Conocimiento	0.4
3	4	Análisis de Métricas y Exportación	0.8
	5	Gestión de Versiones y Publicación	2.5
	6	Gestión Operativa de FM	1.0
4	7	Modelado de Grupos y Relaciones Estructurales	1.0
	8	Edición de la Jerarquía de Características	1.0
5	9	Integración y Sincronización UVL	1.0
	10	Gestión de Restricciones Transversales	0.8
6	11	Validación Lógica y Estructural del Modelo	1.0
7	12	Enriquecimiento con Recursos y Taxonomías	0.8
	13	Validación y Generación Automática de Configuraciones	1.0
	14	Gestión de Configuraciones	1.0
Total			13.0

2.3.2. Desarrollo del plan de iteraciones

Luego de estimar el tiempo de desarrollo de las HU se realizará el plan de iteraciones, mediante este se seleccionan las HU que se desarrollarán en cada una de las etapas de entregas pendientes. La tabla 2.6 muestra el plan de iteraciones con el tiempo empleado en cada una de ellas y las HU que las componen.

Cuadro 2.6. Plan de duración de las iteraciones

Iteración	Historias de usuario		Duración (semanas)
1	1	Administración de Usuarios y Permisos	3
	2	Autenticación y Autogestión de cuenta	
2	3	Gestión de Dominios de Conocimiento	0.4
3	4	Análisis de Métricas y Exportación	4.3
	5	Gestión de Versiones y Publicación	
	6	Gestión Operativa de FM	
4	7	Modelado de Grupos y Relaciones Estructurales	2.0
	8	Edición de la Jerarquía de Características	
5	9	Integración y Sincronización UVL	1.8
	10	Gestión de Restricciones Transversales	
6	11	Validación Lógica y Estructural del Modelo	1.0
7	12	Enriquecimiento con Recursos y Taxonomías	2.8
	13	Validación y Generación Automática de Configuraciones	
	14	Gestión de Configuraciones	
Total			15.3

2.3.3. Desarrollo del plan de entregas

Una vez definido el plan de iteraciones, que organiza el desarrollo en ciclos temporales consecutivos, es necesario establecer el plan de entregas, el cual agrupa las iteraciones en hitos de mayor significación para el cliente o los *stakeholders* del proyecto (**beck2004extreme**). Mientras que las iteraciones representan ciclos internos de desarrollo orientados a la generación incremental de funcionalidad, las entregas constituyen versiones del producto que son puestas a disposición del usuario final, ya sea para su validación, para su uso en entornos de prueba o para su despliegue en producción. En la metodología XP, se recomienda que las entregas sean frecuentes y de corto ciclo, permitiendo así una retroalimentación temprana por parte del cliente y una adaptación continua de los requisitos (**wells2009extreme**). En la tabla 2.7 se presenta el plan de entregas definido para el desarrollo del servicio *backend*, especificando las iteraciones que componen cada entrega, la versión del producto obtenida y las fechas estimadas.

Cuadro 2.7. Plan de entregas.

Versión	Iteración	Fecha de inicio	Fecha de fin
Versión 0.1.1	Iteración 1	20 de diciembre del 2025	9 de enero del 2026
Versión 0.1.2	Iteración 2	10 de enero del 2026	30 de enero del 2026
Versión 0.1.3	Iteración 3	31 de enero del 2026	20 de febrero del 2026
Versión 0.1.4	Iteración 4	21 de febrero del 2026	13 de marzo del 2026
Versión 0.1.5	Iteración 5	14 de marzo del 2026	3 de abril del 2026
Versión 0.1.6	Iteración 6	4 de abril del 2026	24 de abril del 2026
Versión 0.1.7	Iteración 7	25 de abril del 2026	15 de mayo del 2026

2.4. Tarjetas Clase-Responsabilidad-Colaboración (CRC)

Las tarjetas **CRC** son una técnica utilizada en el diseño orientado a objetos para definir clases, asignar responsabilidades y establecer las colaboraciones necesarias entre ellas. Esta herramienta, de naturaleza visual y participativa, involucra activamente a los diseñadores en la exploración de comportamientos e interacciones del sistema. Cada tarjeta representa una clase y resume su nombre, funciones principales y otras clases con las que debe interactuar. Este enfoque promueve el pensamiento en términos de comportamiento más que de estructura de datos, y facilita la comprensión global del sistema desde etapas tempranas del desarrollo. (**paciencia2015metodologias**)

A continuación, se presenta un ejemplo de las tarjetas **CRC** correspondientes a clases del sistema desarrollado, el resto se puede encontrar en el anexo **B**.

Cuadro 2.8. Tarjeta CRC # 1

Tarjeta CRC	
Clase: FeatureModel	
Responsabilidad	Colaboración
• Representar un FM (nombre, descripción, dominio). • Enlazar propietario y versiones. • Mantener colaboradores.	Domain User FeatureModelVersion FeatureModelCollaborator

2.5. Arquitectura de software

La arquitectura de software constituye el conjunto de decisiones de diseño fundamentales que definen la estructura global de un sistema informático. Comprende la identificación y organización de los componentes que lo conforman, las relaciones de comunicación y dependencia entre ellos, así como los principios y patrones que guían su evolución a lo largo del tiempo (**bass2012software**). Más allá de un mero diagrama de bloques, la arquitectura representa el

puente entre los requisitos funcionales y no funcionales del sistema y su implementación concreta, estableciendo las bases para alcanzar atributos de calidad como la escalabilidad, el mantenimiento, la seguridad, el rendimiento y la interoperabilidad (**clements2010documenting**).

La importancia de definir adecuadamente la arquitectura de software radica en que esta determina, en gran medida, el éxito o fracaso del proyecto a largo plazo. Una arquitectura bien concebida permite gestionar la complejidad del sistema, facilita la comunicación entre los miembros del equipo de desarrollo, reduce los riesgos asociados a decisiones técnicas críticas y posibilita la reutilización de componentes en futuros proyectos (**rojas2021arquitectura**). Por el contrario, una arquitectura deficiente o implícita puede derivar en sistemas difíciles de mantener, propensos a errores y costosos de extender. En el contexto del desarrollo ágil, la arquitectura no se define completamente al inicio, sino que emerge y se refina iterativamente a lo largo del proyecto, manteniendo un equilibrio entre la planificación anticipada y la adaptabilidad frente a requisitos cambiantes (**fowler2003design**).

En esta sección se justifica la elección del patrón específico adoptado para este proyecto. Además, se explica cómo su estructura se alinea con la naturaleza de la aplicación y facilita el cumplimiento de los objetivos de diseño planteados.

2.5.1. Representación arquitectónica de la solución

La arquitectura en capas, también conocida como arquitectura n-capas o arquitectura multitier, es un patrón arquitectónico que organiza los componentes de un sistema en niveles horizontales, donde cada capa cumple una función específica y se comunica únicamente con las capas adyacentes (**bass2012software**). Este enfoque sigue el principio de separación de preocupaciones, promoviendo la cohesión interna de cada capa y reduciendo el acoplamiento entre ellas (**gamma1995design**). Como resultado, las modificaciones en una capa tienen un impacto limitado sobre las demás, lo que facilita el mantenimiento, las pruebas y la evolución del sistema.

La arquitectura en capas ofrece múltiples beneficios que la convierten en una elección adecuada para la solución propuesta (**rojas2021arquitectura**):

- Separación de responsabilidades: Cada capa se especializa en una función específica (presentación, lógica de negocio, acceso a datos), lo que facilita la comprensión del sistema y la asignación de tareas de desarrollo.
- Mantenibilidad: Los cambios en una capa (por ejemplo, migrar de **BD**) no afectan necesariamente a las demás, siempre que se respeten las interfaces de comunicación.
- Reutilizabilidad: Las capas inferiores (como la de persistencia) pueden ser reutilizadas por múltiples aplicaciones que requieran las mismas funcionalidades.
- Testabilidad: Cada capa puede ser probada de forma independiente mediante pruebas unitarias o de integración, facilitando la detección temprana de errores.
- Escalabilidad: Es posible escalar capas específicas según la demanda (por ejemplo, replicar la capa de servicios sin replicar la **BD**).

Capas definidas para la propuesta de solución

La solución se implementa como un monolito modular estructurado en capas, que combina principios de la arquitectura en capas con una organización orientada al dominio y mecanismos de procesamiento asíncrono. En cuanto al código, hay tres capas bien diferenciadas, cada una con responsabilidades claramente definidas. Además, se cuenta con una cuarta capa que encapsula los servicios externos de infraestructura que necesita el sistema para su correcto

funcionamiento. La figura 2.2 ilustra la organización general de la arquitectura, el flujo de comunicación entre las capas y algunos componentes clave dentro de cada una de ellas.

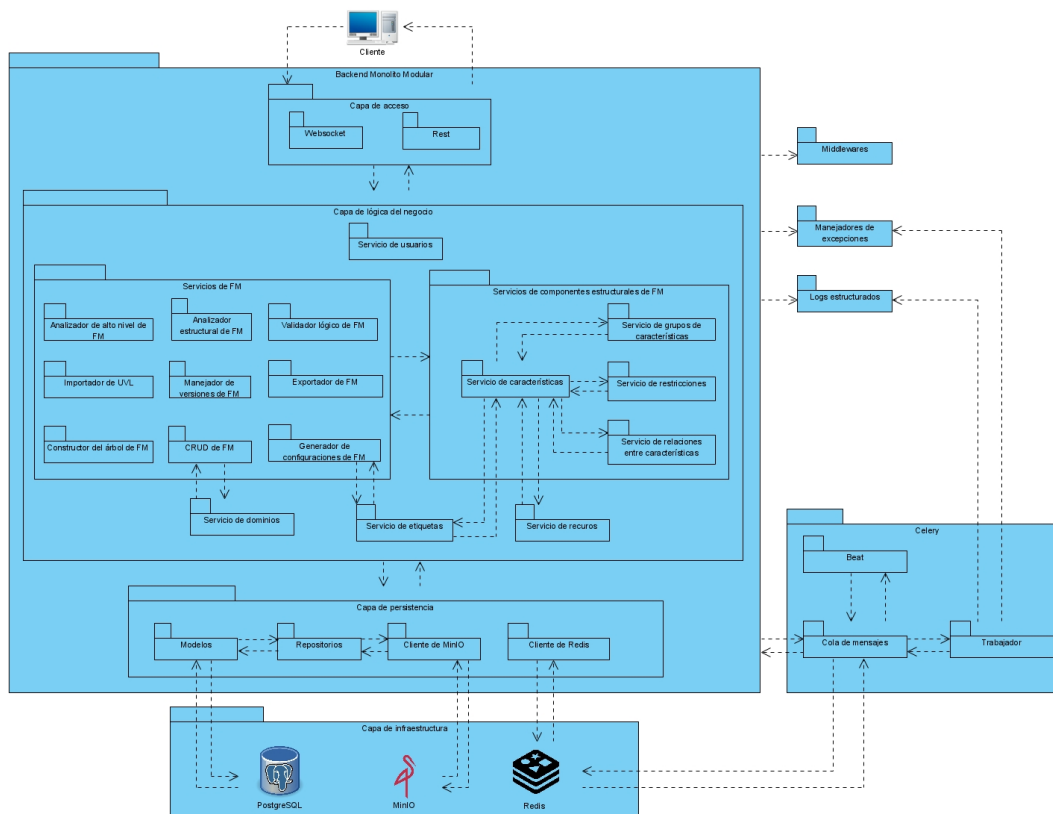


Figura 2.2. Arquitectura en capas de la propuesta de solución. Fuente: elaboración propia.

A continuación, se describen cada una de las capas que componen la arquitectura:

- **Capa de acceso (controladores):** Constituye el punto de entrada al sistema. Expone los *endpoints* de la [API](#) que reciben las peticiones de los clientes. Es responsable de:
 - Recibir y validar las solicitudes [HTTP](#).
 - Autenticar y autorizar a los usuarios mediante los mecanismos de control de acceso.
 - Enrutar las peticiones hacia los servicios correspondientes en la capa de lógica de negocio.
 - Formatear las respuestas (generalmente en [JSON](#)) y manejar los códigos de estado [HTTP](#).
 - Brindar una comunicación en tiempo real para obtener información estadística de los [FM](#).
- **Capa de lógica de negocio (servicios):** Constituye el núcleo funcional del sistema. Contiene toda la lógica relacionada con la gestión de [FM](#), la variabilidad curricular y elementos complementarios. Esta capa está estructurada siguiendo principios de [Diseño Orientado al Dominio \(DDD\)](#), organizando la lógica en servicios especializados que encapsulan reglas complejas del sistema de [FM](#) y de los componentes estructurales del mismo. Sus responsabilidades incluyen:
 - Implementar las operaciones [CRUD](#) de [FM](#).
 - Gestionar las relaciones estructurales y las restricciones de los [FM](#).

- Gestionar el versionamiento de los modelos.
 - Ejecutar los análisis automatizados sobre los modelos (validación de consistencia, detección de características muertas, conteo de configuraciones, etc).
 - Aplicar las reglas de negocio específicas del dominio educativo (prerrequisitos, correlatividades, límites de créditos, cantidad de horas clases, etc).
 - Orquestrar la generación de configuraciones válidas a partir de los modelos.
 - Exportación de **FM** e importación de **UVL**.
 - Gestión de recursos, etiquetas, dominios y usuarios.
- **Capa de persistencia (repositorios y clientes):** Se encarga de gestionar todo el almacenamiento de datos del sistema, ya sea de larga duración (**BD**, almacenamiento de archivos) o de corta duración (caché, sesiones). Su objetivo es abstraer a las capas superiores (lógica de negocio y acceso) de los detalles técnicos de dónde y cómo se guarda cada tipo de información al proporcionar una interfaz uniforme para las operaciones de acceso a datos. Sus funciones son:
 - Gestionar el almacenamiento de los **FM** en su representación interna.
 - Almacenar las configuraciones generadas y los metadatos asociados (usuario propietario, fechas, dominios).
 - Gestionar las entidades referentes a etiquetas, recursos, dominios, usuarios y las correspondientes al núcleo del sistema.
 - Almacenar archivos de exportación de **FM**.
 - Almacenar recursos educativos (conferencias, clases prácticas, etc) asociados a las características.
 - Gestionar la memoria caché.
- **Capa de infraestructura (**BD**, **MinIO**, **Redis**):** Constituye el nivel más bajo de la arquitectura del sistema. Es la responsable de proporcionar todos los componentes tecnológicos subyacentes que hacen posible el almacenamiento, la recuperación y el procesamiento eficiente de los datos. Esta capa es completamente transparente para las capas superiores (acceso, lógica de negocio y persistencia), las cuales interactúan con ella únicamente a través de los repositorios y clientes definidos en la capa de persistencia. Sus responsabilidades son:
 - Almacenar físicamente los datos en estructuras optimizadas (tablas, índices).
 - Garantizar la integridad referencial y la consistencia de los datos.
 - Ejecutar consultas eficientes sobre grandes volúmenes de información.
 - Gestionar transacciones y asegurar la durabilidad de los datos.
 - Almacenar físicamente los archivos de exportación de **FM** y recursos educativos.
 - Descargar objetos mediante **URLs** prefirmadas.
 - Organizar los objetos en *buckets*.
 - Guardar respaldos significativos de la **BD**.
 - Cachear consultas frecuentes y resultados de operaciones de alto costo computacional para optimizar el rendimiento y reducir la latencia del sistema.

Flujo de comunicación entre capas

La comunicación entre capas sigue una dirección descendente: la capa de acceso recibe la petición del cliente y la delega a la capa de lógica de negocio; esta, a su vez, solicita los datos necesarios a la capa de persistencia; finalmente,

la capa de persistencia interactúa con la infraestructura. La respuesta sigue el camino inverso, ascendiendo desde el nivel más bajo hasta el cliente.

Este flujo unidireccional garantiza que no existan dependencias circulares, lo que simplifica el mantenimiento y facilita la sustitución de implementaciones en cada nivel. Por ejemplo, es posible cambiar el [Sistema de Gestión de Bases de Datos \(SGBD\)](#) (de PostgreSQL a MySQL) sin modificar la capa de lógica de negocio, siempre que la capa de persistencia mantenga la misma interfaz.

Arquitectura de Celery

Para atender los requisitos de procesamiento asíncrono y ejecución de tareas de larga duración (como la validación de [FM](#) complejos, el conteo de configuraciones mediante solucionadores [SAT](#) o la generación masiva de planes de estudio, entre otras tareas), la solución incorpora un sistema de colas de mensajes basado en Celery. Celery es un sistema de procesamiento de tareas distribuidas y asíncronas, ampliamente utilizado en aplicaciones Python, que permite descargar operaciones costosas del flujo principal de la aplicación, mejorando así la capacidad de respuesta del servicio y la experiencia del usuario ([celery2025docs](#)). La arquitectura de Celery presenta cuatro componentes fundamentales que interactúan entre sí para garantizar la ejecución confiable y distribuida de las tareas, tal como se ilustra en la figura 2.3.

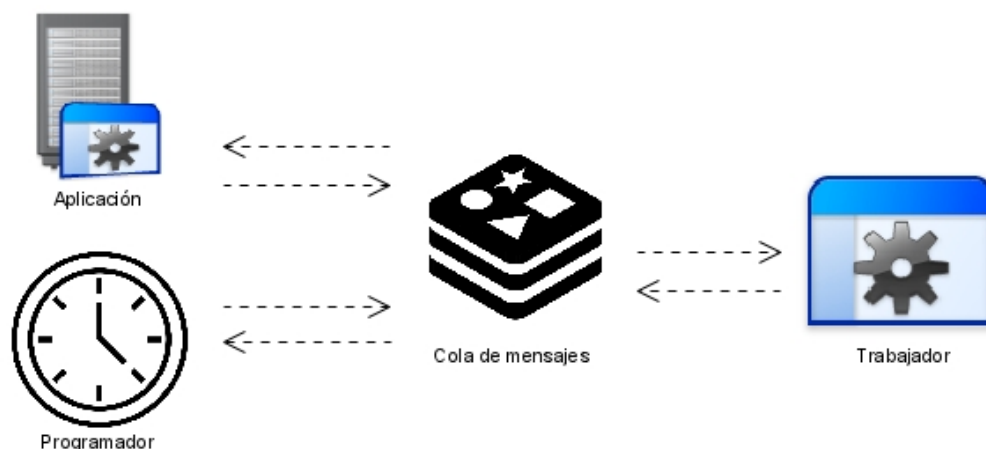


Figura 2.3. Arquitectura de Celery. Fuente: elaboración propia.

A continuación, se describen cada uno de estos componentes:

- **Aplicación:** Constituye el punto de entrada principal para la definición y configuración de las tareas. Es el componente que se integra directamente con el código del servicio *backend*, permitiendo invocar funciones como tareas asíncronas. La aplicación define qué tareas están disponibles, cómo deben serializarse los argumentos y qué cola de mensajes utilizar para la comunicación ([celery2025firststeps](#)).
- **Cola de mensajes:** Actúa como intermediario entre la aplicación que publica tareas y los trabajadores que las ejecutan. Cuando la aplicación invoca una tarea asíncrona, esta se empaqueta como un mensaje y se publica en la cola. Es responsable de almacenar temporalmente estos mensajes y distribuirlos a los trabajadores disponibles. En esta solución se emplea Redis como cola de mensajes, aprovechando su alto rendimiento y su capacidad para gestionar grandes volúmenes de mensajes ([celery2025redis](#)).

- **Trabajador:** Es el proceso que consume los mensajes de la cola y ejecuta las tareas definidas en la aplicación. Los trabajadores operan de forma independiente y pueden ejecutarse en uno o múltiples nodos, permitiendo escalar horizontalmente el procesamiento de tareas según la demanda del sistema. Cada trabajador puede gestionar múltiples tareas en paralelo mediante el uso de procesos hijos (**celery2025workers**).
- **Programador:** Es un componente especializado que actúa como planificador de tareas periódicas. Genera tareas de forma automática en intervalos de tiempo predefinidos (por ejemplo, cada hora, cada día) y las publica en la cola de mensajes para que sean ejecutadas por los trabajadores. En el contexto del servicio *backend*, este componente resulta útil para tareas como la limpieza periódica de modelos temporales, el envío de notificaciones programadas o la ejecución de análisis recurrentes sobre los **FM** (**celery2025beat**).

La integración de Celery en la arquitectura del servicio *backend* aporta múltiples ventajas al sistema:

- **Mejora de la experiencia de usuario:** Las operaciones costosas (como la validación de un **FM** con cientos de características) se ejecutan en segundo plano, devolviendo al cliente una respuesta inmediata con el identificador de la tarea. El cliente puede consultar posteriormente el estado y los resultados de la tarea sin bloquear la interfaz.
- **Escalabilidad horizontal:** Al aumentar la carga de trabajo, es posible añadir más trabajadores (incluso en servidores distintos) para procesar las tareas en paralelo, sin modificar el código de la aplicación principal.
- **Resiliencia:** Si un trabajador falla durante la ejecución de una tarea, Celery puede reencolar automáticamente la tarea para que sea ejecutada por otro trabajador, garantizando la finalización de las operaciones críticas.
- **Desacoplamiento:** La lógica de negocio que invoca tareas asíncronas no necesita conocer los detalles de cómo, cuándo o dónde se ejecutan, lo que reduce el acoplamiento entre componentes.

Ventajas de la arquitectura para la propuesta de solución

La elección de la arquitectura responde directamente a las necesidades identificadas en el análisis de requisitos. Esta representación arquitectónica permite abordar de manera efectiva los desafíos específicos, tales como:

- **Integración con sistemas existentes:** La capa de acceso permite que sistemas como los mencionados en el estudio de homólogos u otros consuman las funcionalidades del *backend* de variabilidad curricular sin necesidad de modificaciones profundas.
- **Evolución del modelo curricular:** La separación entre la lógica de negocio y la persistencia facilita la implementación del versionamiento de modelos, permitiendo almacenar múltiples versiones históricas sin afectar las operaciones actuales.
- **Seguridad y auditoría:** La capa de lógica de negocio centraliza el control de acceso y la validación de permisos, garantizando que solo usuarios autorizados puedan modificar los modelos.
- **Reutilización de componentes:** La capa de persistencia puede ser reutilizada en futuros proyectos que requieran almacenar **FM**, mientras que la capa de lógica de negocio encapsula las reglas de análisis de variabilidad.
- **Rendimiento y escalabilidad:** La incorporación de un sistema de procesamiento asíncrono basado en Celery, junto con una estrategia de caché implementada sobre Redis, permite que la solución maneje eficientemente cargas de trabajo intensivas en cómputo sin degradar la experiencia del usuario.

2.6. Patrones de diseños

Mientras el patrón de arquitectura define la macro-estructura del sistema, los patrones de diseño ofrecen soluciones a problemas de diseño recurrentes a un nivel de granularidad menor, específicamente en el diseño de componentes y sus interacciones. La aplicación consciente de estos patrones es crucial para lograr un código flexible, reusable y fácil de mantener ([aratchige2024overview](#)). Esta sección se divide en dos sub-apartados para abordar las dos categorías principales de patrones utilizadas en el desarrollo de la plataforma.

2.6.1. Patrones GRASP

Los patrones [GRASP](#) constituyen un conjunto de principios y pautas fundamentales para la asignación de responsabilidades en el diseño orientado a objetos ([larman2004applying](#)). A diferencia de los patrones de diseño concretos, los [GRASP](#) no ofrecen soluciones específicas codificadas, sino que proporcionan un marco conceptual que guía a los diseñadores en la toma de decisiones sobre qué clase debe ser responsable de qué tarea o información dentro del sistema ([begel2020grasp](#)). Su propósito es lograr un diseño con alta cohesión, bajo acoplamiento y una clara separación de preocupaciones, atributos esenciales para la mantenibilidad y extensibilidad del software. A continuación se muestran los patrones [GRASP](#) aplicados en el diseño de la propuesta de solución y sus usos específicos:

- Repository (Repositorio): Aísla el acceso a datos detrás de una interfaz/repositorio, separando persistencia de lógica de negocio.
 - Repositorio base y concretos para cada tabla de la [BD](#), con métodos [CRUD](#) genéricos y específicos.
 - Inyección de repositorios vía dependencias.
- Service Layer (Capa de Servicios): Encapsula lógica de negocio en servicios de aplicación, evitando que los controladores conozcan detalles de dominio/persistencia.
 - Lógica de negocio referente a los servicios de los [FM](#).
- Controller (Controlador): Centraliza la gestión de eventos y solicitudes, delegando a servicios para procesar la lógica.
 - Presente en los *routers* de la capa de presentación, recibiendo peticiones HTTP, inyectando dependencias, validando permisos y delegando a servicios/repositorios.
- Low Coupling (Bajo Acoplamiento): Minimiza las dependencias entre componentes, facilitando el mantenimiento y la evolución del sistema.
 - En la creación de las dependencias del sistema, las cuales son inyectadas en los controladores, repositorios y servicios.
 - Sesión de [BD](#) inyectada en los repositorios, evitando que la lógica de negocio tenga conocimiento de la infraestructura de persistencia.
- High Cohesion (Alta Cohesión): Asegura que cada componente tenga una única responsabilidad bien definida, mejorando la claridad y mantenibilidad del código.
 - Servicio enfocado en el versionado de los [FM](#).
 - Servicio enfocado en la construcción del árbol de los [FM](#).
 - Repositorios bases para cada tabla de la [BD](#), con métodos específicos para cada entidad que contienen métodos de validación y soporte.

2.6.2. Patrones GoF

Los patrones de diseño GoF son soluciones clásicas y ampliamente reconocidas para problemas específicos de diseño en software orientado a objetos. **gamma1995design** subdividieron esta categoría de patrones en tres grupos según su propósito. En este sub-apartado se detallan los patrones GoF específicos que se han empleado en el proyecto, explicando el problema que resuelve cada uno en el contexto de la aplicación.

Patrones creacionales

Los patrones creacionales abstractizan el proceso de instanciación de objetos, encapsulando el conocimiento sobre qué clases concretas crear y cómo se relacionan entre sí (**gamma1995design**). Su objetivo principal es hacer que el sistema sea independiente de cómo se crean, componen y representan sus objetos, favoreciendo la reutilización del código y reduciendo el acoplamiento entre las clases que utilizan los objetos y las clases que los instancian (**seguridad2022patrones**).

- Singleton (Instancia Única): Controla una única instancia compartida para acceso global.
 - Empleado para instanciar únicamente los clientes de MinIO, Redis y Celery, evitando múltiples conexiones y garantizando un punto de acceso centralizado a estos servicios.
 - Instancia global para la configuración del sistema, permitiendo que todas las partes del código accedan a los mismos parámetros de configuración sin necesidad de pasarlos explícitamente.
- Factory Method (Método de Fábrica): Delega la creación de objetos a métodos especializados para desacoplar construcción de uso.
 - Empleado para instanciar componentes reutilizables y servicios de infraestructura bajo una política de creación centralizada.
 - Evita la duplicación de lógica de inicialización y reduce el acoplamiento entre consumidores e implementaciones concretas.
- Builder (Constructor): Separa la construcción de un objeto complejo de su representación final, permitiendo crear diferentes versiones del mismo objeto usando el mismo proceso de construcción paso a paso.
 - Empleado en la construcción del árbol completo del FM.

Patrones estructurales

Los patrones estructurales se centran en la composición de clases y objetos para formar estructuras más grandes y flexibles, manteniendo la eficiencia y la independencia de los componentes involucrados (**gamma1995design**). Estos patrones permiten resolver problemas relacionados con la interfaz que exponen los objetos, la manera en que se agregan responsabilidades adicionales y cómo se adaptan componentes con interfaces incompatibles para que puedan colaborar (**refactoring2021patrones**).

- Facade (Fachada): Proporciona una interfaz simplificada y unificada a un conjunto de interfaces más complejas dentro de un subsistema.
 - Fachada de análisis que combina validación lógica, estructural y manejo de UVL con el sistema homólogo FlamapyIDE.

- *Decorator* (Decorador): Permite añadir responsabilidades adicionales a un objeto de forma dinámica, sin modificar su estructura interna ni afectar a otros objetos de la misma clase.
 - Decoradores de caché y ruteo para los controladores.
 - Decoradores para registrar tareas en Celery.

Patrones de comportamiento

Los patrones de comportamiento gestionan la comunicación y asignación de responsabilidades entre objetos, encapsulando la lógica de interacción y distribución de tareas dentro del sistema (**gamma1995design**). Su propósito es elevar el nivel de abstracción en el que se describe el flujo de control, permitiendo que los objetos se comuniquen de manera flexible y desacoplada, sin necesidad de conocer explícitamente las implementaciones concretas de sus colaboradores (**seguridad2022patrones**).

- *Strategy* (Estrategia): Define una familia de algoritmos intercambiables, los encapsula individualmente y permite que el algoritmo varíe de forma independiente a los clientes que lo usan.
 - Generación/optimización de configuraciones con estrategias.
 - Validación lógica multi-motor (*sympy/pysat/z3*) como estrategia interna.
- *State* (Estado): Permite que un objeto modifique su comportamiento cuando su estado interno cambia. El objeto parecerá cambiar su clase.
 - Transiciones de estado para los recursos y los **FM** (ambos tienen los siguientes estados: borrador, en revisión, publicado, archivado).

2.7. Conclusiones del capítulo

Luego del desarrollo de este capítulo se arribaron a las siguientes conclusiones:

- La solución cuenta con 88 **RF** encapsulados en 14 **HU**, organizadas en 7 iteraciones con un tiempo total estimado de desarrollo de 15 semanas aproximadamente. Se definieron 29 **RnF** que abarcan categorías de rendimiento, seguridad, usabilidad, integración, mantenibilidad, disponibilidad, fiabilidad, monitoreo, escalabilidad y portabilidad.
- El diseño del sistema responsable de propiciar una base sólida para la implementación de la aplicación se describe a través de una arquitectura en capas, apoyada con una arquitectura de Celery para potenciar el rendimiento.
- La aplicación de los principios **GRASP** y los patrones **GoF** eleva significativamente la mantenibilidad, flexibilidad y escalabilidad de la solución propuesta, facilitando la modularidad del código y la adaptación a futuros cambios en los requisitos.

Implementación, despliegue y pruebas

El presente capítulo materializa los esfuerzos de análisis y diseño expuestos en los capítulos anteriores, trasladando las especificaciones técnicas a una implementación concreta y funcional. Aquí se detallan las **Tareas de Ingenierías (TI)** ejecutadas para construir el servidor *backend*, el diagrama de despliegue que ilustra la distribución física de los componentes del sistema en el entorno de ejecución, así como el proceso de pruebas al que fue sometido el sistema para garantizar su calidad, rendimiento, seguridad y conformidad con los requisitos establecidos. La verificación y validación sistemáticas mediante diversas estrategias y tipos de pruebas son fundamentales para asegurar que la solución desarrollada satisface las necesidades del cliente final y se encuentre lista para su despliegue en un entorno productivo.

3.1. Fase de implementación

Dentro del ciclo de vida de un sistema informático se encuentra la fase de implementación. Esta es la fase más costosa y que consume más tiempo. Se dice costosa ya que muchas personas, herramientas y recursos están involucrados. La metodología **XP** propone que las historias de usuario deben ser implementadas en dependencia de la iteración a la que pertenezcan y las mismas se descomponen en tareas de ingeniería (**maida2015**).

3.1.1. Tareas de ingeniería

Las **TI** constituyen un artefacto fundamental dentro de la metodología **XP**. Representan la descomposición de las **HU** en actividades técnicas concretas y accionables por parte de los desarrolladores. Mientras que una **HU** describe una funcionalidad desde la perspectiva del cliente, una **TI** detalla el “cómo” se implementará a nivel técnico (**pressman2014se**). En esta sección se enumeran y describen estas tareas, que guiaron el trabajo de codificación diario durante las iteraciones, permitiendo una planificación granular y una asignación clara de responsabilidades dentro del equipo de desarrollo.

A continuación, se presenta un ejemplo de las **TI** correspondientes a las **HU** definidas, el resto se puede encontrar en el anexo **C**.

Cuadro 3.1. Tarea de ingeniería # 1

Tarea	
Número de tarea: 1	Número de Historia de usuario: 4
Nombre de la tarea: Exponer endpoints de ciclo operativo de FM	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 14 de mayo de 2026	Fecha de fin: 15 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de modelos de características para alta, consulta, actualización y activación/desactivación, con soporte de filtros por dominio, paginación y políticas de autorización según rol funcional.	

3.2. Despliegue del sistema

El despliegue constituye la fase del ciclo de vida del software en la que el sistema desarrollado se pone a disposición de los usuarios finales en un entorno de ejecución real o de pruebas. Esta etapa implica no solo la instalación y configuración de los componentes de software en los servidores correspondientes, sino también la verificación de que todas las dependencias tecnológicas (BD, sistemas de almacenamiento, colas de mensajes, etc.) se encuentren operativas y correctamente parametrizadas (bass2012software). Un despliegue exitoso garantiza que el sistema se comporte en producción de acuerdo con las expectativas definidas durante las primeras fases del desarrollo de software, y constituye el paso previo indispensable para la ejecución de las pruebas de aceptación por parte del cliente.

3.2.1. Diagrama de despliegue

El diagrama de despliegue es un artefacto de modelado utilizado en el UML para representar la arquitectura física de un sistema, mostrando la disposición de los nodos de hardware, los componentes de software que se ejecutan en cada uno de ellos y los canales de comunicación que los interconectan (booch2005uml). Su propósito es visualizar la topología de ejecución del sistema, identificar posibles cuellos de botella en la comunicación entre nodos y planificar la distribución de los componentes en entornos productivos. La figura 3.1 ilustra el diagrama de despliegue correspondiente a la propuesta de solución desarrollada.

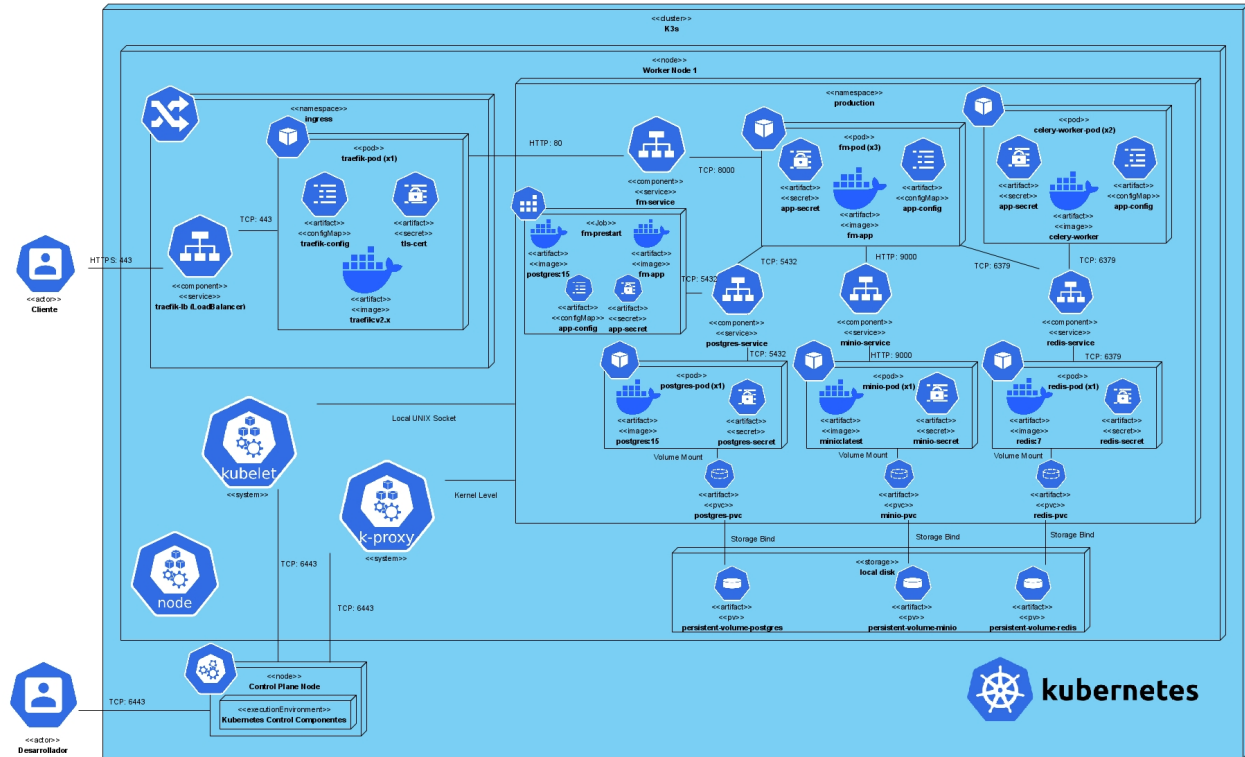


Figura 3.1. Diagrama de despliegue. Fuente: Elaboración propia.

Cliente y desarrollador

El actor desarrollador es un usuario avanzado que interactúa directamente con el clúster a través del Nodo de Plano de Control para gestionar las aplicaciones. Su rol es fundamental para la operación del sistema, pero no para el tráfico de usuarios finales. Entre sus principales acciones se tiene:

ejemplo, una nueva versión del *backend* FastAPI o de Redis) mediante [actualizaciones de rodillo](#) o recreando los [Pods](#).

Clúster K3s

El sistema se despliega sobre un [Clúster K3s](#), una distribución certificada de Kubernetes de código abierto, ligera y certificada, especialmente diseñada para entornos de borde y despliegues con recursos limitados. El [Clúster](#) está compuesto por un [nodo](#) de [plano de control](#) y múltiples [nodos](#) trabajadores. El plano de control aloja los componentes de orquestación de Kubernetes, mientras que los [nodos](#) trabajadores ejecutan las cargas de trabajo del sistema. Esta arquitectura permite escalar horizontalmente el sistema añadiendo más [nodos](#) trabajadores según los requisitos de carga y disponibilidad.

Nodo de Plano de Control

El diagrama muestra un [nodo](#) dedicado exclusivamente a la gestión del clúster. Este [nodo](#) aloja los componentes de control de Kubernetes, que son el cerebro del orquestador. A diferencia del [nodo](#) trabajador, este [nodo](#) no ejecuta cargas de trabajo de la aplicación (como FastAPI o PostgreSQL), sino que se encarga de la administración global del estado del clúster.

Los componentes clave que residen en este [nodo](#) son:

- [kube-apiserver](#): Es la puerta de entrada al clúster. Todas las operaciones de gestión (crear, eliminar, modificar recursos) pasan por él. Tanto el [kubelet](#) de los [nodos](#) trabajadores como el [Kubect](#)l del desarrollador se comunican con él a través del puerto 6443.
- [etcd](#): [BD](#) clave-valor (no visible explícitamente en el diagrama pero implícita en los componentes de control) que almacena el estado completo del clúster. Es la única fuente de verdad.
- [kube-scheduler](#): Decide en qué [nodo](#) trabajador se desplegará un nuevo [Pod](#), basándose en los recursos disponibles y las restricciones (recursos, afinidad, etc.).
- [kube-controller-manager](#): Ejecuta controladores que gestionan el estado del clúster (ej. si un [Pod](#) falla y debe ser reemplazado).

Nodo trabajador. Componentes del sistema

Dentro del [nodo](#) trabajador, los componentes del sistema se organizan en dos [Espacio de nombres](#) lógicos: [Ingress](#) y producción; además, se tiene los elementos del sistema [kubelet](#) y [kube-proxy](#). Estas piezas tienen como responsabilidades:

- [Espacio de nombres Ingress](#): Aloja el controlador de entrada [Traefik](#), que actúa como punto único de entrada para todo el tráfico externo hacia el clúster. [Traefik](#) funciona como un balanceador de carga y enrutador de capa 7, que recibe las peticiones [HTTPS](#) del cliente y las dirige al servicio correspondiente dentro del clúster. Adicionalmente, gestiona los certificados [TLS](#) para la terminación de las conexiones seguras, simplificando la configuración de seguridad.
- [Espacio de nombres](#) producción: Contiene todos los [Servicios](#) y cápsulas ([Pods](#)) que implementan la funcionalidad del sistema. Se organiza como una arquitectura de microservicios donde cada componente se ejecuta en su propia cápsula o en réplicas distribuidas. La comunicación interna entre estas cápsulas se realiza a través de

servicios internos tipo **ClusterIP**, que proporcionan direcciones **IP** virtuales estables y equilibran la carga entre las réplicas disponibles.

- **kubelet**: es el agente de Kubernetes que se ejecuta en cada **nodo** trabajador y tiene como tareas:
 - Escuchar las instrucciones del Plano de Control.
 - Asegurarse de que los **Pods** (contenedores) estén ejecutándose realmente según lo definido.
 - Reportar el estado del **nodo**, los **Pods** y el uso de recursos al Plano de Control.
 - Gestiona los **Volumenes persistentes** (monta los **Volumenes persistentes** en los **Pods**).
- **kube-proxy**: encargado de implementar las reglas de red en el **nodo** trabajador. Es responsable de que el tráfico enviado al servicio **fm-service** (**ClusterIP**) sea correctamente redirigido a uno de los **Pods** de FastAPI. Sin **kube-proxy**, los **Servicios ClusterIP** no funcionarían.

Componentes del despliegue

El diagrama de despliegue evidencia cómo se distribuyen los componentes del sistema en el **Clúster K3s**. Ninguno de los componentes descritos a continuación se despliega como una única instancia aislada, sino que se integran mediante servicios internos que proporcionan direcciones de red estables y balanceo de carga.

- Servicio FastAPI y réplicas: El *backend* desarrollado con FastAPI se despliega mediante tres réplicas idénticas (**fm-pod**), distribuidas entre los **nodos** trabajadores del clúster. Estas réplicas se agrupan tras el **Servicio** interno **fm-service** de tipo **ClusterIP**, que expone una dirección **IP** virtual y distribuye el tráfico entrante entre las tres instancias mediante balanceo (*round-robin*). Esta disposición garantiza alta disponibilidad (si una réplica falla, el servicio sigue operativo) y escalabilidad horizontal (el número de réplicas puede ajustarse dinámicamente según la demanda).
- Trabajadores Celery: Dos réplicas de trabajadores Celery operan de forma independiente al servicio FastAPI, procesando las tareas asíncronas de larga duración. Se comunican con el servicio **redis-service** (tipo **ClusterIP**), que les proporciona un punto de acceso estable al **Pod** de Redis, actuando como intermediario de mensajes y almacén de resultados.
- Inicialización del sistema: Para garantizar que el entorno de **BD** esté completamente preparado antes de que el servidor comience a operar, se ha definido un recurso de **trabajo** de Kubernetes llamado **fm-prestart**. Este **trabajo** se ejecuta una única vez durante el despliegue inicial (o cuando se fuerza su recreación) y es responsable de orquestar la configuración inicial de la **BD** PostgreSQL. Entre las principales acciones a realizar por el **trabajo** están:
 - Verificar si la **BD** ya existe en el servidor PostgreSQL. Si no existe, la crea automáticamente.
 - Garantizar que PostgreSQL acepte conexiones antes de continuar.
 - Generar la documentación técnica del proyecto. Este paso es no crítico (si falla, el **trabajo** continúa).
 - Sembrar datos iniciales en la **BD**.
- **Servicios** de infraestructura: Tres **Servicios** internos (**postgres-service**, **redis-service** y **minio-service**) exponen respectivamente los **Pods** de PostgreSQL, Redis y MinIO. Cada uno se despliega como un **Pod** individual con su correspondiente servicio de tipo **ClusterIP**, lo que permite al *backend* y a los trabajadores Celery comunicarse con ellos sin conocer sus direcciones **IP** concretas. Los datos de estos tres componentes se respaldan en **Volumenes persistentes**, desacoplados de los contenedores mediante reclamaciones (**Reclamación de volumen persistente**) que garantizan su supervivencia más allá del ciclo de vida de los **Pods**.

- **ConfigMaps**: En el diagrama se ven `app-config` (para FastAPI), `postgres-config`, etc. Los **ConfigMaps** se usan para inyectar configuración no confidencial (variables de entorno, archivos de configuración) en los **Pods** sin tener que reconstruir la imagen Docker. Esto permite modificar la configuración sin reiniciar los contenedores.
- **Secretos**: Se ven `app-secret`, `postgres-secret`, `redis-secret`, `minio-secret`. Los **Secretos** son objetos de Kubernetes para almacenar datos sensibles (contraseñas de bases de datos, tokens, claves de **API**). Se inyectan en los **Pods** de forma segura (no en texto plano en los manifiestos **YAML**).

Todos los **Servicios** internos del **Clúster** son del tipo **ClusterIP**, lo que significa que solo son accesibles desde dentro del **Clúster**. El único punto de entrada externo es el **Servicio** balanceador `traefik-lb` (tipo **balanceador de carga**), que recibe las peticiones **HTTPS** del cliente y las enruta al **Servicio** FastAPI a través del controlador de entrada, manteniendo el resto de la infraestructura aislada de accesos externos directos.

Volúmenes persistentes y almacenamiento

Cada uno de los **Servicios** de infraestructura (PostgreSQL, Redis y MinIO) requiere almacenamiento persistente para preservar los datos más allá del ciclo de vida de los contenedores. Para ello, se definen reclamaciones de **Volúmenes persistentes** (**Reclamación de volumen persistente**) que se vinculan a **Volúmenes persistentes** provisionados a nivel de **Clúster**. Estos volúmenes se almacenan en un sistema de archivos compartido o en discos de red (según la infraestructura subyacente), garantizando que los datos sobrevivan a reinicios, caídas o reubicaciones de los **Pods** entre **nodos**.

Flujo de comunicación

El flujo de comunicación entre los componentes sigue la siguiente secuencia:

1. El cliente envía una petición **HTTPS** a la dirección **IP** pública del **Clúster**.
2. El **Servicio** balanceador `traefik-lb` recibe la petición y la dirige al **Pod** de `traefik`.
3. `traefik` inspecciona la ruta de la petición y la enruta al **Servicio** `fm-service` según las reglas definidas en el **Ingress**.
4. El **Servicio** `fm-service` distribuye la petición entre las tres réplicas del *backend* FastAPI mediante balanceo de carga *round-robin*.
5. Dependiendo de la operación solicitada, el **Pod** de **FM** puede:
 - Consultar o modificar datos en PostgreSQL a través del **Servicio** `postgres-service`.
 - Leer o escribir en la caché de Redis mediante el **Servicio** `redis-service`.
 - Almacenar o recuperar archivos de MinIO a través del **Servicio** `minio-service`.
 - Publicar una tarea asíncrona en la cola de Redis, que será consumida por los trabajadores Celery.
6. La respuesta sigue el camino inverso hasta llegar al cliente.

3.3. Fase de pruebas

La prueba es el proceso de ejecución de un programa con la intención de encontrar errores. Es crucial para validar el correcto funcionamiento del sistema y asegurar su calidad. Esta sección presenta la estrategia general de

pruebas y los resultados obtenidos al aplicar un conjunto de técnicas de verificación. El objetivo es demostrar que la aplicación cumple con los requisitos funcionales y no funcionales definidos, es confiable bajo diferentes condiciones de operación.

3.3.1. Estrategia de pruebas

Una estrategia de pruebas es un plan de alto nivel que define el enfoque, los objetivos, los recursos y el calendario para las actividades de evaluación de la calidad del software. Su propósito es garantizar que las pruebas sean sistemáticas, eficientes y efectivas para identificar defectos. ([garcia2023](#))

Se adoptará una estrategia de pruebas basada en riesgos y requisitos. La ejecución seguirá un orden progresivo: comenzando con las Pruebas Unitarias para cimentar la calidad del código, seguidas de las Pruebas de Integración para verificar la comunicación entre módulos. En paralelo, se realizarán Pruebas de Seguridad para validar los controles de acceso. Posteriormente, se ejecutarán Pruebas de Rendimiento para evaluar la estabilidad bajo carga. El ciclo culminará con las Pruebas de Aceptación para validar el cumplimiento de las necesidades del usuario final. Como parte integral del proceso, al final de cada iteración se ejecutaron Pruebas de Regresión para verificar que las nuevas funcionalidades no afectaran el comportamiento existente del sistema. Esta aproximación por capas asegura una cobertura comprehensiva y una detección temprana de errores en todas las dimensiones de calidad del sistema.

3.3.2. Pruebas unitarias

Las pruebas unitarias son un tipo de prueba de software donde se verifican unidades individuales o componentes del código fuente (como una función, un método o una clase) de forma aislada. El objetivo es validar que cada parte del software funciona correctamente por sí misma ([bedoyaAlzate2021](#)). En este proyecto, se implementaron pruebas unitarias para cubrir los componentes clave del sistema, asegurando que cada parte funcione según lo esperado bajo diversas condiciones. Los módulos seleccionados para las pruebas unitarias son los: servicios del sistema, repositorios, excepciones personalizadas y utilidades de validación. Estos componentes son fundamentales para la lógica de negocio, la interacción con la base de datos y la gestión de errores, por lo que su correcto funcionamiento es crítico para la estabilidad y confiabilidad del sistema.

Para cada componente, se definieron casos de prueba que cubren tanto escenarios típicos como condiciones límite o errores esperados. Se utilizaron técnicas de aislamiento mediante *mocks* para simular dependencias externas como la [BD](#), permitiendo así verificar el comportamiento interno sin efectos colaterales ni dependencias del entorno. A continuación, se presenta un ejemplo de los casos de pruebas unitarias, el resto se puede encontrar en el anexo [D](#).

Cuadro 3.2. Caso de Prueba Unitaria No1

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>FeatureModelVersionManager.create_new_version(source_version=None)</code> crea una versión en estado DRAFT con <code>version_number</code> consecutivo.
Continúa en la siguiente página	

Cuadro 3.2 – Continuación de la página anterior.

Caso de Prueba Unitaria	
Entrada:	• <code>feature_model.id</code> válido. • Mock de <code>repository.get_latest_version_number()</code> retornando 3. • <code>source_version=None</code>.
Salida esperada:	• Nueva versión con <code>version_number=4</code>. • <code>status=DRAFT</code>. • Se ejecuta <code>session.commit()</code> y <code>session.refresh()</code>.
Evaluación de la prueba:	• Pasa si se cumplen valores y llamadas a sesión. • Falla si no incrementa versión o no persiste cambios.

Resultados de las pruebas unitarias

El proceso de pruebas unitarias se ejecutó de forma iterativa a lo largo de las siete iteraciones de desarrollo, siguiendo la metodología **XP** que promueve la escritura de pruebas antes del código y la **Integración Continua (CI)** (**beck2000extreme**). Este enfoque permitió detectar y corregir defectos de forma temprana, garantizando la estabilidad y calidad del código en cada incremento funcional.

Se diseñaron y ejecutaron iterativamente hasta 29 casos de prueba unitaria que cubrieron la totalidad de los componentes críticos del sistema. Las pruebas se estructuraron para validar tanto los escenarios positivos (funcionamiento normal de los servicios) como los negativos (manejo de errores y validaciones), asegurando una evaluación comprehensiva de la lógica de negocio. Se llevó a cabo la ejecución completa de estas pruebas en ocho iteraciones, para verificar el cumplimiento de los requisitos técnicos desde la perspectiva del desarrollador.

El total de fallas detectadas (16) durante el proceso de pruebas unitarias se clasificaron en cinco categorías principales. La tabla 3.3 muestra la distribución de los tipos de error en relación con los componentes afectados, lo que permitió identificar áreas críticas y orientar las correcciones de manera eficiente al registrar también cuál componente falló.

Cuadro 3.3. Matriz de tipo de error contra componentes. Fuente: Elaboración propia

-	Servicios	Repositorios	Excepciones	Detalles
Lógica de negocio:	4	1	0	Los servicios que fallaron fueron el de contruir el árbol de características, el validador lógico de modelos, el exportador de modelos y el generador de configuraciones. Además, el repositorio de características falló al no impedir que una característica sea su propio padre (autorreferencia).
Validación de datos:	1	2	0	EL servicio de importador de UVL tuvo un error durante la validación de UVL para generar estructura y versión. El repositorio de restricciones presentó problemas para registrar correctamente los datos y el repositorio de usuarios presentó un fallo durante el registro de usuarios por incompatibilidad en el nombre de los roles.
Escritura/lectura de la BD :	2	2	0	Los servicios de constuir el árbol de características y el exportador de modelos también presentaron fallas al devolver incorrectamente la información recuperada en la BD . El repositorio de las restricciones presentó un fallo al devolver incorrectamente los datos y el repositorio de características falló a la hora de mapear los metadatos.
Inconsistencia en las excepciones:	1	0	1	El servicio de analizador estructural de FM empleó una excepción incorrecta. La excepción <code>DomainNotFoundException</code> devolvió un estado no esperado.
Continúa en la siguiente página				

Cuadro 3.3 – Continuación de la página anterior.

-	Servicios	Repositorios	Excepciones	Detalles
Respuesta de las excepciones:	1	0	1	El servicio de gestionar el versionado de los FM devolvió un mensaje incorrecto al lanzar el identificador del modelo en vez del número de la versión cuando se intentaba crear una nueva. La excepción FeatureAccessDeniedException devolvió el identificador del modelo como objeto en lugar de la cadena de caracteres.
Total:	9	5	2	Se encontraron 16 fallas

Cada una de estas fallas fue abordada mediante correcciones específicas en el código, seguidas de la re-ejecución de las pruebas para confirmar su resolución. La octava y última iteración consistió en la ejecución de las 29 pruebas unitarias para confirmar la estabilidad final y la resolución completa de los fallos. Al concluir esta fase, no quedaron fallos pendientes (0 fallos), alcanzando el 100 % de aprobación en todas las pruebas. La figura 3.2 resume el resultado de estas pruebas.

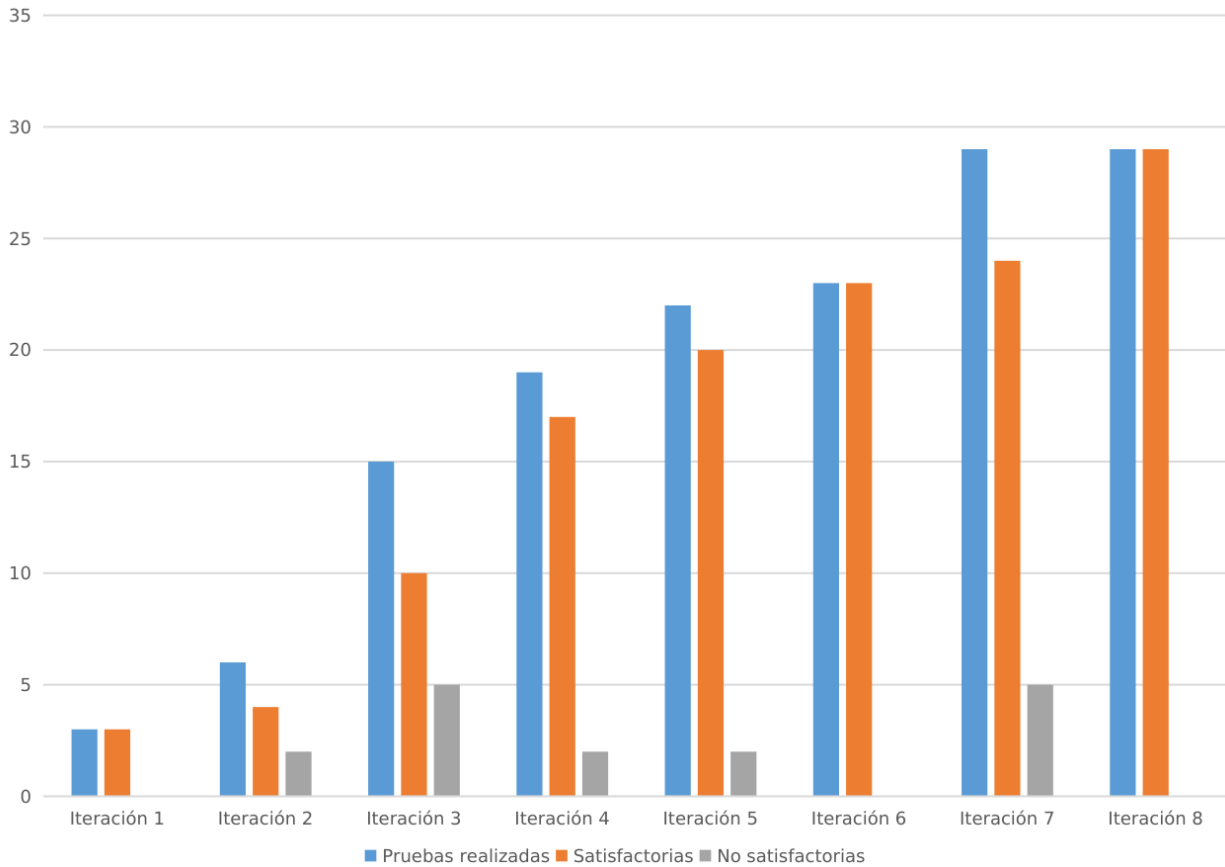


Figura 3.2. Resultados de las Pruebas Unitarias. Fuente: Elaboración propia.

3.3.3. Pruebas de integración

Las pruebas de integración verifican la interacción y comunicación correcta entre diferentes módulos o componentes de software una vez que han sido integrados. El objetivo es encontrar fallos en las interfaces y en la interacción entre unidades integradas ([paz2016](#)). En el contexto de este proyecto que es un servidor *backend*, las pruebas de integración se realizarán sobre los controladores para verificar que las rutas, la lógica de negocio y la interacción con los servicios y repositorios funcionen correctamente. Se diseñaron casos de prueba que simulen escenarios reales de uso, incluyendo la validación de datos, la gestión de errores y la respuesta del sistema ante diferentes tipos de solicitudes.

Los casos de pruebas de integración se aplicarán sobre las rutas críticas del sistema, las cuales son los controladores asociados a los servicios de [FM](#) (entre otros). Se diseñaron 18 casos de prueba que abordan las integraciones más críticas del sistema. A continuación, se presenta un ejemplo de las pruebas de integración, el resto se puede encontrar en el anexo [E](#).

Cuadro 3.4. Caso de Prueba de Integración No1

Caso de Prueba de Integración	
ID de prueba: HU4-P1:	HU: 4
Nombre:	Crear FM en dominio válido
Descripción:	Verificar que POST /feature-models/ crea un FM y persiste metadatos correctamente.
Condiciones de ejecución:	• Sistema levantado con datos de prueba • BD limpia o controlada • Usuario autenticado con rol MODEL_DESIGNER o ADMIN • Existe domain_id activo
Pasos de ejecución:	1. Enviar POST /api/v1/feature-models/ con name, description, domain_id 2. Validar HTTP 201 3. Verificar respuesta con id, owner_id, domain_id, is_active=true 4. Consultar GET /api/v1/feature-models/id 5. Confirmar persistencia en BD (registro creado)
Resultados esperados:	• Se crea el FM correctamente • El owner_id corresponde al usuario autenticado • Se registra created_at/updated_at
Evaluación de la prueba:	Aprobada

Resultados de las pruebas de integración

Las pruebas de integración se centraron en validar la correcta interacción entre los diferentes componentes del sistema a través de la API REST, abarcando desde la recepción de peticiones HTTP en los controladores hasta la respuesta final generada por la capa de servicios, pasando por la comunicación con los repositorios y la BD. Este proceso de verificación se ejecutó de forma iterativa a lo largo del ciclo de desarrollo, totalizando 6 iteraciones de pruebas, de las cuales 5 se realizaron durante el desarrollo activo de las funcionalidades y una iteración final de validación y regresión al concluir la implementación. Para garantizar la calidad de las integraciones, se diseñaron 18 casos de prueba que cubren las rutas críticas del sistema, específicamente los controladores asociados a los servicios de FM.

Durante la ejecución de las pruebas de integración se detectaron un total de 8 fallas, las cuales se identificaron en las siguientes funcionalidades:

- Crear [FM](#) en dominio válido (HU4-P1).
- Reubicar feature con padre inválido (HU7-P2).
- Crear grupo OR con cardinalidades válidas (HU8-P1).
- Crear nueva versión con (*Copy-on-Write*) desde origen (esta funcionalidad falló en 3 ocasiones) (HU5-P1).
- Aplicar UVL a estructura creando nueva versión (HU10-P2).
- Generación automática de configuraciones (HU13-P1).

Cada una de estas fallas fue documentada, corregida y sometida a una nueva ronda de pruebas para confirmar su resolución. A diferencia de las pruebas unitarias, que se centran en componentes aislados, las pruebas de integración revelaron problemas que solo se manifiestan cuando los componentes interactúan entre sí, como la propagación incorrecta de errores entre capas y la falta de validaciones de negocio en los controladores.

La sexta y última iteración consistió en la ejecución completa de los 18 casos de prueba sobre el sistema ya estabilizado, verificando que las correcciones aplicadas no introdujeran nuevos defectos ni afectaran funcionalidades previamente validadas. Al concluir esta fase, no quedaron fallos pendientes (0 fallos), alcanzando el 100 % de aprobación en todas las pruebas ejecutadas. La figura [3.3](#) resume el resultado de estas pruebas, mostrando la evolución de la tasa de éxito a lo largo de las iteraciones y la resolución progresiva de las fallas detectadas.

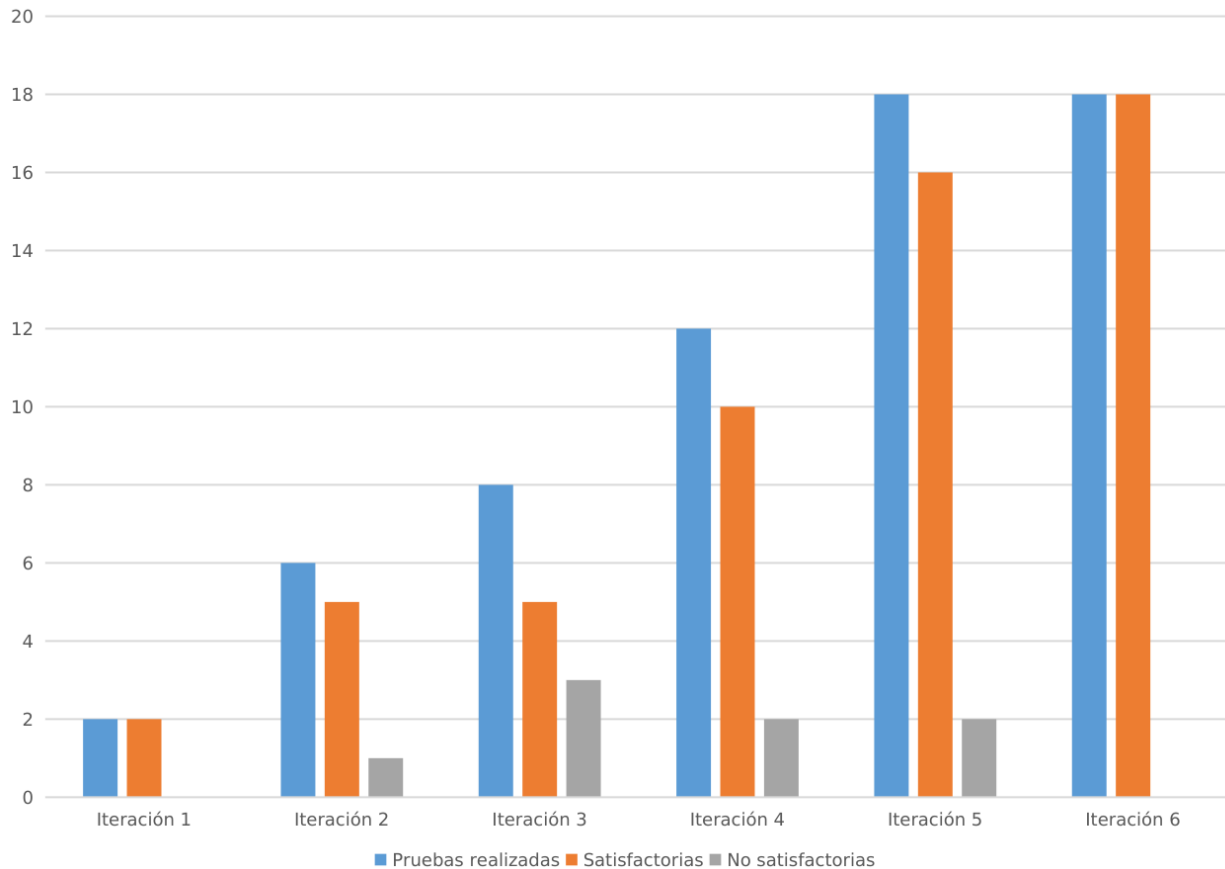


Figura 3.3. Resultados de las Pruebas de Integración. Fuente: Elaboración propia.

3.3.4. Pruebas de seguridad

Las pruebas de seguridad constituyen un conjunto de técnicas de verificación orientadas a identificar vulnerabilidades, debilidades y riesgos en un sistema software que puedan ser explotados por actores maliciosos (**felderer2016security**). A diferencia de las pruebas funcionales, que verifican que el sistema hace lo que debe hacer, las pruebas de seguridad buscan aquello que el sistema no debería permitir: accesos no autorizados, fugas de información, ejecución de código malicioso o violaciones de las políticas de control de acceso (**felderer2015survey**). En el contexto de un servicio *backend* que gestiona **FM** que tienen información curricular sensible y que pudiera integrarse con sistemas institucionales existentes, la seguridad se convierte en un atributo de calidad crítico que debe ser validado rigurosamente.

Los principales objetivos de las pruebas de seguridad incluyen: identificar activos digitales que requieren protección, clasificar vulnerabilidades potenciales, evaluar las consecuencias de una posible explotación, y proporcionar recomendaciones para la remediación de las fallas detectadas (**moradov2023security**). En este proyecto, las pruebas de seguridad se centran en tres dimensiones fundamentales: la autenticación y gestión de sesiones, la autorización basada en roles estáticos, y la protección de los endpoints de la **API REST** contra ataques comunes (inyección, manipulación de parámetros, exposición de información sensible). Esta sección presenta los resultados de la verificación sistemática de los criterios de seguridad definidos, utilizando para ello una lista de verificación estructurada que permite evaluar de manera objetiva el cumplimiento de cada uno de los requisitos establecidos. Dicha lista se evidencia

en el anexo F.

Resultados de las pruebas de seguridad

Las pruebas de seguridad aplicadas al sistema son fundamentales, ya que se enfocan en garantizar la protección de los datos y el control de acceso a las funcionalidades. Para evaluar estos aspectos, se utilizó la lista de chequeo de seguridad definida en el anexo F, obteniendo en la primera iteración los resultados evidenciados en la tabla 3.5.

Cuadro 3.5. Resultados de pruebas de seguridad - Primera iteración. Fuente: Elaboración propia.

Categoría de las pruebas fallidas	Cantidad de errores
Autenticación y control de acceso	2
Protección de datos y comunicaciones	2
Validación de entrada y protección contra ataques	2
manipulación de parámetros	1
total	7

Las pruebas fallidas identificadas incluyeron:

- Acceso no autorizado a rutas protegidas mediante manipulación de URLs.
- Tiempo de expiración de sesiones configurado de manera muy permisiva.
- Falta de lista negra para tokens de autenticación comprometidos.
- Exposición de información sensible en mensajes de error.
- Inyección de código a través de parámetros no validados en *endpoints* de creación y actualización de FM.
- Manipulación del parámetro de estado para los FM saltándose las validaciones del negocio.

Esto llevó a la implementación de *middlewares* más estrictos y a la ocultación de campos sensibles en los esquemas de serialización de la API. Se implementó un controlador para invalidar tokens de autenticación comprometidos mediante una lista negra, y se ajustaron las políticas de expiración de sesiones para reducir la ventana de oportunidad de ataques. Se ajustaron los mensajes de error para evitar la exposición de información sensible que pudiera ser utilizada por atacantes para identificar vulnerabilidades. Las dependencias de los controladores para el acceso de recurso por rol se repasaron y reforzaron para evitar el acceso inadecuado de los recursos del sistema. Asimismo, se desacopló la actualización de metadatos de las transiciones de estado crítico, implementando validaciones forzadas en la capa de servicio que impiden el cambio a estados como PUBLISHED sin la ejecución previa y exitosa de los motores de análisis lógico. Tras aplicar estas correcciones, se ejecutó una segunda iteración de pruebas de seguridad, obteniendo un resultado de 0 fallas, lo que indica que el sistema ahora cumple con los criterios de seguridad establecidos. La figura 3.4 resume la evolución de los resultados de las pruebas de seguridad a lo largo de las iteraciones.

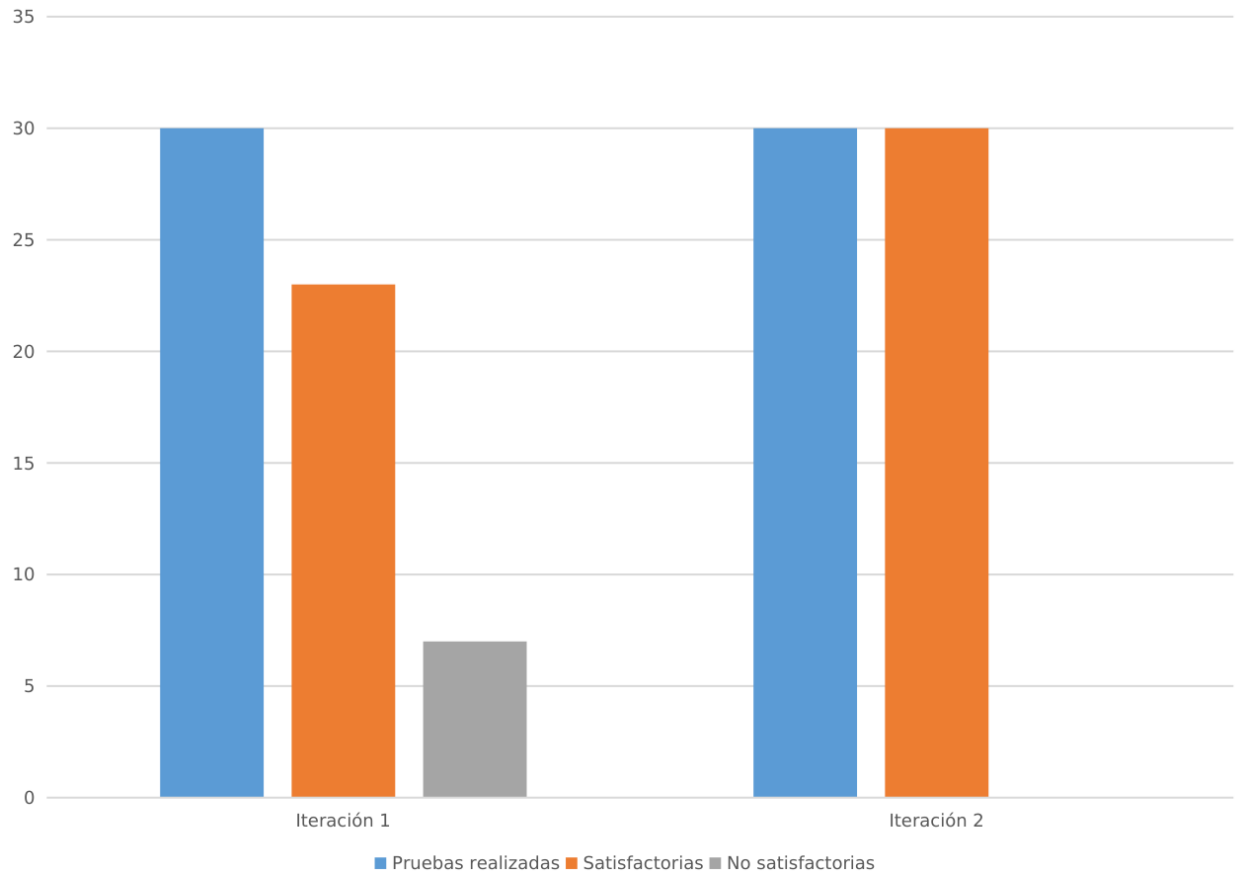


Figura 3.4. Resultados de las Pruebas de Seguridad. Fuente: Elaboración propia.

3.3.5. Pruebas de rendimiento

Las pruebas de rendimiento evalúan la velocidad, capacidad de respuesta, estabilidad, confiabilidad y uso de recursos de un sistema bajo una carga de trabajo determinada. Su objetivo no es encontrar defectos funcionales, sino identificar y eliminar cuellos de botella en el rendimiento ([diaz2008](#)). En el contexto del servicio *backend* para la gestión de variabilidad curricular, estas pruebas se centran en validar el cumplimiento de los requisitos no funcionales establecidos, particularmente aquellos relacionados con los tiempos de respuesta de la [API](#), capacidad de manejar alto tráfico y el tiempo de latencia entre los repositorios y la [BD](#). Esta sección presenta los resultados de la verificación sistemática de los criterios de rendimiento definidos, utilizando para ello métricas objetivas que permiten evaluar el cumplimiento de cada uno de los requisitos establecidos.

La herramienta empleada para la ejecución de estas pruebas fue Apache JMeter, que permite evaluar el comportamiento del sistema bajo carga repetitiva. Se escogieron como casos de pruebas los controladores más críticos del sistema, específicamente aquellos asociados a los servicios de [FM](#), debido a que gestionan la lógica de negocio más compleja y podrían presentar los mayores desafíos de rendimiento. La selección de estos controladores respondió a criterios como su frecuencia de uso, el costo computacional de las operaciones involucradas, el impacto en la experiencia del usuario y la complejidad de las transformaciones de datos requeridas.

Los controladores seleccionados para las pruebas de rendimiento fueron:

1. Obtención del árbol completo del modelo (GET /{model_id}/versions/latest/complete): Este controlador es considerado el más crítico del sistema, pues constituye la base de la mayoría de las operaciones realizadas desde el cliente visual, incluyendo la representación gráfica del árbol de características. Su importancia radica en que una demora en su respuesta afecta directamente toda la interacción del usuario con el modelo. Internamente, consume una lógica compleja que involucra la construcción del árbol completo, la serialización de la estructura jerárquica (características, grupos, restricciones) y el cálculo de estadísticas asociadas.
2. Análisis estructural completo del modelo (POST /{model_id}/versions/{version_id}/analysis/summary): Este controlador ejecuta operaciones de alta intensidad computacional, incluyendo la detección de características muertas, la identificación de características obligatorias, el análisis de satisfactibilidad mediante solucionadores SAT, el cálculo de conjuntos atómicos y la medición de la complejidad del modelo. Su criticidad deriva de ser una operación bloqueante (sin procesamiento asíncrono) que puede tardar varios segundos en modelos de gran tamaño, lo que impacta directamente en la fluidez del trabajo del gestor curricular cuando solicita un análisis profundo de su modelo.
3. Generación de configuraciones válidas (POST /configurations/generate): Este controlador aborda uno de los problemas centrales de la investigación: la derivación de planes de estudio válidos a partir de un modelo de características. Su complejidad es exponencial respecto al número de características y restricciones definidas, y ofrece múltiples estrategias de generación (ávida, aleatoria, genética multiobjetivo), cada una con diferente costo computacional. La experiencia del usuario es particularmente sensible en este punto, ya que la generación de un número elevado de configuraciones puede implicar un bucle intensivo de validación mediante resolución SAT.
4. Exportación del modelo a múltiples formatos (GET /{model_id}/versions/{version_id}/export/{format}): Este controlador permite exportar el modelo a ocho formatos diferentes (XML, JSON, DIMACS, UVL, Mermaid, DOT, TVL, SPLOT). Su criticidad radica en las operaciones de entrada y salida intensivas que realiza: lectura de datos desde la base de datos, transformación de la estructura completa del árbol según el formato destino, serialización y almacenamiento temporal en el sistema de archivos, y envío al cliente. Los formatos orientados a visualización (Mermaid, Graphviz) requieren transformaciones adicionales que incrementan el costo computacional, y los modelos de gran tamaño generan archivos voluminosos cuyo envío debe realizarse de forma eficiente.
5. Análisis asíncrono por lotes (POST /{model_id}/versions/{version_id}/analysis/batch): Este controlador actúa como puerta de entrada para tareas de larga duración delegadas al sistema de colas Celery. Su importancia estratégica deriva de que cualquier fallo en este punto implica la pérdida de solicitudes de análisis por parte de los usuarios. Las pruebas de rendimiento deben verificar el comportamiento del sistema cuando múltiples usuarios lanzan análisis en paralelo, evaluando la capacidad de la cola de mensajes (Redis) y de los trabajadores Celery para gestionar la contención.
6. Obtención de estadísticas agregadas (GET /{model_id}/versions/latest/statistics): Este controlador devuelve métricas resumidas del modelo (cantidad de características, restricciones, profundidad del árbol, entre otras). Aunque cada consulta tiene un costo computacional moderado (requiere recorrer múltiples relaciones en la base de datos), su alta frecuencia de invocación —cada vez que un usuario accede al tablero principal o abre un modelo— puede generar un impacto acumulativo significativo en la carga del sistema, especialmente cuando múltiples usuarios trabajan simultáneamente.
7. Validación completa del modelo (POST /{model_id}/versions/{version_id}/validation/full): Este controlador combina tres servicios de alta complejidad en una sola operación: la validación estructural del árbol,

la validación lógica de las restricciones y el análisis de propiedades del modelo. Su criticidad reside en que se ejecuta justo antes de que un gestor curricular publique una nueva versión de un plan de estudio, constituyendo un punto de bloqueo en el flujo de trabajo. Una demora excesiva en este punto puede generar frustración en el usuario e interrumpir el ciclo de publicación de cambios curriculares.

8. Validación de una configuración específica (POST /configurations/{id}/validate): Este controlador se invoca antes de persistir cada configuración generada, verificando que la selección de características cumpla con todas las restricciones definidas en el modelo. Su frecuencia de uso es muy elevada, especialmente en escenarios donde los usuarios generan y evalúan decenas o cientos de configuraciones de forma interactiva. Aunque cada validación individual tiene un costo moderado, la acumulación de estas operaciones puede representar una carga significativa para el sistema.

Para cada uno de estos controladores, se definieron tres escenarios de prueba progresivos que permiten evaluar el comportamiento del sistema bajo diferentes condiciones de carga:

1. Escenario base: Simula una carga normal de operación, con 10 usuarios concurrentes trabajando sobre modelos de 50 características. Este escenario establece la línea base de rendimiento esperado en condiciones habituales de uso.
2. Escenario pico: Representa situaciones de alta demanda, con 100 usuarios concurrentes operando sobre modelos de más de 100 características. Este escenario evalúa la capacidad del sistema para mantener tiempos de respuesta aceptables durante horas de máxima concurrencia, como períodos de planificación curricular intensiva.
3. Escenario de estrés: Somete al sistema a condiciones extremas, con 1.000 usuarios simultáneos y modelos complejos que incluyen restricciones anidadas y estructuras de características profundas. Este escenario permite identificar el punto de quiebre del sistema, los cuellos de botella en la infraestructura (base de datos, caché, cola de mensajes) y el comportamiento bajo condiciones de sobrecarga.

Resultados de las pruebas de rendimiento

Las pruebas de rendimiento se ejecutaron en dos iteraciones completas sobre los ocho controladores seleccionados. En cada iteración, cada controlador fue sometido a los tres escenarios de carga definidos (base, pico y estrés), con el objetivo de evaluar el comportamiento del sistema, identificar cuellos de botella y verificar el cumplimiento de los requisitos no funcionales establecidos. La primera iteración tuvo un carácter diagnóstico, permitiendo detectar las operaciones con mayores tiempos de respuesta y los componentes que limitaban el rendimiento del sistema. Con base en los hallazgos obtenidos, se aplicaron un conjunto de técnicas y estrategias de optimización, tras lo cual se ejecutó una segunda iteración para validar la efectividad de las correcciones implementadas.

Durante la primera iteración, se evidenció que varios controladores presentaban tiempos de respuesta superiores a los límites aceptables, especialmente en los escenarios pico y de estrés. Los principales problemas detectados fueron:

- El controlador de obtención del árbol completo mostró una degradación significativa en modelos de gran tamaño, debido a la construcción recursiva del árbol sin estrategias de caché efectivas.
- El controlador de análisis estructural completo presentó demoras excesivas (superiores a 30 segundos en escenario de estrés) por la ejecución bloqueante de los solucionadores [SAT](#).
- El controlador de generación de configuraciones evidenció tiempos de respuesta variables según la estrategia seleccionada, con la estrategia genética multiobjetivo siendo la más afectada por el aumento de restricciones.
- El controlador de exportación mostró problemas de entrada y salida al generar archivos de gran tamaño, con tiempos de transferencia que impactaban negativamente en la experiencia del usuario.

La figura 3.5 presenta un resumen de los tiempos de respuesta obtenidos para cada controlador en los tres escenarios de carga durante la primera iteración.

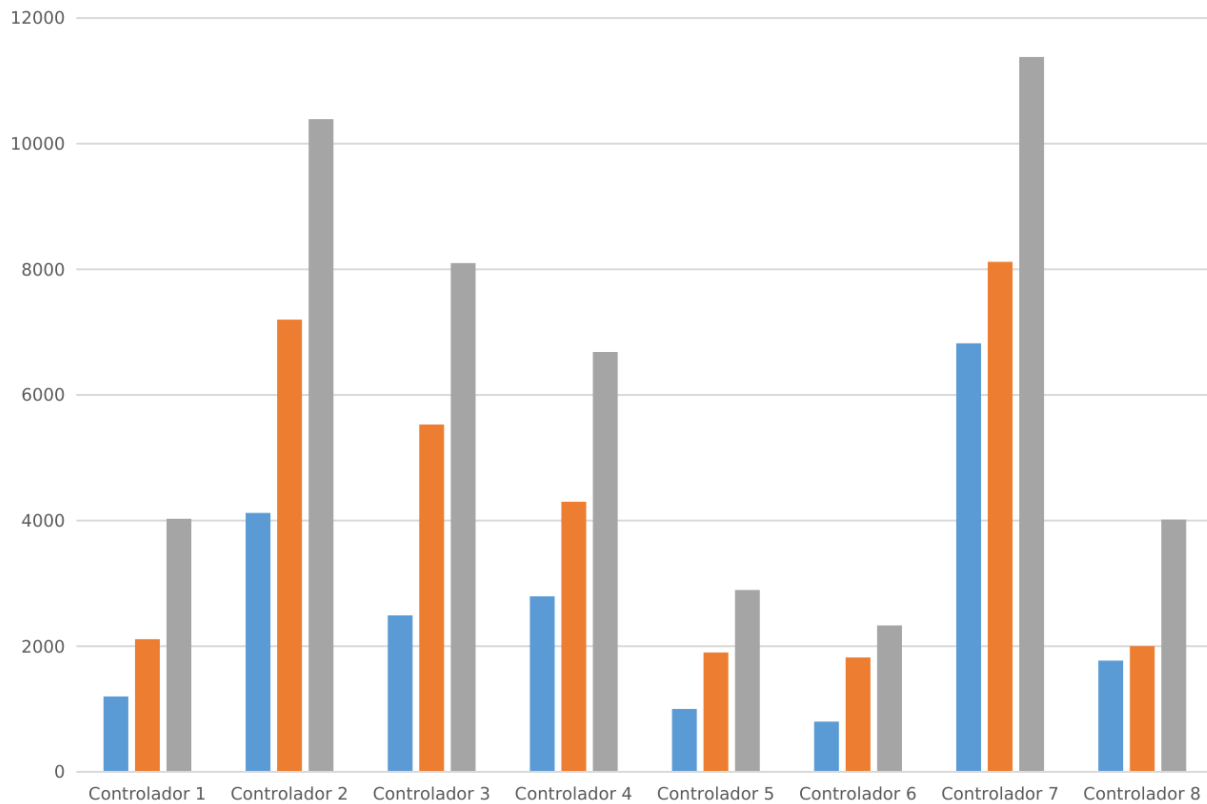


Figura 3.5. Resultados de la primera iteración de pruebas de rendimiento. Fuente: Elaboración propia.

Con el objetivo de reducir los tiempos de respuesta y mejorar la escalabilidad del sistema, se implementaron las siguientes técnicas y estrategias de optimización:

- **Implementación de caché distribuida:** Se incorporó Redis como sistema de caché para almacenar los resultados de operaciones costosas y frecuentes, como la obtención del árbol completo y las estadísticas agregadas. La estrategia de caché implementada serializa la respuesta completa del árbol como objeto JSON en Redis, evitando reconstrucciones innecesarias. Se definieron tiempos de expiración diferenciados según el estado del modelo: 3600 segundos (una hora) para los modelos publicados, cuya estructura cambia con poca frecuencia, y 300 segundos (cinco minutos) para los modelos en edición, que son inherentemente más volátiles. Adicionalmente, se implementó una invalidación inteligente que borra únicamente las claves afectadas por cada operación de escritura, en lugar de invalidar todo el caché globalmente.
- **Procesamiento asíncrono con Celery:** Las operaciones de análisis estructural completo y generación masiva de configuraciones, que involucran la ejecución de solucionadores SAT con complejidad exponencial, fueron delegadas a trabajadores en segundo plano mediante el sistema de colas Celery. Esta estrategia libera el flujo principal de la aplicación, permitiendo que el usuario reciba una respuesta inmediata con el identificador de la tarea, mientras que el procesamiento pesado se ejecuta en segundo plano. Se configuraron colas con diferentes prioridades: una cola de alta prioridad para análisis solicitados interactivamente por el usuario, y una cola de

baja prioridad para exportaciones masivas y tareas de limpieza periódica. Adicionalmente, se implementó un tiempo máximo de ejecución (timeout) de 5 segundos para el solucionador SAT, evitando que operaciones que no convergen bloqueen indefinidamente a los trabajadores.

- Optimización de consultas a la base de datos: Se implementaron múltiples mejoras en el acceso a PostgreSQL. En primer lugar, se añadieron índices estratégicos en las tablas de mayor volumen y consulta, incluyendo índices compuestos en las relaciones entre características y restricciones, así como índices descendentes para la búsqueda de versiones recientes. Estas optimizaciones redujeron significativamente los recorridos de tablas completas en operaciones de construcción del árbol de características. En segundo lugar, se implementó el mecanismo de carga diferida, que redujo el número de consultas a la base de datos en modelos grandes de aproximadamente; esto permitió Agrupar consultar relacionadas en una sola transacción. Adicionalmente, se configuró el grupo de conexiones a la base de datos con parámetros optimizados (`pool_size=20`, `max_overflow=40`), permitiendo gestionar eficientemente escenarios de alta concurrencia.
- Serialización eficiente y compresión de respuestas: Se optimizó el proceso de serialización de respuestas en los controladores más críticos. En lugar de serializar la totalidad de la estructura del árbol, las características, las relaciones y las restricciones de una sola vez, se implementó una serialización diferida que solo procesa los componentes necesarios para la vista inicial del usuario, posponiendo la carga de elementos profundos hasta que sean solicitados explícitamente. Esta estrategia redujo el tamaño de las respuestas JSON en aproximadamente un 60 %. Complementariamente, se habilitó la compresión GZIP mediante el middleware de FastAPI para todas las respuestas con tamaño superior a 1 kilobyte, reduciendo el volumen de datos transmitidos en la red hasta en un 80 %, especialmente beneficioso para modelos curriculares extremadamente grandes.
- Optimización del flujo de validación y análisis: Se refactorizó el proceso de validación lógica de restricciones para ejecutar las comprobaciones en paralelo mediante tareas asíncronas, aprovechando el multiprocesamiento en entornos con múltiples núcleos de CPU. Para operaciones que tradicionalmente eran bloqueantes, como el análisis del solucionador SAT, se incorporó un límite de tiempo máximo de ejecución, más allá del cual la operación retorna un estado de "desconocido." "no determinado." lugar de esperar indefinidamente. Esta estrategia garantiza que la latencia máxima del sistema se mantenga controlada, incluso ante modelos con restricciones particularmente complejas que podrían llevar a los solucionadores a explorar espacios de búsqueda combinatoriamente explosivos.
- Para el controlador de generación de configuraciones, se implementaron múltiples estrategias que el usuario puede seleccionar según sus necesidades: una estrategia ávida (*greedy*) para obtener configuraciones válidas de forma rápida (ideal para exploración inicial), una estrategia aleatoria basada en solucionadores SAT para un muestreo más diverso, y una estrategia de optimización multiobjetivo mediante algoritmos genéticos NSGA-II para escenarios donde se requiere maximizar simultáneamente varios criterios (por ejemplo, minimizar carga académica y maximizar afinidad con competencias deseadas); este enfoque permite permitiendo al usuario seleccionar el equilibrio entre velocidad y calidad de los resultados. Adicionalmente, se implementó un modo de generación por lotes con procesamiento asíncrono, donde el usuario puede solicitar la generación de hasta 100 configuraciones en una sola petición, siendo cada validación procesada de forma concurrente por los trabajadores de Celery.
- Escalado horizontal mediante contenedores: Empleando Docker y Kubernetes, el sistema puede crear réplicas dinámicas del backend bajo alta demanda, distribuyendo la carga mediante un balanceador. Adicionalmente, la base de datos PostgreSQL se configuró con réplicas de solo lectura para escalar las operaciones de consulta, garantizando tiempos de respuesta aceptables en escenarios de máxima concurrencia sin rediseñar la arquitec-

tura.

Tras aplicar las correcciones y estrategias de optimización descritas, se ejecutó una segunda iteración de pruebas de rendimiento, sometiendo nuevamente los ocho controladores a los tres escenarios de carga. Los resultados obtenidos mostraron una mejora significativa en todos los indicadores evaluados. Los controladores que anteriormente presentaban tiempos de respuesta críticos redujeron su latencia en más de un 70 %, y ninguno de ellos superó los límites establecidos en los requisitos no funcionales.

La figura 3.6 presenta los resultados de la segunda iteración, evidenciando la estabilidad y el rendimiento alcanzado por el sistema.

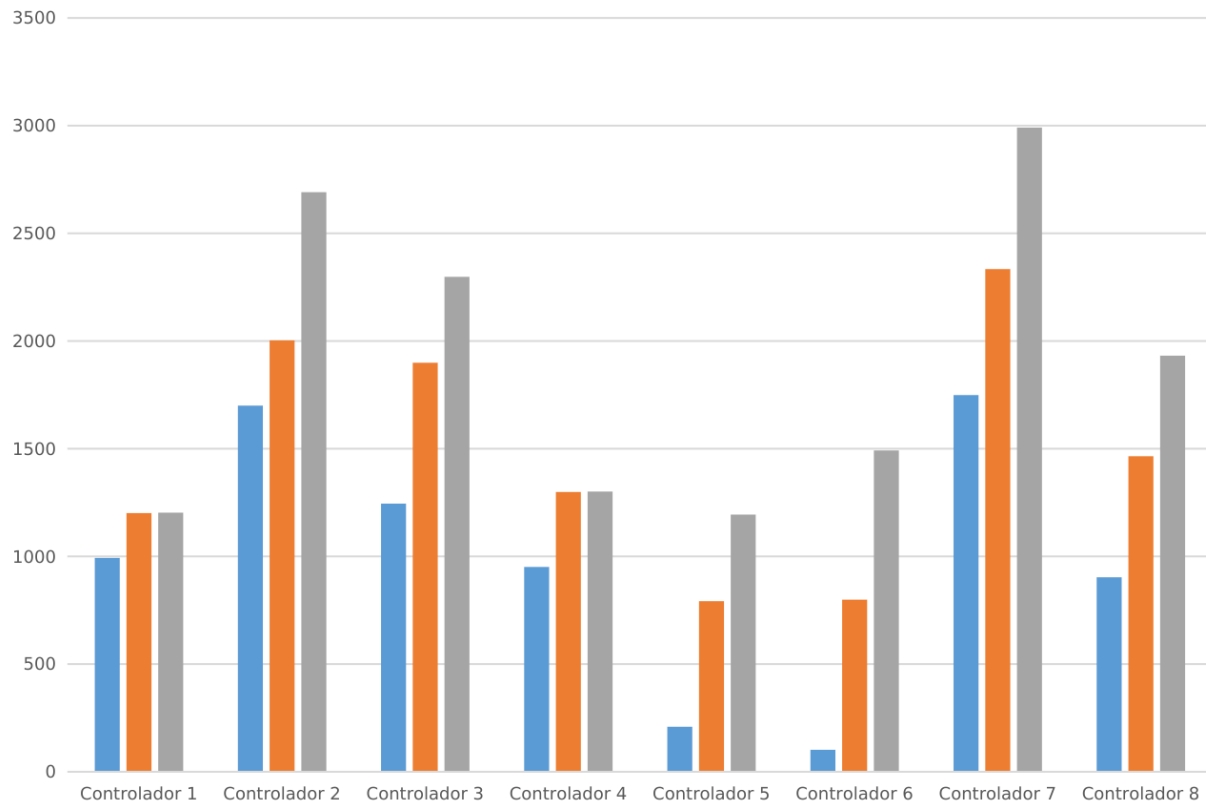


Figura 3.6. Resultados de la segunda iteración de pruebas de rendimiento. Fuente: Elaboración propia.

Los resultados obtenidos en la segunda iteración demuestran que el sistema cumple satisfactoriamente con los requisitos no funcionales de rendimiento definidos. En el escenario base (10 usuarios concurrentes), todos los controladores respondieron en menos de 2 segundos. En el escenario pico (100 usuarios concurrentes), los tiempos de respuesta se mantuvieron por debajo de 2.5 segundos para las operaciones síncronas, con solo dos controladores superiores a los 2 segundos, mientras que las operaciones asíncronas retornaron una respuesta inmediata con el identificador de la tarea. En el escenario de estrés (1.000 usuarios simultáneos), el sistema demostró una capacidad de manejo de carga satisfactoria, con tiempos de respuesta por debajo de 3 segundos para las operaciones críticas, sin errores ni caídas del servicio.

3.3.6. Pruebas de aceptación

Las pruebas de aceptación constituyen la fase final del proceso de verificación y validación del software, en la cual se determina si el sistema satisface los criterios de aceptación establecidos por el cliente y se encuentra apto para su despliegue en producción (**pressman2014se**). A diferencia de las pruebas funcionales y no funcionales ejecutadas por el equipo de desarrollo, las pruebas de aceptación son típicamente realizadas por el cliente o por usuarios finales en un entorno controlado que simula las condiciones reales de operación (**crispin2008agile**). Su propósito fundamental es generar confianza en que el software resuelve efectivamente los problemas del dominio para el cual fue concebido y cumple con las expectativas de sus *stakeholders*.

En el contexto del servicio *backend* para la gestión de variabilidad curricular, las pruebas de aceptación se centran en validar los criterios de aceptación definidos en las **HU**, principalmente en aquellas orientadas a modelar y representar fielmente los **FM** y generar configuraciones válidas que correspondan a itinerarios formativos coherentes. Estas pruebas se ejecutan sobre escenarios de uso reales, utilizando modelos curriculares de asignaturas y carreras. Se consideran exitosas cuando todas las historias de usuario priorizadas en el plan de entregas han sido satisfactoriamente validadas por el cliente. A continuación, se presenta un ejemplo de los casos de pruebas definidos, el resto se pueden consultar en el anexo **G**.

Cuadro 3.6. Prueba de aceptación # 1

Caso de prueba de aceptación	
Código: HU6_P2	Historia de usuario: 6
Nombre: Exportar modelo en formato UVL	
Descripción: Validar que el modelo se puede exportar en formato UVL válido.	
Condiciones de ejecución: • Existe un modelo con versión PUBLISHED. • La versión tiene estructura completa. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro 3.6. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar GET a <code>/api/v1/models/{model_id}/export?format=uvl</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que el <code>Content-Type</code> es <code>text/plain</code> o <code>application/x-uvl</code>. 4. Validar que el contenido es UVL válido (comienza con características o <i>namespace</i>, tiene sintaxis correcta). 5. Guardar contenido en archivo <code>.uvl</code> y validar.
Resultados esperados: • Estado HTTP: 200 OK. • <code>Content-Type</code> correcto. • UVL contiene sintaxis válida. • Todas las características, grupos y restricciones están representadas. • Puede descargarse como archivo. • Re-importación genera estructura idéntica.

Resultados de las pruebas de aceptación

Las pruebas de aceptación se ejecutaron en tres iteraciones sobre un total de 63 casos de prueba diseñados a partir de las historias de usuario priorizadas en el plan de entregas. Estas pruebas fueron ejecutadas en Postman. Durante la primera ejecución de las pruebas de aceptación, se detectaron 11 fallas en el comportamiento del sistema. Estas fallas se clasificaron en las siguientes categorías:

- Fallas en la validación de restricciones (4 casos): El sistema permitía la creación de restricciones que referenciaban características inactivas o que generaban conflictos lógicos no detectados durante la validación estructural.
- Inconsistencias en la exportación a **UVL** (3 casos): En modelos con grupos anidados de características, la exportación generaba una sintaxis **UVL** mal formada que no podía ser reimportada correctamente.
- Errores en la generación de configuraciones (2 casos): El generador de configuraciones producía en ocasiones configuraciones que violaban restricciones de exclusión mutua entre características.
- Problemas de autorización (2 casos): Algunos *endpoints* permitían la modificación de modelos por parte de usuarios con rol de observador, violando la política de privilegios definida.

Todas estas fallas fueron corregidas mediante modificaciones directas en el código afectado, principalmente en los servicios de validación lógica, en el exportador de modelos y en las dependencias de autorización de los controladores.

Tras aplicar las correcciones derivadas de la primera iteración, se ejecutó una nueva ronda de pruebas de aceptación. En esta ocasión, el número de fallas se redujo significativamente a solo 3 casos, todos ellos relacionados con aspectos periféricos del sistema:

- Envío de respuestas en formatos incorrectos (2 casos): Ciertos *endpoints* devolvían códigos de estado [HTTP](#) inapropiados para situaciones de error (por ejemplo, retornar 400 en lugar de 404 cuando un recurso no existía).
- Mensajes de error poco informativos (1 caso): En una validación fallida, el mensaje de error no especificaba qué característica era la causa del conflicto, dificultando la corrección por parte del usuario.

Estas fallas fueron igualmente corregidas, ajustando el manejo de excepciones en los controladores y mejorando la retroalimentación proporcionada al usuario en las respuestas de error. Con el objetivo de garantizar que las correcciones aplicadas no introdujeran nuevos defectos ni afectaran funcionalidades previamente validadas, se ejecutó una tercera iteración de pruebas de aceptación, esta vez con un enfoque de regresión completo. En esta iteración, se volvieron a ejecutar la totalidad de los 63 casos de prueba, obteniendo como resultado cero fallas. La figura 3.7 resume el comportamiento del sistema a lo largo de las tres iteraciones, mostrando la evolución positiva en la tasa de éxito de las pruebas de aceptación.

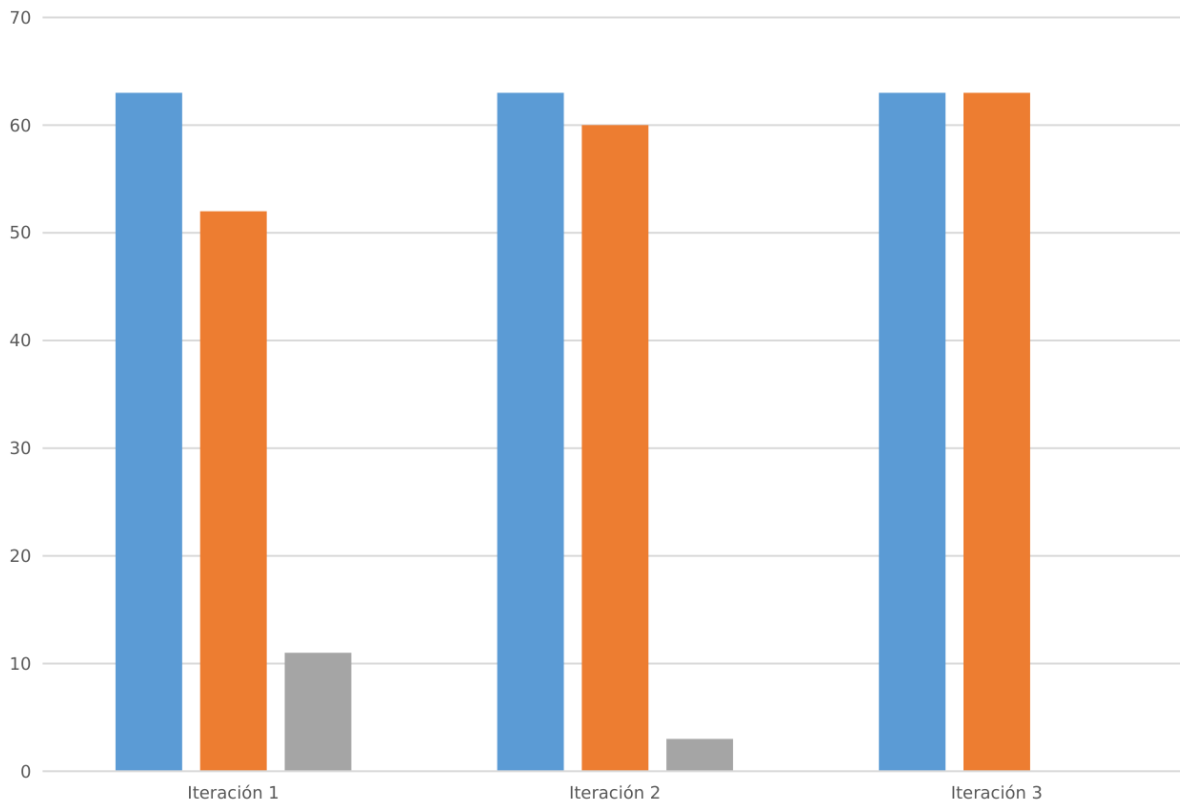


Figura 3.7. Resultados de las Pruebas Aceptación. Fuente: Elaboración propia.

3.4. Conclusiones del capítulo

Luego del desarrollo del presente capítulo se arribaron a las siguientes conclusiones:

- La fase de implementación demostró que la adopción de la metodología **XP** con descomposición de historias de usuario en **TI** fue acertada para un proyecto de esta naturaleza. La granularidad de las tareas permitió un seguimiento preciso del avance, la integración continua de nuevas funcionalidades, una detección temprana de desviaciones en los plazos estimados y una distribución equilibrada de la carga de trabajo. Las 72 **TI** ejecutadas a lo largo de siete iteraciones evidenciaron que las mayores complejidades técnicas se concentraron en los servicios de análisis estructural de **FM**, generación de configuraciones y gestión de versiones, lo que orientó los esfuerzos de optimización hacia estas áreas críticas.
- La arquitectura de despliegue garantiza que el sistema sea escalable, resiliente a fallos, seguro (mediante **HTTPS** y gestión centralizada de secretos) y fácil de administrar gracias a la naturaleza declarativa de Kubernetes.
- Las pruebas unitarias y de integración, al ejecutarse de forma iterativa durante todo el desarrollo, permitieron detectar y corregir 24 fallas (16 en pruebas unitarias y 8 en pruebas de integración) antes de que estas alcanzaran las pruebas de aceptación. Esta detección temprana redujo drásticamente la cantidad de errores severos encontrados por el cliente en la fase final, donde solo se reportaron 11 fallas iniciales, en su mayoría relacionadas con aspectos periféricos y no con la lógica central del sistema.
- Las estrategias de optimización aplicadas (caché distribuida con Redis, procesamiento asíncrono con Celery, optimización de consultas a la **BD**, compresión de respuestas y escalado horizontal con Kubernetes) lograron una reducción promedio del 61 % en los tiempos de respuesta de los ocho controladores críticos evaluados. En el escenario de estrés (1000 usuarios simultáneos), las mejoras más significativas se observaron en los controladores de validación completa (reducción del 74 %, de 11380 ms a 2991 ms) y de análisis estructural (reducción del 72 %, de 10390 ms a 2691 ms), que inicialmente presentaban los mayores cuellos de botella. En consecuencia, se concluye que el servicio *backend* satisface los requisitos no funcionales de rendimiento establecidos, garantizando una experiencia de usuario fluida incluso bajo condiciones de alta demanda y en escenarios de trabajo con modelos curriculares de gran complejidad.
- Las pruebas de seguridad confirmaron que el sistema satisface la totalidad de los requisitos no funcionales de seguridad definidos. La implementación de autenticación mediante tokens **JWT** con ciclo de vida completo, lista negra para tokens comprometidos, comunicación cifrada mediante **HTTPS**, uso exclusivo de **UUID** como identificadores, control de acceso basado en roles y eliminación lógica mediante cambio de estado garantiza la protección de la información curricular sensible frente a accesos no autorizados.
- La culminación de las pruebas de aceptación con un 100 % de casos exitosos en la tercera iteración (0 fallas sobre 63 casos) confirma que el servicio *backend* satisface los criterios de aceptación definidos conjuntamente con el cliente. Todas las **HU** priorizadas en el plan de entregas fueron validadas satisfactoriamente, demostrando que el sistema resuelve efectivamente el problema de gestión de variabilidad curricular para el cual fue concebido.

Tras el desarrollo integral de esta investigación, se arriba a las siguientes conclusiones fundamentales:

- La solución demuestra que es posible captar la variabilidad curricular mediante la técnica de los **FM** de la **SPLE**. El servicio *backend* implementado permite representar formalmente planes de estudio, definir relaciones de obligatoriedad, optatividad y prerrequisitos como restricciones formales, y derivar configuraciones válidas que corresponden a itinerarios formativos personalizados. En consecuencia, se cumple satisfactoriamente el objetivo general planteado al inicio de la investigación.
- El servicio *backend* desarrollado se posiciona estratégicamente en el mercado educativo, constituyendo una innovación con pertinencia tecnológica y académico-formativa. Al cubrir la brecha existente entre los sistemas de gestión académica tradicionales (incapaces de modelar variabilidad) y las potentes herramientas de ingeniería de líneas de productos (diseñadas para expertos en software), la solución aporta un enfoque accesible para gestores curriculares no especializados en ingeniería de software, democratizando el uso de técnicas avanzadas de variabilidad en el dominio educativo.
- La adopción de la metodología ágil **XP** fue acertada al facilitar un desarrollo iterativo y colaborativo, permitiendo una adaptación continua a los requisitos del cliente y una detección temprana de errores. La estrecha participación del usuario durante todo el proceso aseguró que el sistema final se alinee plenamente con las necesidades reales.
- La arquitectura en capas adoptada, combinada con los principios de diseño **GRASP** y los patrones **GoF** proporcionó una base sólida para la implementación del sistema. Esta organización permitió una clara separación de responsabilidades, facilitó la mantenibilidad del código y la modularidad del mismo para luego ser sometido a pruebas sin complicaciones.
- El diagrama de despliegue evidencia que el sistema se despliega sobre un clúster K3s, una distribución ligera de Kubernetes, garantizando alta disponibilidad, escalabilidad automática y tolerancia a fallos. La disposición de los componentes —tres réplicas del *backend* FastAPI, dos trabajadores Celery y los servicios de infraestructura (PostgreSQL, Redis, MinIO) organizados en los espacios de nombres `ingress` y `production`— asegura que el tráfico externo entre exclusivamente a través del controlador Traefik, manteniendo el resto de la infraestructura aislada de accesos externos directos. Los volúmenes persistentes y las reclamaciones de volumen persistente garantizan la supervivencia de los datos más allá del ciclo de vida de los contenedores.
- Gracias a los resultados obtenidos en las pruebas de rendimiento, el sistema es capaz de mantenerse operativo en altas condiciones de demanda, soportando hasta 1000 usuarios simultáneos sin caídas del servicio y con tiempos de respuesta inferiores a 3 segundos en el escenario de estrés para las operaciones críticas. Los tiempos de respuesta obtenidos garantizan una experiencia de usuario fluida incluso durante períodos de planificación curricular intensiva.

Como aporte al continuo proceso de mejora y evolución que caracteriza a cualquier sistema de software, se sugieren las siguientes acciones:

- Desarrollar un *frontend* que consuma el servicio *backend* obtenido como resultado, permitiendo a las instituciones académicas interactuar con los [FM](#) mediante una interfaz gráfica intuitiva y así captar la variabilidad de sus planes de formación.
- Incorporar técnicas de inteligencia artificial y aprendizaje automático para asistir a los gestores curriculares en el proceso de modelado de la variabilidad, sugiriendo estructuras de características, relaciones y restricciones a partir del análisis de planes de estudio existentes o de estándares curriculares de referencia, reduciendo así la curva de aprendizaje asociada al uso de los [FM](#).
- Incorporar mecanismos de colaboración en tiempo real sobre los [FM](#), permitiendo que múltiples gestores académicos trabajen simultáneamente en el diseño curricular, con funcionalidades de bloqueo de elementos bajo edición, fusión automática de cambios y registro detallado de contribuciones para garantizar la trazabilidad y la integridad del modelo.

actualizaciones de rodillo Estrategia de actualización que reemplaza gradualmente instancias antiguas de una aplicación por instancias nuevas sin causar tiempo de inactividad. Kubernetes actualiza progresivamente los pods, generalmente reemplazando uno a uno y verificando la salud antes de continuar. Permite reversiones rápidas si algo sale mal y asegura disponibilidad continua del servicio (**kubernetes2025rolling-updates**). [93](#)

balanceador de carga Tipo de servicio Kubernetes que expone un servicio externamente mediante un balanceador de carga proporcionado por el proveedor de nube (AWS, GCP, Azure) o configurado manualmente en entornos on-premise. Asigna una dirección IP pública (o accesible desde fuera del clúster) y distribuye el tráfico entrante entre los pods que respaldan el servicio (**kubernetes2025loadbalancer**). [95](#)

ClusterIP Tipo de servicio por defecto en Kubernetes que expone el servicio mediante una dirección IP interna accesible únicamente desde dentro del clúster. Es útil para la comunicación entre diferentes componentes internos del sistema (por ejemplo, FastAPI comunicándose con PostgreSQL) sin exponerlos directamente al exterior (**kubernetes2025clusterip**). [94](#), [95](#)

Clúster Conjunto de nodos (máquinas físicas o virtuales) que ejecutan aplicaciones contenerizadas gestionadas por Kubernetes. Un clúster consta de al menos un nodo maestro (plano de control) y varios nodos trabajadores. El plano de control gestiona el estado del clúster, mientras que los nodos trabajadores ejecutan las cargas de trabajo (**kubernetes2025components**). [93–95](#)

ConfigMap Recurso que permite desacoplar la configuración del código de las aplicaciones contenerizadas. Almacena pares clave-valor con información de configuración no sensible (como rutas, nombres de servicios, etc.) que puede ser inyectada en los pods como variables de entorno o archivos montados (**kubernetes2025configmap**). [95](#)

Espacio de nombres Mecanismo de aislamiento lógico dentro de un clúster Kubernetes que permite dividir los recursos del clúster entre múltiples usuarios, equipos o entornos (desarrollo, pruebas, producción). Los recursos dentro de un mismo espacio de nombres pueden referenciarse entre sí mediante nombres cortos, mientras que entre espacios diferentes se requiere calificación (**kubernetes2025namespaces**). [93](#)

etcd Base de datos clave-valor distribuida de código abierto que actúa como almacén de datos consistente del plano de control de Kubernetes. Almacena la configuración completa, el estado de todos los objetos del clúster (pods, servicios, volúmenes, etc.) y es la única fuente de verdad. Proporciona garantías de consistencia y puede configurarse para alta disponibilidad con replicación (**kubernetes2025etcd**). [93](#)

Ingress Recurso de Kubernetes que gestiona el acceso externo a los servicios dentro del clúster, típicamente a través de [HTTPS](#) o [HTTP](#). Permite definir reglas de enrutamiento basadas en el nombre de host y la ruta de la URL, así como configurar la terminación de certificados [TLS](#) (**kubernetes2025ingress**). [92](#), [93](#), [95](#)

- K3s** Distribución certificada de Kubernetes de código abierto, ligera y compatible con el estándar Kubernetes, diseñada específicamente para entornos de borde, IoT y despliegues con recursos limitados. Requiere menos memoria y CPU que una instalación estándar de Kubernetes, lo que la hace ideal para clústeres pequeños y servidores de gama baja ([k3s2025docs](#)). [93](#), [94](#)
- kube-apiserver** Servidor API de Kubernetes que actúa como la puerta de entrada principal al clúster. Es el componente central que expone la [API](#) de Kubernetes a través de una interfaz [REST](#). Todos los clientes (kubect, kubelet, controladores) comunican con él para crear, modificar, eliminar y consultar objetos del clúster. Utiliza etcd como almacén de datos persistente ([kubernetes2025apiserver](#)). [92](#), [93](#)
- kube-controller-manager** Componente del plano de control que ejecuta múltiples controladores responsables de regular el estado del clúster. Cada controlador monitorea el estado actual y trabaja para llevarlo al estado deseado. Entre ellos están el controlador de replicación (asegura que haya el número correcto de réplicas), controlador de nodos y otros que gestionan recursos y ciclos de vida ([kubernetes2025controller-manager](#)). [93](#)
- kube-proxy** Componente de red que se ejecuta en cada nodo y es responsable de implementar las reglas de enrutamiento de servicios de Kubernetes. Mantiene las reglas de red (iptables, ipvs o userspace) que permiten que el tráfico destinado a un servicio sea correctamente redirigido a los pods correspondientes. Sin kube-proxy, los servicios ClusterIP no funcionarían ([kubernetes2025kube-proxy](#)). [93](#), [94](#)
- kube-scheduler** Componente del plano de control que asigna pods a nodos. Observa los pods recién creados sin nodo asignado y selecciona nodos adecuados basándose en restricciones de recursos (CPU, memoria), políticas de afinidad, anti-afinidad y otras estrategias de planificación ([kubernetes2025scheduler](#)). [93](#)
- Kubect** Herramienta de línea de comandos de Kubernetes que permite a los administradores y desarrolladores interactuar con el clúster. Envía comandos al kube-apiserver para crear, inspeccionar, actualizar y eliminar recursos de Kubernetes. Puede usarse para desplegar aplicaciones, inspeccionar y gestionar recursos del clúster, ver logs y ejecutar comandos dentro de contenedores ([kubernetes2025kubect](#)). [92](#), [93](#)
- kubelet** Agente que se ejecuta en cada nodo del clúster y es responsable de asegurar que los contenedores especificados en los manifiestos de pods se ejecuten realmente en el nodo. Se comunica constantemente con el plano de control a través del kube-apiserver, reporta el estado del nodo y de los pods, gestiona volúmenes persistentes y ejecuta sondas de salud ([kubernetes2025kubelet](#)). [93](#), [94](#)
- nodo** Máquina física o virtual que forma parte de un clúster Kubernetes. Cada nodo contiene los servicios necesarios para ejecutar pods, incluyendo el kubelet (agente que se comunica con el plano de control), el tiempo de ejecución de contenedores (por ejemplo, Docker) y el proxy de red kube-proxy ([kubernetes2025nodes](#)). [93–95](#)
- plano de control** Conjunto de componentes que gestionan el estado global del clúster Kubernetes. Incluye el servidor API (kube-apiserver), el almacén clave-valor etcd, el controlador de planificación (kube-scheduler) y los controladores integrados (kube-controller-manager). Es el cerebro del clúster que toma decisiones sobre escalado, reinicio y distribución de pods ([kubernetes2025controlplane](#)). [92](#), [93](#)
- Pod** Unidad mínima y desplegable en Kubernetes que encapsula uno o más contenedores (generalmente Docker), almacenamiento, una dirección IP única y opciones de configuración. Los contenedores dentro de un mismo pod comparten el ciclo de vida, el espacio de red y pueden comunicarse mediante localhost ([kubernetes2025pods](#)). [92–95](#)
- Reclamación de volumen persistente** Solicitud de almacenamiento realizada por un usuario o un pod. La PVC especifica la cantidad de almacenamiento requerido y el modo de acceso (lectura-escritura por un solo nodo,

lectura-escritura por múltiples nodos, etc.). Kubernetes vincula la PVC con un volumen persistente (PV) que cumple con los requisitos solicitados (**kubernetes2025pvc**). [94](#), [95](#)

Secreto Recurso similar al ConfigMap pero destinado a almacenar información sensible, como contraseñas, tokens de autenticación, claves SSH o credenciales de bases de datos. Los datos se codifican en base64 y Kubernetes puede gestionar su acceso de forma más restrictiva que los ConfigMaps convencionales (**kubernetes2025secrets**). [95](#)

Servicio Abstracción que define un conjunto lógico de pods y una política de acceso a ellos. Un servicio expone una dirección IP virtual y un puerto que permanecen estables incluso cuando los pods subyacentes se crean, destruyen o reubican. Los tipos más comunes son ClusterIP (accesible solo dentro del clúster) y LoadBalancer (accesible desde fuera del clúster) (**kubernetes2025services**). [92–95](#)

trabajo Recurso de Kubernetes que crea uno o más pods y se asegura de que un número específico de ellos se complete exitosamente. A diferencia de otros controladores como Deployment o ReplicaSet, que mantienen pods en ejecución continua, los Jobs están diseñados para tareas que tienen un inicio y un fin definidos (procesamiento por lotes, migraciones de base de datos, exportaciones programadas, etc.). Un Job crea los pods necesarios y monitorea su finalización; si un pod falla, el Job lo recrea automáticamente hasta alcanzar el número de terminaciones exitosas especificado en su configuración. (**kubernetes2025jobs**). [94](#)

Traefik Controlador de Ingress (Ingress Controller) de código abierto que actúa como enrutador y balanceador de carga de capa 7 para clústeres Kubernetes, Docker y otros orquestadores. En el despliegue propuesto, Traefik funciona como punto único de entrada externa, recibiendo peticiones [HTTPS](#) del cliente y enrutándolas al servicio FastAPI según las reglas definidas. Gestiona automáticamente los certificados [TLS](#) y proporciona terminación de conexiones seguras (**traefik2025docs**). [93](#)

Volumenes persistentes Recursos de almacenamiento aprovisionado en el clúster, independiente del ciclo de vida de cualquier pod que lo utilice. Representa un recurso de almacenamiento real en la infraestructura subyacente (disco en la nube, almacenamiento en red, volumen local, etc.) y puede ser aprovisionado estáticamente por el administrador o dinámicamente mediante clases de almacenamiento (**kubernetes2025pv**). [92](#), [94](#), [95](#)

YAML Lenguaje de serialización de datos legible por humanos, cuyo acrónimo recursivo significa *YAML Ain't Markup Language* (YAML no es un lenguaje de marcado). Se utiliza ampliamente en archivos de configuración debido a su sintaxis limpia y expresiva, que emplea indentación para representar estructuras jerárquicas (diccionarios, listas y tipos escalares). En el ecosistema Kubernetes, todos los recursos del clúster (pods, servicios, configmaps, secretos, volúmenes persistentes, entre otros) se definen declarativamente mediante archivos YAML, los cuales describen el estado deseado del sistema. La naturaleza declarativa y legible de YAML facilita la administración, el control de versiones y la automatización de despliegues mediante prácticas de [IaC](#) (**yaml2025spec**; **kubernetes2025yaml**). [92](#), [95](#)

- AAFM** Análisis Automatizado de Feature Models. [13](#)
- ACID** Atomicidad, Consistencia, Aislamiento, Durabilidad. [34](#)
- API** Interfaz de Programación de Aplicaciones. [29](#), [30](#), [33](#), [38](#), [39](#), [45](#), [76](#), [83](#), [92](#), [95](#), [101](#), [103–105](#), [118](#)
- BD** Base de datos. [76](#), [77](#), [82](#), [84](#), [87](#), [91](#), [93](#), [94](#), [96](#), [98](#), [101](#), [105](#), [114](#), [169](#), [181](#), [182](#), [185](#), [187](#), [189](#), [193](#), [196–199](#), [201](#), [202](#), [204–206](#), [208](#), [210](#), [211](#), [220](#), [222](#), [223](#), [233](#), [239](#), [241](#), [245–249](#)
- BDD** Diagrama de Decisión Binaria. [29](#), [61](#)
- BPMN** Notación de Modelado de Procesos de Negocio. [44](#)
- CASE** Ingeniería de Software Asistida por Computadora. [44](#), [45](#)
- CI** Integración Continua. [97](#), [197](#)
- CI/CD** Integración Continua y Despliegue Continuo. [36](#), [43](#), [44](#), [76](#)
- CRC** Clase, Responsabilidad y Colaboración. [52](#), [54](#), [81](#)
- CRUD** Creación, Lectura, Actualización y Eliminación. [58](#), [83](#), [87](#), [197](#)
- DDD** Diseño Orientado al Dominio. [83](#)
- DEAP** Algoritmos Evolutivos Distribuidos en Python. [61](#)
- DIMACS** Formateo DIMACS. [60](#), [66](#), [127](#)
- DOT** Lenguaje DOT de Graphviz. [60](#), [66](#), [127](#), [137](#), [149](#), [218](#)
- DRF** Django REST Framework. [33](#)
- FM** Modelo de Características. [vii](#), [x](#), [xi](#), [2](#), [3](#), [6–16](#), [19](#), [20](#), [23–34](#), [37–42](#), [52](#), [54](#), [56](#), [58–60](#), [63–67](#), [77–81](#), [83–89](#), [95](#), [98–105](#), [111](#), [114–116](#), [124–126](#), [134](#), [137](#), [181–184](#), [188](#), [193](#), [194](#), [209](#)
- FODA** Análisis de Dominio Orientado a Características. [8](#)
- GoF** Banda de los Cuatro. [viii](#), [54](#), [88](#), [89](#), [115](#)
- GRASP** Patrones de Software Generales para la Asignación de Responsabilidades. [54](#), [87](#), [89](#), [115](#)
- HTTP** Protocolo de Transferencia de Hipertexto. [29](#), [39](#), [45](#), [76](#), [83](#), [101](#), [112](#), [113](#), [117](#), [197–250](#)
- HTTPS** Protocolo de Transferencia de Hipertexto Seguro. [76](#), [92](#), [93](#), [95](#), [114](#), [117](#), [119](#), [197](#)
- HU** Historias de Usuario. [50–52](#), [54](#), [77–80](#), [89](#), [90](#), [101](#), [111](#), [114](#), [181–195](#)
- I/O** Entrada/Salida. [33](#), [40](#)
- IaC** Infraestructura como Código. [77](#), [119](#)
- IDE** Entorno de Desarrollo Integrado. [46](#)
- IP** Protocolo de Internet. [36](#), [94](#), [95](#)

JSON Notación de Objetos JavaScript. [34](#), [60](#), [66](#), [76](#), [77](#), [83](#), [109](#), [127](#), [137](#), [217](#)

JSONB JSON Binario. [34](#)

JWT JSON Web Token. [76](#), [114](#), [196](#)

LMS Sistema de Gestión del Aprendizaje. [27](#), [28](#), [30](#)

MOOC Curso en Línea Masivo y Abierto. [2](#)

NSGA2 Algoritmo Genético de Ordenamiento No-Dominado II. [61](#)

OR-Tools Herramientas de Investigación Operativa. [61](#)

ORM Mapeo Objeto - Relacional. [34](#)

PK Clave Primaria. [197](#)

RAM Memoria de Acceso Aleatorio. [37](#)

REST Transferencia de Estado Representacional. [29](#), [30](#), [45](#), [76](#), [101](#), [103](#), [118](#)

RF Requisitos Funcionales. [61–75](#), [89](#)

RnF Requisitos no Funcionales. [75–77](#), [89](#), [196](#), [197](#)

S3 Simple Storage Service. [38](#)

SAT Satisfacibilidad Booleana. [11](#), [38](#), [42](#), [59–61](#), [79](#), [85](#), [106](#), [107](#), [130](#), [137](#)

SGBD Sistema de Gestión de Bases de Datos. [85](#)

SHARPSAT Conteo de Modelos. [11](#)

SMT Satisfacibilidad Módulo Teorías. [38](#), [42](#), [59](#), [137](#)

SPL Línea de Productos de Software. [xi](#), [2](#), [3](#), [5–7](#), [13](#), [16–20](#), [26](#), [28–30](#), [32](#), [42](#)

SPLE Ingeniería de Líneas de Productos de Software. [3](#), [5](#), [6](#), [8](#), [18–20](#), [25](#), [27–29](#), [52](#), [54](#), [115](#)

SQL Lenguaje de Consulta Estructurado. [34](#), [35](#)

SSL Capa de Sockets Seguros (predecesor de TLS). [76](#), [197](#)

TI Tareas de Ingenierías. [90](#), [114](#)

TIC Tecnologías de la Información y Comunicación. [2](#)

TLS Seguridad de la Capa de Transporte. [76](#), [93](#), [117](#), [119](#), [197](#)

TTL Tiempo de Vida. [38](#)

TVL Lenguaje Textual de Variabilidad. [60](#), [66](#), [127](#)

UML Lenguaje de Modelado Unificado. [44](#), [91](#)

URL Localizador Uniforme de Recursos. [38](#), [76](#), [84](#), [104](#), [197](#)

UUID Identificador Único Universal. [60](#), [76](#), [114](#), [197](#)

UVL Lenguaje de Variabilidad Universal. [xix](#), [12](#), [14](#), [15](#), [20](#), [26](#), [29](#), [30](#), [32](#), [52](#), [54](#), [59](#), [60](#), [66](#), [78–80](#), [84](#), [88](#), [98](#), [106](#), [111](#), [112](#), [127](#), [129](#), [130](#), [134](#), [137](#), [138](#), [216](#), [217](#), [231–233](#)

VPS Servidores Privados Virtuales. [36](#)

XML Lenguaje de Marcado Extensible. [60](#), [66](#), [127](#)

XP Extreme Programming. [48](#), [50–53](#), [77](#), [79](#), [80](#), [90](#), [97](#), [114](#), [115](#)

Apéndices

Historias de usuario

Épica 1: Gestión de Identidad y Acceso

Cuadro A.1. Historia de usuario # 2

Historia de usuario	
Número: 2	Nombre: Autenticación y Autogestión de cuenta
Épica: Gestión de Identidad y Acceso	
Prioridad en negocio: Alta	Riesgo en desarrollo: Media
Puntos estimados: 0.4	
RFs cubiertos: 1, 2, 3, 4, 5, 8	
Programador responsable: Jose Luis Echemendia López	
Descripción: El usuario del sistema debe poder iniciar sesión, cerrar sesión, gestionar su perfil y recuperar su acceso para garantizar la seguridad de sus datos y su identidad en la plataforma.	
Criterios de aceptación: • Permitir registro e inicio con correo y contraseña. • Procesar recuperación de acceso mediante token enviado por correo. • Permitir la edición del perfil propio sin intervención administrativa.	
Observaciones: La historia concentra autenticación, recuperación de acceso y autogestión del perfil como capacidades base del módulo de identidad. Campos de entrada asociados: RF#1 y RF#3 (correo electrónico, contraseña), RF#5 (token y nueva contraseña; o contraseña actual y nueva contraseña), RF#8 (correo electrónico opcional).	

Cuadro A.2. Historia de usuario # 3

Historia de usuario	
Número: 3	Nombre: Administración de Usuarios y Permisos
Épica: Gestión de Identidad y Acceso	
Prioridad en negocio: Alta	Riesgo en desarrollo: Media
Puntos estimados: 0.3	
RFs cubiertos: 6, 7, 9, 10, 11	
Programador responsable: Jose Luis Echemendia López	
Descripción: El administrador debe poder gestionar el listado de usuarios, sus roles y su estado activo o inactivo para controlar quién accede a la plataforma y qué acciones puede realizar según la jerarquía estática.	
Criterios de aceptación: • Listar y filtrar usuarios por rol o nombre. • Modificar el rol asignado a cada usuario. • Desactivar y activar cuentas sin eliminar la información persistida.	
Observaciones: La gestión de cuentas debe preservar la trazabilidad administrativa y evitar borrados físicos sobre usuarios existentes. Campos de entrada asociados: RF#9 (identificador de usuario en ruta y rol en consulta).	

Épica 2: Gestión de Dominios Curriculares

Cuadro A.3. Historia de usuario # 4

Historia de usuario	
Número: 4	Nombre: Gestión de Dominios de Conocimiento
Épica: Gestión de Dominios Curriculares	
Prioridad en negocio: Alta	Riesgo en desarrollo: Media
Puntos estimados: 0.4	
RFs cubiertos: 12, 13, 14, 15, 16, 17, 18, 19	
Programador responsable: Jose Luis Echemendia López	
Descripción: El administrador debe poder crear, actualizar y supervisar los dominios curriculares para establecer el contexto de alto nivel donde se alojarán los FM .	

Continúa en la próxima página

Cuadro A.3. Continuación de la página anterior.

Criterios de aceptación: • Crear dominios con nombre y descripción. • Activar y desactivar dominios conforme a su estado de uso. • Obtener cada dominio junto con sus modelos asociados.
Observaciones: El dominio funciona como unidad de organización para agrupar y consultar los modelos vinculados. Campos de entrada asociados: RF#14 (nombre, descripción opcional) y RF#15 (identificador de dominio en ruta; nombre y descripción opcionales).

Épica 3: Ciclo de Vida del FM

Cuadro A.4. Historia de usuario # 5

Historia de usuario	
Número: 5	Nombre: Gestión Operativa de FM
Épica: Ciclo de Vida del FM	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 20, 21, 22, 23, 24, 25	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador de modelo debe poder crear, listar y modificar los metadatos de los modelos de características dentro de un dominio para mantener un catálogo organizado de planes de estudio o arquitecturas curriculares.	
Criterios de aceptación: • Crear modelos asociados a un dominio válido. • Actualizar metadatos preservando la integridad del modelo. • Ofrecer paginación y filtrado para la consulta de modelos.	
Observaciones: Esta historia cubre la administración operativa del catálogo de modelos, sin entrar todavía en su estructura interna. Campos de entrada asociados: RF#21 (nombre, descripción opcional, identificador de dominio) y RF#23 (identificador de modelo en ruta; nombre, descripción y estado activo opcionales).	

Cuadro A.5. Historia de usuario # 6

Historia de usuario	
Número: 6	Nombre: Gestión de Versiones y Publicación
Épica: Ciclo de Vida del FM	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 2.5	
RFs cubiertos: 26, 27, 28, 29, 30, 31, 32	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador de modelo y el revisador deben poder crear versiones, publicar instantáneas y archivar versiones antiguas para garantizar la trazabilidad y evolución del currículo sin afectar configuraciones previas.	
Criterios de aceptación: • Generar nuevas versiones mediante técnica de copia al escribir. • Validar la versión antes de su publicación oficial. • Permitir la recuperación de versiones archivadas.	
Observaciones: La publicación debe conservar el historial del modelo y permitir volver a estados previos sin pérdida de información. Campos de entrada asociados: RF#28 (identificador de modelo en ruta; identificador de versión origen y descripción opcionales).	

Cuadro A.6. Historia de usuario # 7

Historia de usuario	
Número: 7	Nombre: Análisis de Métricas y Exportación
Épica: Ciclo de Vida del FM	
Prioridad en negocio: Media	Riesgo en desarrollo: Media
Puntos estimados: 0.8	
RFs cubiertos: 33, 34	
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro A.6. Continuación de la página anterior.

Descripción: El diseñador y el observador deben poder obtener estadísticas y exportar el modelo en múltiples formatos para compartirlo con otras herramientas o analizar su complejidad. Los formatos a soportar son: • UVL. • XML. • TVL. • DIMACS. • JSON. • DOT. • mermaid.
Criterios de aceptación: • Calcular métricas de la versión publicada más reciente o de una versión específica. • Exportar el modelo en formatos técnicos como UVL y dimacs. • Exportar el modelo en formatos visuales como mermaid y dot. • Guardar el resultado de la exportación en un archivo.
Observaciones: La exportación debe ser consistente con la versión consultada y servir tanto para intercambio técnico como para inspección visual.

Épica 4: Modelado de Variabilidad Estructural

Cuadro A.7. Historia de usuario # 8

Historia de usuario	
Número: 8	Nombre: Edición de la Jerarquía de Características
Épica: Modelado de Variabilidad Estructural	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 53, 54, 55, 56, 60, 61, 62, 63	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador y el editor de modelo deben poder crear, mover, actualizar y desactivar características para construir el árbol de variabilidad que representa el plan de estudios.	

Continúa en la próxima página

Cuadro A.7. Continuación de la página anterior.

Criterios de aceptación: • Reubicar una <i>feature</i> cambiando su padre sin perder consistencia. • Recuperar subárboles completos de una característica. • Aplicar desactivación lógica con soporte de versionado.
Observaciones: Las operaciones sobre la jerarquía deben preservar integridad estructural y trazabilidad de cambios. Campos de entrada asociados: RF#53 (nombre, tipo, identificador de versión, y opcionales de padre, grupo, recurso y propiedades) y RF#60 (identificador de característica en ruta; nombre, tipo, padre y grupo opcionales).

Cuadro A.8. Historia de usuario # 9

Historia de usuario	
Número: 9	Nombre: Modelado de Grupos y Relaciones Estructurales
Épica: Modelado de Variabilidad Estructural	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador y el editor de modelo deben poder definir grupos de características, como XOR y OR, y relaciones jerárquicas para restringir cómo se pueden combinar los módulos educativos.	
Criterios de aceptación: • Crear grupos con validaciones sobre el tipo de agrupación. • Definir relaciones semánticas entre características. • Permitir la definición de cardinalidades mínimas y máximas. • Mantener consistencia al activar, desactivar o actualizar dichas estructuras.	
Observaciones: Esta historia cubre la semántica estructural del árbol y la coherencia entre agrupaciones y relaciones. Campos de entrada asociados: RF#66 (tipo de grupo, característica padre, cardinalidades), RF#67 (identificador de grupo en ruta y campos opcionales), RF#72 (tipo, origen, destino) y RF#73 (identificador de relación en ruta y campos opcionales).	

Épica 5: Restricciones y Lógica Avanzada

Cuadro A.9. Historia de usuario # 10

Historia de usuario	
Número: 10	Nombre: Gestión de Restricciones Transversales
Épica: Restricciones y Lógica Avanzada	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 0.8	
RFs cubiertos: 76, 77, 78, 79, 80, 81	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador de modelo debe poder crear y gestionar reglas de exclusión e implicación complejas para modelar dependencias que no son jerárquicas, por ejemplo, si se elige A, no se puede elegir B.	
Criterios de aceptación: • Permitir crear restricciones complejas con validación de sintaxis. • Permitir expresiones lógicas complejas. • Mantener control de versiones sobre las restricciones creadas.	
Observaciones: Las restricciones deben integrarse con el modelo sin romper la evolución histórica del mismo. Campos de entrada asociados: RF#78 (identificador de versión, expresión lógica y descripción opcional) y RF#79 (identificador de restricción en ruta; expresión lógica y descripción opcionales).	

Cuadro A.10. Historia de usuario # 11

Historia de usuario	
Número: 11	Nombre: Integración y Sincronización UVL
Épica: Restricciones y Lógica Avanzada	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 36, 37, 38, 39	
Programador responsable: Jose Luis Echemendia López	
Descripción: El diseñador deben poder operar directamente sobre el lenguaje UVL o sincronizarlo desde la interfaz gráfica para mantener la potencia del modelado por código y la facilidad del modelado visual.	
Criterios de aceptación: • Sincronizar el modelo visual al editar el código UVL. • Persistir el UVL analizado y validado. • Aplicar UVL a la estructura para regenerar el árbol desde el código UVL.	

Continúa en la próxima página

Cuadro A.10. Continuación de la página anterior.

Observaciones: La sincronización debe mantener equivalencia entre la representación textual y la representación visual del modelo. Campos de entrada asociados: RF#36 y RF#38 (identificador de modelo y versión en ruta, contenido [UVL](#)).

Épica 6: Validación y Análisis de Modelos

Cuadro A.11. Historia de usuario # 12

Historia de usuario	
Número: 12	Nombre: Validación Lógica y Estructural del Modelo
Épica: Validación y Análisis de Modelos	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 39, 40, 41, 42	
Programador responsable: Jose Luis Echemendia López	
Descripción: El revisador y el diseñador deben poder ejecutar análisis de consistencia, detectar características muertas y redundancias para asegurar que el plan de estudios sea lógicamente posible y no presente errores.	
Criterios de aceptación: • Evaluar la satisfacibilidad global del modelo mediante un solucionador SAT. • Detectar ciclos y nodos huérfanos. • Identificar características muertas y redundancias.	
Observaciones: El análisis debe advertir problemas estructurales y semánticos antes de publicar o derivar configuraciones.	

Épica 7: Configuración, Optimización y Recursos

Cuadro A.12. Historia de usuario # 13

Historia de usuario	
Número: 13	Nombre: Gestión de Configuraciones
Épica: Configuración, Optimización y Recursos	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 44, 45, 46, 47, 48, 49	

Continúa en la próxima página

Cuadro A.12. Continuación de la página anterior.

Programador responsable: Jose Luis Echemendia López
Descripción: El configurador debe poder crear, actualizar y persistir selecciones de características para generar itinerarios educativos específicos para un perfil o estudiante.
Criterios de aceptación: • Guardar configuraciones asociadas a una versión publicada. • Ofrecer listado paginado de configuraciones almacenadas. • Mantener operaciones de actualización y activación sobre configuraciones persistidas.
Observaciones: La configuración debe quedar ligada a una versión concreta del modelo para conservar su validez histórica. Campos de entrada asociados: RF#44 (nombre, descripción opcional, identificador de versión y lista de características) y RF#47 (identificador de configuración en ruta; nombre, descripción y lista de características opcionales).

Cuadro A.13. Historia de usuario # 14

Historia de usuario	
Número: 14	Nombre: Validación y Generación Automática de Configuraciones
Épica: Configuración, Optimización y Recursos	
Prioridad en negocio: Alta	Riesgo en desarrollo: Alta
Puntos estimados: 1.0	
RFs cubiertos: 49, 50, 51, 52	
Programador responsable: Jose Luis Echemendia López	
Descripción: El configurador y el estudiante deben poder validar si una selección es consistente y generar automáticamente opciones válidas u optimizadas para encontrar el mejor camino formativo sin violar prerequisites.	
Criterios de aceptación: • Validar selecciones sin necesidad de persistirlas. • Ejecutar algoritmos de optimización para proponer mejores configuraciones.	
Observaciones: La generación automática debe apoyar la toma de decisiones sin obligar a guardar resultados intermedios.	

Cuadro A.14. Historia de usuario # 15

Historia de usuario	
Número: 15	Nombre: Enriquecimiento con Recursos y Taxonomías
Épica: Configuración, Optimización y Recursos	
Prioridad en negocio: Media	Riesgo en desarrollo: Media
Puntos estimados: 0.8	
RFs cubiertos: 57, 58, 59, 82, 83, 84, 85, 86, 87, 88	
Programador responsable: Jose Luis Echemendia López	
Descripción: El editor de modelo y el configurador deben poder etiquetar características y vincular recursos externos, como archivos o enlaces, para añadir contenido pedagógico real a la estructura lógica del modelo.	
Criterios de aceptación: • Asociar y desasociar etiquetas a características. • Cargar recursos y asignar propietarios. • Mantener versionada la información asociada a los recursos. • Aceptar metadatos como idioma, licencia y otros de interés en el registro de los recursos.	
Observaciones: Los recursos y etiquetas deben servir como apoyo semántico y documental para el uso del modelo. Campos de entrada asociados: RF#84 (nombre, descripción opcional), RF#87 (metadatos completos del recurso y contenido) y RF#88 (identificador de recurso en ruta y campos opcionales, con restricción sobre propietario).	

Tarjetas CRC

Cuadro B.1. Tarjeta CRC # 2

Tarjeta CRC	
Clase: User	
Responsabilidad	Colaboración
• Representar usuarios y roles. • Mantener estado activo/inactivo. • Soportar auditoría (creado/actualizado/eliminado por).	FeatureModel (owner/collaborator) FeatureModelCollaborator Repositorios de usuario

Cuadro B.2. Tarjeta CRC # 3

Tarjeta CRC	
Clase: Domain	
Responsabilidad	Colaboración
• Agrupar modelos por dominio. • Mantener metadatos (nombre, descripción). • Gestionar relación con modelos.	FeatureModel

Cuadro B.3. Tarjeta CRC # 4

Tarjeta CRC	
Clase: FeatureModel	

Continúa en la próxima página

Cuadro B.3. Continuación de la página anterior.

Responsabilidad	Colaboración
• Representar un FM (nombre, descripción, dominio). • Enlazar propietario y versiones. • Mantener colaboradores.	Domain User FeatureModelVersion FeatureModelCollaborator

Cuadro B.4. Tarjeta CRC # 5

Tarjeta CRC	
Clase: FeatureModelVersion	
Responsabilidad	Colaboración
• Versionar un FM (estado, snapshot, UVL). • Enlazar estructura completa (features, grupos, relaciones, restricciones y configuraciones).	FeatureModel Feature FeatureGroup FeatureRelation Constraint Configuration

Cuadro B.5. Tarjeta CRC # 6

Tarjeta CRC	
Clase: Feature	
Responsabilidad	Colaboración
• Representar característica (nombre, tipo, propiedades). • Mantener jerarquía padre/hijos. • Enlazar grupo y recurso. • Soportar tags y configuraciones.	FeatureModelVersion FeatureGroup FeatureRelation Tag Configuration Resource

Cuadro B.6. Tarjeta CRC # 7

Tarjeta CRC
Clase: FeatureGroup

Continúa en la próxima página

Cuadro B.6. Continuación de la página anterior.

Responsabilidad	Colaboración
- Definir agrupaciones XOR/OR. - Controlar cardinalidades. - Enlazar feature padre y miembros.	FeatureModelVersion Feature

Cuadro B.7. Tarjeta CRC # 8

Tarjeta CRC	
Clase: FeatureRelation	
Responsabilidad	Colaboración
Representar relaciones cross-tree (requires/excludes).	FeatureModelVersion Feature

Cuadro B.8. Tarjeta CRC # 9

Tarjeta CRC	
Clase: Constraint	
Responsabilidad	Colaboración
Persistir restricciones lógicas complejas (expr_text, expr_cnf).	FeatureModelVersion

Cuadro B.9. Tarjeta CRC # 10

Tarjeta CRC	
Clase: Configuration	
Responsabilidad	Colaboración
- Persistir selección de features. - Enlazar tags. - Mantener estado activo/inactivo.	FeatureModelVersion Feature Tag

Cuadro B.10. Tarjeta CRC # 11

Tarjeta CRC	
Clase: Tag	
Responsabilidad	Colaboración
• Clasificar features/configuraciones. • Mantener unicidad por nombre.	Feature Configuration

Cuadro B.11. Tarjeta CRC # 12

Tarjeta CRC	
Clase: Resource	
Responsabilidad	Colaboración
• Representar recursos educativos. • Mantener metadatos de contenido/licencia. • Enlazar propietario.	User Feature

Cuadro B.12. Tarjeta CRC # 13

Tarjeta CRC	
Clase: FeatureModelVersionManager	
Responsabilidad	Colaboración
• Crear versiones (copy-on-write). • Publicar/archivar/restaurar versiones. • Generar snapshot y validaciones de transición.	FeatureModelVersionRepository FeatureModelVersion Feature User FeatureModel

Cuadro B.13. Tarjeta CRC # 14

Tarjeta CRC	
Clase: FeatureModelExportService	
Responsabilidad	Colaboración

Continúa en la próxima página

Cuadro B.13. Continuación de la página anterior.

• Exportar FM a UVL/XML/TVL/DIMACS/JSON/DOT/FeatureModel • Construir mapeo UUID ↔ int.	FeatureModelVersion Feature Constraint Enums de exportación
---	--

Cuadro B.14. Tarjeta CRC # 15

Tarjeta CRC	
Clase: FeatureModelUVLImporter	
Responsabilidad	Colaboración
• Validar UVL. • Construir estructura desde UVL. • Crear relaciones/restricciones y nueva versión.	FeatureModelVersionManager Feature FeatureGroup FeatureRelation Constraint FeatureModelVersion User

Cuadro B.15. Tarjeta CRC # 16

Tarjeta CRC	
Clase: FeatureModelLogicalValidator	
Responsabilidad	Colaboración
• Ejecutar validación SAT/SMT (sympy/pysat/z3). • Verificar consistencia global y configuraciones.	InvalidConstraintException y otras excepciones Dependencias SAT/SMT

Cuadro B.16. Tarjeta CRC # 17

Tarjeta CRC	
Clase: FeatureModelConfigurationGenerator	
Responsabilidad	Colaboración

Continúa en la próxima página

Cuadro B.16. Continuación de la página anterior.

• Generar configuraciones válidas. • Aplicar estrategias GREEDY/RANDOM/BEAM/GENETIC/NSGA2.	FeatureModelLogicalValidator GenerationStrategy Dependencias DEAP/OR-Tools/BDD
---	--

Cuadro B.17. Tarjeta CRC # 18

Tarjeta CRC	
Clase: FeatureModelStructuralAnalyzer	
Responsabilidad	Colaboración
• Analizar estructura (dead features, redundancias, SCC). • Calcular métricas de complejidad.	AnalysisType networkx Excepciones estructurales

Cuadro B.18. Tarjeta CRC # 19

Tarjeta CRC	
Clase: FeatureModelTreeBuilder	
Responsabilidad	Colaboración
• Construir árbol completo para respuesta. • Generar estadísticas. • Generar UVL efectivo.	FeatureModelVersion Feature FeatureGroup FeatureModelExportService FeatureModelCompleteResponse

Cuadro B.19. Tarjeta CRC # 20

Tarjeta CRC	
Clase: FeatureModelAnalysisFacade	
Responsabilidad	Colaboración
• Orquestar análisis lógico y estructural. • Calcular commonality/core/atomic sets. • Ejecutar validación UVL opcional (Flamapy).	FeatureModelLogicalValidator FeatureModelStructuralAnalyzer FeatureModelUVLImporter

Cuadro B.20. Tarjeta CRC # 21

Tarjeta CRC	
Clase: SettingsService	
Responsabilidad	Colaboración
• Consultar y actualizar settings. • Cachear en Redis.	AppSetting redis_client

de

Cuadro B.21. Tarjeta CRC # 22

Tarjeta CRC	
Clase: AppSetting	
Responsabilidad	Colaboración
• Persistir claves de configuración. • Mantener descripción y valor.	SettingsService Redis cache

Tareas de ingenierías

HU-01: Autenticación y Autogestión de cuenta

Cuadro C.1. Tarea de ingeniería # 2

Tarea	
Número de tarea: 2	Número de Historia de usuario: 1
Nombre de la tarea: Diseñar modelo de token de recuperación	
Tipo de tarea: Definir modelos	Puntos estimados: 6
Fecha de inicio: 20 de abril de 2026	Fecha de fin: 20 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Diseñar y documentar el modelo de token de recuperación de contraseña, incluyendo ciclo de vida, expiración, estado, relación con usuario y campos de auditoría compatibles con la estrategia de persistencia.	

Cuadro C.2. Tarea de ingeniería # 3

Tarea	
Número de tarea: 3	Número de Historia de usuario: 1
Nombre de la tarea: Implementar esquemas de autenticación	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 8
Fecha de inicio: 20 de abril de 2026	Fecha de fin: 21 de abril de 2026
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro C.2. Continuación de la página anterior.

Descripción: Definir esquemas de entrada y salida para registro, inicio de sesión, recuperación de acceso, restablecimiento de contraseña y actualización de perfil, con validaciones explícitas y mensajes de error consistentes.

Cuadro C.3. Tarea de ingeniería # 4

Tarea	
Número de tarea: 4	Número de Historia de usuario: 1
Nombre de la tarea: Implementar servicio de autenticación y cuenta	
Tipo de tarea: Crear servicio	Puntos estimados: 12
Fecha de inicio: 21 de abril de 2026	Fecha de fin: 22 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Diseñar e implementar el servicio de identidad para registro, autenticación, gestión de credenciales, emisión y verificación de tokens, y actualización de perfil, garantizando trazabilidad y cumplimiento de políticas de seguridad.	

Cuadro C.4. Tarea de ingeniería # 5

Tarea	
Número de tarea: 5	Número de Historia de usuario: 1
Nombre de la tarea: Exponer endpoints de auth y perfil	
Tipo de tarea: Crear controlador	Puntos estimados: 10
Fecha de inicio: 22 de abril de 2026	Fecha de fin: 23 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de identidad con endpoints de registro, inicio/cierre de sesión, solicitud de recuperación, restablecimiento de contraseña y edición de perfil propio; incorporar validación de solicitudes, manejo uniforme de errores y contratos de respuesta consistentes.	

Cuadro C.5. Tarea de ingeniería # 6

Tarea	
Número de tarea: 6	Número de Historia de usuario: 1
Nombre de la tarea: Enviar correos de recuperación asincrónamente	

Continúa en la próxima página

Cuadro C.5. Continuación de la página anterior.

Tipo de tarea: Crear tareas de celery	Puntos estimados: 6
Fecha de inicio: 23 de abril de 2026	Fecha de fin: 24 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar la tarea asíncrona de notificación para recuperación de cuenta con políticas de reintento, plantillas de correo versionadas y registro de trazas operativas para auditoría y soporte.	

HU-02: Administración de Usuarios y Permisos

Cuadro C.6. Tarea de ingeniería # 7

Tarea	
Número de tarea: 7	Número de Historia de usuario: 2
Nombre de la tarea: Implementar esquemas para gestión de usuarios	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 6
Fecha de inicio: 27 de abril de 2026	Fecha de fin: 27 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir contratos de entrada y salida para listado paginado, filtrado por rol y término de búsqueda, actualización de rol y cambio de estado de cuenta, con reglas de validación y paginación estandarizadas.	

Cuadro C.7. Tarea de ingeniería # 8

Tarea	
Número de tarea: 8	Número de Historia de usuario: 2
Nombre de la tarea: Implementar repositorio administrativo de usuarios	
Tipo de tarea: Crear repositorio	Puntos estimados: 8
Fecha de inicio: 27 de abril de 2026	Fecha de fin: 28 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el repositorio de usuarios administrativos con consultas paginadas, filtros combinables, búsqueda textual y operaciones de actualización de rol y estado, preservando eliminación lógica y consistencia transaccional.	

Cuadro C.8. Tarea de ingeniería # 9

Tarea	
Número de tarea: 9	Número de Historia de usuario: 2
Nombre de la tarea: Implementar servicio de administración de usuarios	
Tipo de tarea: Crear servicio	Puntos estimados: 10
Fecha de inicio: 28 de abril de 2026	Fecha de fin: 29 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de administración de usuarios para orquestar permisos, reglas de transición de rol y validaciones de seguridad, incorporando controles de autorización y registro de acciones críticas.	

Cuadro C.9. Tarea de ingeniería # 10

Tarea	
Número de tarea: 10	Número de Historia de usuario: 2
Nombre de la tarea: Exponer endpoints de administración de usuarios	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 29 de abril de 2026	Fecha de fin: 30 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador administrativo para listado paginado, filtrado y búsqueda de usuarios, cambio de rol y activación/desactivación de cuentas; aplicar autorización por rol, validación de parámetros de ruta/consulta y trazabilidad de acciones administrativas.	

Cuadro C.10. Tarea de ingeniería # 11

Tarea	
Número de tarea: 11	Número de Historia de usuario: 2
Nombre de la tarea: Definir excepciones de administración de usuarios	
Tipo de tarea: Crear excepciones personalizadas	Puntos estimados: 4
Fecha de inicio: 30 de abril de 2026	Fecha de fin: 30 de abril de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Diseñar excepciones de dominio para transición inválida de rol, usuario inexistente y operación no autorizada, estandarizando códigos de error y semántica de respuesta para la capa API.	

HU-03: Gestión de Dominios de Conocimiento

Cuadro C.11. Tarea de ingeniería # 12

Tarea	
Número de tarea: 12	Número de Historia de usuario: 3
Nombre de la tarea: Ajustar modelo de dominio curricular	
Tipo de tarea: Definir modelos	Puntos estimados: 6
Fecha de inicio: 4 de mayo de 2026	Fecha de fin: 4 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Refinar el modelo de dominio curricular, incorporando restricciones de unicidad, atributos de estado, campos de auditoría y relaciones con modelos para asegurar consistencia semántica y persistencia confiable.	

Cuadro C.12. Tarea de ingeniería # 13

Tarea	
Número de tarea: 13	Número de Historia de usuario: 3
Nombre de la tarea: Implementar esquemas de dominio	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 6
Fecha de inicio: 4 de mayo de 2026	Fecha de fin: 5 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir esquemas para creación, actualización, consulta, búsqueda y detalle de dominios con modelos asociados, estableciendo validaciones de formato, límites y contratos de serialización coherentes.	

Cuadro C.13. Tarea de ingeniería # 14

Tarea	
Número de tarea: 14	Número de Historia de usuario: 3
Nombre de la tarea: Implementar repositorio de dominios	
Tipo de tarea: Crear repositorio	Puntos estimados: 8
Fecha de inicio: 5 de mayo de 2026	Fecha de fin: 6 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el repositorio de dominios con consultas paginadas, criterios de búsqueda por nombre y descripción, y recuperación eficiente de relaciones con modelos asociados.	

Cuadro C.14. Tarea de ingeniería # 15

Tarea	
Número de tarea: 15	Número de Historia de usuario: 3
Nombre de la tarea: Implementar servicio de gobernanza de dominios	
Tipo de tarea: Crear servicio	Puntos estimados: 8
Fecha de inicio: 6 de mayo de 2026	Fecha de fin: 7 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de gobernanza de dominios para gestionar ciclo de vida, validaciones de unicidad e integridad referencial, y reglas de activación/desactivación con trazabilidad administrativa.	

Cuadro C.15. Tarea de ingeniería # 16

Tarea	
Número de tarea: 16	Número de Historia de usuario: 3
Nombre de la tarea: Exponer endpoints de dominios	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 7 de mayo de 2026	Fecha de fin: 8 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de dominios para operaciones de creación, consulta, actualización, búsqueda y cambio de estado, incluyendo endpoint de detalle con modelos asociados; asegurar validación de datos, paginación y estandarización de respuestas HTTP.	

HU-04: Gestión Operativa de FM

Cuadro C.16. Tarea de ingeniería # 17

Tarea	
Número de tarea: 17	Número de Historia de usuario: 4
Nombre de la tarea: Ajustar modelo de FeatureModel y colaboradores	
Tipo de tarea: Definir modelos	Puntos estimados: 8
Fecha de inicio: 11 de mayo de 2026	Fecha de fin: 11 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro C.16. Continuación de la página anterior.

Descripción: Consolidar el modelo de FeatureModel y colaboradores, formalizando relaciones con dominio y propietario, además de reglas de unicidad y consistencia para operaciones de lectura y escritura.

Cuadro C.17. Tarea de ingeniería # 18

Tarea	
Número de tarea: 18	Número de Historia de usuario: 4
Nombre de la tarea: Implementar esquemas operativos de FM	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 8
Fecha de inicio: 11 de mayo de 2026	Fecha de fin: 12 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir esquemas operativos para alta, actualización, listado paginado y consulta de detalle del modelo, incluyendo metadatos obligatorios, validaciones de consistencia y formato de respuesta uniforme.	

Cuadro C.18. Tarea de ingeniería # 19

Tarea	
Número de tarea: 19	Número de Historia de usuario: 4
Nombre de la tarea: Implementar repositorio de FM	
Tipo de tarea: Crear repositorio	Puntos estimados: 8
Fecha de inicio: 12 de mayo de 2026	Fecha de fin: 13 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el repositorio de modelos de características con operaciones de persistencia y consultas por dominio, estado y criterios de búsqueda, optimizando lectura para escenarios paginados.	

Cuadro C.19. Tarea de ingeniería # 20

Tarea	
Número de tarea: 20	Número de Historia de usuario: 4
Nombre de la tarea: Implementar servicio operativo de FM	
Tipo de tarea: Crear servicio	Puntos estimados: 10

Continúa en la próxima página

Cuadro C.19. Continuación de la página anterior.

Fecha de inicio: 13 de mayo de 2026	Fecha de fin: 14 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio operativo del catálogo de modelos para gestionar altas, cambios de metadatos y estado operativo, aplicando validaciones de negocio e integridad de información.	

Cuadro C.20. Tarea de ingeniería # 21

Tarea	
Número de tarea: 21	Número de Historia de usuario: 4
Nombre de la tarea: Exponer endpoints de ciclo operativo de FM	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 14 de mayo de 2026	Fecha de fin: 15 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de modelos de características para alta, consulta, actualización y activación/desactivación, con soporte de filtros por dominio, paginación y políticas de autorización según rol funcional.	

HU-05: Gestión de Versiones y Publicación

Cuadro C.21. Tarea de ingeniería # 22

Tarea	
Número de tarea: 22	Número de Historia de usuario: 5
Nombre de la tarea: Ajustar modelo de versionado de FM	
Tipo de tarea: Definir modelos	Puntos estimados: 8
Fecha de inicio: 18 de mayo de 2026	Fecha de fin: 18 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir el modelo de versionado con estados explícitos, metadatos de snapshot y trazabilidad completa, estableciendo reglas de transición válidas para sostener el ciclo de vida de versiones.	

Cuadro C.22. Tarea de ingeniería # 23

Tarea	
Número de tarea: 23	Número de Historia de usuario: 5
Nombre de la tarea: Implementar repositorio de versiones	
Tipo de tarea: Crear repositorio	Puntos estimados: 10
Fecha de inicio: 18 de mayo de 2026	Fecha de fin: 19 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el repositorio de versiones con consultas por modelo, localización de versión publicada vigente y recuperación de versiones específicas con sus dependencias necesarias.	

Cuadro C.23. Tarea de ingeniería # 24

Tarea	
Número de tarea: 24	Número de Historia de usuario: 5
Nombre de la tarea: Implementar gestor de versionado copy-on-write	
Tipo de tarea: Crear clase	Puntos estimados: 14
Fecha de inicio: 19 de mayo de 2026	Fecha de fin: 21 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Diseñar e implementar la clase FeatureModelVersionManager para orquestar creación de versiones por copy-on-write, publicación, archivado y restauración, con validación estricta de transiciones de estado.	

Cuadro C.24. Tarea de ingeniería # 25

Tarea	
Número de tarea: 25	Número de Historia de usuario: 5
Nombre de la tarea: Exponer endpoints de versiones y publicación	
Tipo de tarea: Crear controlador	Puntos estimados: 10
Fecha de inicio: 21 de mayo de 2026	Fecha de fin: 22 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de versionado para listar/consultar versiones, crear versiones por copy-on-write, publicar, archivar, restaurar y obtener estructura completa de una versión; incluir validación de transiciones de estado y control de permisos por perfil.	

Cuadro C.25. Tarea de ingeniería # 26

Tarea	
Número de tarea: 26	Número de Historia de usuario: 5
Nombre de la tarea: Definir excepciones de transición de versiones	
Tipo de tarea: Crear excepciones personalizadas	Puntos estimados: 4
Fecha de inicio: 22 de mayo de 2026	Fecha de fin: 22 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir excepciones de dominio para transiciones inválidas, condiciones de publicación no satisfechas y conflictos de estado, con mapeo consistente a respuestas HTTP y contexto de diagnóstico.	

HU-06: Análisis de Métricas y Exportación

Cuadro C.26. Tarea de ingeniería # 27

Tarea	
Número de tarea: 27	Número de Historia de usuario: 6
Nombre de la tarea: Implementar servicio de exportación multiformato	
Tipo de tarea: Crear servicio	Puntos estimados: 16
Fecha de inicio: 25 de mayo de 2026	Fecha de fin: 26 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de exportación multiformato para generar representaciones UVL, XML, TVL, DIMACS, JSON, DOT y Mermaid, incorporando mapeo estable de identificadores y validación de consistencia del artefacto generado.	

Cuadro C.27. Tarea de ingeniería # 28

Tarea	
Número de tarea: 28	Número de Historia de usuario: 6
Nombre de la tarea: Implementar esquemas de métricas y exportación	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 6
Fecha de inicio: 26 de mayo de 2026	Fecha de fin: 26 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro C.27. Continuación de la página anterior.

Descripción: Definir esquemas de solicitud y respuesta para consulta de métricas y exportación por versión objetivo o versión vigente publicada, incluyendo parámetros de formato y validaciones de compatibilidad.

Cuadro C.28. Tarea de ingeniería # 29

Tarea	
Número de tarea: 29	Número de Historia de usuario: 6
Nombre de la tarea: Exponer endpoints de estadísticas y exportación	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 26 de mayo de 2026	Fecha de fin: 27 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de análisis y exportación para consultar métricas por versión y generar artefactos en formatos técnicos y visuales; soportar negociación de formato, descarga/streaming seguro y manejo de errores de serialización.	

Cuadro C.29. Tarea de ingeniería # 30

Tarea	
Número de tarea: 30	Número de Historia de usuario: 6
Nombre de la tarea: Implementar tareas asíncronas de exportación	
Tipo de tarea: Crear tareas de celery	Puntos estimados: 8
Fecha de inicio: 27 de mayo de 2026	Fecha de fin: 28 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar tareas asíncronas de exportación para procesar cargas intensivas fuera del hilo de petición, con persistencia de estado, almacenamiento de artefactos y mecanismos de recuperación ante fallos.	

Cuadro C.30. Tarea de ingeniería # 31

Tarea	
Número de tarea: 31	Número de Historia de usuario: 6
Nombre de la tarea: Definir excepciones de exportación	
Tipo de tarea: Crear excepciones personalizadas	Puntos estimados: 4

Continúa en la próxima página

Cuadro C.30. Continuación de la página anterior.

Fecha de inicio: 28 de mayo de 2026	Fecha de fin: 28 de mayo de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir excepciones específicas para formato no soportado, versión inexistente y errores de serialización o generación de artefactos, con semántica clara para observabilidad y soporte operativo.	

HU-07: Edición de la Jerarquía de Características

Cuadro C.31. Tarea de ingeniería # 32

Tarea	
Número de tarea: 32	Número de Historia de usuario: 7
Nombre de la tarea: Ajustar modelo de Feature para edición jerárquica	
Tipo de tarea: Definir modelos	Puntos estimados: 8
Fecha de inicio: 1 de junio de 2026	Fecha de fin: 1 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Ajustar el modelo de Feature para soportar edición jerárquica segura, contemplando relaciones padre-hijo, pertenencia a grupo, vínculo con recursos, propiedades extensibles y estado lógico activo.	

Cuadro C.32. Tarea de ingeniería # 33

Tarea	
Número de tarea: 33	Número de Historia de usuario: 7
Nombre de la tarea: Implementar esquemas de jerarquía de features	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 8
Fecha de inicio: 1 de junio de 2026	Fecha de fin: 2 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir esquemas de entrada y salida para creación, actualización, movimiento, consulta de detalle y subárbol, y cambio de estado de características, con validaciones estructurales y referenciales.	

Cuadro C.33. Tarea de ingeniería # 34

Tarea	
Número de tarea: 34	Número de Historia de usuario: 7
Nombre de la tarea: Implementar repositorio de features	
Tipo de tarea: Crear repositorio	Puntos estimados: 10
Fecha de inicio: 2 de junio de 2026	Fecha de fin: 3 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el repositorio de características con operaciones de recorrido jerárquico, consultas por versión, actualizaciones parciales y desactivación lógica, garantizando integridad de relaciones padre-hijo.	

Cuadro C.34. Tarea de ingeniería # 35

Tarea	
Número de tarea: 35	Número de Historia de usuario: 7
Nombre de la tarea: Implementar servicio de jerarquía de features	
Tipo de tarea: Crear servicio	Puntos estimados: 12
Fecha de inicio: 3 de junio de 2026	Fecha de fin: 4 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de jerarquía de características para validar movimientos en el árbol, prevenir inconsistencias estructurales y ejecutar versionado copy-on-write en operaciones de activación y desactivación.	

Cuadro C.35. Tarea de ingeniería # 36

Tarea	
Número de tarea: 36	Número de Historia de usuario: 7
Nombre de la tarea: Exponer endpoints de features estructurales	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 4 de junio de 2026	Fecha de fin: 5 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de características para creación, consulta de detalle/-subárbol, actualización, movimiento en jerarquía y activación/desactivación lógica; reforzar validaciones de integridad estructural y autorización por rol de edición.	

HU-08: Modelado de Grupos y Relaciones Estructurales

Cuadro C.36. Tarea de ingeniería # 37

Tarea	
Número de tarea: 37	Número de Historia de usuario: 8
Nombre de la tarea: Ajustar modelos de FeatureGroup y FeatureRelation	
Tipo de tarea: Definir modelos	Puntos estimados: 8
Fecha de inicio: 8 de junio de 2026	Fecha de fin: 8 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir y refinar los modelos de grupos y relaciones, incorporando tipo semántico, cardinalidades, extremos de vínculo y estado lógico, junto con reglas de integridad para persistencia y validación.	

Cuadro C.37. Tarea de ingeniería # 38

Tarea	
Número de tarea: 38	Número de Historia de usuario: 8
Nombre de la tarea: Implementar esquemas de grupos y relaciones	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 8
Fecha de inicio: 8 de junio de 2026	Fecha de fin: 9 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir contratos de API para listado, creación, actualización y cambio de estado de grupos y relaciones, incorporando validaciones de cardinalidad, tipos permitidos y consistencia entre nodos.	

Cuadro C.38. Tarea de ingeniería # 39

Tarea	
Número de tarea: 39	Número de Historia de usuario: 8
Nombre de la tarea: Implementar repositorios de grupos y relaciones	
Tipo de tarea: Crear repositorio	Puntos estimados: 10
Fecha de inicio: 9 de junio de 2026	Fecha de fin: 10 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar repositorios de grupos y relaciones con consultas por versión, característica padre y tipo semántico, optimizando la carga de datos para validaciones estructurales posteriores.	

Cuadro C.39. Tarea de ingeniería # 40

Tarea	
Número de tarea: 40	Número de Historia de usuario: 8
Nombre de la tarea: Implementar servicio de semántica estructural	
Tipo de tarea: Crear servicio	Puntos estimados: 12
Fecha de inicio: 10 de junio de 2026	Fecha de fin: 11 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de semántica estructural para validar cardinalidades, coherencia de relaciones cross-tree y reglas de versionado al aplicar cambios sobre la estructura del modelo.	

Cuadro C.40. Tarea de ingeniería # 41

Tarea	
Número de tarea: 41	Número de Historia de usuario: 8
Nombre de la tarea: Exponer endpoints de grupos y relaciones	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 11 de junio de 2026	Fecha de fin: 12 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de grupos y relaciones para gestión completa de agrupaciones (incluyendo cardinalidades) y relaciones cross-tree; validar reglas semánticas del modelo y aplicar autorización granular para operaciones de edición.	

HU-09: Gestión de Restricciones Transversales

Cuadro C.41. Tarea de ingeniería # 42

Tarea	
Número de tarea: 42	Número de Historia de usuario: 9
Nombre de la tarea: Ajustar modelo de restricciones lógicas	
Tipo de tarea: Definir modelos	Puntos estimados: 6
Fecha de inicio: 15 de junio de 2026	Fecha de fin: 15 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir el modelo de restricciones lógicas con representación textual y normalizada, estado de vigencia y metadatos de versión, habilitando trazabilidad y análisis posterior.	

Cuadro C.42. Tarea de ingeniería # 43

Tarea	
Número de tarea: 43	Número de Historia de usuario: 9
Nombre de la tarea: Implementar esquemas de restricciones	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 6
Fecha de inicio: 15 de junio de 2026	Fecha de fin: 16 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir esquemas de solicitud y respuesta para ciclo completo de restricciones, incluyendo validaciones sintácticas de expresión, serialización uniforme y estructura de errores de dominio.	

Cuadro C.43. Tarea de ingeniería # 44

Tarea	
Número de tarea: 44	Número de Historia de usuario: 9
Nombre de la tarea: Implementar repositorio de restricciones	
Tipo de tarea: Crear repositorio	Puntos estimados: 8
Fecha de inicio: 16 de junio de 2026	Fecha de fin: 17 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el repositorio de restricciones con filtros por versión y estado, búsqueda paginada y recuperación de expresiones normalizadas para procesos de validación lógica.	

Cuadro C.44. Tarea de ingeniería # 45

Tarea	
Número de tarea: 45	Número de Historia de usuario: 9
Nombre de la tarea: Implementar servicio de restricciones transversales	
Tipo de tarea: Crear servicio	Puntos estimados: 12
Fecha de inicio: 17 de junio de 2026	Fecha de fin: 18 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de restricciones transversales para validar sintaxis, transformar expresiones a forma operativa y preservar trazabilidad de cambios mediante reglas de versionado.	

Cuadro C.45. Tarea de ingeniería # 46

Tarea	
Número de tarea: 46	Número de Historia de usuario: 9
Nombre de la tarea: Exponer endpoints de restricciones	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 18 de junio de 2026	Fecha de fin: 19 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de restricciones lógicas para alta, consulta, actualización y cambio de estado, asegurando validación sintáctica de expresiones, consistencia de versión y respuestas de error homogéneas para fallos de lógica.	

HU-10: Integración y Sincronización UVL

Cuadro C.46. Tarea de ingeniería # 47

Tarea	
Número de tarea: 47	Número de Historia de usuario: 10
Nombre de la tarea: Implementar importador UVL a estructura	
Tipo de tarea: Crear clase	Puntos estimados: 14
Fecha de inicio: 22 de junio de 2026	Fecha de fin: 23 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Diseñar e implementar la clase FeatureModelUVLImporter para parseo, validación semántica y construcción de estructura interna a partir de UVL, con reporte de errores orientado a diagnóstico técnico.	

Cuadro C.47. Tarea de ingeniería # 48

Tarea	
Número de tarea: 48	Número de Historia de usuario: 10
Nombre de la tarea: Implementar esquemas de sincronización UVL	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 6
Fecha de inicio: 23 de junio de 2026	Fecha de fin: 23 de junio de 2026
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro C.47. Continuación de la página anterior.

Descripción: Definir esquemas de entrada y salida para persistencia de UVL, sincronización desde estructura y aplicación sobre versión destino, con validaciones de compatibilidad y tamaño de contenido.

Cuadro C.48. Tarea de ingeniería # 49

Tarea	
Número de tarea: 49	Número de Historia de usuario: 10
Nombre de la tarea: Implementar servicio de sincronización UVL	
Tipo de tarea: Crear servicio	Puntos estimados: 12
Fecha de inicio: 23 de junio de 2026	Fecha de fin: 24 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de sincronización UVL para garantizar equivalencia entre representación visual y textual, persistir UVL efectivo y gestionar creación controlada de nuevas versiones.	

Cuadro C.49. Tarea de ingeniería # 50

Tarea	
Número de tarea: 50	Número de Historia de usuario: 10
Nombre de la tarea: Exponer endpoints de integración UVL	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 24 de junio de 2026	Fecha de fin: 25 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de integración UVL para guardar contenido validado, sincronizar UVL desde estructura y aplicar UVL sobre una versión destino; contemplar validaciones previas de consistencia y mensajes de error orientados a diagnóstico.	

Cuadro C.50. Tarea de ingeniería # 51

Tarea	
Número de tarea: 51	Número de Historia de usuario: 10
Nombre de la tarea: Definir excepciones de UVL	
Tipo de tarea: Crear excepciones personalizadas	Puntos estimados: 4

Continúa en la próxima página

Cuadro C.50. Continuación de la página anterior.

Fecha de inicio: 25 de junio de 2026	Fecha de fin: 26 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir excepciones especializadas para errores de parseo, inconsistencias de sincronización y conflictos estructurales, estandarizando el reporte técnico para depuración y soporte.	

HU-11: Validación Lógica y Estructural del Modelo

Cuadro C.51. Tarea de ingeniería # 52

Tarea	
Número de tarea: 52	Número de Historia de usuario: 11
Nombre de la tarea: Implementar validador lógico SAT/SMT	
Tipo de tarea: Crear clase	Puntos estimados: 16
Fecha de inicio: 29 de junio de 2026	Fecha de fin: 30 de junio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Diseñar e implementar la clase FeatureModelLogicalValidator para verificar satisfacibilidad global, consistencia lógica y validez de configuraciones propuestas sobre un modelo y versión concretos.	

Cuadro C.52. Tarea de ingeniería # 53

Tarea	
Número de tarea: 53	Número de Historia de usuario: 11
Nombre de la tarea: Implementar analizador estructural	
Tipo de tarea: Crear clase	Puntos estimados: 12
Fecha de inicio: 30 de junio de 2026	Fecha de fin: 1 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Diseñar e implementar el analizador estructural para detección de ciclos, nodos huérfanos, características muertas, redundancias y componentes fuertemente conectados.	

Cuadro C.53. Tarea de ingeniería # 54

Tarea	
Número de tarea: 54	Número de Historia de usuario: 11
Nombre de la tarea: Implementar fachada de análisis del FM	
Tipo de tarea: Crear servicio	Puntos estimados: 10
Fecha de inicio: 1 de julio de 2026	Fecha de fin: 2 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar la fachada de análisis para orquestar validaciones lógicas y estructurales, consolidar resultados en un único contrato y normalizar severidad, trazas y recomendaciones técnicas.	

Cuadro C.54. Tarea de ingeniería # 55

Tarea	
Número de tarea: 55	Número de Historia de usuario: 11
Nombre de la tarea: Implementar esquemas de respuesta de análisis	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 6
Fecha de inicio: 2 de julio de 2026	Fecha de fin: 2 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir esquemas de respuesta para reportes de análisis lógico y estructural, incluyendo severidad, evidencia, recomendaciones y metadatos de ejecución para consumo por clientes y observabilidad.	

Cuadro C.55. Tarea de ingeniería # 56

Tarea	
Número de tarea: 56	Número de Historia de usuario: 11
Nombre de la tarea: Exponer endpoints de validación y análisis	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 2 de julio de 2026	Fecha de fin: 3 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de validación para análisis lógico y estructural del modelo, validación de configuraciones propuestas y reporte de hallazgos; incorporar ejecución segura con límites de tiempo y manejo controlado de excepciones de análisis.	

HU-12: Gestión de Configuraciones

Cuadro C.56. Tarea de ingeniería # 57

Tarea	
Número de tarea: 57	Número de Historia de usuario: 12
Nombre de la tarea: Ajustar modelo de Configuration	
Tipo de tarea: Definir modelos	Puntos estimados: 8
Fecha de inicio: 6 de julio de 2026	Fecha de fin: 6 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir y ajustar el modelo de Configuration con vínculo explícito a versión objetivo, selección de características, estado operativo e historial de cambios para preservar validez temporal.	

Cuadro C.57. Tarea de ingeniería # 58

Tarea	
Número de tarea: 58	Número de Historia de usuario: 12
Nombre de la tarea: Implementar esquemas de configuraciones persistentes	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 8
Fecha de inicio: 6 de julio de 2026	Fecha de fin: 7 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir esquemas para creación, consulta, listado, actualización y cambio de estado de configuraciones persistentes, así como validación de propuestas no persistidas con contratos de salida normalizados.	

Cuadro C.58. Tarea de ingeniería # 59

Tarea	
Número de tarea: 59	Número de Historia de usuario: 12
Nombre de la tarea: Implementar repositorio de configuraciones	
Tipo de tarea: Crear repositorio	Puntos estimados: 8
Fecha de inicio: 7 de julio de 2026	Fecha de fin: 8 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el repositorio de configuraciones con persistencia paginada, filtros por versión y estado, y carga eficiente de asociaciones con características seleccionadas.	

Cuadro C.59. Tarea de ingeniería # 60

Tarea	
Número de tarea: 60	Número de Historia de usuario: 12
Nombre de la tarea: Implementar servicio de gestión de configuraciones	
Tipo de tarea: Crear servicio	Puntos estimados: 12
Fecha de inicio: 8 de julio de 2026	Fecha de fin: 9 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de configuraciones para validar consistencia funcional, persistir selecciones por versión objetivo y gestionar reglas de activación/desactivación con control de estado.	

Cuadro C.60. Tarea de ingeniería # 61

Tarea	
Número de tarea: 61	Número de Historia de usuario: 12
Nombre de la tarea: Exponer endpoints de configuraciones	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 9 de julio de 2026	Fecha de fin: 10 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de configuraciones para creación persistente, consulta por identificador, listado paginado, actualización y activación/desactivación; incluir endpoint de validación de selección no persistente con contratos claros de entrada y salida.	

HU-13: Validación y Generación Automática de Configuraciones

Cuadro C.61. Tarea de ingeniería # 62

Tarea	
Número de tarea: 62	Número de Historia de usuario: 13
Nombre de la tarea: Implementar núcleo de generación de configuraciones	
Tipo de tarea: Crear clase	Puntos estimados: 10
Fecha de inicio: 13 de julio de 2026	Fecha de fin: 14 de julio de 2026
Programador responsable: Jose Luis Echemendia López	

Continúa en la próxima página

Cuadro C.61. Continuación de la página anterior.

Descripción: Diseñar e implementar la clase base de generación de configuraciones válidas, incorporando estrategias iniciales deterministas y aleatorias, validación previa de restricciones y criterios de factibilidad sobre la versión objetivo del modelo.

Cuadro C.62. Tarea de ingeniería # 63

Tarea	
Número de tarea: 63	Número de Historia de usuario: 13
Nombre de la tarea: Implementar estrategia BEAM para generación válida	
Tipo de tarea: Crear clase	Puntos estimados: 8
Fecha de inicio: 14 de julio de 2026	Fecha de fin: 14 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Extender el generador con búsqueda BEAM, incluyendo funciones de puntuación, poda y control de diversidad, para producir candidatos válidos con mejor cobertura y calidad estructural.	

Cuadro C.63. Tarea de ingeniería # 64

Tarea	
Número de tarea: 64	Número de Historia de usuario: 13
Nombre de la tarea: Implementar optimización mono-objetivo	
Tipo de tarea: Crear servicio	Puntos estimados: 8
Fecha de inicio: 14 de julio de 2026	Fecha de fin: 15 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de optimización mono-objetivo con algoritmos genéticos, definiendo función de aptitud, operadores evolutivos y criterios de convergencia para seleccionar configuraciones válidas de mayor calidad.	

Cuadro C.64. Tarea de ingeniería # 65

Tarea	
Número de tarea: 65	Número de Historia de usuario: 13
Nombre de la tarea: Implementar optimización multiobjetivo	
Tipo de tarea: Crear servicio	Puntos estimados: 8

Continúa en la próxima página

Cuadro C.64. Continuación de la página anterior.

Fecha de inicio: 15 de julio de 2026	Fecha de fin: 15 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de optimización multiobjetivo con enfoque de frentes de Pareto, equilibrando criterios de calidad contrapuestos y garantizando diversidad y estabilidad en el conjunto de soluciones candidatas.	

Cuadro C.65. Tarea de ingeniería # 66

Tarea	
Número de tarea: 66	Número de Historia de usuario: 13
Nombre de la tarea: Implementar esquemas de generación/optimización	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 6
Fecha de inicio: 15 de julio de 2026	Fecha de fin: 16 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir contratos de API para validación de propuestas, generación automática y optimización, incluyendo estrategia seleccionada, parámetros de ejecución, límites operativos y estructura homogénea de resultados.	

Cuadro C.66. Tarea de ingeniería # 67

Tarea	
Número de tarea: 67	Número de Historia de usuario: 13
Nombre de la tarea: Exponer endpoints de validación y generación automática	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 16 de julio de 2026	Fecha de fin: 17 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de configuración avanzada para validar propuestas, generar configuraciones y ejecutar optimización con parametrización controlada; estandarizar respuestas y trazas de ejecución para consumo cliente.	

Cuadro C.67. Tarea de ingeniería # 68

Tarea	
Número de tarea: 68	Número de Historia de usuario: 13

Continúa en la próxima página

Cuadro C.67. Continuación de la página anterior.

Nombre de la tarea: Ejecutar optimización intensiva en background	
Tipo de tarea: Crear tareas de celery	Puntos estimados: 8
Fecha de inicio: 17 de julio de 2026	Fecha de fin: 18 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar flujo asíncrono de optimización intensiva con colas de trabajo, control de reintentos, persistencia de estado, almacenamiento de resultados y reanudación por identificador de ejecución.	

HU-14: Enriquecimiento con Recursos y Taxonomías

Cuadro C.68. Tarea de ingeniería # 69

Tarea	
Número de tarea: 69	Número de Historia de usuario: 14
Nombre de la tarea: Ajustar modelos de Tag y Resource	
Tipo de tarea: Definir modelos	Puntos estimados: 10
Fecha de inicio: 20 de julio de 2026	Fecha de fin: 21 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir y ajustar los modelos de Tag y Resource con reglas de unicidad, metadatos extendidos y asociaciones con características y propietario, garantizando consistencia funcional y persistencia estable.	

Cuadro C.69. Tarea de ingeniería # 70

Tarea	
Número de tarea: 70	Número de Historia de usuario: 14
Nombre de la tarea: Implementar esquemas de etiquetas y recursos	
Tipo de tarea: Crear esquemas de validación entrada de la api y salida	Puntos estimados: 8
Fecha de inicio: 21 de julio de 2026	Fecha de fin: 21 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Definir esquemas para gestión completa de etiquetas y gestión parcial/completa de recursos, además de operaciones de asociación y desasociación con características, con validaciones de negocio y formato.	

Cuadro C.70. Tarea de ingeniería # 71

Tarea	
Número de tarea: 71	Número de Historia de usuario: 14
Nombre de la tarea: Implementar repositorios de taxonomías y recursos	
Tipo de tarea: Crear repositorio	Puntos estimados: 10
Fecha de inicio: 21 de julio de 2026	Fecha de fin: 22 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar repositorios de taxonomías y recursos con búsquedas paginadas, filtros por metadatos y operaciones de vinculación entre entidades, asegurando integridad relacional y consistencia de lectura.	

Cuadro C.71. Tarea de ingeniería # 72

Tarea	
Número de tarea: 72	Número de Historia de usuario: 14
Nombre de la tarea: Implementar servicio de enriquecimiento semántico	
Tipo de tarea: Crear servicio	Puntos estimados: 12
Fecha de inicio: 22 de julio de 2026	Fecha de fin: 23 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el servicio de enriquecimiento semántico para aplicar reglas de propiedad, versionado de metadatos y consistencia de asociaciones entre etiquetas, características y recursos educativos.	

Cuadro C.72. Tarea de ingeniería # 73

Tarea	
Número de tarea: 73	Número de Historia de usuario: 14
Nombre de la tarea: Exponer endpoints de tags y recursos	
Tipo de tarea: Crear controlador	Puntos estimados: 8
Fecha de inicio: 23 de julio de 2026	Fecha de fin: 24 de julio de 2026
Programador responsable: Jose Luis Echemendia López	
Descripción: Implementar el controlador de taxonomías y recursos para gestión de etiquetas, asociación/desasociación con características y administración de recursos educativos con metadatos; aplicar reglas de propietario, validaciones de negocio y respuestas consistentes para operaciones parciales y completas.	

Pruebas unitarias

Servicios

Cuadro D.1. Caso de Prueba Unitaria No1

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>FeatureModelVersionManager.create_new_version(source_version=None)</code> crea una versión en estado DRAFT con <code>version_number</code> consecutivo.
Entrada:	• <code>feature_model.id</code> válido. • Mock de <code>repository.get_latest_version_number()</code> retornando 3. • <code>source_version=None</code>.
Salida esperada:	• Nueva versión con <code>version_number=4</code>. • <code>status=DRAFT</code>. • Se ejecuta <code>session.commit()</code> y <code>session.refresh()</code>.
Evaluación de la prueba:	• Pasa si se cumplen valores y llamadas a sesión. • Falla si no incrementa versión o no persiste cambios.

Cuadro D.2. Caso de Prueba Unitaria No2

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>publish_version()</code> rechaza publicar si la versión no está en DRAFT.
Entrada:	• Versión con <code>status=PUBLISHED</code>. • <code>validate=True</code>.
Salida esperada:	• Se lanza <code>InvalidVersionStateException</code>. • No se hace <code>commit</code>.
Evaluación de la prueba:	• Pasa si lanza excepción exacta y no persiste. • Falla si permite transición inválida.

Cuadro D.3. Caso de Prueba Unitaria No3

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>archive_version()</code> cambia PUBLISHED ->ARCHIVED.
Entrada:	• Versión con <code>status=PUBLISHED</code>.
Salida esperada:	• <code>status=ARCHIVED</code>. • Se ejecuta <code>commit</code> y <code>refresh</code>.
Evaluación de la prueba:	• Pasa si transición y persistencia son correctas. • Falla si estado final no es ARCHIVED.

Cuadro D.4. Caso de Prueba Unitaria No4

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>FeatureModelExportService.export()</code> falla con formato inválido.
Entrada:	• Valor de formato no incluido en <code>ExportFormat</code> soportados.
Salida esperada:	• <code>ValueError</code> con mensaje de formato no soportado.
Evaluación de la prueba:	• Pasa si se eleva <code>ValueError</code> y no retorna contenido. • Falla si intenta exportar sin validar formato.

Cuadro D.5. Caso de Prueba Unitaria No5

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>export(ExportFormat.JSON)</code> retorna contenido serializable.
Entrada:	• Versión con features/constraints mínimas.
Salida esperada:	• Cadena JSON válida. • Incluye metadatos de modelo y versión.
Evaluación de la prueba:	• Pasa si <code>json.loads()</code> no falla y contiene campos esperados. • Falla si retorna contenido inválido o incompleto.

Cuadro D.6. Caso de Prueba Unitaria No6

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>FeatureModelUvLImporter.validate_uv1_only()</code> valida estructura UVL y no persiste datos.
Entrada:	• UVL válido con sección <code>features</code> y opcional <code>constraints</code>.
Salida esperada:	• Resultado con <code>is_valid=True</code>, <code>root</code>, conteo de <code>features</code> y <code>constraints</code>. • Sin llamadas a <code>session.commit()</code>.
Evaluación de la prueba:	• Pasa si valida y no modifica <code>BD</code>. • Falla si persiste o retorna estructura incorrecta.

Cuadro D.7. Caso de Prueba Unitaria No7

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>_parse_uv1()</code> dispara error cuando la indentación no es múltiplo de 4.
Entrada:	• UVL con línea indentada con 2 espacios.
Salida esperada:	• <code>UVLParseError</code> con detalle de indentación inválida.
Evaluación de la prueba:	• Pasa si se captura <code>UVLParseError</code>. • Falla si el parser acepta contenido mal indentado.

Cuadro D.8. Caso de Prueba Unitaria No8

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>apply_uvl()</code> crea una nueva versión, estructura de features y guarda <code>uvl_content</code> .
Entrada:	• UVL válido. • Mocks de sesión y <code>FeatureModelVersionManager.create_new_version()</code>.
Salida esperada:	• Nueva versión con <code>uvl_content</code> persistido. • Features creadas con jerarquía válida. • <code>commit</code> y <code>refresh</code> ejecutados.
Evaluación de la prueba:	• Pasa si crea estructura y persiste contenido. • Falla si no crea versión o no guarda UVL.

Cuadro D.9. Caso de Prueba Unitaria No9

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>analyze_version()</code> combina resultados lógicos/estructurales y añade <code>uvl_validation</code> cuando hay contenido UVL.
Entrada:	• Versión con features y <code>uvl_content</code>. • Mocks de validadores y <code>validate_uvl_only</code>.
Salida esperada:	• <code>AnalysisSummary</code> con <code>satisfiable</code>, <code>dead_features</code>, <code>commonality</code>, <code>complexity_metrics</code>. • <code>uvl_validation</code> presente.
Evaluación de la prueba:	• Pasa si el summary contiene datos agregados esperados. • Falla si omite integración de validadores.

Cuadro D.10. Caso de Prueba Unitaria No10

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>compare_versions()</code> calcula deltas de <code>dead_features</code> , <code>core_features</code> y configuraciones.
Entrada:	• <code>base_version</code> y <code>target_version</code> con diferencias controladas (mocks).
Salida esperada:	• Diccionario con claves <code>base</code>, <code>target</code>, <code>delta</code>. • Listas de <code>*_added</code> y <code>*_removed</code> coherentes.
Evaluación de la prueba:	• Pasa si los deltas coinciden con el escenario. • Falla si compara mal o mezcla resultados.

Repositorios

Cuadro D.11. Caso de Prueba Unitaria No11

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>BaseUserRepository.validate_email_unique()</code> lanza error si existe usuario.
Entrada:	• <code>existing_user</code> no nulo.
Salida esperada:	• <code>ValueError</code> con mensaje de email ya registrado.
Evaluación de la prueba:	• Pasa si bloquea duplicado. • Falla si permite continuar.

Cuadro D.12. Caso de Prueba Unitaria No12

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>BaseDomainRepository.validate_name_unique()</code> permite mismo nombre cuando <code>existing_domain.id == current_domain.id</code> .
Entrada:	• <code>existing_domain.id</code> igual a <code>current_domain.id</code>.
Salida esperada:	• No excepción.
Evaluación de la prueba:	• Pasa si no falla en auto-actualización. • Falla si bloquea un caso válido.

Cuadro D.13. Caso de Prueba Unitaria No13

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>validate_parent_not_self(feature_id, parent_id)</code> rechaza autorreferencia.
Entrada:	• <code>feature_id == parent_id</code>.
Salida esperada:	• <code>ValueError</code> indicando que no puede ser su propio padre.
Evaluación de la prueba:	• Pasa si detecta autorreferencia. • Falla si permite ciclo trivial.

Cuadro D.14. Caso de Prueba Unitaria No14

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>build_feature_tree()</code> transforma lista plana en árbol por <code>parent_id</code> .
Entrada:	• Lista de features con una raíz y dos hijos.
Salida esperada:	• Lista con 1 raíz y 2 nodos en <code>children</code>.
Evaluación de la prueba:	• Pasa si jerarquía coincide. • Falla si pierde nodos o relaciones.

Cuadro D.15. Caso de Prueba Unitaria No15

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>validate_same_version()</code> falla cuando origen/destino pertenecen a distintas versiones.
Entrada:	• <code>source_feature.feature_model_version_id != target_feature.feature_model_version_id</code>.
Salida esperada:	• <code>ValueError</code> indicando incompatibilidad de versión.
Evaluación de la prueba:	• Pasa si bloquea relaciones entre versiones distintas. • Falla si permite inconsistencia.

Cuadro D.16. Caso de Prueba Unitaria No16

Caso de Prueba Unitaria	
Descripción:	Verificar <code>DomainRepository.create()</code> con datos válidos.
Continúa en la siguiente página	

Cuadro D.16 – Continuación de la página anterior.

Caso de Prueba Unitaria	
Entrada:	• DomainCreate(name, description) válido.
Salida esperada:	• Entidad Domain creada y persistida. • is_active=True por defecto (si aplica en modelo).
Evaluación de la prueba:	• Pasa si persiste y retorna entidad consistente. • Falla si omite persistencia o retorna datos incompletos.

Cuadro D.17. Caso de Prueba Unitaria No17

Caso de Prueba Unitaria	
Descripción:	Verificar que UserRepository.authenticate(email, password) retorna usuario al recibir credenciales válidas.
Entrada:	• Usuario existente. • Mock de verificación de hash retornando True.
Salida esperada:	• Retorna instancia de User.
Evaluación de la prueba:	• Pasa si autentica correctamente. • Falla si retorna None con credenciales válidas.

Cuadro D.18. Caso de Prueba Unitaria No18

Caso de Prueba Unitaria	
Descripción:	Verificar que authenticate() retorna None con password inválida.
Continúa en la siguiente página	

Cuadro D.18 – Continuación de la página anterior.

Caso de Prueba Unitaria	
Entrada:	• Usuario existente. • Verificador de hash retornando False.
Salida esperada:	• None.
Evaluación de la prueba:	• Pasa si no autentica credenciales inválidas. • Falla si devuelve usuario.

Cuadro D.19. Caso de Prueba Unitaria No19

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>FeatureModelVersionRepository.get_latest_version_number()</code> retorna <code>0</code> cuando no hay versiones y el máximo correcto cuando sí existen.
Entrada:	• Escenario A: sin registros. • Escenario B: versiones con <code>version_number</code> [1,2,4].
Salida esperada:	• A: <code>0</code>. • B: <code>4</code>.
Evaluación de la prueba:	• Pasa si calcula correctamente en ambos escenarios. • Falla si retorna <code>None</code> o valor incorrecto.

Cuadro D.20. Caso de Prueba Unitaria No20

Caso de Prueba Unitaria	
Descripción:	Verificar <code>get_statistics(version_id)</code> con features, grupos, constraints y relaciones.
Entrada:	• Versión con datos de prueba predecibles.
Salida esperada:	• Diccionario con contadores correctos (<code>features</code>, <code>groups</code>, <code>constraints</code>, <code>relations</code>, etc.).
Evaluación de la prueba:	• Pasa si contadores coinciden con fixtures. • Falla si hay desalineación en métricas.

3 Excepciones

Cuadro D.21. Caso de Prueba Unitaria No21

Caso de Prueba Unitaria	
Descripción:	Verificar normalización de objeto a partir de path y método HTTP.
Entrada:	• Request con path <code>/api/v1/models/123/versions</code> y método GET.
Salida esperada:	• Objeto <code>model.get</code>.
Evaluación de la prueba:	• Pasa si remueve prefijos (<code>api</code>, <code>v1</code>) y singulariza recurso. • Falla si retorna objeto no normalizado.

Cuadro D.22. Caso de Prueba Unitaria No22

Caso de Prueba Unitaria	
Descripción:	Verificar que un <code>RequestValidationError</code> genera respuesta estandarizada HTTP 422.
Entrada:	Request válido + excepción de validación con errores de campos.
Salida esperada:	<code>JSONResponse</code> con <code>status_code=422</code>. <code>message.category=request_validation</code>. <code>message.error_code=1001</code>.
Evaluación de la prueba:	- Pasa si estructura y código de error son correctos. - Falla si cambia contrato de error.

Cuadro D.23. Caso de Prueba Unitaria No23

Caso de Prueba Unitaria	
Descripción:	Verificar mapeo de códigos para <code>HTTPException</code> (ej. 404).
Entrada:	<code>HTTPException(status_code=404, detail="Not Found")</code>.
Salida esperada:	<code>JSONResponse</code> con <code>code=404</code>, <code>error_code=1005</code>, <code>category=http_error</code>.
Evaluación de la prueba:	- Pasa si usa mapa <code>_ERROR_CODE_MAP</code>. - Falla si devuelve código/mensaje inconsistente.

Cuadro D.24. Caso de Prueba Unitaria No24

Caso de Prueba Unitaria	
Descripción:	Verificar respuesta controlada para errores no esperados.
Entrada:	• <code>Exception("boom")</code>.
Salida esperada:	• HTTP 500. • Mensaje genérico sin filtrar detalle interno. • <code>error_code=5000</code>.
Evaluación de la prueba:	• Pasa si no expone detalles sensibles y respeta contrato. • Falla si filtra stack/error interno al cliente.

Cuadro D.25. Caso de Prueba Unitaria No25

Caso de Prueba Unitaria	
Descripción:	Verificar que <code>NotFoundException</code> inicializa con código 404.
Entrada:	• <code>NotFoundException("X no encontrado")</code>.
Salida esperada:	• <code>status_code=404, detail="X no encontrado"</code>.
Evaluación de la prueba:	• Pasa si hereda y configura correctamente. • Falla si código HTTP no coincide.

Cuadro D.26. Caso de Prueba Unitaria No26

Caso de Prueba Unitaria	
Descripción:	Verificar inicialización de excepción de negocio.
Continúa en la siguiente página	

Cuadro D.26 – Continuación de la página anterior.

Caso de Prueba Unitaria	
Entrada:	• BusinessException(Regla inválida).
Salida esperada:	• status_code=400, detail correcto.
Evaluación de la prueba:	• Pasa si estado y detalle son correctos. • Falla si usa código distinto.

Cuadro D.27. Caso de Prueba Unitaria No27

Caso de Prueba Unitaria	
Descripción:	Verificar mensaje contextual cuando dominio existe.
Entrada:	• domain_name="Ingeniería de Software".
Salida esperada:	• Excepción tipo ConflictException (409). • Mensaje incluyendo nombre del dominio conflictivo.
Evaluación de la prueba:	• Pasa si conserva semántica y contexto. • Falla si mensaje pierde identificador clave.

Cuadro D.28. Caso de Prueba Unitaria No28

Caso de Prueba Unitaria	
Descripción:	Verificar composición de mensaje para transición inválida de versión.
Entrada:	• current_state="published", required_state="draft", operation="publish version".
Continúa en la siguiente página	

Cuadro D.28 – Continuación de la página anterior.

Caso de Prueba Unitaria	
Salida esperada:	• Excepción de negocio con mensaje que incluya estado actual, requerido y operación.
Evaluación de la prueba:	• Pasa si mensaje es trazable para depuración. • Falla si omite contexto de transición.

Cuadro D.29. Caso de Prueba Unitaria No29

Caso de Prueba Unitaria	
Descripción:	Verificar que se lanza excepción procesable cuando no existe raíz en el árbol.
Entrada:	• Instancia directa de <code>MissingRootFeatureException()</code>.
Salida esperada:	• Tipo <code>UnprocessableEntityException</code> con HTTP 422.
Evaluación de la prueba:	• Pasa si mantiene clasificación semántica (error estructural 422). • Falla si cambia a otro código.

Pruebas de integración

Cuadro E.1. Caso de Prueba de Integración HU4-P1

Caso de Prueba de Integración	
ID de prueba: HU4-P1	HU: 4
Nombre:	Crear FM en dominio válido
Descripción:	Verificar que POST /feature-models/ crea un FM y persiste meta-datos correctamente.
Condiciones de ejecución:	• Sistema levantado con datos de prueba. • BD limpia o controlada • Usuario autenticado con rol MODEL_DESIGNER o ADMIN • Existe domain_id activo
Pasos de ejecución:	1. Enviar POST /api/v1/feature-models/ con name, description, domain_id. 2. Validar HTTP 201. 3. Verificar respuesta con id, owner_id, domain_id, is_active=true. 4. Consultar GET /api/v1/feature-models/{id}. 5. Confirmar persistencia en BD (registro creado).
Continúa en la siguiente página	

Cuadro E.1 – Continuación de la página anterior.

Caso de Prueba de Integración	
Resultados esperados:	• Se crea el FM correctamente. • El <code>owner_id</code> corresponde al usuario autenticado. • Se registra <code>created_at/updated_at</code>.
Evaluación de la prueba:	Aprobada

Cuadro E.2. Caso de Prueba de Integración HU4-P2

Caso de Prueba de Integración	
ID de prueba: HU4-P2	HU : 4
Nombre:	Crear FM con dominio inexistente
Descripción:	Verificar manejo de error de negocio cuando <code>domain_id</code> no existe.
Condiciones de ejecución:	• Usuario autenticado con permisos de diseño • <code>domain_id</code> inexistente
Pasos de ejecución:	1. Enviar POST <code>/api/v1/feature-models/</code> con <code>domain_id</code> inválido. 2. Validar respuesta de error. 3. Verificar que no se crea ningún registro en BD.
Resultados esperados:	• HTTP 404 (o error de dominio equivalente). • Mensaje claro de dominio no encontrado. • Sin efectos colaterales en BD.
Evaluación de la prueba:	Aprobada

Cuadro E.3. Caso de Prueba de Integración HU5-P1

Caso de Prueba de Integración	
ID de prueba: HU5-P1	HU: 5
Nombre:	Crear nueva versión COW desde versión origen
Descripción:	Verificar que POST /feature-models/{model_id}/versions crea nueva versión por copy-on-write.
Condiciones de ejecución:	• Existe FM con versión origen • Usuario propietario del FM o superusuario
Pasos de ejecución:	1. Enviar POST /api/v1/feature-models/{model_id}/versions con source_version_id. 2. Validar respuesta exitosa. 3. Verificar que version_number se incrementa. 4. Confirmar que la versión origen permanece inmutable.
Resultados esperados:	• HTTP 200/201 según contrato actual. • Nueva versión creada en estado editable. • Estructura clonada sin modificar versión fuente.
Evaluación de la prueba:	Aprobada

Cuadro E.4. Caso de Prueba de Integración HU5-P2

Caso de Prueba de Integración	
ID de prueba: HU5-P2	HU: 5
Nombre:	Publicar versión válida
Descripción:	Verificar que PATCH /feature-models/{model_id}/versions/{version_id}/publish cambia estado y persiste publicación.
Continúa en la siguiente página	

Cuadro E.4 – Continuación de la página anterior.

Caso de Prueba de Integración	
Condiciones de ejecución:	• Versión existente y publicable • Usuario propietario o superusuario
Pasos de ejecución:	1. Ejecutar <code>PATCH /api/v1/feature-models/{model_id}/versions/{version_id}/publish</code>. 2. Validar respuesta. 3. Consultar versión por GET. 4. Verificar estado final y timestamp de publicación.
Resultados esperados:	• Estado pasa a PUBLISHED. • Se guarda <code>published_at</code> (si aplica). • Integridad de datos preservada.
Evaluación de la prueba:	Aprobada

Cuadro E.5. Caso de Prueba de Integración HU7-P1

Caso de Prueba de Integración	
ID de prueba: HU7-P1	HU: 7
Nombre:	Crear feature en versión de FM
Descripción:	Verificar que <code>POST /features/</code> crea feature y respeta validaciones de parent/version.
Condiciones de ejecución:	• Existe versión destino • Usuario con rol MODEL_DESIGNER/ADMIN/DEVELOPER
Continúa en la siguiente página	

Cuadro E.5 – Continuación de la página anterior.

Caso de Prueba de Integración	
Pasos de ejecución:	1. Enviar POST /api/v1/features/ con feature_model_version_id, name, type. 2. Validar HTTP 201. 3. Consultar GET /api/v1/features/{feature_id}. 4. Validar persistencia y pertenencia a la versión.
Resultados esperados:	• Feature creada con ID único. • Datos consistentes en BD. • Sin corrupción del árbol.
Evaluación de la prueba:	Aprobada

Cuadro E.6. Caso de Prueba de Integración HU7-P2

Caso de Prueba de Integración	
ID de prueba: HU7-P2	HU : 7
Nombre:	Reubicar feature con padre inválido
Descripción:	Verificar rechazo de PATCH /features/{feature_id}/move cuando el nuevo padre no pertenece a la misma versión.
Condiciones de ejecución:	• Feature origen existente • parent_id de otra versión o inexistente
Pasos de ejecución:	1. Enviar PATCH /api/v1/features/{feature_id}/move con parent_id inválido. 2. Verificar código de error. 3. Consultar la feature original.
Continúa en la siguiente página	

Cuadro E.6 – Continuación de la página anterior.

Caso de Prueba de Integración	
Resultados esperados:	• HTTP 400. • Mensaje de inconsistencia de padre. • Estructura original se mantiene intacta.
Evaluación de la prueba:	Aprobada

Cuadro E.7. Caso de Prueba de Integración HU8-P1

Caso de Prueba de Integración	
ID de prueba: HU8-P1	HU: 8
Nombre:	Crear grupo OR con cardinalidades válidas
Descripción:	Verificar integración de POST /feature-groups/ con reglas de cardinalidad y versionado COW.
Condiciones de ejecución:	• Feature padre existente • Usuario propietario del modelo o superusuario
Pasos de ejecución:	1. Enviar POST /api/v1/feature-groups/ con group_type=OR, min_cardinality, max_cardinality, parent_feature_id. 2. Validar HTTP 201. 3. Consultar grupo creado. 4. Verificar nueva versión derivada del modelo (si aplica por COW).
Resultados esperados:	• Grupo creado correctamente. • Cardinalidades persistidas. • Nueva versión consistente del modelo.
Evaluación de la prueba:	Aprobada

Cuadro E.8. Caso de Prueba de Integración HU8-P2

Caso de Prueba de Integración	
ID de prueba: HU8-P2	HU : 8
Nombre:	Crear relación EXCLUDES válida
Descripción:	Verificar que POST /feature-relations/ crea relación cross-tree y la persiste.
Condiciones de ejecución:	• Features source/target existentes en misma versión • Usuario con permisos
Pasos de ejecución:	1. Enviar POST /api/v1/feature-relations/ con type=EXCLUDES. 2. Validar HTTP 201. 3. Consultar GET /api/v1/feature-relations/{relation_id}. 4. Verificar que la relación queda activa y vinculada a la versión.
Resultados esperados:	• Relación creada y consistente. • IDs de source/target correctos. • Registro trazable en BD.
Evaluación de la prueba:	Aprobada

Cuadro E.9. Caso de Prueba de Integración HU9-P1

Caso de Prueba de Integración	
ID de prueba: HU9-P1	HU : 9
Nombre:	Crear constraint con expresión válida
Descripción:	Verificar que POST /constraints/ integra validación y persistencia de restricciones.
Continúa en la siguiente página	

Cuadro E.9 – Continuación de la página anterior.

Caso de Prueba de Integración	
Condiciones de ejecución:	• Versión de FM existente • Usuario dueño o superusuario
Pasos de ejecución:	1. Enviar POST /api/v1/constraints/ con expr_text válido. 2. Validar HTTP 201. 3. Leer constraint por ID. 4. Confirmar persistencia y estado activo.
Resultados esperados:	• Constraint creada correctamente. • Se mantiene consistencia del modelo/versionado COW.
Evaluación de la prueba:	Aprobada

Cuadro E.10. Caso de Prueba de Integración HU9-P2

Caso de Prueba de Integración	
ID de prueba: HU9-P2	HU: 9
Nombre:	Crear constraint con sintaxis inválida
Descripción:	Verificar control de errores al intentar persistir una restricción inválida.
Condiciones de ejecución:	• Versión existente • Usuario con permisos
Pasos de ejecución:	1. Enviar POST /api/v1/constraints/ con expr_text inválido. 2. Validar respuesta de error. 3. Confirmar que no se crea registro.
Continúa en la siguiente página	

Cuadro E.10 – Continuación de la página anterior.

Caso de Prueba de Integración	
Resultados esperados:	• HTTP 400. • Mensaje de validación semántica/sintáctica. • BD sin nuevos constraints inválidos.
Evaluación de la prueba:	Aprobada

Cuadro E.11. Caso de Prueba de Integración HU10-P1

Caso de Prueba de Integración	
ID de prueba: HU10-P1	HU: 10
Nombre:	Guardar UVL y disparar análisis asíncrono
Descripción:	Verificar que PUT /feature-models/{model_id}/versions/{version_id}/uvl persiste UVL y retorna analysis_task_id.
Condiciones de ejecución:	• Versión existente del modelo • Usuario propietario/superusuario • Celery/Redis disponibles
Pasos de ejecución:	1. Enviar PUT /api/v1/feature-models/{model_id}/versions/{version_id}/uvl con UVL válido. 2. Validar HTTP 200. 3. Verificar source="stored" y analysis_task_id en respuesta. 4. Consultar GET /api/v1/feature-models/{model_id}/versions/{version_id}/uvl. 5. Confirmar contenido persistido.
Resultados esperados:	• UVL guardado en versión. • Se encola tarea de análisis. • Recuperación posterior coincide con contenido enviado.
Continúa en la siguiente página	

Cuadro E.11 – Continuación de la página anterior.

Caso de Prueba de Integración	
Evaluación de la prueba:	Aprobada

Cuadro E.12. Caso de Prueba de Integración HU10-P2

Caso de Prueba de Integración	
ID de prueba: HU10-P2	HU: 10
Nombre:	Aplicar UVL a estructura creando nueva versión
Descripción:	Verificar que POST /feature-models/{model_id}/versions/{version_id}/uvl/apply-to-structure crea versión estructurada desde UVL.
Condiciones de ejecución:	• Versión base existente • UVL de entrada válido
Pasos de ejecución:	1. Enviar POST /api/v1/feature-models/{model_id}/versions/{version_id}/uvl/apply-to-structure. 2. Validar HTTP 201. 3. Verificar nuevo version_id retornado. 4. Consultar estructura de la nueva versión (features/grupos/relaciones/constraints).
Resultados esperados:	• Nueva versión creada con estructura derivada del UVL. • Versión fuente no se altera. • Se retorna analysis_task_id para procesamiento posterior.
Evaluación de la prueba:	Aprobada

Cuadro E.13. Caso de Prueba de Integración HU11-P1

Caso de Prueba de Integración	
ID de prueba: HU11-P1	HU: 11
Nombre:	Validación integral del modelo
Descripción:	Verificar que POST /feature-models/{model_id}/versions/{version_id}/validation/full devuelve resultado compuesto (lógica + estructura + análisis).
Condiciones de ejecución:	• Versión existente con features y restricciones • Usuario autenticado
Pasos de ejecución:	1. Enviar POST /api/v1/feature-models/{model_id}/versions/{version_id}/validation/full. 2. Validar HTTP 200. 3. Confirmar presencia de secciones logical, structure, analysis. 4. Revisar coherencia de is_valid y listas de errores/warnings.
Resultados esperados:	• Contrato de respuesta completo y consistente. • Sin errores de serialización. • Métricas/issue list alineadas con datos del modelo.
Evaluación de la prueba:	Aprobada

Cuadro E.14. Caso de Prueba de Integración HU11-P2

Caso de Prueba de Integración	
ID de prueba: HU11-P2	HU: 11
Nombre:	Resumen de análisis avanzado
Descripción:	Verificar POST /feature-models/{model_id}/versions/{version_id}/analysis/summary y su payload analítico.
Continúa en la siguiente página	

Cuadro E.14 – Continuación de la página anterior.

Caso de Prueba de Integración	
Condiciones de ejecución:	• Versión existente y accesible • Usuario autorizado
Pasos de ejecución:	1. Enviar POST /api/v1/feature-models/{model_id}/versions/{version_id}/analysis/summary con max_solutions y analysis_types opcionales. 2. Validar HTTP 200. 3. Verificar campos: satisfiable, dead_features, core_features, complexity_metrics, estimated_configurations.
Resultados esperados:	• Resumen calculado correctamente. • Campos obligatorios presentes y tipados. • Resultado reproducible con mismo input/base de datos.
Evaluación de la prueba:	Aprobada

Cuadro E.15. Caso de Prueba de Integración HU6-P1

Caso de Prueba de Integración	
ID de prueba: HU6-P1	HU: 6
Nombre:	Exportar última versión publicada en Mermaid
Descripción:	Verificar GET /feature-models/{model_id}/versions/latest/export/mermaid incluyendo persistencia de artefacto de exportación.
Condiciones de ejecución:	• Existe al menos una versión PUBLISHED • MinIO y Redis operativos
Continúa en la siguiente página	

Cuadro E.15 – Continuación de la página anterior.

Caso de Prueba de Integración	
Pasos de ejecución:	1. Ejecutar <code>GET /api/v1/feature-models/{model_id}/versions/latest/export/mermaid</code>. 2. Validar HTTP 200 y Content-Type de texto. 3. Verificar cabecera Content-Disposition. 4. Consultar <code>GET /api/v1/feature-models/{model_id}/exports</code> para confirmar cache/indexado de export.
Resultados esperados:	• Contenido Mermaid válido. • Export registrado en cache y almacenamiento de objetos. • URL de descarga disponible en listado de exports.
Evaluación de la prueba:	Aprobada

Cuadro E.16. Caso de Prueba de Integración HU6-P2

Caso de Prueba de Integración	
ID de prueba: HU6-P2	HU : 6
Nombre:	Obtener estadísticas de última versión publicada
Descripción:	Verificar <code>GET /feature-models/{model_id}/versions/latest/statistics</code> y exactitud mínima de métricas.
Condiciones de ejecución:	• FM existente con versión PUBLISHED
Pasos de ejecución:	1. Ejecutar <code>GET /api/v1/feature-models/{model_id}/versions/latest/statistics</code>. 2. Validar HTTP 200. 3. Verificar campos esperados (total_features, total_groups, total_relations, total_constraints, max_tree_depth, etc.). 4. Contrastar al menos 2 métricas con consulta directa de BD.
Continúa en la siguiente página	

Cuadro E.16 – Continuación de la página anterior.

Caso de Prueba de Integración	
Resultados esperados:	• Métricas consistentes con la estructura almacenada. • Contrato de respuesta completo. • Sin desalineación entre repositorio y cálculo estadístico.
Evaluación de la prueba:	Aprobada

Cuadro E.17. Caso de Prueba de Integración HU12-P1

Caso de Prueba de Integración	
ID de prueba: HU12-P1	HU: 12
Nombre:	Crear configuración válida
Descripción:	Verificar que POST /configurations/ valida selección de features y persiste configuración solo si es válida.
Condiciones de ejecución:	• Versión de FM existente • Conjunto de features que cumple restricciones
Pasos de ejecución:	1. Enviar POST /api/v1/configurations/ con feature_model_version_id y feature_ids válidos. 2. Validar HTTP 200/201 según contrato actual. 3. Recuperar configuración por ID. 4. Verificar relación con versión y features persistidos.
Resultados esperados:	• Configuración creada y almacenada. • Validación lógica aplicada antes de persistir. • Estructura de respuesta correcta.
Evaluación de la prueba:	Aprobada

Cuadro E.18. Caso de Prueba de Integración HU13-P1

Caso de Prueba de Integración	
ID de prueba: HU13-P1	HU: 13
Nombre:	Generación automática de configuraciones
Descripción:	Verificar POST /configurations/generate para múltiples resultados y métrica de calidad.
Condiciones de ejecución:	• Versión satisfacible • Motor de generación habilitado
Pasos de ejecución:	1. Enviar POST /api/v1/configurations/generate con feature_model_version_id, count, strategy. 2. Validar HTTP 200. 3. Verificar que results tenga tamaño $\leq$ count y elementos con success/selected_features. 4. Verificar objeto quality no vacío.
Resultados esperados:	• Se devuelven configuraciones válidas. • El límite solicitado se respeta. • Métricas de calidad disponibles para evaluación.
Evaluación de la prueba:	Aprobada

Lista de verificación de seguridad

Cuadro F.1. Lista de Verificación de Seguridad

Verificación	Cumple
RnF#1: El sistema debe implementar autenticación basada en tokens JWT para garantizar la seguridad de las sesiones de usuario	
Iniciar sesión emite un token de acceso firmado	✓
El token incluye identidad del usuario	✓
El sistema valida firma y espira el token en cada petición autenticada	✓
Se bloquea acceso con token inválido/expirado	✓
Se bloquea acceso para usuario inactivo	✓
RnF#2: El sistema debe implementar un ciclo de vida completo para los tokens JWT que incluya operaciones de emisión, renovación e invalidación, garantizando que los tokens tengan un tiempo de expiración limitado y puedan ser revocados explícitamente por el usuario	
Existe emisión de token de acceso y token para refrescar	✓
Existe <i>endpoint</i> de renovación de token	✓
Existe <i>endpoint</i> de invalidación explícita	✓
Se verifica expiración del token de refrescar	✓
Se usa lista negra para revocar tokens de refrescar	✓
RnF#3: El sistema debe implementar un mecanismo de eliminación lógica mediante un cambio de estado (activo/inactivo) para conservar la integridad referencial de la BD	
Las entidades del sistema incluyen <code>is_active</code>	✓
Existen operaciones de activación/desactivación	✓
Existen consultas que filtran por <code>is_active</code> en listados	✓
Todos las eliminaciones del sistema usan solo eliminación lógica	✓
Continúa en la siguiente página	

Cuadro F.1 – Continuación de la página anterior.

Verificación	Cumple
Existe política única documentada para cuándo es permitido un eliminado fuerte	✓
RnF#4: Uso exclusivo de UUID como tipo de dato para llaves primarias en las tablas de la BD , no valores numéricos. Su generación se realizará automáticamente a nivel de aplicación o BD . Esto garantiza unicidad global y mejora la seguridad al no exponer patrones secuenciales	
Las tablas usan UUID como Clave Primaria (PK)	✓
<i>Endpoints</i> usan UUID en parámetros/ <i>path</i> para entidades principales	✓
RnF#5: Solo usuarios autorizados deben poder crear, modificar y eliminar registros del sistema	
Existen mecanismos de autorización por rol reutilizables	✓
Existen rutas críticas con protección por rol	✓
Todas las rutas CRUD tienen dependencia explícita de autenticación y autorización	✓
Hay pruebas de autorización por <i>endpoint</i> y rol para todas las rutas de escritura	✓
Hay pruebas de denegación (403) por rol insuficiente en todos los CRUD	✓
RnF#6: El acceso al sistema a través de cualquier cliente debe ser mediante una comunicación segura TLS/SSL para proteger la confidencialidad e integridad de los datos transmitidos	
El despliegue productivo define <i>routers</i> HTTPS	✓
Se configura redirección de HTTP a HTTPS	✓
Se validan cabeceras de seguridad TLS/SSL en tiempo de ejecución	✓
RnF#7: Toda configuración variable (BD , credenciales, URLs externas) debe inyectarse vía variables de entorno, no hardcodeada	
Variables sensibles se leen desde Settings/env	✓
Despliegue productivo inyecta variables vía GitHub	✓
Hay validadores para secretos inseguros en producción	✓
No existen credenciales hardcodeadas en el repositorio	✓
Existe escaneo automático de secretos en CI	✓

Pruebas de aceptación

Épica 1: Gestión de Identidad y Acceso

HU-01: Autenticación y Autogestión de cuenta

Cuadro G.1. Prueba de aceptación # 2

Caso de prueba de aceptación	
Código: HU1_P1	Historia de usuario: 1
Nombre: Registro de usuario con correo y contraseña	
Descripción: Validar que un usuario nuevo pueda registrarse en el sistema con correo y contraseña válidos.	
Condiciones de ejecución: • El servidor del sistema está disponible. • La BD está accesible. • No existe un usuario registrado con el correo proporcionado.	
Pasos de ejecución: 1. Enviar POST a /api/v1/auth/register con carga útil: {".email": "usuario@test.com", "password": "Pass123!@"}. 2. Validar que la respuesta contenga estado HTTP 201. 3. Validar que la respuesta incluya el token de autenticación. 4. Validar que se creó el usuario en la BD.	

Continúa en la próxima página

Cuadro G.1. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 201 Created. • Respuesta contiene access_token, refresh_token y user_id. • Usuario registrado en BD con is_active=True. • Contraseña almacenada como hash (no en texto plano).

Cuadro G.2. Prueba de aceptación # 3

Caso de prueba de aceptación	
Código: HU1_P2	Historia de usuario: 1
Nombre: Inicio de sesión con correo y contraseña	
Descripción: Validar que un usuario registrado pueda iniciar sesión correctamente.	
Condiciones de ejecución: • El servidor del sistema está disponible. • Existe un usuario registrado con correo test@example.com y contraseña conocida.	
Pasos de ejecución: 1. Enviar POST a /api/v1/auth/login con carga útil: {".email": "test@example.com", "password": "Pass123!@"}. 2. Validar que la respuesta contenga estado HTTP 200. 3. Verificar que se devuelve un access_token válido.	
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta contiene access_token, refresh_token y user_id. • El token es válido para futuras peticiones autenticadas.	

Cuadro G.3. Prueba de aceptación # 4

Caso de prueba de aceptación	
Código: HU1_P3	Historia de usuario: 1
Nombre: Recuperación de acceso mediante token por correo	

Continúa en la próxima página

Cuadro G.3. Continuación de la página anterior.

Descripción: Validar que un usuario pueda solicitar recuperación de contraseña y recibir token por correo.
Condiciones de ejecución: • El servidor del sistema está disponible. • El servicio de correo está configurado. • Existe un usuario con correo <code>usuario@test.com</code>. • El usuario se ha registrado previamente.
Pasos de ejecución: 1. Enviar POST a <code>/api/v1/auth/forgot-password</code> con carga útil: <code>{.email": "usuario@test.com"}</code>. 2. Validar que la respuesta contenga estado HTTP 200. 3. Verificar que se envió un correo con token de recuperación. 4. Extraer el token del correo y enviar PUT a <code>/api/v1/auth/reset-password</code> con carga útil: <code>{"token": "token", "new_password": "NewPass456!@"}</code>. 5. Validar que la contraseña se actualizó.
Resultados esperados: • Estado HTTP: 200 OK (solicitud). • Se envía correo con token de recuperación. • Estado HTTP: 200 OK (restablecimiento). • Usuario puede iniciar sesión con nueva contraseña. • El token anterior se invalida.

Cuadro G.4. Prueba de aceptación # 5

Caso de prueba de aceptación	
Código: HU1_P4	Historia de usuario: 1
Nombre: Edición de perfil propio sin intervención administrativa	
Descripción: Validar que un usuario autenticado pueda actualizar su perfil (correo, nombre) sin necesidad de administrador.	

Continúa en la próxima página

Cuadro G.4. Continuación de la página anterior.

Condiciones de ejecución: • Usuario está autenticado con token válido. • El nuevo correo no está registrado en otro usuario. • El usuario tiene permisos para editar su propio perfil.
Pasos de ejecución: 1. Enviar PUT a /api/v1/auth/profile con cabeceras de autenticación y carga útil: {".email": "nuevo@test.com", "first_name": "Juan", "last_name": "Pérez"}. 2. Validar que la respuesta contenga estado HTTP 200. 3. Consultar GET a /api/v1/auth/profile para confirmar cambios. 4. Validar que los datos actualizados coinciden.
Resultados esperados: • Estado HTTP: 200 OK. • Perfil actualizado en BD. • Respuesta contiene los datos actualizados. • El token sigue siendo válido. • Cambios persistidos en BD.

Cuadro G.5. Prueba de aceptación # 6

Caso de prueba de aceptación	
Código: HU1_P5	Historia de usuario: 1
Nombre: Cierre de sesión	
Descripción: Validar que un usuario pueda cerrar sesión invalidando su token.	
Condiciones de ejecución: • Usuario está autenticado con token válido. • El servidor mantiene una lista de tokens revocados.	

Continúa en la próxima página

Cuadro G.5. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar POST a <code>/api/v1/auth/logout</code> con cabeceras de autenticación. 2. Validar que la respuesta contenga estado HTTP 200. 3. Intentar acceder a un <i>endpoint</i> protegido con el token anterior. 4. Validar que se rechaza la petición.
Resultados esperados: • Estado HTTP: 200 OK. • El token se añade a lista de revocación. • Peticiones posteriores con ese token retornan HTTP 401 Unauthorized. • Mensaje: "Token revoked or expired".

HU-02: Administración de Usuarios y Permisos

Cuadro G.6. Prueba de aceptación # 7

Caso de prueba de aceptación	
Código: HU2_P1	Historia de usuario: 2
Nombre: Listar usuarios con filtro por rol	
Descripción: Validar que un administrador puede obtener listado de usuarios filtrados por rol.	
Condiciones de ejecución: • Usuario es administrador (rol=ADMIN). • Existen usuarios con diferentes roles en la BD. • Usuario está autenticado con token de administrador.	
Pasos de ejecución: 1. Enviar GET a <code>/api/v1/admin/users?role=EDITOR&skip=0&limit=10</code> con cabeceras de autenticación. 2. Validar que la respuesta contenga estado HTTP 200. 3. Verificar que todos los usuarios retornados tienen <code>role="EDITOR"</code>. 4. Validar que la respuesta incluye total, skip, limit.	

Continúa en la próxima página

Cuadro G.6. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Respuesta contiene arreglo de usuarios filtrados por rol. • Cada usuario incluye: id, email, first_name, last_name, role, is_active. • Paginación correcta.

Cuadro G.7. Prueba de aceptación # 8

Caso de prueba de aceptación	
Código: HU2_P2	Historia de usuario: 2
Nombre: Listar usuarios con filtro por nombre	
Descripción: Validar que un administrador puede filtrar usuarios por nombre completo.	
Condiciones de ejecución: • Usuario es administrador. • Existen usuarios registrados con nombres distintos. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar GET a /api/v1/admin/users?name=Juan&skip=0&limit=10 con cabeceras de autenticación. 2. Validar que la respuesta contenga estado HTTP 200. 3. Verificar que todos los usuarios retornados contienen "Juan" en nombre o apellido.	
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta contiene usuarios cuyo nombre/apellido contiene la búsqueda. • Búsqueda sin distinción de mayúsculas/minúsculas.	

Cuadro G.8. Prueba de aceptación # 9

Caso de prueba de aceptación	
Código: HU2_P3	Historia de usuario: 2
Nombre: Modificar rol de usuario	

Continúa en la próxima página

Cuadro G.8. Continuación de la página anterior.

Descripción: Validar que un administrador puede cambiar el rol asignado a un usuario.
Condiciones de ejecución: • Usuario es administrador. • Existe un usuario con rol VIEWER. • Usuario está autenticado con credenciales de <i>admin</i>.
Pasos de ejecución: 1. Obtener ID de usuario a modificar. 2. Enviar PUT a /api/v1/admin/users/{user_id}/role con carga útil: {role": ".EDITOR"}. 3. Validar que la respuesta contenga estado HTTP 200. 4. Consultar GET a /api/v1/admin/users/{user_id} para confirmar cambio. 5. Verificar que role=.EDITOR".
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta contiene usuario actualizado con nuevo rol. • La BD refleja el cambio. • Permisos del usuario se actualizan dinámicamente.

Cuadro G.9. Prueba de aceptación # 10

Caso de prueba de aceptación	
Código: HU2_P4	Historia de usuario: 2
Nombre: Desactivar cuenta de usuario sin eliminar información	
Descripción: Validar que un administrador puede desactivar una cuenta sin eliminar los datos.	
Condiciones de ejecución: • Usuario es administrador. • Existe un usuario activo. • No se permiten borrados físicos en el sistema.	

Continúa en la próxima página

Cuadro G.9. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de usuario a desactivar. 2. Enviar PUT a /api/v1/admin/users/{user_id}/status con carga útil: {is_active": false}. 3. Validar que la respuesta contenga estado HTTP 200. 4. Intentar que el usuario desactivado inicie sesión. 5. Verificar que el inicio de sesión falla con mensaje de cuenta desactivada.
Resultados esperados: • Estado HTTP: 200 OK. • Usuario marcado como is_active=False en BD. • Datos del usuario persisten en BD. • Usuario no puede autenticarse. • Mensaje: User account is deactivated".

Cuadro G.10. Prueba de aceptación # 11

Caso de prueba de aceptación	
Código: HU2_P5	Historia de usuario: 2
Nombre: Reactivar cuenta de usuario	
Descripción: Validar que un administrador puede reactivar una cuenta desactivada.	
Condiciones de ejecución: • Usuario es administrador. • Existe un usuario desactivado (is_active=False). • Los datos del usuario están intactos en BD.	
Pasos de ejecución: 1. Obtener ID de usuario a reactivar. 2. Enviar PUT a /api/v1/admin/users/{user_id}/status con carga útil: {is_active": true}. 3. Validar que la respuesta contenga estado HTTP 200. 4. Intentar autenticar con el usuario reactivado. 5. Verificar que el inicio de sesión es exitoso.	

Continúa en la próxima página

Cuadro G.10. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Usuario marcado como <code>is_active=True</code>. • Datos originales restaurados. • Usuario puede autenticarse normalmente.

Épica 2: Gestión de Dominios Curriculares

HU-03: Gestión de Dominios de Conocimiento

Cuadro G.11. Prueba de aceptación # 12

Caso de prueba de aceptación	
Código: HU3_P1	Historia de usuario: 3
Nombre: Crear dominio con nombre y descripción	
Descripción: Validar que un administrador puede crear un nuevo dominio curricular.	
Condiciones de ejecución: • Usuario es administrador. • Usuario está autenticado. • No existe un dominio con el mismo nombre.	
Pasos de ejecución: 1. Enviar POST a <code>/api/v1/domains</code> con carga útil: <code>{"name": "Ingeniería de Software", "description": "Dominio para estudios en ingeniería de software"}</code>. 2. Validar que la respuesta contenga estado HTTP 201. 3. Verificar que se retorna <code>id</code>, <code>name</code>, <code>description</code>, <code>is_active</code>, <code>created_at</code>. 4. Consultar BD para confirmar persistencia.	
Resultados esperados: • Estado HTTP: 201 Created. • Respuesta contiene ID generado y campos completos. • Dominio se crea con <code>is_active=True</code> por defecto. • <code>created_at</code> y <code>updated_at</code> se establecen automáticamente.	

Cuadro G.12. Prueba de aceptación # 13

Caso de prueba de aceptación	
Código: HU3_P2	Historia de usuario: 3
Nombre: Obtener dominio con modelos asociados	
Descripción: Validar que se puede obtener un dominio junto con todos sus modelos vinculados.	
Condiciones de ejecución: • Existe un dominio con ID conocido. • El dominio tiene modelos asociados. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar GET a /api/v1/domains/{domain_id}?include_models=true con cabeceras de autenticación. 2. Validar que la respuesta contenga estado HTTP 200. 3. Verificar que la respuesta incluye id, name, description, is_active. 4. Verificar que incluye arreglo models con todos los modelos del dominio. 5. Cada modelo debe contener: id, name, domain_id, created_at.	
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta contiene información del dominio. • Arreglo models contiene todos los modelos vinculados. • Modelos activos e inactivos se incluyen (según filtro).	

Cuadro G.13. Prueba de aceptación # 14

Caso de prueba de aceptación	
Código: HU3_P3	Historia de usuario: 3
Nombre: Desactivar dominio	
Descripción: Validar que un administrador puede desactivar un dominio sin eliminar información.	

Continúa en la próxima página

Cuadro G.13. Continuación de la página anterior.

Condiciones de ejecución: • Usuario es administrador. • Existe un dominio activo. • El dominio puede no tener modelos o tenerlos.
Pasos de ejecución: 1. Obtener ID de dominio a desactivar. 2. Enviar PUT a /api/v1/domains/{domain_id} con carga útil: {is_active": false}. 3. Validar que la respuesta contenga estado HTTP 200. 4. Consultar GET a /api/v1/domains/{domain_id} para confirmar. 5. Verificar que is_active=False.
Resultados esperados: • Estado HTTP: 200 OK. • Dominio marcado como is_active=False. • Información persiste en BD. • Modelos del dominio no son eliminados.

Cuadro G.14. Prueba de aceptación # 15

Caso de prueba de aceptación	
Código: HU3_P4	Historia de usuario: 3
Nombre: Listar todos los dominios	
Descripción: Validar que se puede obtener un listado paginado de dominios.	
Condiciones de ejecución: • Existen múltiples dominios en la BD. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.14. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar GET a /api/v1/domains?skip=0&limit=10 con cabeceras de autenticación. 2. Validar que la respuesta contenga estado HTTP 200. 3. Verificar que se retorna arreglo de dominios. 4. Cada dominio incluye: id, name, description, is_active, created_at. 5. Validar paginación: total, skip, limit.
Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de dominios ordenados por fecha. • Paginación correcta. • Total de registros se incluye.

Épica 3: Ciclo de Vida del **FM**

HU-04: Gestión Operativa de **FM**

Cuadro G.15. Prueba de aceptación # 16

Caso de prueba de aceptación	
Código: HU4_P1	Historia de usuario: 4
Nombre: Crear modelo asociado a dominio válido	
Descripción: Validar que se puede crear un nuevo modelo de características en un dominio existente.	
Condiciones de ejecución: • Usuario es diseñador o administrador. • Existe un dominio activo. • Usuario está autenticado. • No existe modelo con igual nombre en el dominio.	

Continúa en la próxima página

Cuadro G.15. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de dominio válido. 2. Enviar POST a /api/v1/models con carga útil: {"name": "Plan 2024", "description": "Modelo curricular 2024", "domain_id": «domain_id»}. 3. Validar que la respuesta contenga estado HTTP 201. 4. Verificar que se retorna id, name, domain_id, created_at. 5. Validar que existe en BD.
Resultados esperados: • Estado HTTP: 201 Created. • Modelo creado con is_active=True. • Respuesta incluye ID generado. • domain_id está vinculado correctamente. • created_at y updated_at se establecen.

Cuadro G.16. Prueba de aceptación # 17

Caso de prueba de aceptación	
Código: HU4_P2	Historia de usuario: 4
Nombre: Actualizar metadatos del modelo	
Descripción: Validar que los metadatos de un modelo pueden actualizarse sin afectar la estructura.	
Condiciones de ejecución: • Existe un modelo con ID conocido. • Usuario es diseñador del modelo o administrador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.16. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de modelo. 2. Enviar PUT a <code>/api/v1/models/{model_id}</code> con carga útil: <code>{"name": "Plan 2024 v2", "description": "Modelo actualizado"}</code>. 3. Validar que la respuesta contenga estado HTTP 200. 4. Consultar GET a <code>/api/v1/models/{model_id}</code> para confirmar. 5. Verificar que los metadatos están actualizados y la estructura intacta.
Resultados esperados: • Estado HTTP: 200 OK. • Metadatos actualizados en BD. • Estructura del modelo no se modifica. • <code>updated_at</code> se actualiza. • Versiones no se ven afectadas.

Cuadro G.17. Prueba de aceptación # 18

Caso de prueba de aceptación	
Código: HU4_P3	Historia de usuario: 4
Nombre: Listar modelos con paginación y filtrado	
Descripción: Validar que se pueden listar modelos con paginación, ordenamiento y filtros.	
Condiciones de ejecución: • Existen múltiples modelos en la BD. • Modelos están vinculados a dominios. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar GET a <code>/api/v1/models?domain_id=<id>&skip=0&limit=10&sort=-created_at</code> con cabeceras de autenticación. 2. Validar que la respuesta contenga estado HTTP 200. 3. Verificar que se retorna arreglo de modelos. 4. Validar paginación: total, skip, limit. 5. Verificar que el ordenamiento es correcto (descendente por <code>created_at</code>).	

Continúa en la próxima página

Cuadro G.17. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de modelos paginados. • Filtrado por dominio correctamente aplicado. • Ordenamiento correcto. • Total de registros incluido.
--

HU-05: Gestión de Versiones y Publicación

Cuadro G.18. Prueba de aceptación # 19

Caso de prueba de aceptación	
Código: HU5_P1	Historia de usuario: 5
Nombre: Crear nueva versión mediante copia al escribir	
Descripción: Validar que se puede crear una nueva versión duplicando una versión existente (Copy-on-Write).	
Condiciones de ejecución: • Existe un modelo con versión publicada. • Usuario es diseñador del modelo. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de modelo y ID de versión origen. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions</code> con carga útil: <code>{"version_origin_id": «version_id», "description": "Versión 2.0"}</code>. 3. Validar que la respuesta contenga estado HTTP 201. 4. Verificar que se crea versión con estado DRAFT. 5. Validar que la estructura se copió desde versión origen.	

Continúa en la próxima página

Cuadro G.18. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 201 Created. • Nueva versión creada con estado DRAFT. • Copia completa de la estructura anterior. • Versión origen no se modifica. • Versión nueva tiene <code>version_number</code> incrementado.
--

Cuadro G.19. Prueba de aceptación # 20

Caso de prueba de aceptación	
Código: HU5_P2	Historia de usuario: 5
Nombre: Validar versión antes de publicación	
Descripción: Validar que se ejecutan validaciones antes de permitir publicación de versión.	
Condiciones de ejecución: • Existe una versión en estado DRAFT. • Versión tiene estructura válida o inválida. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de versión. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/validate</code> con cabeceras de autenticación. 3. Validar que la respuesta contenga estado HTTP 200. 4. Respuesta incluye: <code>is_valid</code>, <code>errors</code> (si los hay), <code>warnings</code>. 5. Si es válida, usar para publicación; si no, mostrar errores.	
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta incluye estado de validación. • Listado de errores estructurales/lógicos si existen. • Listado de advertencias. • Puede proceder a publicación si <code>is_valid=true</code>.	

Cuadro G.20. Prueba de aceptación # 21

Caso de prueba de aceptación	
Código: HU5_P3	Historia de usuario: 5
Nombre: Publicar versión de modelo	
Descripción: Validar que una versión validada puede publicarse y pasar a estado PUBLISHED.	
Condiciones de ejecución: • Existe versión en estado DRAFT validada. • Versión no tiene errores críticos. • Usuario es revisor o administrador.	
Pasos de ejecución: 1. Obtener ID de versión en estado DRAFT. 2. Enviar POST a /api/v1/models/{model_id}/versions/{version_id}/publish con cabeceras de autenticación. 3. Validar que la respuesta contenga estado HTTP 200. 4. Verificar que el estado cambia a PUBLISHED. 5. Consultar la versión para confirmar cambio de estado.	
Resultados esperados: • Estado HTTP: 200 OK. • Estado cambia a PUBLISHED. • published_at se establece con marca de tiempo actual. • Versión anterior (si existe) pasa a estado ARCHIVED (opcional). • Configuraciones pueden vincularse a esta versión.	

Cuadro G.21. Prueba de aceptación # 22

Caso de prueba de aceptación	
Código: HU5_P4	Historia de usuario: 5
Nombre: Recuperar versiones archivadas	
Descripción: Validar que se pueden listar y recuperar versiones archivadas.	

Continúa en la próxima página

Cuadro G.21. Continuación de la página anterior.

Condiciones de ejecución: • Existen versiones en estado ARCHIVED. • Usuario está autenticado. • Usuario tiene permisos para ver historial.
Pasos de ejecución: 1. Enviar GET a /api/v1/models/{model_id}/versions?status=ARCHIVED con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que se retornan solo versiones ARCHIVED. 4. Cada versión incluye: id, version_number, status, archived_at, created_at. 5. Seleccionar una versión y enviar GET a /api/v1/models/{model_id}/versions/{version_id}.
Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de versiones archivadas. • Información completa de cada versión. • Marcas de tiempo de archivación incluidas. • Puede recuperar estructura de versión archivada.

HU-06: Análisis de Métricas y Exportación

Cuadro G.22. Prueba de aceptación # 23

Caso de prueba de aceptación	
Código: HU6_P1	Historia de usuario: 6
Nombre: Calcular métricas de versión publicada	
Descripción: Validar que se pueden calcular métricas de la versión publicada más reciente.	
Condiciones de ejecución: • Existe un modelo con versión PUBLISHED. • La versión tiene estructura completa. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.22. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar GET a <code>/api/v1/models/{model_id}/metrics</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que incluye métricas: <code>feature_count</code>, <code>group_count</code>, <code>constraint_count</code>, <code>cyclomatic_complexity</code>, <code>tree_depth</code>. 4. Verificar que se calcula sobre versión PUBLISHED.
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta incluye todas las métricas. • Valores son números enteros ≥ 0. • Cálculo se realiza sobre versión publicada. • Valores coinciden con estructura del modelo.

Cuadro G.23. Prueba de aceptación # 24

Caso de prueba de aceptación	
Código: HU6_P2	Historia de usuario: 6
Nombre: Exportar modelo en formato UVL	
Descripción: Validar que el modelo se puede exportar en formato UVL válido.	
Condiciones de ejecución: • Existe un modelo con versión PUBLISHED. • La versión tiene estructura completa. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar GET a <code>/api/v1/models/{model_id}/export?format=uvl</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que Content-Type es <code>text/plain</code> o <code>application/x-uvl</code>. 4. Validar que el contenido es UVL válido (comienza con <code>features</code> o <code>namespace</code>, tiene sintaxis correcta). 5. Guardar contenido en archivo <code>.uvl</code> y validar.	

Continúa en la próxima página

Cuadro G.23. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Content-Type correcto. • UVL contiene sintaxis válida. • Todas las características, grupos y restricciones están representadas. • Puede descargarse como archivo. • Reimportación genera estructura idéntica.

Cuadro G.24. Prueba de aceptación # 25

Caso de prueba de aceptación	
Código: HU6_P3	Historia de usuario: 6
Nombre: Exportar modelo en formato JSON	
Descripción: Validar que el modelo se puede exportar en formato JSON.	
Condiciones de ejecución: • Existe un modelo con versión PUBLISHED. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar GET a /api/v1/models/{model_id}/export?format=json con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que Content-Type es application/json. 4. Validar que es JSON válido (se puede analizar). 5. Verificar que incluye: model, version, features, groups, constraints.	
Resultados esperados: • Estado HTTP: 200 OK. • JSON válido y bien formado. • Estructura incluye todos los componentes del modelo. • Puede descargarse como .json.	

Cuadro G.25. Prueba de aceptación # 26

Caso de prueba de aceptación	
Código: HU6_P4	Historia de usuario: 6
Nombre: Exportar modelo en formato DOT	
Descripción: Validar que el modelo se puede exportar en formato DOT para visualización.	
Condiciones de ejecución: • Existe un modelo con versión PUBLISHED. • Usuario está autenticado. • El sistema tiene funcionalidad de generación de grafos.	
Pasos de ejecución: 1. Enviar GET a /api/v1/models/{model_id}/export?format=dot con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que Content-Type es text/plain o text/vnd.graphviz. 4. Validar sintaxis DOT (comienza con digraph o graph). 5. Intentar renderizar con Graphviz para validar.	
Resultados esperados: • Estado HTTP: 200 OK. • Sintaxis DOT válida. • Nodos representan características. • Aristas representan relaciones jerárquicas. • Puede visualizarse con herramientas Graphviz.	

Cuadro G.26. Prueba de aceptación # 27

Caso de prueba de aceptación	
Código: HU6_P5	Historia de usuario: 6
Nombre: Exportar modelo en formato Mermaid	
Descripción: Validar que el modelo se puede exportar en formato Mermaid para diagramas interactivos.	
Condiciones de ejecución: • Existe un modelo con versión PUBLISHED. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.26. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar GET a /api/v1/models/{model_id}/export?format=mermaid con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Validar sintaxis Mermaid (comienza con graph o flowchart). 4. Verificar que incluye características como nodos. 5. Verificar que relaciones están correctamente representadas.
Resultados esperados: • Estado HTTP: 200 OK. • Sintaxis Mermaid válida. • Diagrama representa jerarquía del modelo. • Compatible con Mermaid.js.

Épica 4: Modelado de Variabilidad Estructural

HU-07: Edición de la Jerarquía de Características

Cuadro G.27. Prueba de aceptación # 28

Caso de prueba de aceptación	
Código: HU7_P1	Historia de usuario: 7
Nombre: Crear característica en versión de modelo	
Descripción: Validar que se puede crear una nueva característica en una versión específica.	
Condiciones de ejecución: • Existe un modelo con versión DRAFT. • Usuario es diseñador del modelo. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.27. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de modelo e ID de versión. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/features</code> con carga útil: <code>{"name": "Módulo A", "type": "CONCRETE"}</code>. 3. Validar que la respuesta contenga estado 201. 4. Verificar que se retorna <code>id, name, type, version_id, parent_id</code>. 5. Validar que existe en BD.
Resultados esperados: • Estado HTTP: 201 Created. • Característica creada con ID generado. • Tipo es CONCRETE, ABSTRACT u OPTIONAL según especificación. • Sin padre (raíz) inicialmente. • Marcas de tiempo se establecen.

Cuadro G.28. Prueba de aceptación # 29

Caso de prueba de aceptación	
Código: HU7_P2	Historia de usuario: 7
Nombre: Reubicar característica cambiando padre	
Descripción: Validar que una característica puede ser reubicada cambiando su padre.	
Condiciones de ejecución: • Existe una característica con padre. • Existe otra característica que será el nuevo padre. • Cambio no crearía ciclo. • Usuario es diseñador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.28. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de la característica a mover y ID del nuevo padre. 2. Enviar PUT a /api/v1/models/{model_id}/versions/{version_id}/features/{feature_id} con carga útil: {"parent_id": «new_parent_id»}. 3. Validar que la respuesta contenga estado 200. 4. Consultar característica para confirmar nuevo padre. 5. Verificar que subárbol se movió correctamente.
Resultados esperados: • Estado HTTP: 200 OK. • Padre actualizado. • Subárbol completo se mueve con la característica. • Estructura jerárquica mantiene integridad. • No se crea ciclo.

Cuadro G.29. Prueba de aceptación # 30

Caso de prueba de aceptación	
Código: HU7_P3	Historia de usuario: 7
Nombre: Recuperar subárbol completo de característica	
Descripción: Validar que se puede obtener una característica con toda su estructura descendente.	
Condiciones de ejecución: • Existe una característica con descendientes. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de la característica. 2. Enviar GET a /api/v1/models/{model_id}/versions/{version_id}/features/{feature_id}?include_subtree=true con cabeceras. 3. Validar que la respuesta contenga estado 200. 4. Verificar que incluye la característica y su arreglo de children. 5. Verificar que cada child incluye sus propios children (recursivo).	

Continúa en la próxima página

Cuadro G.29. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Característica principal incluida. • Arreglo <code>children</code> contiene todas las características descendentes. • Estructura jerárquica completa preservada. • Profundidad del árbol representada correctamente.

Cuadro G.30. Prueba de aceptación # 31

Caso de prueba de aceptación	
Código: HU7_P4	Historia de usuario: 7
Nombre: Desactivar característica con soporte de versionado	
Descripción: Validar que una característica puede ser desactivada lógicamente manteniéndola en la BD .	
Condiciones de ejecución: • Existe una característica activa. • La característica no es raíz del árbol. • Usuario es diseñador. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de la característica a desactivar. 2. Enviar PUT a <code>/api/v1/models/{model_id}/versions/{version_id}/features/{feature_id}</code> con carga útil: <code>{is_active": false}</code>. 3. Validar que la respuesta contenga estado 200. 4. Consultar característica para confirmar <code>is_active=false</code>. 5. Verificar que no afecta otras versiones.	
Resultados esperados: • Estado HTTP: 200 OK. • Característica marcada como <code>is_active=false</code>. • Característica permanece en BD. • Otros cambios no afectados. • Versionado mantiene registro del cambio.	

Cuadro G.31. Prueba de aceptación # 32

Caso de prueba de aceptación	
Código: HU7_P5	Historia de usuario: 7
Nombre: Actualizar propiedades de característica	
Descripción: Validar que se pueden actualizar propiedades de una característica.	
Condiciones de ejecución: • Existe una característica. • Usuario es diseñador. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de la característica. 2. Enviar PUT a <code>/api/v1/models/{model_id}/versions/{version_id}/features/{feature_id}</code> con carga útil: <code>{"name": "Nuevo nombre", "type": "BSTRACT"}</code>. 3. Validar que la respuesta contenga estado 200. 4. Consultar característica para confirmar cambios. 5. Verificar que metadata se actualizó.	
Resultados esperados: • Estado HTTP: 200 OK. • Nombre actualizado. • Tipo actualizado. • <code>updated_at</code> se actualiza. • Cambios persistidos en BD.	

HU-08: Modelado de Grupos y Relaciones Estructurales

Cuadro G.32. Prueba de aceptación # 33

Caso de prueba de aceptación	
Código: HU8_P1	Historia de usuario: 8
Nombre: Crear grupo XOR entre características	
Descripción: Validar que se puede crear un grupo XOR (exactamente uno) entre características hermanas.	

Continúa en la próxima página

Cuadro G.32. Continuación de la página anterior.

Condiciones de ejecución: • Existen características hermanas en versión DRAFT. • Las características tienen el mismo padre. • Usuario es diseñador. • Usuario está autenticado.
Pasos de ejecución: 1. Obtener ID de las características a agrupar. 2. Enviar POST a /api/v1/models/{model_id}/versions/{version_id}/groups con carga útil: {"group_type": "XOR", "parent_id": «parent_id»}. 3. Validar que la respuesta contenga estado 201. 4. Verificar que se retorna id, group_type, parent_id, min_cardinality=1, max_cardinality=1. 5. Asignar características al grupo.
Resultados esperados: • Estado HTTP: 201 Created. • Grupo creado con group_type=XOR. • Cardinalidades establecidas: min=1, max=1. • ID generado. • Grupo listo para asignar características.

Cuadro G.33. Prueba de aceptación # 34

Caso de prueba de aceptación	
Código: HU8_P2	Historia de usuario: 8
Nombre: Crear grupo OR con cardinalidades	
Descripción: Validar que se puede crear un grupo OR con cardinalidades mínimas y máximas personalizadas.	
Condiciones de ejecución: • Existen características hermanas en versión DRAFT. • Usuario es diseñador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.33. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de padre. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/groups</code> con carga útil: <code>{"group_type": ".%R", "parent_id": «parent_id», "min_cardinality": 1, "max_cardinality": 3}</code>. 3. Validar que la respuesta contenga estado 201. 4. Verificar que cardinalidades se establecen correctamente. 5. Asignar características al grupo.
Resultados esperados: • Estado HTTP: 201 Created. • Grupo creado con <code>group_type=OR</code>. • Cardinalidades: <code>min=1</code>, <code>max=3</code>. • Validación: <code>min ≥ 1</code> y <code>max ≥ min</code>. • Grupo aceptará entre 1 y 3 características seleccionadas.

Cuadro G.34. Prueba de aceptación # 35

Caso de prueba de aceptación	
Código: HU8_P3	Historia de usuario: 8
Nombre: Crear relación requires entre características	
Descripción: Validar que se puede crear una relación de implicación (requires) entre características.	
Condiciones de ejecución: • Existen dos características en versión DRAFT. • Las características pueden estar en diferentes ramas. • Usuario es diseñador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.34. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de característica origen y destino. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/relationships</code> con carga útil: <code>{relationship_type": "REQUIRES", "source_id": «feature_id_1>", "target_id": «feature_id_2>"}</code>. 3. Validar que la respuesta contenga estado 201. 4. Verificar que se retorna relación con ID correctos. 5. Guardar ID de relación.
Resultados esperados: • Estado HTTP: 201 Created. • Relación creada. • Tipo: REQUIRES (implicación). • source y target correctos. • Bidireccional en términos de consulta.

Cuadro G.35. Prueba de aceptación # 36

Caso de prueba de aceptación	
Código: HU8_P4	Historia de usuario: 8
Nombre: Crear relación excludes entre características	
Descripción: Validar que se puede crear una relación de exclusión entre características.	
Condiciones de ejecución: • Existen dos características en versión DRAFT. • Usuario es diseñador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.35. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de características a relacionar. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/relationships</code> con carga útil: <code>{relationship_type": ".EXCLUDES", "source_id": «id_1>", "target_id": «id_2>"}</code>. 3. Validar que la respuesta contenga estado 201. 4. Verificar que relación se crea. 5. Intentar crear configuración con ambas características (debe fallar).
Resultados esperados: • Estado HTTP: 201 Created. • Relación de exclusión creada. • Ambas características no pueden seleccionarse simultáneamente. • Validación de configuraciones lo verifica.

Cuadro G.36. Prueba de aceptación # 37

Caso de prueba de aceptación	
Código: HU8_P5	Historia de usuario: 8
Nombre: Listar grupos y relaciones de versión	
Descripción: Validar que se pueden obtener todos los grupos y relaciones de una versión.	
Condiciones de ejecución: • Existe versión con grupos y relaciones. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar GET a <code>/api/v1/models/{model_id}/versions/{version_id}/groups</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar arreglo de grupos con id, group_type, parent_id, min_cardinality, max_cardinality. 4. Enviar GET a <code>/api/v1/models/{model_id}/versions/{version_id}/relationships</code>. 5. Verificar arreglo de relaciones.	

Continúa en la próxima página

Cuadro G.36. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK (ambas). • Todos los grupos listados. • Todas las relaciones listadas. • Información completa de cada grupo/relación.

Épica 5: Restricciones y Lógica Avanzada

HU-09: Gestión de Restricciones Transversales

Cuadro G.37. Prueba de aceptación # 38

Caso de prueba de aceptación	
Código: HU9_P1	Historia de usuario: 9
Nombre: Crear restricción simple con sintaxis UVL	
Descripción: Validar que se puede crear una restricción simple con expresión lógica.	
Condiciones de ejecución: • Existe versión DRAFT con características. • Usuario es diseñador. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de versión. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/constraints</code> con carga útil: <code>{.expression": "FeatureA =>FeatureB", "description": "Si A entonces B"}</code>. 3. Validar que la respuesta contenga estado 201. 4. Verificar que se analiza y valida la expresión. 5. Guardar ID de restricción.	

Continúa en la próxima página

Cuadro G.37. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 201 Created. • Restricción creada con ID. • Expresión validada sintácticamente. • Descripción opcional almacenada. • Restricción activa.

Cuadro G.38. Prueba de aceptación # 39

Caso de prueba de aceptación	
Código: HU9_P2	Historia de usuario: 9
Nombre: Crear restricción compleja con múltiples operadores	
Descripción: Validar que se pueden crear restricciones complejas con operadores lógicos.	
Condiciones de ejecución: • Existe versión DRAFT con características. • Usuario es diseñador. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de versión. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/constraints</code> con carga útil: <code>{.expression": "(FeatureA FeatureB) & ~FeatureC =>FeatureD", "description": "Restricción compleja"}</code>. 3. Validar que la respuesta contenga estado 201. 4. Verificar que se valida correctamente. 5. Intentar añadir a validación del modelo.	
Resultados esperados: • Estado HTTP: 201 Created. • Restricción con múltiples operadores aceptada. • Sintaxis: , &, ~, =>, válidas. • Expresión se almacena correctamente.	

Cuadro G.39. Prueba de aceptación # 40

Caso de prueba de aceptación	
Código: HU9_P3	Historia de usuario: 9
Nombre: Actualizar restricción existente	
Descripción: Validar que una restricción puede ser actualizada con nueva expresión.	
Condiciones de ejecución: • Existe restricción en versión DRAFT. • Usuario es diseñador. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de restricción. 2. Enviar PUT a <code>/api/v1/models/{model_id}/versions/{version_id}/constraints/{constraint_id}</code> con carga útil: <code>{".expression": "FeatureA & FeatureB =>FeatureC", "description": ".Expresión actualizada"}</code>. 3. Validar que la respuesta contenga estado 200. 4. Consultar restricción para confirmar cambios. 5. Verificar que versionado registra el cambio.	
Resultados esperados: • Estado HTTP: 200 OK. • Expresión actualizada. • Nueva expresión validada. • Descripción actualizada. • <code>updated_at</code> se modifica.	

Cuadro G.40. Prueba de aceptación # 41

Caso de prueba de aceptación	
Código: HU9_P4	Historia de usuario: 9
Nombre: Listar restricciones de versión	
Descripción: Validar que se pueden obtener todas las restricciones de una versión.	

Continúa en la próxima página

Cuadro G.40. Continuación de la página anterior.

Condiciones de ejecución: • Existe versión con restricciones. • Usuario está autenticado.
Pasos de ejecución: 1. Enviar GET a /api/v1/models/{model_id}/versions/{version_id}/constraints con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar arreglo de restricciones. 4. Cada restricción incluye: id, expression, description, is_active, created_at.
Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de todas las restricciones. • Información completa de cada restricción. • Ordenadas por fecha de creación.

HU-010: Integración y Sincronización UVL

Cuadro G.41. Prueba de aceptación # 42

Caso de prueba de aceptación	
Código: HU10_P1	Historia de usuario: 10
Nombre: Aplicar contenido UVL a estructura de modelo	
Descripción: Validar que se puede cargar un modelo completo desde contenido UVL.	
Condiciones de ejecución: • Existe una versión DRAFT vacía. • Se dispone de contenido UVL válido. • Usuario es diseñador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.41. Continuación de la página anterior.

Pasos de ejecución: 1. Preparar contenido UVL válido (características, grupos, restricciones). 2. Enviar PUT a /api/v1/models/{model_id}/versions/{version_id}/uvl con carga útil: {content": «contenido UVL>"}. 3. Validar que la respuesta contenga estado 200. 4. Consultar la estructura generada del modelo. 5. Verificar que árbol, grupos y restricciones se crearon correctamente.
Resultados esperados: • Estado HTTP: 200 OK. • Estructura del modelo se genera desde UVL. • Todas las características se crean. • Todos los grupos se crean. • Todas las restricciones se crean. • Estructura coincide con UVL original.

Cuadro G.42. Prueba de aceptación # 43

Caso de prueba de aceptación	
Código: HU10_P3	Historia de usuario: 10
Nombre: Persistir UVL analizado en versión	
Descripción: Validar que el contenido UVL se analiza, valida y persiste en la BD.	
Condiciones de ejecución: • Existe versión DRAFT. • Se carga contenido UVL válido. • Usuario es diseñador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.42. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar PUT a <code>/api/v1/models/{model_id}/versions/{version_id}/uvl</code> con contenido UVL. 2. Validar que la respuesta contenga estado 200. 3. Consultar GET <code>/api/v1/models/{model_id}/versions/{version_id}/uvl</code>. 4. Verificar que el UVL almacenado coincide con el original. 5. Validar que existe en BD.
Resultados esperados: • Estado HTTP: 200 OK (ambas). • UVL almacenado en BD. • Análisis y tokens se persisten. • Puede recuperarse completo. • Válido para reimportación.

Épica 6: Validación y Análisis de Modelos

HU-11: Validación Lógica y Estructural del Modelo

Cuadro G.43. Prueba de aceptación # 44

Caso de prueba de aceptación	
Código: HU11_P1	Historia de usuario: 11
Nombre: Evaluar satisfacibilidad global del modelo	
Descripción: Validar que se puede ejecutar un análisis SAT sobre el modelo completo.	
Condiciones de ejecución: • Existe versión con estructura y restricciones. • Sistema tiene integración con solucionador SAT. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.43. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/validate/satisfiability</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que respuesta incluye: <code>is_satisfiable</code>, <code>error_message</code> (si aplica). 4. Si es satisfiable, debe existir al menos una configuración válida. 5. Si no es satisfiable, revisar restricciones.
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta incluye <code>is_satisfiable</code> (true/false). • Si false, mensaje de error explicando por qué. • Cálculo se ejecuta sobre modelo completo. • Tiempo de ejecución razonable.

Cuadro G.44. Prueba de aceptación # 45

Caso de prueba de aceptación	
Código: HU11_P2	Historia de usuario: 11
Nombre: Detectar ciclos en la jerarquía	
Descripción: Validar que se detectan ciclos en las relaciones entre características.	
Condiciones de ejecución: • Existe versión con estructura. • Estructura puede contener ciclos (intencional para testing). • Usuario está autenticado.	
Pasos de ejecución: 1. Crear escenario con ciclo: A->B->C->A. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/validate/structure</code> con cabeceras. 3. Validar que la respuesta contenga estado 200. 4. Verificar que se reporta error: <code>Cycle detected: A->B->C->A</code>. 5. Listar características participantes.	

Continúa en la próxima página

Cuadro G.44. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Respuesta incluye lista de ciclos detectados. • Cada ciclo lista características que participan. • Mensaje claro de error.

Cuadro G.45. Prueba de aceptación # 46

Caso de prueba de aceptación	
Código: HU11_P3	Historia de usuario: 11
Nombre: Identificar características muertas	
Descripción: Validar que se detectan características que nunca pueden ser seleccionadas.	
Condiciones de ejecución: • Existe versión con restricciones que hacen imposible seleccionar cierta característica. • Usuario está autenticado.	
Pasos de ejecución: 1. Crear estructura con característica imposible de seleccionar. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/analyze/dead-features</code> con cabeceras. 3. Validar que la respuesta contenga estado 200. 4. Verificar arreglo de características muertas. 5. Cada característica incluye razón por la que es muerta.	
Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de características muertas identificadas. • Explicación de por qué es muerta. • Análisis completo y correcto.	

Cuadro G.46. Prueba de aceptación # 47

Caso de prueba de aceptación	
Código: HU11_P4	Historia de usuario: 11
Nombre: Detectar nodos huérfanos	
Descripción: Validar que se detectan características que no tienen ruta a raíz.	
Condiciones de ejecución: • Existe versión con estructura. • Se intenta crear característica sin padre (huérfana). • Usuario está autenticado.	
Pasos de ejecución: 1. Buscar características sin ruta a raíz. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/validate/structure</code> con cabeceras. 3. Validar que la respuesta reporta nodos huérfanos. 4. Verificar que lista los IDs de nodos huérfanos. 5. Mensaje explicativo.	
Resultados esperados: • Estado HTTP: 200 OK. • Lista de nodos huérfanos detectados. • Explicación clara. • Recomendación de reparación.	

Cuadro G.47. Prueba de aceptación # 48

Caso de prueba de aceptación	
Código: HU11_P5	Historia de usuario: 11
Nombre: Ejecutar validación completa	
Descripción: Validar que se ejecuta el conjunto completo de validaciones en un paso.	
Condiciones de ejecución: • Existe versión con estructura. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.47. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/validate/full</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Respuesta incluye: <code>satisfiable</code>, <code>cycles</code>, <code>dead_features</code>, <code>orphan_nodes</code>, <code>warnings</code>. 4. Cada sección tiene lista de errores/advertencias. 5. Resumen general de validación.
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta incluye todas las validaciones. • Formato consistente. • Fácil de interpretar. • Guía para correcciones.

Épica 7: Configuración, Optimización y Recursos

HU-12: Gestión de Configuraciones

Cuadro G.48. Prueba de aceptación # 49

Caso de prueba de aceptación	
Código: HU12_P1	Historia de usuario: 12
Nombre: Crear configuración en versión publicada	
Descripción: Validar que se puede crear una configuración (selección de características) en una versión publicada.	
Condiciones de ejecución: • Existe versión PUBLISHED con características. • Se dispone de lista válida de características. • Usuario es configurador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.48. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de características válidas. 2. Enviar POST a /api/v1/configurations con carga útil: {"name": Configuración 1, "description": Configuración de prueba, "version_id": «version_id», "features": [«id_1», «id_2»]}. 3. Validar que la respuesta contenga estado 201. 4. Verificar que se crea configuración con ID. 5. Validar que se vincula a versión PUBLISHED.
Resultados esperados: • Estado HTTP: 201 Created. • Configuración creada con ID. • Vinculada a versión PUBLISHED. • Selección de características persistida. • Marcas de tiempo se establecen.

Cuadro G.49. Prueba de aceptación # 50

Caso de prueba de aceptación	
Código: HU12_P2	Historia de usuario: 12
Nombre: Actualizar configuración existente	
Descripción: Validar que se puede actualizar nombre, descripción y características de una configuración.	
Condiciones de ejecución: • Existe configuración persistida. • Usuario es propietario o administrador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.49. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de configuración. 2. Enviar PUT a <code>/api/v1/configurations/{config_id}</code> con carga útil: <code>{"name": "Nuevo nombre", "description": "Descripción actualizada", "features": [«nuevo_id_1>», «nuevo_id_2>"]}</code>. 3. Validar que la respuesta contenga estado 200. 4. Consultar configuración para confirmar cambios. 5. Verificar que las características se actualizaron.
Resultados esperados: • Estado HTTP: 200 OK. • Configuración actualizada en BD. • Nombre y descripción cambiados. • Lista de características actualizada. • <code>updated_at</code> se modifica.

Cuadro G.50. Prueba de aceptación # 51

Caso de prueba de aceptación	
Código: HU12_P3	Historia de usuario: 12
Nombre: Listar configuraciones paginadas	
Descripción: Validar que se pueden obtener configuraciones con paginación.	
Condiciones de ejecución: • Existen múltiples configuraciones persistidas. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar GET a <code>/api/v1/configurations?skip=0&limit=10</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar arreglo de configuraciones. 4. Cada configuración incluye: <code>id</code>, <code>name</code>, <code>description</code>, <code>version_id</code>, <code>created_at</code>, <code>updated_at</code>. 5. Validar paginación: <code>total</code>, <code>skip</code>, <code>limit</code>.	

Continúa en la próxima página

Cuadro G.50. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de configuraciones paginadas. • Información completa de cada configuración. • Paginación funciona correctamente. • Total de registros incluido.

Cuadro G.51. Prueba de aceptación # 52

Caso de prueba de aceptación	
Código: HU12_P4	Historia de usuario: 12
Nombre: Filtrar configuraciones por versión	
Descripción: Validar que se pueden filtrar configuraciones por ID de versión.	
Condiciones de ejecución: • Existen configuraciones de múltiples versiones. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de versión específica. 2. Enviar GET a /api/v1/configurations?version_id=<version_id>&skip=0&limit=10 con cabeceras. 3. Validar que la respuesta contenga estado 200. 4. Verificar que todas las configuraciones retornadas tienen version_id correcto. 5. Validar paginación.	
Resultados esperados: • Estado HTTP: 200 OK. • Solo configuraciones de versión especificada. • Paginación correcta. • Filtrado preciso.	

Cuadro G.52. Prueba de aceptación # 53

Caso de prueba de aceptación	
Código: HU12_P5	Historia de usuario: 12
Nombre: Desactivar configuración	
Descripción: Validar que se puede desactivar una configuración sin eliminarla.	
Condiciones de ejecución: • Existe configuración activa. • Usuario es propietario o administrador. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de configuración. 2. Enviar PUT a /api/v1/configurations/{config_id} con carga útil: {is_active": false}. 3. Validar que la respuesta contenga estado 200. 4. Consultar configuración para confirmar is_active=false. 5. Verificar que datos persisten en BD.	
Resultados esperados: • Estado HTTP: 200 OK. • Configuración marcada como is_active=false. • Datos persistidos. • No aparece en listados de activas (opcional).	

HU-13: Validación y Generación Automática de Configuraciones

Cuadro G.53. Prueba de aceptación # 54

Caso de prueba de aceptación	
Código: HU13_P1	Historia de usuario: 13
Nombre: Validar selección sin persistirla	
Descripción: Validar que se puede verificar si una selección es consistente sin guardarla.	

Continúa en la próxima página

Cuadro G.53. Continuación de la página anterior.

Condiciones de ejecución: • Existe versión PUBLISHED con estructura y restricciones. • Se dispone de lista de características. • Usuario está autenticado.
Pasos de ejecución: 1. Enviar POST a /api/v1/configurations/validate con carga útil: {"version_id": «version_id», "features": [«id_1», «id_2»]}. 2. Validar que la respuesta contenga estado 200. 3. Respuesta incluye: is_valid, errors (si existen), warnings. 4. Verificar que no se crea configuración persistida. 5. Mensaje claro de validación.
Resultados esperados: • Estado HTTP: 200 OK. • Respuesta indica validez. • Lista de errores si los hay. • Lista de advertencias. • No se persiste la configuración.

Cuadro G.54. Prueba de aceptación # 55

Caso de prueba de aceptación	
Código: HU13_P2	Historia de usuario: 13
Nombre: Generar configuraciones válidas automáticamente	
Descripción: Validar que el sistema puede generar automáticamente configuraciones válidas.	
Condiciones de ejecución: • Existe versión PUBLISHED satisfiable. • Usuario está autenticado. • Sistema tiene solucionador SAT integrado.	

Continúa en la próxima página

Cuadro G.54. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar POST a <code>/api/v1/configurations/generate</code> con carga útil: <code>{"version_id": «version_id>», "limit": 5}</code>. 2. Validar que la respuesta contenga estado 200. 3. Respuesta incluye arreglo de configuraciones válidas. 4. Cada una con lista de características seleccionadas. 5. Límite de generaciones respetado.
Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de configuraciones generadas. • Cada una es válida según restricciones. • Cantidad $\leq$ <code>limit</code>. • Diversidad de soluciones (si <code>limit > 1</code>).

Cuadro G.55. Prueba de aceptación # 56

Caso de prueba de aceptación	
Código: HU13_P3	Historia de usuario: 13
Nombre: Generar configuración óptima	
Descripción: Validar que se puede generar una configuración optimizada según criterios.	
Condiciones de ejecución: • Existe versión PUBLISHED. • Se definen criterios de optimización. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar POST a <code>/api/v1/configurations/optimize</code> con carga útil: <code>{"version_id": «version_id>», "objective": "maximize_features"}</code>. 2. Validar que la respuesta contenga estado 200. 3. Respuesta incluye configuración optimizada. 4. Incluye métricas de optimización. 5. Configuración es válida.	

Continúa en la próxima página

Cuadro G.55. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 200 OK. • Configuración optimizada devuelta. • Cumple restricciones. • Optimización aplicada. • Métricas incluidas.
--

Cuadro G.56. Prueba de aceptación # 57

Caso de prueba de aceptación	
Código: HU13_P4	Historia de usuario: 13
Nombre: Validar configuración contra restricciones complejas	
Descripción: Validar que la validación considera restricciones complejas con múltiples operadores.	
Condiciones de ejecución: • Existe versión con restricciones complejas. • Se dispone de selección a validar. • Usuario está autenticado.	
Pasos de ejecución: 1. Crear restricción compleja: (A B) & ~C =>D. 2. Enviar POST a /api/v1/configurations/validate con selección que viola. 3. Validar que la respuesta reporta error. 4. Mensaje indica qué restricción se viola. 5. Sugerir alternativa válida.	
Resultados esperados: • Estado HTTP: 200 OK. • is_valid=false. • Error específico reportado. • Restricción violada identificada. • Sugerencia de corrección.	

Épica 6: Validación y Análisis de Modelos

HU-11: Enriquecimiento con Recursos y Taxonomías

Cuadro G.57. Prueba de aceptación # 58

Caso de prueba de aceptación	
Código: HU14_P1	Historia de usuario: 14
Nombre: Crear etiqueta	
Descripción: Validar que se pueden crear etiquetas para clasificar características.	
Condiciones de ejecución: • Existe modelo. • Usuario es diseñador o administrador. • Usuario está autenticado.	
Pasos de ejecución: 1. Enviar POST a /api/v1/tags con carga útil: {"name": "Programación", "color": "#FF5733"}. 2. Validar que la respuesta contenga estado 201. 3. Verificar que se retorna ID de etiqueta. 4. Verificar que se persiste en BD. 5. Guardar ID para asociar a características.	
Resultados esperados: • Estado HTTP: 201 Created. • Etiqueta creada con ID. • Nombre y color se almacenan. • Puede utilizarse para asociaciones.	

Cuadro G.58. Prueba de aceptación # 59

Caso de prueba de aceptación	
Código: HU14_P2	Historia de usuario: 14
Nombre: Asociar etiqueta a característica	
Descripción: Validar que se puede asociar una etiqueta a una característica.	

Continúa en la próxima página

Cuadro G.58. Continuación de la página anterior.

Condiciones de ejecución: • Existe característica en versión. • Existe etiqueta creada. • Usuario es diseñador. • Usuario está autenticado.
Pasos de ejecución: 1. Obtener ID de característica y etiqueta. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/features/{feature_id}/tags/{tag_id}</code> con cabeceras. 3. Validar que la respuesta contenga estado 200. 4. Consultar característica para confirmar asociación. 5. Verificar que la etiqueta aparece en lista de etiquetas de la característica.
Resultados esperados: • Estado HTTP: 200 OK. • Etiqueta asociada a característica. • Persistida en BD. • Consultas posteriores la incluyen.

Cuadro G.59. Prueba de aceptación # 60

Caso de prueba de aceptación	
Código: HU14_P3	Historia de usuario: 14
Nombre: Desasociar etiqueta de característica	
Descripción: Validar que se puede remover una etiqueta asociada a una característica.	
Condiciones de ejecución: • La característica tiene etiquetas asociadas. • Usuario es diseñador. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.59. Continuación de la página anterior.

Pasos de ejecución: 1. Obtener ID de característica y etiqueta asociada. 2. Enviar DELETE a /api/v1/models/{model_id}/versions/{version_id}/features/{feature_id}/tags/{tag_id} con cabeceras. 3. Validar que la respuesta contenga estado 200. 4. Consultar característica para confirmar que la etiqueta no está asociada. 5. Verificar que se removió de BD.
Resultados esperados: • Estado HTTP: 200 OK. • Etiqueta removida de la característica. • BD actualizada. • La etiqueta sigue existiendo (se reutiliza).

Cuadro G.60. Prueba de aceptación # 61

Caso de prueba de aceptación	
Código: HU14_P4	Historia de usuario: 14
Nombre: Crear recurso educativo	
Descripción: Validar que se puede crear un recurso (archivo, enlace) asociado al modelo.	
Condiciones de ejecución: • Existe modelo. • Usuario es editor de recursos. • Usuario está autenticado. • Se dispone de archivo o URL.	
Pasos de ejecución: 1. Enviar POST a /api/v1/resources con carga útil multipart: {name: Introducción a Java", type: "VIDEO", url: "https://...", language: ".es", license: CC-BY-4.0"}. 2. Validar que la respuesta contenga estado 201. 3. Verificar que se retorna ID de recurso. 4. Verificar metadatos almacenados. 5. Guardar ID para asociar a características/configuraciones.	

Continúa en la próxima página

Cuadro G.60. Continuación de la página anterior.

Resultados esperados: • Estado HTTP: 201 Created. • Recurso creado con ID. • Metadatos almacenados. • Propietario registrado automáticamente. • Marcas de tiempo se establecen.
--

Cuadro G.61. Prueba de aceptación # 62

Caso de prueba de aceptación	
Código: HU14_P5	Historia de usuario: 14
Nombre: Asociar recurso a característica	
Descripción: Validar que se puede vincular un recurso educativo a una característica.	
Condiciones de ejecución: • Existe característica en versión. • Existe recurso creado. • Usuario es editor. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de característica y recurso. 2. Enviar POST a <code>/api/v1/models/{model_id}/versions/{version_id}/features/{feature_id}/resources/{resource_id}</code> con cabeceras. 3. Validar que la respuesta contenga estado 200. 4. Consultar característica para verificar asociación. 5. Recurso aparece en lista de recursos de la característica.	
Resultados esperados: • Estado HTTP: 200 OK. • Recurso asociado a característica. • Persistencia en BD. • Versiones se registran.	

Cuadro G.62. Prueba de aceptación # 63

Caso de prueba de aceptación	
Código: HU14_P6	Historia de usuario: 14
Nombre: Actualizar metadatos de recurso	
Descripción: Validar que se pueden actualizar metadatos del recurso (idioma, licencia, etc).	
Condiciones de ejecución: • Existe recurso persistido. • Usuario es propietario del recurso. • Usuario está autenticado.	
Pasos de ejecución: 1. Obtener ID de recurso. 2. Enviar PUT a /api/v1/resources/{resource_id} con carga útil: {"language": ".en", "license": "MIT", "description": "Nueva descripción"}. 3. Validar que la respuesta contenga estado 200. 4. Consultar recurso para confirmar cambios. 5. Verificar que updated_at se modifica.	
Resultados esperados: • Estado HTTP: 200 OK. • Metadatos actualizados en BD. • Historial versionado (opcional). • Cambios persistidos. • Campo updated_at actualizado.	

Cuadro G.63. Prueba de aceptación # 64

Caso de prueba de aceptación	
Código: HU14_P7	Historia de usuario: 14
Nombre: Listar recursos de característica con filtros	
Descripción: Validar que se pueden obtener todos los recursos de una característica con filtros.	
Condiciones de ejecución: • La característica tiene múltiples recursos asociados. • Usuario está autenticado.	

Continúa en la próxima página

Cuadro G.63. Continuación de la página anterior.

Pasos de ejecución: 1. Enviar GET a <code>/api/v1/models/{model_id}/versions/{version_id}/features/{feature_id}/resources?language=es&type=VIDEO</code> con cabeceras. 2. Validar que la respuesta contenga estado 200. 3. Verificar que solo recursos con idioma es y tipo VIDEO se retornan. 4. Cada recurso incluye metadatos completos. 5. Paginación funciona.
Resultados esperados: • Estado HTTP: 200 OK. • Arreglo de recursos filtrados. • Información completa de cada uno. • Filtros aplicados correctamente. • Paginación funciona.